

GUÍA-LIBRO · DEEP RL · EDICIÓN 3

Arkanoid DRL Learning Lab

Una guía para **entender Deep RL** midiendo un Arkanoid desde cero. Veremos un agente que **parecía aprender**, descubriremos **por qué no ganaba** y construiremos una formulación que **sí generaliza** a niveles nunca vistos. Primero la intuición; después el formalismo, el código y los resultados, sin perder rigor.

PPO

DQN

SAC

World Model

World Model RNN

PyTorch · MPS

Protocolo congelado

5 semillas

AUTOR

Roberto Canales Mora · con Claude

WEB

www.robertocanales.com

REPOSITORIO

github.com/rcanalescoder/DRLArkanoid

PROTOCOLO

`frozen_hash a1ab7ce18d7bad6b`

Cómo leer esta guía

Tres niveles de lectura y el sistema de cajas.

Tienes delante un libro que mezcla intuiciones de un videojuego con ecuaciones de Bellman, fragmentos de pseudocódigo y tablas de resultados con cinco semillas. Si eres junior, te asusta el álgebra; si vienes de la investigación, te impacientan las metáforas. La pregunta es: como está organizado este libro para que cada lector lea solo lo que necesita sin perder el hilo?

Cuatro niveles de lectura superpuestos

La misma página leída a distinta profundidad. Para en la capa que te sirva.

● Intuición Para todos · escena de Arkanoid, analogías	Imprescindible
● Técnica Definiciones, variables y fórmulas	Recomendable
● Código Pseudocódigo e idea de implementación	Opcional
● Reproducibilidad Protocolo, semillas, datos	Opcional

Las cuatro capas de lectura conviven en la misma página: la de intuición es autosuficiente; las demás añaden profundidad para quien la quiera. Puedes parar en la capa que te sirva.

Este capítulo todavía no enseña reinforcement learning. Es un capítulo *meta*: explica el **contrato de lectura**, las reglas del juego entre el texto y tu. Igual que antes de jugar a Arkanoid conviene saber que la pala se mueve solo en horizontal y que la bola rebota con ángulos predecibles, antes de leer conviene saber como están dispuestas las piezas del texto. Una vez entiendas el sistema de niveles de lectura, el sistema de cajas y la coreografía que repite cada capítulo, podrás navegar el resto del libro a la velocidad que te corresponda, sin sentir que te dejas algo imprescindible por el camino.

Cuatro niveles de lectura superpuestos

Cada capítulo está escrito en varias capas que conviven en la misma página. No son cuatro libros distintos encuadrados juntos: es el mismo texto leído a distinta profundidad. La idea es que puedas **parar** en la capa que te sirva y seguir avanzando. La capa de intuición es siempre autosuficiente; las otras tres añaden detalle para quien lo quiera, pero ninguna es un requisito para entender la siguiente página.

Capa de intuición	El relato principal: la pregunta que abre el capítulo, la escena de Arkanoid, la idea central en palabras y la conclusión. Un lector junior puede seguir solo esta capa de principio a fin del libro y salir entendiendo que problema resuelve cada técnica y por que. No necesita tocar una sola fórmula.
Capa técnica	El concepto con su definición precisa, sus variables ($Q(s, a)$, $V(s)$, $pi(a s)$, r , γ ...), las fórmulas destacadas y las tablas de comparación. Aquí se dice <i>exactamente</i> que se calcula, no solo a que se parece. Es la capa que separa entender la idea de poder razonar sobre ella.
Capa de código	El pseudocódigo o la idea de implementación, que aparece siempre después de la idea, nunca antes. No es código de producción: es el esqueleto lógico que te permite, si quieres, reescribirlo en el lenguaje que prefieras. Si solo quieres entender, puedes saltarla; si quieres construir, es tu punto de partida.
Capa de reproducibilidad	Los hiperparametros concretos, el protocolo experimental y los resultados con sus condiciones (cuantas semillas, que presupuesto de pasos, validación frente a test). Es la capa que permitiría a otra persona repetir el experimento y comprobar las cifras. Todas las cifras del libro salen de un mismo protocolo congelado; ninguna esta inventada.

TRADUCCIÓN AL JUEGO

Piensa en los cuatro niveles como en cuatro formas de relacionarte con Arkanoid: jugarlo, leer el manual de instrucciones, mirar el código del motor del juego y, por último, abrir la consola de depuración con todas las variables físicas en pantalla (posición de la bola, vector de velocidad, ángulo de rebote). Es el mismo Arkanoid en los cuatro casos; cambia cuanto detalle decides ver. Jugar ya es entender el juego; abrir la consola es para quien necesita reproducir un rebote exacto.

Si lees por primera vez, te recomendamos una primera pasada **solo por la capa de intuición**: pregunta, escena, idea, conclusión. En una segunda vuelta bajas a la capa técnica de los capítulos que te interesen. La capa de código y la de reproducibilidad estan ahí para cuando quieras construir o verificar, no para abrumarte en la primera lectura. Saltarse una capa no es hacer trampa: es usar el libro como esta diseñado.

El sistema de cajas

El texto corrido lleva el hilo principal. Pero hay ideas que conviene *sacar* del flujo: una advertencia, una aclaración verbal de una fórmula, un límite incomodo de un resultado. Para eso usamos **cajas** con etiqueta de color. Cada tipo de caja cumple una función fija y se lee en un momento distinto respecto al texto que la rodea. Esta es la leyenda completa:

Antes de la fórmula	Explica en palabras que calcula una ecuación <i>antes</i> de mostrartela. Si las matemáticas no son lo tuyo, esta caja suele bastar: te dice que hace la fórmula sin pedirte que la descifres. Leela siempre primero; la fórmula viene justo después.
Traducción al juego	Aterrizas el concepto abstracto en Arkanoid concreto: que es el estado, la acción, la recompensa, la matriz 8x10, la red o el buffer en esta partida. Si una idea te suena demasiado teórica, busca esta caja para verla en movimiento.
Error común	La interpretación equivocada típica, la que casi todo el mundo hace la primera vez. Leerla te ahorra cometer el fallo. Si crees que ya lo has entendido, esta caja es justo la que conviene mirar para confirmarlo.
Que mirar	Guía de lectura de una gráfica o una tabla: en qué columna fijarte, qué comparación importa, qué tendencia buscar. Aparece antes del elemento visual. Sin ella, una tabla de seis algoritmos por siete columnas es ruido; con ella, sabes adónde mirar.
Que no demuestra	Los límites de un resultado: que NO puedes concluir aunque el número sea bueno. Es la caja de la honestidad. Una cifra alta en un conjunto no dice nada sobre otro conjunto distinto; esta caja te recuerda hasta dónde llega la evidencia y dónde empieza la especulación.
Para curiosos	Formalismo extra, referencia a un paper o un detalle avanzado. Es opcional : puedes saltártela sin perder el hilo. Esta pensada para quien quiere rascar más allá de lo necesario para entender el capítulo.

Junto a las cajas hay tres bloques de cierre que se repiten capítulo a capítulo y conviene reconocer de un vistazo:

Quédate con esto	La mini-conclusión del capítulo: la frase o dos que deberías recordar si olvidas todo lo demás. Si solo tuvieras tiempo de leer una cosa por capítulo, sería esta.
Autocomprobación	Tres preguntas cortas al final para que verifiques tu mismo si has entendido. No hay nota; son para ti. Si no sabes responderlas, vale la pena releer la capa de intuición antes de seguir.
Ejercicio	Un problema con enunciado, una pista , una respuesta y una frase de qué significa eso para el agente que aprende. Aquí se pasa de leer a hacer: un cálculo, o razonar una transición concreta.

ERROR COMÚN

Pensar que hay que leer todas las cajas de todos los capítulos para "hacerlo bien". No es así. Las cajas son herramientas opcionales según tu nivel y tu objetivo: un junior puede ignorar "Para curiosos" y casi todas las fórmulas; un investigador puede saltarse "Traducción al juego" si ya domina el concepto. El cuerpo del texto corrido es el único camino obligatorio; las cajas son atajos y miradores que tomas cuando los necesitas.

La coreografía de cada capítulo

Los capítulos no están desordenados: siguen casi siempre la misma **coreografía** de nueve pasos. Reconocerla te ayuda a anticipar qué viene y a saltar directamente a la parte que buscas. Es como saber que en cada nivel de Arkanoid primero aparecen los ladrillos, luego sale la bola y luego empieza el rebote: una vez conoces la secuencia, no te pierdes.

1. Pregunta	El capítulo abre con un problema concreto (la frase destacada del principio). Define que vamos a resolver y por que importa.
2. Intuición	Una escena de Arkanoid o una imagen mental que da pie al concepto, antes de cualquier formalismo.
3. Concepto	La definición técnica con sus variables. El corazón de la capa técnica.
4. Traducción al juego	Como se concreta el concepto en nuestra tarea: estado, acción, recompensa, red, buffer, evaluación.
5. Mini-ejemplo	Un calculo, una transición o una lectura paso a paso, hecha despacio y a mano.
6. Codigo	Pseudocodigo o idea de implementación, solo si aplica y siempre después de la idea.
7. Resultado	Lo observado, con su métrica y sus límites, cuando hay un experimento detrás.
8. Lectura critica	Que significa y, sobre todo, que NO significa el resultado.
9. Autocomprobación	Las tres preguntas de cierre.

No todos los capítulos usan los nueve pasos: este mismo, por ser meta, no tiene resultado experimental ni codigo. La coreografía es una guía, no una camisa de fuerza. Pero el patron general (de la pregunta a la lectura critica) se mantiene, y eso hace que, una vez te acostumbras a leer un capítulo, sepas leer todos.

PARA CURIOSOS

Esta estructura no es decorativa: refleja una idea pedagogica concreta. Primero el *por que* (pregunta), luego el *que* (concepto), luego el *donde* (juego), luego el *como* (codigo) y por último el *cuanto* y el *hasta donde* (resultado y lectura critica). Es, en esencia, el orden en que un equipo de investigación descubre las cosas, invertido para contarlas: el resultado se obtiene al final del trabajo, pero la pregunta que lo motivo estaba al principio, y al lector le sirve más empezar por la pregunta.

El pacto de honestidad

Una última pieza del contrato, transversal a todas las capas. Este libro cuenta una historia real de experimentos, y los experimentos reales incluyen fracasos. No los escondemos. Cuando una idea no funciona, lo decimos con el mismo cuidado que cuando funciona, porque el fracaso bien entendido enseña tanto como el éxito. Dos ejemplos que iras encontrando: el World Model con un LSTM recurrente **no mejora** al World Model normal en esta tarea, y un cierto termino de recompensa moldeada (el shaping Phi) resulto **saboteaba** el objetivo, de modo que quitarlo mejoro los resultados. Ambos son resultados negativos, y ambos estan en el libro porque enseñan algo.

Por eso encontraras la caja *Que no demuestra* tan a menudo. Y por eso mantenemos tres distinciones que es muy fácil confundir y que cambian por completo lo que un número significa:

Recompensa frente a éxito	La recompensa acumulada (sumar puntos por rebotar la pala, romper ladrillos, etc.) no es lo mismo que el éxito , que en este libro es limpiar el nivel entero. Una bola puede acumular recompensa rebotando mucho tiempo sin llegar nunca a limpiar la pantalla.
Validación frente a test	Los niveles se reparten en conjuntos disjuntos de entrenamiento, validación y test. El resultado en validación (donde se ajustan decisiones) no es el resultado en test (donde se mide de verdad, sobre niveles no vistos). Confundirlos infla las cifras.
Una semilla frente a la media de cinco	Lo que consigue una ejecución con una semilla concreta no es lo mismo que la media de cinco semillas. El protocolo de este libro usa cinco semillas justamente porque una sola puede tener suerte o mala suerte.

QUE MIRAR

En la tabla siguiente, lee primero el algoritmo (fila), luego cruza con el presupuesto de pasos (columna). No compares números de filas distintas como si midieran lo mismo sin mirar antes la columna: un valor a 700k pasos y un valor a 3M pasos cuentan historias diferentes sobre el mismo algoritmo.

Algoritmo	Éxito a 700k	Éxito a 1.5M	Éxito a 3M
SAC-pure (actor)	25	87	91
SAC-critic-hybrid	4	61	88

Lectura: el mismo número "leído" sin condiciones engaña. SAC-pure pasa de un 25% de éxito a 700k pasos a un 87% a 1.5M: el actor *si* funciona, pero necesita despegar entre esos dos presupuestos. La variante híbrida pasa de un 4% a un 88% en el mismo rango: su rendimiento depende de forma brutal del presupuesto de pasos. Si alguien citara solo "SAC-hybrid = 88%" sin decir "a 3M pasos", estaría contando media verdad. Por eso cada cifra de este libro viene con sus condiciones; y cuando no tenemos un número fiable, lo describimos en palabras y no nos lo inventamos.

Quédate con esto

El libro tiene cuatro niveles de lectura superpuestos (intuición, técnica, código, reproducibilidad): un junior puede seguir **solo** la capa de intuición y entender todo lo esencial. Las cajas de colores son atajos opcionales con función fija; el texto corrido es el único camino obligatorio. Cada capítulo sigue la misma coreografía, de la pregunta a la lectura crítica. Y todas las cifras son reales y vienen con sus condiciones (cuantas semillas, que presupuesto, validación o test), fracasos incluidos.

AUTOCOMPROBACIÓN

1. Eres un lector junior y una fórmula te bloquea. Que capa de lectura puedes seguir sin tocarla, y que caja te explica esa fórmula en palabras justo antes de mostrarla?
2. Cual es la diferencia entre la caja "Que mirar" y la caja "Que no demuestra", y en que momento aparece cada una respecto a una tabla de resultados?
3. Por qué el libro insiste en distinguir "recompensa" de "éxito", "validación" de "test" y "una semilla" de "la media de cinco"? Que parte del contrato de lectura protegen esas tres distinciones?

EJERCICIO

Imagina que abres un capítulo y ves, en este orden: una frase destacada al principio, una escena de la pala rebotando una bola, una caja "Antes de la fórmula", una fórmula centrada, una caja "Traducción al juego", un bloque de pseudocódigo, una tabla precedida de una caja "Que mirar" y, al final, una caja "Que no demuestra" y tres preguntas. Ordena estos elementos según los pasos de la coreografía y di que paso falta. **Pista:** cuenta los pasos del 1 al 9 y comprueba cual no esta representado. **Respuesta:** el orden es 1 (pregunta), 2 (intuición), 3 (concepto, introducido por "Antes de la fórmula" más la fórmula), 4 (juego), 6 (codigo), 7 (resultado, con la tabla y su "Que mirar"), 8 (lectura critica, la caja "Que no demuestra") y 9 (autocomprobación); falta el paso 5, el mini-ejemplo guiado. **Que significa para el agente:** sin el paso 5 te quedas sin ver una transición concreta del agente (un $s \rightarrow a \rightarrow r \rightarrow s$ prima calculado a mano), que es justo donde se entiende como el número abstracto se convierte en una decisión real sobre mover la pala.

El sistema de cajas: el idioma visual del libro

Cada recuadro tiene un color y una función. No hace falta memorizarlos: aquí están todos.

Antes de la fórmula
qué calcula, en palabras

Traducción al juego
cómo se ve en Arkanoid

Error común
la confusión típica

Qué mirar
cómo leer una gráfica

Que no demuestra
los límites del dato

Para curiosos
formalismo opcional

Quédate con esto
la idea para llevar

Autocomprobación
3 preguntas de repaso

Ejercicio
practica el concepto

Y ahora, para expertos
el mismo tema, técnico

El idioma visual del libro: cada recuadro tiene color y función fijos. No hay que memorizarlo — aquí están los diez tipos, del «error común» (ámbar) al rojo «para expertos».

La coreografía de cada capítulo

Todos los capítulos siguen los mismos 9 pasos. Sabes siempre dónde estás.



Todos los capítulos siguen los mismos nueve pasos, de la pregunta a la autocomprobación. Saber la estructura te dice siempre dónde estás y qué viene después.

Y AHORA, PARA EXPERTOS NIVEL EXPERTO

A partir de aquí, cada tema se cierra con un recuadro rojo como este. La idea es leer en doble pista: primero explicamos todo de forma pedagógica —la intuición, la escena, el mini-ejemplo— para que cualquiera siga el hilo; y luego, en este recuadro, repasamos exactamente lo mismo de forma comprimida y técnica, con el formalismo, las ecuaciones y los nombres propios que el lector avanzado espera encontrar.

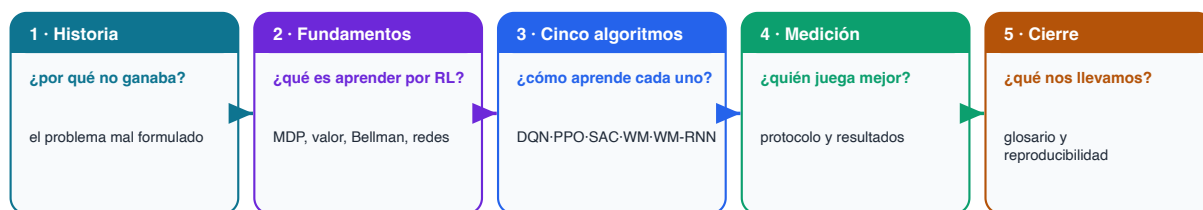
Si vienes de junior, puedes saltarte estos recuadros sin perder nada esencial: el cuerpo del libro es autocontenido y la narración no depende de ellos. Y si vienes de experto, aquí tienes la versión densa — definiciones precisas, referencias y notación — sin tener que esperar a la prosa. Misma materia, dos profundidades; tú eliges a qué velocidad la recorres.

El mapa del laboratorio

Todo el recorrido de un vistazo.

Si abres un proyecto que mezcla un videojuego de los años ochenta con cinco algoritmos de aprendizaje por refuerzo, redes convolucionales y un protocolo de evaluación congelado, lo primero que necesitas no es el primer detalle: es el mapa. Donde empieza la historia, donde acaba, y por qué ruta vamos a pasar de un agente que rompió 0 ladrillos en los niveles difíciles a uno que limpia el 91% de niveles del mismo tipo que jamás había visto durante el entrenamiento?

El mapa del laboratorio: la ruta completa



El gran arco: agente ciego 0% en dispersos → reformular la tarea → PPO 91% en niveles NUNCA vistos

La ruta completa del libro en cinco tramos, con la pregunta que responde cada uno. Abajo, el gran arco: del agente ciego (0%) a PPO (91% en niveles nunca vistos).

Este capítulo es esa vista de pajarero. No vamos a demostrar nada todavía ni a entrar en una sola ecuación: vamos a recorrer el arco completo del libro, presentar de pasada a los personajes (quien es el agente, quien el entorno, que es un estado o una recompensa) y dejar marcado el viaje que haremos después con calma. Piensa en ello como el plano que cuelga en la pared de un laboratorio antes de encender ninguna máquina: no te dice como funciona cada aparato, pero te dice donde esta cada cosa y en que orden la vas a usar.

De qué va el laboratorio

El banco de pruebas es **Arkanoid**, en su variante tipo Breakout: una pala abajo, una bola que rebota y una pared de ladrillos arriba. En nuestra versión la pared es una rejilla de $8 \times 10 = 80$ ladrillos, y el jugador solo puede hacer tres cosas: mover la pala a la **izquierda**, **mantenerla** quieta o moverla a la **derecha**. Tres acciones, ni una más. La pregunta del proyecto es si un programa puede aprender a jugar a esto *solo*, sin que nadie le explique las reglas, a base de probar acciones y recibir señales de acierto o de error.

Esa idea (aprender de la consecuencia de las propias acciones, no de ejemplos etiquetados por una persona) es el corazón del **aprendizaje por refuerzo**, o RL por sus siglas en ingles. No hay un profesor que diga "en esta pantalla la jugada correcta es mover a la derecha"; solo hay un marcador que sube

cuando rompes ladrillos y baja cuando pierdes la bola. Conviene poner cara ya a los personajes que aparecen en cada página del libro, porque son siempre los mismos.

TRADUCCIÓN AL JUEGO

El **agente** es el cerebro que decide mover la pala. El **entorno** es el juego: la física de la bola, los ladrillos, las paredes. En cada instante el agente recibe un **estado** (lo que sabe del mundo: donde esta la bola, hacia donde va, donde esta la pala, que ladrillos siguen vivos), elige una **acción** (izquierda, mantener, derecha) y el entorno le devuelve una **recompensa** (un número: positivo si rompe un ladrillo, negativo si pierde la bola) y un estado nuevo. Repetir ese bucle (estado -> acción -> recompensa -> estado nuevo) millones de veces es, literalmente, todo lo que pasa durante el entrenamiento.

Agente	el que decide. Recibe estados y elige acciones. Es lo único que entrenamos.
Entorno	el mundo con sus reglas: aquí, el juego de Arkanoid y su física. No se entrena; reacciona.
Estado s	la información que describe la situación actual (bola, pala, ladrillos vivos).
Acción a	la decisión tomada. Aquí hay tres posibles.
Recompensa r	la señal numerica que mide si lo ocurrido tras la acción fue bueno o malo.
Política $\pi(a s)$	la estrategia del agente: que acción elegir en cada estado.
Valor $V(s)$ y $Q(s,a)$	una estimación de cuanta recompensa total cabe esperar a futuro desde un estado, o desde un estado tomando una acción concreta.

Estos siete nombres son el alfabeto del libro. Cada algoritmo que veremos no es más que una forma distinta de aprender la política π o de estimar el valor Q . No hace falta entenderlos del todo ahora; basta con saber que existen y que volveremos a ellos una y otra vez.

TRADUCCIÓN AL JUEGO

Para que los terminos no queden en abstracto, leamos una transición real del bucle. La bola viene cayendo hacia la esquina inferior derecha; ese es el **estado s** (incluye su posición, su velocidad y la posición de la pala). El agente elige la **acción** "derecha" para colocar la pala bajo la bola. El entorno aplica su física: la pala intercepta la bola, que rebota hacia arriba, golpea un ladrillo y lo destruye. La **recompensa** de ese paso es $r = +1.0$ por el ladrillo roto (más un pequeño extra por haber rebotado en la pala). El agente recibe el **estado nuevo s'**, con la bola subiendo y un hueco más en la pared. Ese único paso ya contiene a los cuatro personajes principales en acción.

El arco del libro, capa a capa

El recorrido tiene seis tramos. Cada uno responde a una pregunta y prepara el siguiente. Lo presento como un itinerario, no como un índice cerrado.

1. La historia del problema	como empezo todo y por que casi no funciona. Un primer agente ciego que solo veía seis números cinemáticos, sin saber que ladrillos quedaban. Aquí nace la pregunta central del libro: cuando el agente falla, el problema, ¿es del algoritmo o de como le planteamos la tarea?
2. Los fundamentos de RL	el vocabulario y las ideas que sostienen todo: estado, acción, recompensa, política, valor y la ecuación que relaciona el ahora con el futuro. Es la teoría mínima para leer el resto sin perderse.
3. El laboratorio Arkanoid	como convertimos un juego en un problema medible: que ve el agente (los 86 números del estado, con la matriz de ladrillos incluida), que recompensas damos, cuanto tiempo le dejamos antes de cortar la partida, y como generamos niveles distintos para entrenar y para examinar.
4. Los cinco algoritmos	cinco maneras de aprender a jugar: DQN, PPO, SAC, World Model (Dyna-Q) y su variante con memoria, World Model RNN . Unos aprenden estimando el valor de cada acción; otros ajustan directamente la estrategia; otros se construyen un modelo mental del juego para "imaginar" jugadas. Los compararemos en igualdad de condiciones.
5. La medición y los resultados	la parte más seria del libro. Un protocolo congelado (mismas reglas para todos, cinco semillas aleatorias, evaluación sobre niveles <i>no vistos</i>) que evita engañarnos a nosotros mismos. Aquí salen los números que de verdad importan, incluidos los resultados negativos.
6. El código	como esta construido todo por dentro: la red de dos ramas, el bucle de entrenamiento, el banco de evaluación. La idea es que cualquiera pueda abrir el proyecto, leerlo y reproducirlo.

■ El gran arco de resultados: del 0% al 91%

Si hubiera que resumir el libro en una sola frase, seria esta: **lo difícil no fue elegir el algoritmo perfecto, fue formular bien la tarea**. Y la mejor forma de verlo es comparar el punto de partida con el de llegada.

TRADUCCIÓN AL JUEGO

En la **era ciega** (el primer intento historico), el agente jugaba en una rejilla más pequeña de $4 \times 7 = 28$ ladrillos y veía solo seis números sobre la bola y la pala: no sabia que ladrillos seguian en pie. El resultado fue pobre: **0%** de éxito en los niveles dispersos y, de media, apenas un par de ladrillos rotos sobre 28. Lo único que parecía "funcionar" (un **56%** de "éxito" en la rejilla totalmente llena) era un espejismo: con la pared completa, una bola rebotando durante muchisimos pasos acaba tocando casi todo por pura física, no por puntería. Era supervivencia, no estrategia.

El punto de llegada es otro mundo. Tras reformular la tarea (darle al agente una matriz que le dice que ladrillos siguen vivos, un reloj de partida proporcional al número de ladrillos, una recompensa con la meta limpia y un encoder convolucional que entiende el espacio del tablero), el mejor algoritmo, **PPO**, alcanza un **91%** de éxito en niveles del mismo tipo que los de entrenamiento. Y no se desmorona ante lo desconocido: cuando se le examina con patrones de ladrillos que *nunca* había visto, todavía limpia el **89%**. El salto del 0% al 91% no vino de entrenar más tiempo ni de un algoritmo magico: vino de plantear bien la pregunta.

ANTES DE LA FÓRMULA

Una de esas reformulaciones es el "reloj" de la partida, y se entiende con una cuenta de una línea. En la era ciega había un timeout *constante*: 600 pasos, dieras los ladrillos que dieras. Ahora el timeout es **proporcional** al tamaño del nivel: se conceden 90 pasos por ladrillo. En la rejilla de 80 ladrillos eso son $90 \times 80 = 7200$ pasos. La idea es dar siempre un plazo justo para la tarea que toca: ni tan corto que sea imposible limpiar un nivel grande, ni tan largo que premie quedarse rebotando sin objetivo. Es un cambio en como *describimos* la tarea, no en el algoritmo.

$\text{timeout} = 90 \text{ pasos/ladrillo} \times 80 \text{ ladrillos} = 7200 \text{ pasos}$

ERROR COMÚN

Es tentador leer "PPO 91%, World Model 55%" y concluir que PPO es "mejor algoritmo" en abstracto. No es lo que dice el libro. Esos números se obtienen *después* de formular bien la tarea, y cada uno es la **media de cinco semillas** sobre un conjunto concreto de niveles, no el resultado de una única ejecución afortunada. Cambiar la formulación (por ejemplo, quitarle al agente la visión de los ladrillos) hundiría a todos por igual. El protagonista de esta historia es la **tarea bien planteada**; los algoritmos son los actores de reparto.

ERROR COMÚN

Otra confusión frecuente: mezclar **recompensa** con **éxito**. La recompensa es la señal interna que guía el aprendizaje paso a paso (+1.0 por ladrillo, -1.0 por perder la bola). El éxito es la métrica externa con la que *juzgamos* al agente: el porcentaje de la pared que consigue limpiar en una evaluación. Son cosas distintas, y a lo largo del libro veremos un caso donde subir la recompensa con un truco de shaping no solo no subía el éxito, sino que lo empeoraba. Cuando leas un número, pregunta siempre: ¿esto es recompensa o es éxito?

QUE NO DEMUESTRA

Que PPO llegue al 91% no demuestra que el problema este "resuelto" en general. Es un 91% en un tipo concreto de niveles (los del mismo patrón visto en entrenamiento), con una arquitectura concreta y un presupuesto concreto de pasos. En los niveles más exigentes la cifra baja, y hay algoritmos que se estancan o que a veces colapsan. Tampoco hay que confundir la **validación** (lo que se mira durante el desarrollo para ajustar) con el **test** sobre niveles nunca vistos (el examen final): el 91% es de test. Una vista de pájaro sirve para situarse, no para sacar conclusiones; los matices, los límites y los fracasos honestos (como el del modelo con memoria, que no mejora a su versión simple) llegan en sus capítulos.

■ Por qué vale la pena el viaje

Para tener una referencia de escala: **DeepMind** hizo famoso este tipo de juego en 2015, cuando su agente DQN aprendió a jugar a **49 juegos** de Atari (Breakout entre ellos) mirando directamente los píxeles de la pantalla, con redes convolucionales y del orden de **50 millones de fotogramas** por juego. Aquel trabajo demostró que *se podía*. Nuestro laboratorio no compite con eso ni lo reproduce: hace otra cosa. Toma cinco familias de algoritmos, las pone a competir en el *mismo* banco de pruebas con reglas idénticas, y usa ese banco para responder a una pregunta más fina y, quizá, más útil: cuando un agente

de RL falla, ¿es culpa del método o de como hemos descrito el mundo? La respuesta corta es la tesis de todo el libro. La respuesta larga empieza en el siguiente capítulo.

Quédate con esto

El libro recorre seis tramos (historia, fundamentos, laboratorio, cinco algoritmos, medición, código) alrededor de un mismo elenco: agente, entorno, estado, acción, recompensa, política y valor. El gran arco va de un agente ciego con 0% en niveles dispersos a PPO con 91% en niveles del mismo tipo y 89% en patrones nunca vistos, siempre como media de cinco semillas. La moraleja no es "este algoritmo gana", sino "formular bien la tarea es lo que desbloquea el aprendizaje".

AUTOCOMPROBACIÓN

1. ¿Cuáles son los siete "personajes" que aparecen en todo el libro, y cual de ellos es el único que de verdad entrenamos?
2. El agente de la era ciega lograba un 56% de "éxito" en la rejilla llena. ¿Por qué ese número no significa que jugara bien, y que diferencia hay entre recompensa y éxito?
3. Si el salto del 0% al 91% no vino de cambiar de algoritmo ni de entrenar más, ¿de qué vino, según la tesis del libro?

Arkanoid DRL Learning Lab

Estudia el bucle agente-entorno, los datos que genera y cómo aprende cada algoritmo · DQN · PPO · SAC · World Model

backend: webgpu

en pausa

Paso

Reiniciar

Entrenar

Motivación del laboratorio. Esta interfaz está pensada como guía de estudio: no solo muestra al agente jugando, también el **mecanismo interno**. Por eso hay un icono de información (i) junto a cada grupo y cada dato importante. **Pulsa cualquier i** para abrir una ficha completa que explica qué es, por qué importa y, cuando aplica, su fórmula y sus datos.

PARTIDA OBSERVADA

ENV 0

jugando

Recompensa episodio: **-0.03** Ladrillos: 0/28 Pasos: 4

MÉTRICAS CLAVE

RECOMPENSA-100

1.295

▲ 0.000

PÉRDIDA

0.000

E EXPLORACIÓN

0.050

TD-ERROR

0.000

BUFFER

38.400

TASA DE ÉXITO

0.0%

Inspector DQN

Q(s,a) — valor esperado de cada acción

←

0.000

→

0.000

→

0.000

ε exploración 0.050

TD-error medio 0.0000

Buffer 38.400 (38%)

CURVAS DE ENTRENAMIENTO

RECOMPENSA / ÉXITO

reward-100 éxito-100

PÉRDIDA (HUBER)

loss

TAMAÑO DEL REPLAY BUFFER

buffer

EXPLORACIÓN (E)

ε

TRANSICIÓN ACTUAL

s (0.59, 0.53) → a ← → r -0.008 → s' (0.57, 0.51) → done false

VERSIÓN DEL EXPERIMENTO

DQN

Model-free · basado en valor · off-policy

recomendado

PPO

Model-free · actor-crítico · on-policy

política

SAC

Model-free · actor-crítico · off-policy

avanzado

World Model

Model-based · off-policy

model-based

CONTROL DE EJECUCIÓN

Velocidad de simulación

4x

Entornos de entrenamiento (headless)

256

Entornos visuales

8

Reward shaping potencial

☒

ENTORNOS EN PARALELO

env. 0

env. 1

env. 2

env. 3

env. 4

env. 5

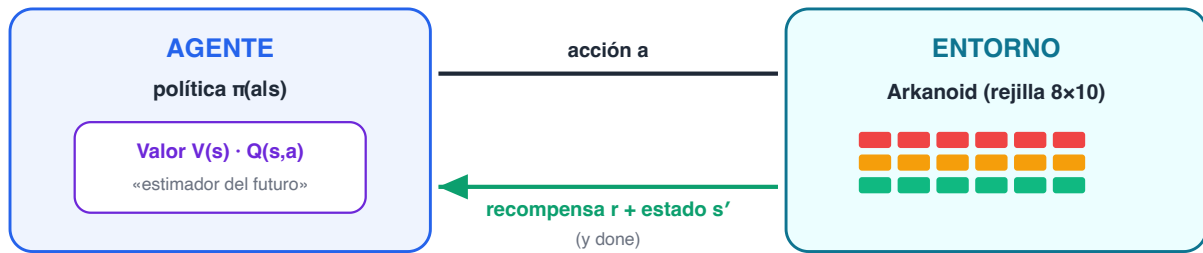
env. 6

env. 7

El laboratorio web de un vistazo: a la izquierda la **partida observada** sobre la rejilla 8×10; en el centro las **métricas clave** y las **curvas de entrenamiento**; a la derecha el **inspector**, que abre la red y muestra los valores $Q(s, a)$ de cada acción. Abajo, el selector de algoritmo y los controles. Todo lo que cuenta este libro se puede ver aquí en vivo.

Los siete personajes del bucle

Reaparecen en todo el libro. El valor vive dentro del agente, estimando el futuro.



estado $s \rightarrow$ agente/política $\rightarrow$ acción $a \rightarrow$ entorno $\rightarrow$ recompensa r + nuevo estado $s' \rightarrow \dots$

Los siete protagonistas y cómo se encadenan: el agente decide una acción, el entorno responde con recompensa y nuevo estado. El valor vive dentro del agente, estimando el futuro.

El gran arco de resultados: del 0% al 91%

ERA CIEGA

- observación: 6 números (sin ladrillos)
- rejilla 4x7 · reloj fijo de 600 pasos
- 0% en niveles dispersos
- ~2 de 28 ladrillos rotos
- 56% solo en rejilla llena = espejismo



FORMULACIÓN CORREGIDA

- observación: 6 cinemáticos + matriz 8x10
- timeout proporcional · sin shaping Φ · conv
- 5 semillas · evaluación GREEDY
- medido en niveles NUNCA vistos
- PPO 91% TEST-ID (0 colapsos)

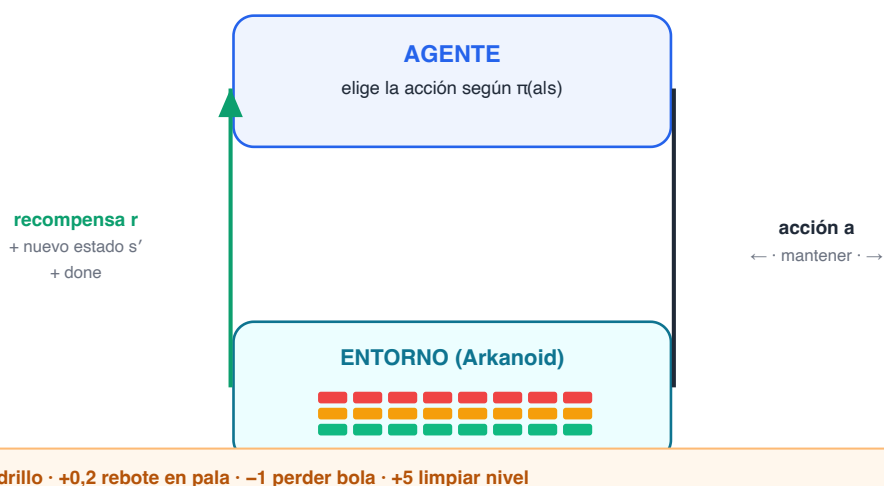
La tesis del libro en una imagen: el salto del 0% al 91% no vino de un algoritmo mágico, sino de reformular la tarea (observación, reloj y meta).

Aprender por consecuencias

Que es el aprendizaje por refuerzo, con un Arkanoid.

Nadie le enseña a un niño a montar en bici dándole una lista de fotos etiquetadas "esto es equilibrio" y "esto es caída". El niño lo prueba, se cae, ajusta, y vuelve a probar. Aprende por las consecuencias de sus actos. La pregunta de este libro entero es: si un agente de software solo recibe la consecuencia de lo que hace (un número que sube o baja), sin que nadie le diga nunca cual era la jugada correcta, puede aprender a jugar bien a Arkanoid?

El bucle agente-entorno: el corazón de todo



El bucle agente-entorno, el diagrama base de todo el libro: el agente actúa, el entorno devuelve recompensa y nuevo estado, y el ciclo se repite millones de veces.

Aprendizaje supervisado

datos con la respuesta correcta

"esto es un gato"

Aprendizaje no supervisado

datos sin etiqueta; busca estructura

"estos se parecen"

Aprendizaje por refuerzo (RL)

premios y castigos al actuar

"+1 si ganas"

El RL no aprende de respuestas correctas, sino de la recompensa que recibe al probar.

Las tres familias del aprendizaje automático. En el **supervisado** cada ejemplo trae su respuesta correcta; en el **no supervisado** solo hay datos sin etiqueta; en el **refuerzo** no hay respuestas, solo una recompensa que llega —a veces tarde— como consecuencia de actuar. Esa diferencia define todo el libro.

Esa es, en una frase, la idea del **aprendizaje por refuerzo** (en inglés *reinforcement learning*, RL). No le damos al agente las respuestas. Le damos un mundo donde actuar, una señal de recompensa que premia lo bueno y castiga lo malo, y muchísimos intentos. El agente tiene que descubrir, por su cuenta, que conviene hacer. En este capítulo construimos esa idea desde cero, usando Arkanoid como banco de pruebas, sin dar por sabido nada.

La escena: una partida de Arkanoid

Recordemos el juego. Hay una **pala** que se mueve por la parte de abajo de la pantalla. Hay una **bola** que rebota. Y arriba hay una pared de **ladrillos**. Cada vez que la bola golpea un ladrillo, el ladrillo desaparece. Si la bola se te escapa por abajo porque la pala no llega, pierdes. El objetivo es romper todos los ladrillos sin perder la bola.

Para un humano esto es trivial: miras la bola, la sigues con la pala, intentas dirigir los rebotes hacia donde quedan ladrillos. Pero fíjate en lo que acabo de hacer: he dicho "miras", "sigues", "intentas dirigir". He descrito una *estrategia*. El agente de RL no parte de ninguna estrategia. Parte de cero. No sabe que la pala se usa para no perder la bola. Ni siquiera sabe que perder la bola es malo. No sabe que esos rectángulos de arriba son "ladrillos", ni que romperlos sea la meta. Todo eso lo tiene que **deducir de las consecuencias**.

Las piezas del problema

Para hablar con precisión necesitamos cinco palabras. Son el esqueleto de todo el libro, así que merece la pena fijarlas bien.

Agente	quien decide y aprende. En nuestro caso, el programa que controla la pala. Es lo único que entrenamos; todo lo demás es el mundo donde vive.
Entorno	todo lo que no es el agente: la física de la bola, las paredes, los ladrillos, las reglas del juego. El entorno recibe la acción del agente y le devuelve una nueva situación y un número.
Estados	la descripción de la situación actual que el agente recibe en cada instante. Es lo que el agente "ve". Más abajo veremos que, en este proyecto, ese estado son 86 números.
Acciones	lo que el agente puede hacer. En Arkanoid son solo tres opciones: mover la pala a la izquierda , mantenerla quieta o moverla a la derecha .
Recompensas	el número que el entorno entrega después de cada acción. Positivo cuando pasa algo bueno, negativo cuando pasa algo malo. Es la única nota que recibe el agente. No hay más profesor que este número.

Y con estas piezas se monta una sexta idea, la del tiempo:

Episodio	una partida completa, de principio a fin. Empieza con el nivel lleno de ladrillos y termina cuando el agente limpia el nivel, pierde la bola o se acaba el tiempo. Aprender en RL es jugar muchísimos episodios, uno detrás de otro.
-----------------	--

El bucle: el corazón de todo

Con esas piezas, el aprendizaje por refuerzo es un **bucle** que se repite paso a paso. En cada paso ocurre exactamente lo mismo, en el mismo orden.

ANTES DE LA FÓRMULA

El agente observa la situación actual s , elige una acción a , y el entorno le responde con dos cosas: cuanto ha ganado o perdido en ese paso (la recompensa r) y cual es la nueva situación (el siguiente estado s'). También le dice si la partida ha terminado (la señal *done*, que vale 1 cuando el episodio acaba y 0 mientras sigue). Después, ese siguiente estado se convierte en el estado actual del paso siguiente, y vuelta a empezar.

$$s \rightarrow a \rightarrow (r, s', \text{done}) \rightarrow s \leftarrow s' \rightarrow a \rightarrow \dots$$

Eso es el bucle **estado** → **acción** → **recompensa** → **siguiente estado** → **repetir**. No hay nada más. Toda la complejidad del libro (las redes neuronales, los algoritmos con nombres raros como DQN o PPO) son maneras distintas de responder a una sola pregunta dentro de este bucle: *dada la situación s que veo ahora, que acción a me conviene elegir para que la suma de recompensas futuras sea lo más alta posible?*

Esa palabra, **futuras**, es la clave y la dificultad. El agente no quiere maximizar la recompensa de este paso, sino la de toda la partida. Y muchas veces la acción que más conviene ahora (acercar la pala a la bola, aunque no rompa ningún ladrillo y de recompensa cero en este paso) es buena solo por lo que permite **después**: no perder la bola y poder seguir rompiendo ladrillos. A la regla con la que el agente decide que acción tomar en cada estado la llamaremos **política** ($\pi(a|s)$): es, literalmente, su forma de jugar, y es lo que el entrenamiento va puliendo. Aprender a valorar el futuro a partir de una señal que llega paso a paso es el problema central del aprendizaje por refuerzo. Lo retomaremos con cuidado en los capítulos siguientes.

■ Cómo se ve esto en nuestro Arkanoid

Hasta aquí ha sido abstracto. Bajemoslo a números concretos, los de este proyecto, para que las cinco palabras dejen de ser definiciones y sean cosas que se pueden tocar.

TRADUCCIÓN AL JUEGO

El **tablero** es una rejilla de ladrillos de 8 filas por 10 columnas: 80 ladrillos en total cuando el nivel esta lleno.

El **estado** s que recibe el agente son **86 números**. Seis describen la física del momento: la posición de la bola (sus coordenadas x e y), su velocidad (v_x , v_y), la posición de la pala (x) y la distancia entre la bola y la pala. Los otros 80 son una **matriz de ocupación** de 8x10: una casilla vale 1 si su ladrillo sigue vivo y 0 si ya esta roto. Así el agente no solo sabe donde esta la bola, sino también que ladrillos quedan por romper. (Más adelante veremos que esto no siempre fue así: en el primer intento del proyecto el agente solo recibia los seis números cinemáticos, sin la matriz de ladrillos, y la diferencia resulto enorme.)

La **acción** a es una de tres: izquierda, mantener, derecha.

La **recompensa** r se reparte así: **+1.0** por cada ladrillo roto, **+0.2** por rebotar la bola con la pala, **-1.0** por perder la bola y **+5.0** por limpiar el nivel entero. Hay además un bonus de combo de **+0.5** por cada ladrillo extra que se rompe en una misma cadena sin que la bola vuelva a tocar la pala; con los cuatro números principales basta para entender la idea, pero conviene saber que ese bonus existe.

Un **episodio** termina al limpiar el nivel, al perder la bola, o si se agota un límite de pasos. Ese límite es proporcional al tamaño del nivel: 90 pasos por ladrillo, lo que da 7200 pasos en una rejilla 8x10. Es el "reloj" de la partida, y más adelante veremos que ajustar bien ese reloj fue una de las decisiones que más influyo en el resultado.

Mini-escenario: diez pasos vistos por dentro

Nada fija mejor el bucle que recorrerlo a mano. Imaginemos un agente todavía poco entrenado, en mitad de una partida. Veamos diez pasos: en cada uno, que ve (resumido, porque ver los 86 números enteros seria ilegible), que acción toma, que recompensa recibe y, sobre todo, **que lección deja** ese paso para el aprendizaje. Recuerda: el agente no sabe de antemano que ninguna acción es buena; solo después, viendo la recompensa, puede empezar a asociar "en una situación como esa, esa acción salio bien (o mal)".

QUE MIRAR

Lee la tabla por filas, como una pelicula a camara lenta. Fijate en la columna de recompensa: casi siempre es cero. La señal útil es escasa. El agente pasa la mayor parte del tiempo actuando sin recibir ninguna nota, y solo de vez en cuando llega un +1.0 o un -1.0 que lo cambia todo.

Paso	Que ve el agente (s)	Acción (a)	Recompensa (r)	Que deja como lección
1	bola bajando por el centro, pala a la izquierda	derecha	0	nada todavía; la pala se acerca a la bola, pero el premio (si llega) vendrá después
2	bola más abajo, pala casi bajo la bola	derecha	0	nada aun; sigue colocandose
3	bola justo encima de la pala	mantener	+0.2	rebote logrado: "estar bajo la bola y mantener" dio premio
4	bola subiendo hacia los ladrillos	mantener	0	nada; la bola viaja sola, la acción casi da igual ahora
5	bola llega a un ladrillo	izquierda	+1.0	ladrillo roto: premio grande, aunque la acción de la pala no fue la causa directa
6	bola rebotada, bajando otra vez	izquierda	0	nada; vuelve a tocar colocar la pala
7	bola bajando rápido a la derecha, pala a la izquierda	izquierda	0	error en marcha: la pala se aleja de la bola, pero aun no hay castigo
8	bola muy abajo, pala lejos a la izquierda	derecha	0	reacción tardía; intenta volver, pero quizá no llegue
9	bola por debajo de la pala, fuera de alcance	derecha	-1.0	bola pérdida: castigo. La culpa real estaba en los pasos 7-8, no en este
10	nivel reiniciado, bola nueva saliendo	mantener	0	empieza otro episodio; arrastra lo aprendido en este

Un detalle para no enganarnos al leer la tabla: en el paso 5 la recompensa de +1.0 se le entrega justo después de la acción "izquierda", pero romper ese ladrillo fue cosa de la física de la bola, no de mover la pala en ese instante. El entorno no explica *por que* da el premio; solo lo da. Atribuir correctamente cada recompensa a las acciones que de verdad la causaron es trabajo del agente, no del juego. Esto enlaza con las dos grandes dificultades que esconde la tabla, y conviene nombrarlas ya aunque las desarrollemos después.

La primera: la **recompensa es escasa**. De diez pasos, siete dieron cero. El agente actúa mucho y recibe nota poco. Tiene que aprender a partir de señales muy espaciadas en el tiempo.

La segunda, más sutil: el **problema de la asignación de crédito**. El castigo de -1.0 llegó en el paso 9, pero el verdadero error fue elegir "izquierda" en el paso 7, cuando la bola ya bajaba hacia la derecha. La consecuencia mala apareció dos pasos después de la causa. El agente tiene que aprender a "repartir la culpa" hacia atrás, a las decisiones que de verdad provocaron el desenlace. Eso es justo lo que harán los algoritmos del libro, cada uno a su manera.

Lo que cree un humano frente a lo que recibe el agente

Aquí está, a mi juicio, el malentendido más común de quien empieza en RL. Tendemos a suponer que el agente "entiende" el juego como nosotros. No es así. Nosotros le ponemos significado a las cosas; el solo recibe números. Esta tabla contrapone las dos miradas.

Lo que cree un humano	Lo que de verdad recibe el agente
"Hay una pala, una bola y ladrillos."	Un vector de 86 números. Que algunos sean "la bola" o "los ladrillos" es interpretación nuestra, no del agente.
"Mover la pala sirve para no perder la bola."	Tres acciones sin nombre ni propósito. Que "izquierda" sirva para algo solo se descubre por sus consecuencias.
"Romper ladrillos es el objetivo."	Un +1.0 que aparece a veces. El objetivo emerge de perseguir ese número, no de una meta declarada.
"Perder la bola es un fallo."	Un -1.0 y el fin del episodio. "Fallo" es una etiqueta humana; para el agente es solo una recompensa negativa que conviene evitar.
"El agente sabe que va ganando."	Solo conoce el estado actual y la última recompensa. No tiene marcador ni narrativa; tiene números y lo que haya aprendido a estimar sobre el futuro.

ERROR COMÚN

Pensar que diseñar la recompensa es un detalle menor, "ponerle puntos al juego". Es justo al revés: como la recompensa es la **única** información sobre que es bueno, el agente optimizara exactamente lo que premias, no lo que tu querías premiar. Si premias mal, aprendera a hacer algo brillante y completamente inútil. En este proyecto eso paso de verdad: una recompensa auxiliar mal planteada (un termino de *shaping*) empujaba al agente en la dirección equivocada, y quitarla mejoro el resultado. Lo veremos con datos más adelante. De momento, quédate con la moraleja: el agente toma la recompensa al pie de la letra.

RL frente al aprendizaje supervisado y no supervisado

Para situar bien el aprendizaje por refuerzo conviene compararlo con las otras dos grandes familias del aprendizaje automático. La diferencia esencial esta en **que dato recibe cada una** durante el entrenamiento.

Aprendizaje supervisado	recibe ejemplos con la respuesta correcta ya puesta: pares (entrada, etiqueta). "Esta foto es un gato", "este correo es spam". El profesor te dice la respuesta y tu aprendes a imitarla. Necesita que alguien etiquete los datos de antemano.
Aprendizaje no supervisado	recibe datos sin etiqueta y busca estructura por su cuenta: grupos, patrones, regularidades. "Agrupa estos clientes en perfiles parecidos." No hay respuesta correcta que imitar; hay datos que organizar.
Aprendizaje por refuerzo	no recibe ni la respuesta correcta ni datos sueltos, sino una recompensa tras cada acción en un entorno. Nadie le dice cual era la jugada buena; solo si lo que hizo salio mejor o peor. Y, a diferencia de los otros dos, sus propias acciones determinan que datos vera después: si mueve mal la pala, no llegara a estados con muchos ladrillos rotos y nunca aprendera de ellos.

La diferencia se ve clara con Arkanoid. Un enfoque **supervisado** necesitaria una base de datos enorme de situaciones con la acción correcta anotada por un experto ("en esta posición, mover a la derecha"). Nadie tiene eso, y generarlo seria carisimo. Un enfoque **no supervisado** podría agrupar situaciones parecidas, pero por si solo no sabria *jugar*: no hay noción de objetivo. El aprendizaje por **refuerzo** encaja con el problema porque no exige respuestas dadas: solo necesita un entorno donde probar y una

recompensa que premie romper ladrillos y castigue perder la bola. El agente genera su propia experiencia jugando.

PARA CURIOSOS

La frontera no es un muro. La recompensa de RL es una forma de supervisión, pero **evaluativa** (te dice como de bien lo has hecho) en lugar de **instructiva** (te dice que deberías haber hecho). Y dentro de un agente de RL pueden aparecer piezas supervisadas: en el capítulo del *World Model* de este libro entrenaremos una red para que **prediga** el siguiente estado a partir del actual y la acción, lo cual es un problema supervisado clásico (entrada conocida, objetivo conocido) metido dentro de un agente de refuerzo.

La idea central: aprender por consecuencias

Recogiendo todo: el agente no recibe instrucciones, recibe consecuencias. Actúa, observa que número llega y en que situación queda, y poco a poco ajusta su comportamiento (su política) para que la suma de esos números a lo largo de la partida sea lo más alta posible. No memoriza respuestas correctas porque no las tiene. Las **descubre** probando, equivocándose y reforzando lo que funciona. De ahí el nombre: refuerzo.

Una última precaución de vocabulario, que sostendrá todo el libro: no es lo mismo **recompensa** que **éxito**. La recompensa es la señal interna que el agente persigue paso a paso (esos +1.0, +0.2, -1.0). El éxito es lo que a nosotros nos importa de verdad: limpiar el nivel. Están relacionados, pero no son lo mismo, y más adelante veremos casos en los que un agente acumula recompensa razonable sin llegar a jugar bien. Cuando demos resultados en este libro, diremos siempre cual de las dos cosas estamos midiendo.

Quédate con esto

El aprendizaje por refuerzo es un bucle: el agente ve un estado s , elige una acción a , recibe una recompensa r y un nuevo estado s' , y repite. No hay profesor que de la respuesta correcta; solo una recompensa que premia lo bueno y castiga lo malo. El agente aprende, por las consecuencias de sus actos, a maximizar la recompensa a lo largo de toda la partida, no solo en el paso actual. Esa es, en una frase, la idea que sostiene el libro entero.

AUTOCOMPROBACIÓN

1. Enumera las cinco piezas del bucle de RL y di, para cada una, cual es en nuestro Arkanoid (con sus números concretos cuando los tengas).
2. En el mini-escenario, el castigo llegó en el paso 9, pero el error real estaba en el paso 7. Como se llama este problema y por qué complica el aprendizaje?
3. Que dato recibe durante el entrenamiento cada una de las tres familias (supervisado, no supervisado, refuerzo)? Por qué el supervisado no es práctico para Arkanoid?

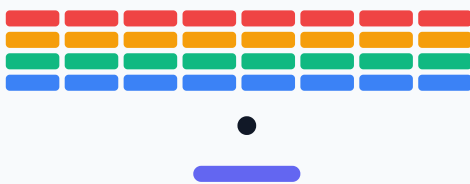
EJERCICIO

Diseña tu una recompensa "lógica pero rota" para Arkanoid: en vez de premiar ladrillos rotos, da **+0.2 cada vez que la pala rebota la bola** y nada más (ni premio por ladrillos ni castigo por perder). Que aprendería a hacer el agente para maximizar ese número? **Pista:** piensa cual es la conducta que más rebotes acumula a lo largo del tiempo, sin importar los ladrillos. **Respuesta:** aprendería a *mantener la bola en juego eternamente*, golpeandola una y otra vez con la pala, sin ninguna intención de romper ladrillos, porque cada rebote suma y romper ladrillos no aporta nada extra. Maximizar esa recompensa lleva a un superviviente pasivo, no a un buen jugador. **Que significa para el agente:** que el agente nunca persigue "ganar" en abstracto; persigue exactamente el número que le pongas. Diseñar bien la recompensa no es un adorno, es definir cual es el problema.

Lo que ve un humano vs lo que recibe el agente

El agente no «entiende» el juego: solo recibe números y una recompensa.

EL HUMANO VE



«hay ladrillos arriba, devuelvo la bola y apunto»

EL AGENTE RECIBE

6 cinemáticos:

bola(0.59,0.53) v(-.02,.04) pala 0.61 d 0.18

matriz 8x10 (1=ladrillo, 0=hueco):

```
1 1 0 1 1 1 0 1
1 0 0 1 1 0 0 1
0 1 1 1 0 1 1 0
1 1 0 0 1 1 0 1
```

recompensa: +1 ocasional · -1 al perder

El agente no «entiende» el juego como nosotros: donde el humano ve un tablero, el agente recibe 6 números cinemáticos + una matriz 8x10 de unos y ceros, y una recompensa escasa.

Y AHORA, PARA EXPERTOS NIVEL EXPERTO

«Aprender por consecuencias» se formaliza como un **Proceso de Decisión de Markov** (MDP), la tupla (S, A, P, R, γ) : S es el espacio de estados, A el de acciones, $P(s' | s, a)$ la dinámica de transición (probabilidad de pasar a s' tras ejecutar a en s), $R(s, a, s')$ la función de recompensa y $\gamma \in [0, 1)$ el factor de descuento. El agente interactúa generando una trayectoria $\tau = (s_0, a_0, r_0, s_1, \dots)$.

Sobre esa trayectoria definimos el **retorno** descontado desde el instante t :

$$G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$$

y el objetivo del aprendizaje es hallar la política π que maximiza el retorno esperado a lo largo de las trayectorias que ella misma induce:

$$J(\pi) = E_{\{\tau \sim \pi\}}[G_0]$$

Toda la teoría posterior —valor, Bellman, exploración— se construye sobre esta única definición de éxito.

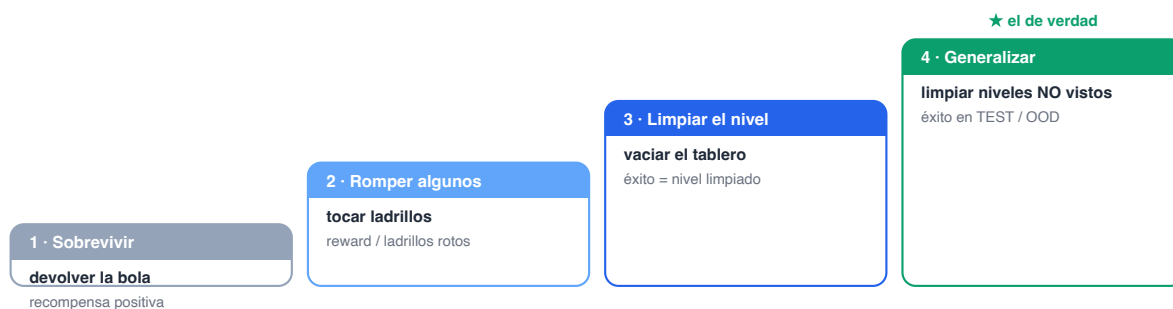
Sobrevivir no es resolver

Por que recompensa alta no es ganar.

Un agente lleva miles de partidas. Su recompensa media sube semana a semana y la curva de aprendizaje es una pendiente bonita hacia arriba. Entonces le pones delante un nivel normal y no rompe casi ningún ladrillo. La pregunta de este capítulo es incomoda: si la recompensa sube, ¿por qué el agente no resuelve la tarea? ¿Y como distinguimos, mirando solo los números, entre un agente que de verdad ha aprendido a jugar y uno que se ha limitado a aprender a sobrevivir?

Cuatro niveles de exigencia (no uno)

Recompensa al alza puede quedarse en el primer escalón. El examen es el cuarto.



Cuatro niveles de competencia fáciles de confundir. Una recompensa al alza puede quedarse en el primer escalón; el examen de verdad es el cuarto: generalizar a niveles no vistos.

Esta es, para quien empieza, la idea más importante de todo el libro. No trata de un algoritmo ni de un hiperparametro: trata de no engañarse a uno mismo. La trampa es sutil porque el número que sube es real; lo que falla es lo que creemos que ese número significa. Antes de poner una sola fórmula, conviene tener una escena en la cabeza.

TRADUCCIÓN AL JUEGO

Imagina dos partidas. En la primera, la pala persigue la bola con soltura y la devuelve una y otra vez; la bola viaja por el centro, rebota en las paredes, vuelve, y el muro de ladrillos apenas se toca. En la segunda, la pala coloca la bola con ángulo para que suba a una esquina, abre un hueco, mete la bola por el pasillo y va vaciando la rejilla fila a fila. Las dos pueden durar lo mismo y las dos pueden sumar recompensa parecida. Solo una esta *resolviendo* el nivel. Distinguir esas dos partidas mirando una única cifra es justo lo que vamos a aprender a no hacer mal.

Cuatro niveles de exigencia, no uno

El error de partida es pensar que "jugar bien a Arkanoid" es una sola cosa que se tiene o no se tiene. En realidad son cuatro escalones de dificultad creciente, y cada uno exige bastante más que el anterior.

Conviene tenerlos nombrados, porque casi todas las discusiones sobre si un agente "funciona" se aclaran en cuanto preguntas: ¿de cual de los cuatro escalones estamos hablando?

- 1 · Sobrevivir** Devolver la bola con la pala el mayor tiempo posible. No exige puntería: exige no perder. Un agente que solo aprende esto mantiene la bola en juego, pero podría no estar destruyendo nada a proposito.
- 2 · Romper algunos ladrillos** Hacer caer una parte de la rejilla. Ya implica cierta intención: colocar la pala para que la bola viaje hacia donde hay ladrillos. Es el primer escalon donde la *puntería* empieza a contar.
- 3 · Limpiar el nivel entero** Destruir los 80 ladrillos de la rejilla 8x10. Exige planificación sostenida: rematar las esquinas, no dejar ladrillos sueltos inaccesibles y mantener el control de la bola hasta el final. Es el objetivo real de la tarea.
- 4 · Generalizar a niveles no vistos** Limpiar disposiciones de ladrillos que el agente **nunca** ha visto durante el entrenamiento. No vale memorizar un nivel concreto: hay que haber aprendido una **política** $\pi(a|s)$ que funcione sobre patrones nuevos. Es la única forma de saber que ha aprendido a jugar y no a recitar.

Los escalones son acumulativos en dificultad, pero no necesariamente en lo que el agente ha entendido. Y aquí esta el nudo del capítulo: **se puede subir el primer escalon sin tocar los demás**, y aun así cosechar recompensa. Sobrevivir paga. Si la recompensa premia, aunque sea de forma indirecta, el seguir vivo, un agente puede volverse muy bueno en no perder la bola sin haber aprendido jamás a apuntar.

TRADUCCIÓN AL JUEGO

En nuestra recompensa actual hay un $+0.2$ por cada rebote con la pala. Es un termino pensado para dar señal temprana: al principio el agente no rompe nada y, sin ese pequeño premio por rebotar, no tendría ninguna pista de que tocar la bola es bueno. Pero fíjate en la tensión que crea. Romper un ladrillo da $+1.0$ y rebotar da $+0.2$. Si una partida dura cientos de pasos y el agente rebota la bola decenas de veces, la suma de muchos $+0.2$ puede competir con la de unos pocos $+1.0$. Para un agente que mide el mundo en recompensa acumulada, "rebotar mucho y romper poco" puede ser una estrategia perfectamente rentable. La recompensa sube; el nivel no se limpia. Más adelante veremos que incluso un termino bienintencionado de *shaping* (el potencial Φ) llego a sabotear el objetivo y mejoramos quitandolo; aquí basta con quedarse con la idea: la señal que premias no es lo mismo que la meta que quieres.

El caso real: el agente ciego que "tenía un 56% de éxito"

Esto no es un peligro teorico. Nos paso. El primer agente de este proyecto, el de la **era ciega**, es el ejemplo perfecto de métrica que enganaba, y por eso lo contamos sin maquillarlo.

Aquel agente jugaba en una versión reducida del problema: una rejilla de $4 \times 7 = 28$ ladrillos en lugar de la actual de 80, una observación de solo **6 números cinemáticos** (posición de la bola, su velocidad y la posición de la pala) y un **timeout constante de 600 pasos** por partida. Lo decisivo es lo que *no* tenía en la observación: no veía los ladrillos. Cero información sobre que celdas seguian llenas y cuales ya estaban vacias. Para el, el campo de juego era una bola y una pala moviendose; el muro de ladrillos era invisible.

Cuando lo evaluamos, los números parecian contar dos historias contradictorias a la vez.

QUE MIRAR

En la tabla siguiente, no mires solo la columna de éxito. Mira la **diferencia** entre tipos de nivel. Un agente que de verdad supiera jugar tendría un rendimiento parecido en niveles dispersos y en niveles llenos: la habilidad de apuntar no debería depender de cuantos ladrillos haya. Si el número se dispara solo en un tipo concreto de nivel, eso es una pista de que no es habilidad lo que estamos midiendo, sino otra cosa.

Situación evaluada	Ladrillos rotos / resultado	Lectura honesta
Niveles dispersos (pocos ladrillos, separados)	0% de éxito	Cuando hay que apuntar, no acierta nada.
Rendimiento típico de destrucción	~2 de 28 ladrillos	Rompe lo justo por casualidad geométrica.
Rejilla completamente llena	~56% de "éxito"	El número alto que enganaba.

Visto de golpe, el contraste es brutal: el mismo agente pasa de un **0%** en niveles dispersos a un **56%** en la rejilla llena. Si solo hubieramos publicado el 56%, habríamos anunciado un agente que "resuelve más de la mitad de las partidas". Sería cierto en la letra y completamente falso en el espíritu. Y conviene ser preciso con las palabras: ese 56% es porcentaje de partidas consideradas éxito en un único escenario, el de rejilla llena; no es la media sobre los tipos de nivel ni, mucho menos, prueba de puntería.

Por qué el 56% era un espejismo: supervivencia degenerada

La explicación está en la física del juego, no en la inteligencia del agente. Llamamos a esto **supervivencia degenerada**: el éxito aparece como efecto secundario de seguir vivo, no de tener un plan.

ANTES DE LA FÓRMULA

La idea es contar oportunidades de choque. Si una bola rebota durante muchísimos pasos dentro de un campo **completamente lleno** de ladrillos, cada tramo de su trayectoria tiene casi garantizado cruzarse con un ladrillo: no hay huecos por los que colarse sin tocar nada. El número de impactos no depende entonces de hacia donde apunte el agente, sino simplemente de cuanto tiempo aguante la bola en juego y de lo denso que este el muro. Cuanto más llena la rejilla y más larga la partida, más ladrillos caen por pura acumulación de rebotes. La fórmula de abajo no es una ecuación exacta de física, sino una regla de proporcionalidad para entender de donde sale el número.

$$\text{ladrillos rotos} \approx (\text{pasos que sobrevive la bola}) \times (\text{densidad de ladrillos en el campo})$$

En la rejilla llena, el agente ciego conseguía que la bola rebotara del orden de **2200 pasos**. Una bola viva durante 2200 pasos en un muro sin huecos acaba tocando casi todo, lo apunte quien lo apunte. El agente no estaba resolviendo el nivel: estaba sobreviviendo mucho rato en un escenario donde sobrevivir basta para que la física limpie el muro por ti. Por eso el número se hundía a **0%** en cuanto los ladrillos estaban dispersos: ahí ya no valía rebotar sin más, había que dirigir la bola hacia ladrillos concretos separados por huecos, y eso es exactamente lo que el agente no sabía hacer (ni podía: no los veía).

PARA CURIOSOS

Fija las dos puntas de la regla de proporcionalidad para ver por que el termino "densidad" lo decide todo. En la rejilla llena, la densidad esta cerca de 1 (casi no hay huecos), así que el primer factor, los pasos que sobrevive la bola, domina y muchos rebotes se traducen en muchos impactos. En un nivel disperso, la densidad es muy baja: la mayor parte de cada trayectoria atraviesa hueco, y por muchos pasos que aguante la bola, el producto se queda pequeño salvo que *dirijas* la bola hacia los pocos ladrillos que hay. Ese "salvo que dirijas" es precisamente la punteria, el factor que la fórmula no captura y que el agente ciego no tenía. La regla explica el 56% y el 0% con la misma idea: cuando la densidad es alta, sobrevivir disfraza la falta de punteria; cuando es baja, la deja al descubierto.

ERROR COMÚN

El error es leer "**recompensa al alza**" o "**56% de éxito**" como "**el agente ha aprendido a jugar**". Son cosas distintas. La recompensa mide lo que *premiarnos*; el éxito en limpiar niveles mide lo que *queremos*. Cuando esos dos no coinciden, el agente, que solo optimiza lo que premiamos, encontrara la vía barata: sobrevivir, rebotar, acumular pequeños premios. Quien empieza y confunde "mi curva de recompensa sube" con "mi agente funciona" puede pasarse días celebrando un progreso que no existe en la tarea que de verdad importa. La curva subía de verdad. La conclusión era falsa.

I Recompensa, éxito y por que medimos en niveles no vistos

De aquí salen dos distinciones que usaremos en todo el libro y que conviene fijar ya.

La primera: **recompensa no es éxito**. La **recompensa** es la señal interna *r* que el agente intenta maximizar, paso a paso; el **éxito** es una métrica externa que define el evaluador (aquí: ¿se ha limpiado el nivel, si o no?). Un agente puede acumular recompensa sin tener éxito, como acabamos de ver. Por eso nuestra **métrica de cabecera no es la recompensa media, sino el porcentaje de niveles limpiados**. Y por eso miramos siempre la recompensa con desconfianza: es útil como señal de entrenamiento, pero traidora como medida de logro.

La segunda: el cuarto escalon, generalizar, es el único que de verdad descarta el autoengano. Un agente puede aprenderse de memoria un punado de niveles concretos y "limpiarlos" sin haber entendido nada transferible. La única forma de saber si ha aprendido a *jugar* es enseñarle niveles que no ha visto nunca. Por eso, más adelante, todos nuestros resultados de cabecera se miden sobre conjuntos de **test** con disposiciones de ladrillos **nuevas**, separadas de las de entrenamiento. Es el equivalente a no dejar que el alumno vea el examen antes de hacerlo. Y conviene no mezclar dos cosas que parecen iguales: el conjunto de **validación** sirve para ajustar y elegir durante el desarrollo, mientras que el de **test** se reserva para la cifra final; además ese test se desglosa en familias distintas (patrones del mismo tipo ya visto, patrones no vistos y densidades no vistas), de modo que un buen número global no esconda un fallo en lo verdaderamente nuevo.

QUE NO DEMUESTRA

Que el agente ciego fracasara **no** demuestra que su algoritmo fuera malo, ni que hiciera falta entrenar más tiempo, ni que se necesitara un método más sofisticado. Ese agente estaba resolviendo, lo mejor que podía, el problema que le habíamos planteado: sobrevivir sin ver los ladrillos. El fallo no estaba en como aprendía, sino en como habíamos **formulado la tarea**. Esa es la tesis que recorre el libro entero, y este capítulo es su primera prueba: el progreso no llegó cambiando de algoritmo ni entrenando más, sino planteando bien la pregunta (reloj justo, meta limpia, observación con ladrillos, escala de señal adecuada, encoder adecuado y evaluación honesta sobre niveles no vistos). Aquí solo lo enunciamos; los capítulos siguientes lo demuestran ingrediente a ingrediente.

Mini-ejemplo: leer una partida sin dejarse engañar

Imagina dos agentes evaluados sobre el mismo lote de niveles. Te dan solo una cifra de cada uno y tienes que decidir cual ha aprendido a jugar.

Agente A	Recompensa media por partida: alta. Pero casi toda procede de rebotes (+0.2) y de partidas largas en niveles densos. En niveles dispersos: 0% limpiados.
Agente B	Recompensa media por partida: más baja que la de A. Pero procede sobre todo de ladrillos rotos (+1.0), combos (el bonus +0.5 por ladrillo extra encadenado sin tocar la pala) y limpiezas de nivel (+5.0), y su porcentaje de niveles limpiados es parecido en dispersos y en llenos.

La intuición ingenua elige a A: tiene más recompensa. La lectura correcta elige a B. La recompensa de A esta *inflada* por la vía barata de sobrevivir y rebotar; la de B, aunque menor, esta hecha de la moneda que de verdad cuenta —romper, encadenar y limpiar— y además no se desploma cuando cambia el tipo de nivel. La lección practica: nunca compares agentes por una sola cifra de recompensa. Desglosa de donde viene esa recompensa y comprueba si el éxito aguanta al cambiar de escenario.

Quédate con esto

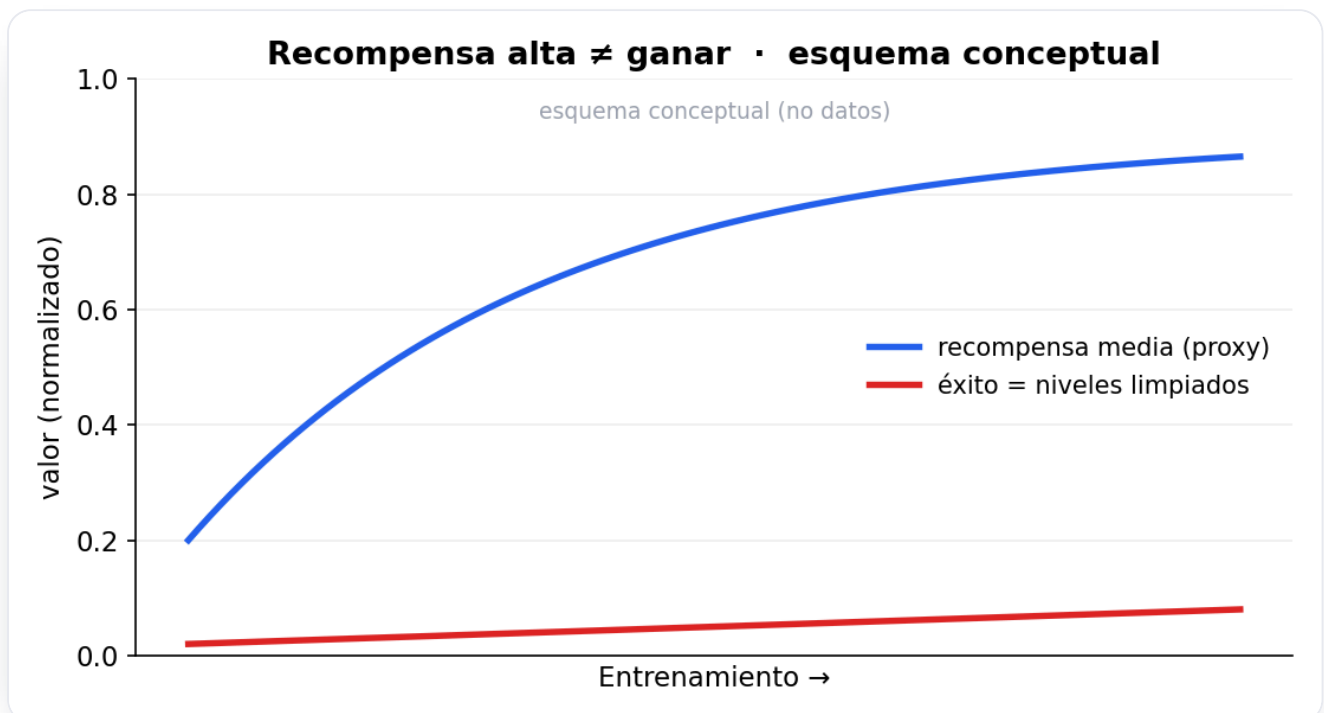
Sobrevivir no es resolver. Hay cuatro escalones, no uno: sobrevivir, romper algunos, limpiar el nivel y generalizar a niveles nuevos. Una recompensa al alza puede venir de subir solo el primer escalon. El agente ciego lo demostro de la peor manera honesta posible: **0%** en niveles dispersos, ~2 de 28 ladrillos rotos, y un **56%** de "éxito" que era solo física (una bola rebotando ~2200 pasos en una rejilla llena), no puntería. Por eso medimos el **porcentaje de niveles limpiados**, y lo medimos sobre niveles **no vistos**: es la única forma de no confundir aprendizaje aparente con logro real.

AUTOCOMPROBACIÓN

1. El agente ciego marcaba 56% en rejilla llena y 0% en niveles dispersos. ¿Qué propiedad del campo de juego (presente en la regla de proporcionalidad) explica esa diferencia, y por qué ninguna de las dos cifras mide puntería de verdad?
2. Un compañero te enseña una curva de recompensa que sube de forma sostenida y concluye que su agente "ya resuelve la tarea". ¿Qué dos preguntas le harías antes de creerle?
3. ¿Por qué el cuarto escalon (generalizar a niveles no vistos, medido sobre el conjunto de test) es el único que descarta de verdad que el agente solo este memorizando, y no aprendiendo a jugar?

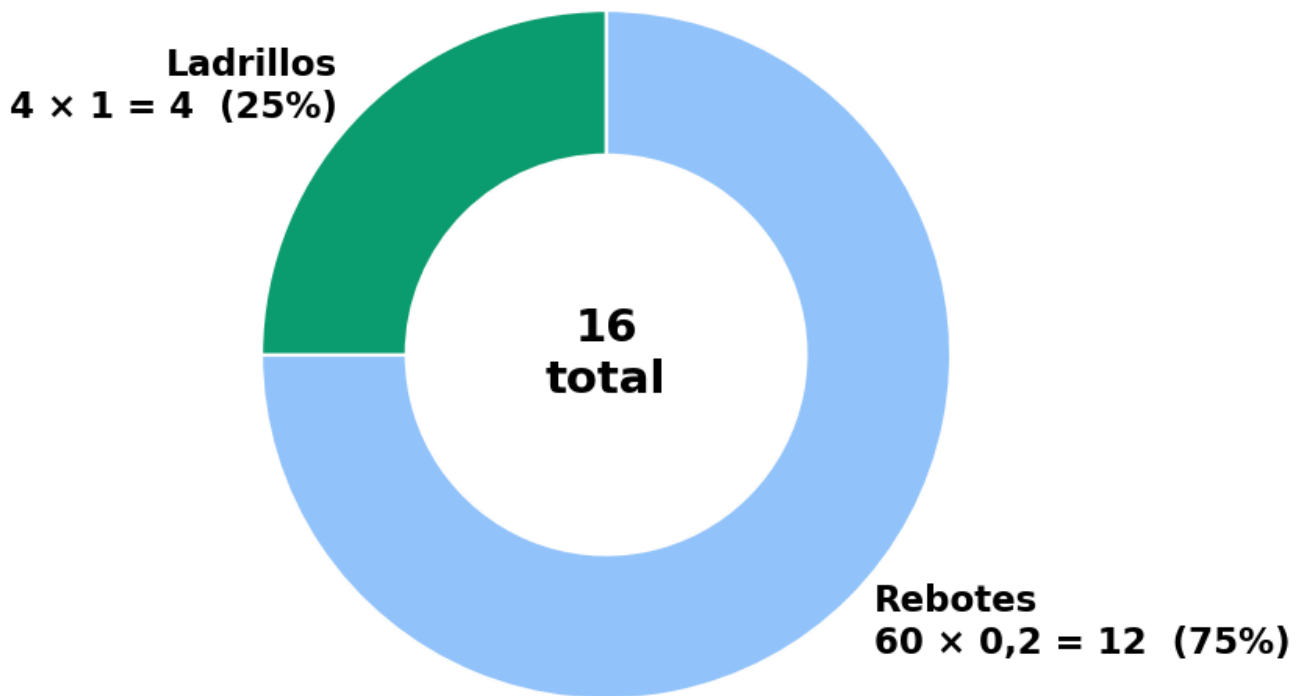
EJERCICIO

Un agente sobrevive de media unos 2000 pasos por partida. En la recompensa, cada rebote con la pala vale $+0.2$ y cada ladrillo roto vale $+1.0$. Supon que en una partida larga rebota la bola 60 veces y rompe 4 ladrillos. ¿Qué fracción de su recompensa total (ignorando los demás terminos, como el -1.0 por perder o el $+5.0$ por limpiar) procede de *sobrevivir/rebotar* y cual de *romper*? **Pista:** calcula los dos sumandos por separado, 60×0.2 frente a 4×1.0 , y divide cada uno entre la suma. **Respuesta:** 12.0 de rebotes frente a 4.0 de ladrillos; sobre un total de 16.0, eso es el 75% de la recompensa procedente de rebotar y solo el 25% de romper. *Que significa para el agente:* si optimiza la recompensa total, le sale a cuenta volverse experto en rebotar antes que en apuntar; la señal le esta diciendo, sin querer, que sobrevivir importa más que resolver. Ese desajuste entre lo que premiamos y lo que queremos es justo lo que hay que vigilar.



Esquema conceptual del mensaje central: la recompensa media (proxy) puede subir mientras el éxito real —limpiar niveles— apenas se mueve. Por eso medimos el logro, no el proxy.

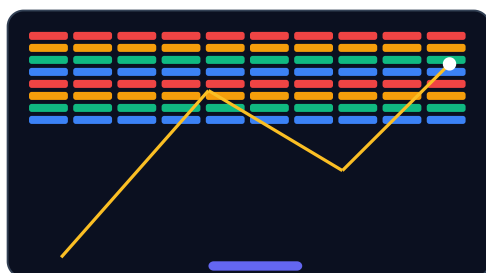
Una recompensa «razonable» puede premiar sobrevivir (ejemplo)



Por qué una recompensa «razonable» puede premiar sobrevivir: 60 rebotes a +0,2 pesan 12 puntos frente a los 4 de romper ladrillos. El agente aprende a no perder, no a ganar.

Por qué el 56% era un espejismo

Con el muro lleno, la bola rompe ladrillos por física aunque el agente no apunte.



MURO LLENO → toca ladrillos casi siempre



NIVEL DISPERSO → atraviesa huecos si no apunta

La clave del espejismo del 56%: con el muro lleno la bola rompe ladrillos por física aunque el agente no apunte; en un nivel disperso, atraviesa los huecos si no apunta.



Y AHORA, PARA EXPERTOS NIVEL EXPERTO

«Sobrevivir no es resolver» es, en términos formales, el problema de las **métricas proxy** y la **ley de Goodhart**: «cuando una medida se convierte en objetivo, deja de ser una buena medida». Premiar la supervivencia (recompensa por no perder la bola) define un proxy cuya optimización no implica resolver la tarea real, que es limpiar el nivel; el agente puede maximizar el proxy quedándose en bucles seguros que devuelven la bola eternamente sin romper ladrillos.

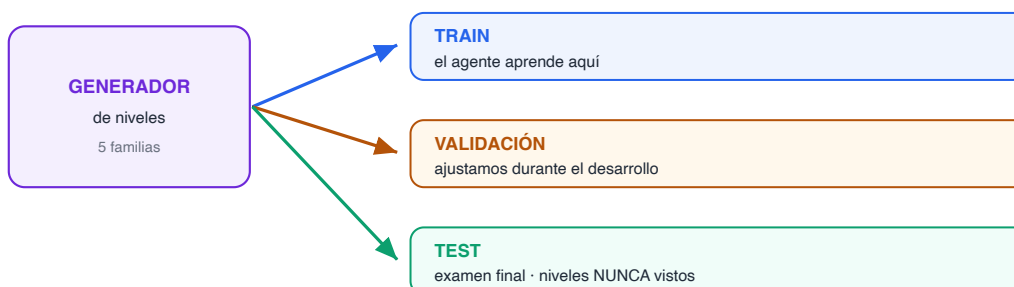
Esto es una forma de *reward hacking*: la política explota la letra de la recompensa, no su espíritu. La consecuencia metodológica es separar la señal de aprendizaje de la métrica de logro: la recompensa puede ser un proxy denso y útil para guiar el gradiente, pero la métrica con la que **juzgamos** al agente debe ser el **éxito** (*success rate*, fracción de niveles completados), no la recompensa acumulada. Optimizar una cosa y evaluar por otra es justo lo que evita engañarnos.

El examen de los niveles no vistos

Train, validación y test disjuntos.

Un agente termina su entrenamiento rompiendo el 95% de los ladrillos en un nivel concreto. Lo ponemos en ese mismo nivel y, en efecto, lo limpia casi siempre. ¿Hemos demostrado que el agente *sabe jugar a Arkanoid*, o solo que se ha aprendido de memoria ese nivel en particular? ¿Y cómo distinguimos una cosa de la otra?

Conjuntos disjuntos: estudiar y examinarse con cosas distintas



El generador reparte los niveles en tres bolsas disjuntas: se aprende en TRAIN, se decide en VALIDACIÓN y se declara el resultado solo en TEST (niveles nunca vistos).

Esta pregunta no es un tecnicismo: es el filo entre un resultado real y un autoengaño. Un agente que memoriza una solución no ha aprendido nada transferible; ha grabado una secuencia de movimientos que solo vale para el examen que ya conoce. Para no caer en esa trampa necesitamos una regla de oro que recorre todo el libro: **nunca se evalúa en lo que se ha entrenado**. Este capítulo fija esa regla, define la métrica principal del proyecto y monta el aparato de evaluación —tres tipos de examen y un protocolo de juego— sobre el que descansan todos los números que verás más adelante. Conviene insistir en algo: sin este aparato, ninguna de las tablas de los capítulos siguientes significaría nada.

La escena: estudiar con unos ejercicios, examinarse con otros

Imagina que preparas un examen de matemáticas con una colección de problemas resueltos. Hay dos formas de prepararte. La primera es entender el método: aprendes a resolver ecuaciones de segundo grado, y entonces da igual qué coeficientes te pongan en el examen. La segunda es memorizar: te aprendes de carrerilla que "el problema 7 da $x=3$ ", sin entender por qué. Si el examen repite el problema 7, las dos estrategias aprueban. Pero si el examen cambia los números —y un buen examen siempre los cambia—, solo sobrevive quien entendió el método.

Un buen profesor lo sabe, y por eso **el examen nunca contiene exactamente los mismos ejercicios que el cuaderno de prácticas**. Esa separación es precisamente lo que vamos a imponer en Arkanoid. El "cuaderno de prácticas" será un conjunto de niveles donde el agente entrena; el "examen" será otro conjunto de niveles, *distintos*, que el agente no ha visto jamás. Si rinde bien en niveles nuevos, ha aprendido algo parecido al método. Si solo rinde bien en los niveles de entrenamiento, ha memorizado.

La analogía termina aquí: en lo que sigue dejamos el cuaderno y volvemos a los pesos de la red, los conjuntos de niveles y la matriz de ocupación.

El concepto técnico: conjuntos disjuntos y generalización

En aprendizaje automático, este principio se formaliza partiendo los datos en conjuntos que no se solapan.

Conjunto de entrenamiento (train)	los niveles que el agente ve durante el aprendizaje. Sobre ellos ajusta sus pesos: aquí ocurre toda la mejora.
Conjunto de validación	niveles que el agente NO usa para aprender, pero que nosotros consultamos durante el desarrollo para tomar decisiones (elegir hiperparámetros, decidir cuándo parar). Es el "examen de prueba" que hacemos antes del examen de verdad.
Conjunto de test	niveles reservados, que no se tocan hasta el final, y que solo sirven para medir el resultado que se reporta. Es el "examen oficial".
Generalización	la capacidad de rendir bien en datos no vistos. Es la propiedad que de verdad nos importa: lo contrario de memorizar.

La palabra clave es **disjuntos**: los tres conjuntos no comparten ningún nivel. Un nivel que está en train no aparece ni en validación ni en test, y al revés. Si se filtra aunque sea un nivel de test al entrenamiento, el examen deja de medir generalización y empieza a premiar la memoria; en la jerga, se dice que hay *fuga* (contaminación) entre conjuntos, y todos los números quedan inflados.

ERROR COMÚN

Creer que validación y test son lo mismo y se pueden usar indistintamente. No lo son. La validación se mira *muchas veces* durante el desarrollo, así que, sin querer, acabamos ajustando nuestras decisiones a ella —se "gasta"—. El test se mira *una vez*, al final, justamente para que conserve su valor como juez imparcial. Reportar como resultado el número de validación, en lugar del de test, es una forma sutil pero real de hacer trampa: el agente no ha visto esos niveles, pero *nosotros* sí, y los hemos usado para afinar el modelo.

Traducción al juego: niveles generados, no dibujados a mano

¿De dónde sacamos suficientes niveles distintos para llenar tres conjuntos sin solaparlos? No los dibujamos uno a uno: usamos un **generador procedural**, un programa que fabrica niveles automáticamente sobre la rejilla de $8 \times 10 = 80$ ladrillos. Cada nivel decide qué celdas tienen ladrillo (un 1 en la matriz de ocupación) y cuáles están vacías (un 0). El generador no produce caos uniforme, sino niveles agrupados en **familias** según su estructura espacial:

dispersión	ladrillos esparcidos, sin patrón geométrico claro.
filas	franjas horizontales de ladrillos.
columnas	franjas verticales.
bloque	un rectángulo macizo de ladrillos en alguna zona.
simétrico	patrones con simetría izquierda-derecha.

Estas familias son el vocabulario estructural del problema. Y aquí entra el matiz importante: los conjuntos train, validación y test se reparten en niveles **disjuntos**, pero podemos elegir *cómo* de distinto es el

examen respecto al cuaderno. Esa elección da lugar a tres tipos de test, que son el corazón de este capítulo.



Las cinco familias de niveles que produce el generador procedural, cada una en su recuadro: dispersión, filas, columnas, bloque y simétrico. Cada cuadradito de color es un ladrillo vivo sobre la rejilla 8×10 ; los huecos son celdas vacías (el 0 de la matriz de ocupación que recibe la red). Lo que interpreta la figura es que cada familia tiene una *firma espacial* distinta: "filas", "columnas" y "bloque" exhiben geometría muy marcada, "dispersión" parece ruido sin estructura y "simétrico" tiene un eje espejo. Generalizar es funcionar en instancias nuevas de estas formas —y, en el caso más exigente, en formas que el agente nunca vio.

Los tres exámenes: TEST-ID, OOD-patrón y OOD-dificultad

No todos los exámenes miden lo mismo. Aprobar uno donde los números cambian poco es fácil; aprobar uno con un tipo de problema que nunca practicaste es harina de otro costal. Definimos tres niveles de exigencia, de menor a mayor distancia respecto al entrenamiento:

- TEST-ID** de *in-distribution*, "dentro de la distribución". Niveles del **mismo tipo de patrón** que se vio en entrenamiento, pero instancias concretas nuevas (otra colocación de los ladrillos dentro de la misma familia). Es el examen con los mismos métodos pero distintos números.
- OOD-patrón** de *out-of-distribution*, "fuera de la distribución". **Patrones no vistos**: familias o estructuras espaciales que no aparecieron en entrenamiento. Es el examen donde te ponen un tipo de problema que no estaba en el cuaderno.
- OOD-dificultad** también fuera de la distribución, pero variando la **densidad**: cantidades de ladrillos (niveles más llenos o más vacíos) que no se entrenaron. Es el mismo tipo de problema, pero mucho más largo o mucho más corto de lo que practicaste.

TRADUCCIÓN AL JUEGO

Supongamos que el agente entrenó con niveles de la familia "filas" de densidad media. Un nivel **TEST-ID** sería otra disposición de filas de densidad parecida: reconocible. Un nivel **OOD-patrón** podría ser un "bloque" macizo o un patrón nunca generado para train: la geometría es ajena. Un nivel **OOD-dificultad** mantendría las filas, pero con muchísimos más ladrillos de los vistos, alargando la partida hacia territorio desconocido. Los tres son disjuntos del entrenamiento; lo que cambia es *cuánto* se alejan.

Esta gradación es valiosa porque convierte "¿generaliza?" en una pregunta con resolución fina. Un agente puede aprobar TEST-ID y suspender OOD: eso nos dice que aprendió el patrón concreto pero no la *idea* de despejar ladrillos. Veremos exactamente ese efecto en los resultados: por ejemplo, en la hoja

de resultados a 1,5 millones de pasos, el mejor agente (PPO) saca 91 en TEST-ID, 89 en OOD-patrón y 86 en OOD-dificultad —una caída suave, señal de que generaliza bien—, mientras que otros agentes se desploman mucho más al salir de lo conocido. La distancia entre las tres columnas es, en sí misma, una medida de cuánto entendió el agente frente a cuánto memorizó.

La métrica principal: éxito, no recompensa

¿Qué número leemos en cada examen? Aquí conviene una distinción que el libro mantendrá siempre: **recompensa no es lo mismo que éxito**. La recompensa es la señal interna que guía el aprendizaje (sumar puntos por romper ladrillos, restar por perder la bola); es lo que el agente intenta maximizar paso a paso. Pero un agente podría acumular recompensa decente sin *limpiar* ningún nivel —rebotando mucho, rascando ladrillos sueltos—. Por eso la **métrica principal** de evaluación es la **tasa de éxito**: el porcentaje de niveles que el agente consigue limpiar por completo. Es un criterio binario y honesto por nivel —se limpió o no se limpió— promediado sobre el conjunto de test.

ANTES DE LA FÓRMULA

La tasa de éxito no pondera cuántos ladrillos rompiste: cuenta cuántos niveles cerraste del todo. Si N es el número de niveles del examen y limpio_i vale 1 cuando el nivel i quedó completamente despejado y 0 en otro caso, la métrica es simplemente la fracción de unos.

$$\text{éxito} = (1/N) \cdot \text{Sigma}_i \text{limpio}_i$$

Romper 79 de 80 ladrillos y perder la bola cuenta, para esta métrica, exactamente igual que romper cero: el nivel no se limpió. Puede parecer severo, pero es justo lo que queremos medir —terminar la tarea, no quedarse cerca— y es mucho más difícil de falsear que una curva de puntos que sube.

ERROR COMÚN

Mirar la curva de recompensa, verla subir y concluir "ya aprende". La recompensa puede subir por motivos degenerados. En la era antigua del proyecto ocurrió justo esto: con una rejilla llena, una bola rebotando durante miles de pasos tocaba casi todos los ladrillos por pura física, no por puntería, y arrojaba un ~56% de "éxito" engañoso —y solo en la rejilla llena; en niveles dispersos el resultado era del 0%—. La supervivencia no es destreza. La tasa de limpieza sobre niveles variados y no vistos es mucho más difícil de falsear.

El protocolo de juego: evaluación GREEDY

Falta decidir *cómo* juega el agente durante el examen. Durante el entrenamiento, los agentes exploran: prueban acciones subóptimas a propósito para descubrir el entorno (en DQN, por ejemplo, con una probabilidad ϵ de actuar al azar; en los métodos de política, muestreando de una distribución de probabilidad sobre las acciones). Esa aleatoriedad es buena para aprender, pero contamina la medición: no queremos puntuar la suerte de un dado, sino la política aprendida.

Por eso la evaluación es **greedy** (codiciosa, determinista): en cada estado el agente elige **siempre la mejor acción según lo aprendido**, sin explorar.

ANTES DE LA FÓRMULA

La acción greedy es, simplemente, "de las tres acciones posibles, quédate con la que tu red considera mejor". En los métodos basados en valor (DQN, World Model) eso significa la acción con mayor valor estimado $Q(s, a)$. En los métodos con política explícita (PPO, SAC) significa la acción más probable según el actor. No hay azar: mismo estado, misma decisión.

$$a^* = \operatorname{argmax}_a Q(s, a)$$

Jugar greedy hace la evaluación **reproducible**: si congelamos el agente y los niveles de test, el resultado no baila de una ejecución a otra por ruido de exploración. Y mide lo que de verdad importa —la calidad de la política aprendida— en lugar de premiar tropiezos afortunados.

PARA CURIOSOS

Una sola partida greedy no elimina toda la aleatoriedad del problema: el nivel inicial, los rebotes y el punto de partida de la bola introducen varianza propia del entorno. Por eso el éxito se promedia sobre *muchos* niveles del conjunto de test y, además, sobre **5 semillas** de entrenamiento independientes (semillas 0 a 4). Una semilla es una ejecución completa del entrenamiento con una inicialización aleatoria distinta; promediar cinco evita que un golpe de suerte —o de mala suerte— en un único entrenamiento se cuele como conclusión. En este libro, cuando leas un porcentaje de éxito, salvo que se diga lo contrario, es la *media de 5 semillas*, no el mejor caso.

El oráculo de limpiabilidad: que el examen sea justo

Hay un peligro escondido en generar niveles automáticamente: el generador podría fabricar un nivel **imposible** de limpiar —una configuración donde, por geometría de los rebotes, algún ladrillo nunca puede ser alcanzado—. Si colásemos esos niveles en el test, estaríamos penalizando al agente por no lograr lo imposible, y la tasa de éxito quedaría artificialmente hundida. Es como meter en un examen un problema sin solución: cualquier nota que saque el alumno deja de ser informativa.

Para evitarlo, el proyecto usa un **oráculo de limpiabilidad**: un verificador que, antes de admitir un nivel, comprueba que *existe* alguna forma de limpiarlo, y **descarta los niveles imposibles**. Así, cuando el agente falla, es por su política, no porque la tarea no tuviera salida. El oráculo no le dice al agente cómo jugar —no es una ayuda durante la partida—; solo garantiza que el examen es resoluble. Mantener el examen justo es condición previa para que la métrica signifique algo.

Mini-ejemplo: leer una fila de resultados

Juntemos las piezas leyendo una línea real de la tabla principal (media de 5 semillas, evaluación greedy, presupuesto de 1,5 millones de pasos):

QUE MIRAR

En la tabla siguiente, las tres primeras columnas son los tres exámenes. Fíjate en **dos cosas a la vez**: el nivel absoluto (¿qué porcentaje saca?) y, sobre todo, la *pendiente* de izquierda a derecha (¿cuánto cae al alejarse de lo entrenado?). Una caída pequeña indica que el agente capturó la idea general; una caída grande, que dependía mucho del patrón concreto. La columna de colapsos avisa si alguna semilla se hundió por debajo del 10% de éxito —el umbral que en este libro define un "colapso".

Agente	TEST-ID	OOD-patrón	OOD-dificultad	% colapsos
PPO	91	89	86	0%
DQN	77	74	64	0%
World Model (Dyna-Q)	55	53	42	0%

Lectura: PPO no solo saca la nota más alta, sino que apenas pierde 5 puntos al pasar del examen conocido (TEST-ID, 91) al más alejado (OOD-dificultad, 86): generaliza con holgura. DQN parte más abajo (77) y cae más (hasta 64): aprendió, pero con menos margen fuera de lo familiar. El World Model se queda en un techo bajo (55) y también se resiente en OOD-dificultad (42). Ninguno de estos tres sufre colapsos (0%), es decir, ninguna de sus cinco semillas se hundió por debajo del 10%; eso habla de estabilidad, una propiedad distinta de la nota media. Toda esta lectura es posible *solo porque* el examen está separado del entrenamiento; si hubiéramos evaluado en los niveles de train, las tres filas marcarían casi 100 y no distinguiríamos al que entiende del que memoriza.

QUE NO DEMUESTRA

Que un agente saque buena nota en estos tres exámenes **no demuestra** que jugaría bien en un Arkanoid arbitrario fuera de este generador procedural, ni con otras físicas, ni en la rejilla pequeña de la era antigua. "Fuera de distribución" aquí significa fuera de *esta* distribución concreta de patrones y densidades; no es generalización universal. Tampoco demuestra comprensión en sentido humano: el agente puede haber aprendido regularidades estadísticas muy eficaces sin "entender" el juego. La métrica acota lo que sabemos —rinde en niveles nuevos de estas familias—, ni más ni menos.

Quédate con esto

La métrica principal es la **tasa de éxito** (niveles limpiados del todo, no recompensa acumulada), medida con juego **greedy**, promediada sobre niveles de test y **5 semillas**. Los niveles se generan automáticamente, se agrupan en familias y se reparten en conjuntos **disjuntos** train / validación / test, con un **oráculo** que descarta niveles imposibles para que el examen sea justo. Hay tres exámenes de exigencia creciente —**TEST-ID** (mismo patrón), **OOD-patrón** (patrones nuevos) y **OOD-dificultad** (densidades nuevas)— y la *caída* de nota entre ellos es la medida real de generalización. Entrenar y examinar en el mismo nivel sería memorizar: la trampa que todo este aparato existe para impedir.

AUTOCOMPROBACIÓN

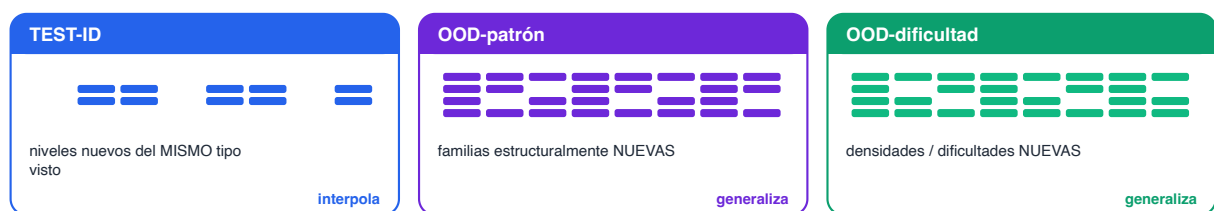
1. ¿Por qué reportar el resultado del conjunto de validación, en vez del de test, se considera una forma de trampa, si en ambos casos son niveles no entrenados?
2. Un agente saca 90 en TEST-ID y 45 en OOD-patrón. ¿Qué te dice esa diferencia sobre lo que aprendió, y por qué no bastaría con mirar solo el 90?
3. ¿Qué problema resolvería el oráculo de limpiabilidad y por qué, sin él, una tasa de éxito baja podría no ser culpa del agente?

EJERCICIO

Diseña, sobre el papel, el reparto de las cinco familias {dispersión, filas, columnas, bloque, simétrico} en train, validación y test de modo que el conjunto de test pueda medir **OOD-patrón** de verdad. **Pista:** para que un examen sea OOD-patrón, el agente no debe haber visto esa familia durante el entrenamiento; piensa qué familia(s) reservar por completo para test. **Respuesta:** basta con dejar fuera del train al menos una familia entera —por ejemplo, entrenar con {dispersión, filas, columnas, simétrico} y reservar "bloque" únicamente para test; las instancias nuevas de las familias entrenadas darían TEST-ID, y las de "bloque" darían OOD-patrón. (Las variantes de densidad no vista sobre familias conocidas formarían, aparte, el OOD-dificultad.) Para el agente, esto significa que el examen distingue entre *reconocer una forma practicada* y *enfrentarse a una geometría nueva*: solo lo segundo prueba que aprendió a despejar ladrillos en general, y no a resolver los patrones que ya tenía memorizados.

Los tres exámenes

No es lo mismo interpolar (variar lo conocido) que generalizar (lo nuevo de verdad).



Tres niveles de exigencia del examen: TEST-ID (mismo tipo, interpolar), OOD-patrón y OOD-dificultad (familias y densidades nuevas, generalizar de verdad).

Y AHORA, PARA EXPERTOS **NIVEL EXPERTO**

«El examen de los niveles no vistos» es el protocolo estándar de **conjuntos disjuntos** de entrenamiento, validación y test: el agente aprende sobre *train*, ajustamos hiperparámetros mirando *validación* y solo medimos el resultado final sobre un *test* que el modelo nunca tocó. La diferencia entre rendimiento en entrenamiento y en test es la **brecha de generalización** (*generalization gap* = rendimiento en train – rendimiento en test); una brecha grande delata **sobreajuste** al conjunto concreto de niveles, es decir, memorización en lugar de comprensión.

Los niveles de test son, por construcción, **fuera de la distribución** de entrenamiento (*OOD, out-of-distribution*): introducen *distribution shift* respecto a lo visto. Por eso el éxito en TEST es la magnitud honesta —mide si la política capturó la estructura subyacente del juego y la transfiere a configuraciones nuevas— mientras que un éxito alto solo en entrenamiento es compatible con haber memorizado trayectorias y no demuestra nada sobre generalización.

El primer intento: el agente ciego

Devolvía la bola, pero no apuntaba.

Si un agente devuelve la bola una y otra vez, rebote tras rebote, y va sumando recompensa partida tras partida, ¿esta de verdad *aprendiendo a jugar*? ¿O solo ha aprendido a sobrevivir, sin tener ni idea de a que ladrillo apunta?

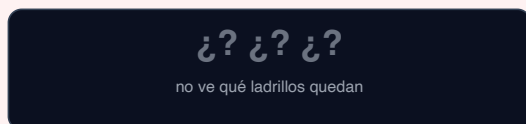
La palabra clave es «observación»

El mismo tablero, dos observaciones. Sin ladrillos en la entrada, apuntar es imposible.

AGENTE CIEGO

observación = 6 números

bola x,y · velocidad · pala x · distancia



→ rebota, pero no puede ELEGIR objetivo

AGENTE CON OJOS

observación = 6 números + matriz 8x10



→ sabe qué queda y puede APUNTAR

La misma escena, dos observaciones. Sin la matriz de ladrillos en la entrada, el agente puede rebotar pero no elegir objetivo: el techo es de información, no de entrenamiento.

Antes de presentar el agente que funciona, conviene contar el que **no** funciona. No por nostalgia, sino porque el primer intento fallido es la prueba más limpia de la tesis de este libro: el problema no se desbloquea cambiando de algoritmo ni entrenando más horas, sino **formulando bien la tarea**. Y para entender por qué importa la formulación, hay que ver de cerca a un agente que recibió una formulación pobre. Lo llamaremos el **agente ciego**: jugaba bien con la pala y, aun así, era incapaz de decidir.

De Breakout a Arkanoid: una idea de cuarenta años

El juego nace en **1976**. Atari lanza *Breakout*: una pala abajo, una bola, un muro de ladrillos arriba. Devuelves la bola con la pala, la bola sube, golpea un ladrillo, lo rompe y baja de nuevo. Si la bola pasa por debajo de la pala, pierdes. La leyenda cuenta que el prototipo lo idearon Nolan Bushnell y Steve Bristow, y que la parte de ingeniería la firmó un joven Steve Wozniak (con Steve Jobs de por medio). La mecánica es mínima y por eso es tan buena: cabe en una frase.

En **1986**, la japonesa Taito publica *Arkanoid* y le anade lo que lo convierte en un clásico: ladrillos de distintos colores y resistencias, capsulas que caen y modifican la pala (más larga, laser, bola pegajosa, vidas extra), varias bolas a la vez, jefes finales. La esencia de Breakout sigue intacta —rebotar para romper—, pero el espacio de decisiones se ensancha. En este libro nos quedamos con el **núcleo común** de ambos: pala, bola, muro de ladrillos. Es lo justo para estudiar el problema de decisión sin que la fontanería del juego nos distraiga.

TRADUCCIÓN AL JUEGO

Nuestra versión: una rejilla de ladrillos arriba, una bola, una pala que solo se mueve en horizontal. El agente elige en cada paso una de **tres acciones**: mover la pala a la izquierda, mantenerla quieta o moverla a la derecha. Todo lo demás —la física de la bola, los rebotes, la detección de ladrillo roto— lo resuelve el entorno. El agente solo decide hacia donde va la pala.

El hito de DeepMind: aprender mirando píxeles

Conviene situar el problema en su contexto histórico, porque ayuda a calibrar lo difícil y lo fácil. En **2015**, DeepMind publica en *Nature* el trabajo del **DQN** (Deep Q-Network): un mismo agente, sin reglas específicas de cada juego, aprende a jugar **49 juegos de Atari** —Breakout entre ellos— mirando directamente **los píxeles de la pantalla**. No recibe la posición de la bola ni la de la pala; recibe la imagen en crudo, como la vería una persona, y una red **convolucional** se encarga de extraer de esos píxeles lo que importa.

QUE NO DEMUESTRA

Ese resultado es un hito, pero tuvo un precio enorme en datos: del orden de **50 millones de fotogramas por juego**, lo que equivale a unos **38 días** de partidas jugadas sin parar. Esa cifra es de DeepMind, **no es un resultado nuestro**: la traemos solo como vara de medir. Lo que demuestra es que *se puede* aprender desde píxeles; lo que **no** demuestra es que sea barato, ni que la forma en que tu le presentas la tarea al agente sea irrelevante. Precisamente de eso va el resto del capítulo.

El contraste con DeepMind nos da el marco. Ellos le dieron al agente **mucha** información (la pantalla entera) y **muchísimos** datos. Nosotros, en el primer intento, hicimos lo contrario: le dimos al agente **poquísima** información. Y ahí empezó el problema.

La primera modelización: una observación mínima

El primer diseño fue deliberadamente austero. Queríamos lo mínimo viable, así que al agente le dimos como observación solo **seis números cinemáticos**: la posición de la bola (x , y), su velocidad (v_x , v_y) y la posición horizontal de la pala. Números sobre *como se mueve* la escena.

ERROR COMÚN

Pensar que «si el agente sabe donde esta la bola, ya sabe lo que tiene que hacer». Saber donde esta la bola basta para **devolverla**, que es un problema de seguimiento. Pero no basta para **elegir a que ladrillo apuntar**, que es un problema de decisión con objetivos. Son dos capacidades distintas, y la observación mínima solo cubre la primera.

La clave esta en lo que **faltaba**: en esos seis números **no aparecen los ladrillos**. Ni cuantos quedan, ni donde, ni cuales ya estan rotos. El agente no tenía *ningún* dato sobre el muro. Por eso lo llamamos ciego: ve la bola y la pala con nitidez, pero el objetivo del juego —el muro que hay que demoler— es para el una pared invisible.

Y había dos decisiones más que, sumadas a la ceguera, completan el cuadro del primer intento:

Rejilla 4x7	El muro tenía 28 ladrillos (4 filas por 7 columnas), bastante más pequeño que el de la versión actual.
Observación de 6 cinemáticos	Solo los seis números de movimiento. Sin matriz de ladrillos.
Timeout constante de 600 pasos	Cada partida se cortaba siempre a los 600 pasos , tuviera el muro 28 ladrillos o 3. Un reloj fijo, igual para niveles llenos que para niveles casi vacíos.

Una partida del ciego, paso a paso

Reconstruyamos una partida típica para separar dos cosas que es fácil confundir: que el agente **se mueva bien** y que el agente **decida bien**.

TRADUCCIÓN AL JUEGO

1. La bola cae hacia la pala. El agente lee x , y , v_x , v_y y la posición de la pala, calcula hacia donde va a llegar la bola y mueve la pala para interceptarla.
2. Rebote correcto: la bola sube. El agente **cobra** la recompensa del rebote con la pala (+0.2). Esto se le da bien: es justo lo que sus seis números le permiten resolver.
3. La bola sube y golpea *algún* ladrillo. Cae uno: recompensa +1.0 por ladrillo roto. Pero ojo: el agente **no eligió** ese ladrillo. La bola subió, tocó lo que había en su trayectoria y lo rompió. El mérito es de la física, no de la puntería.
4. La bola vuelve a bajar. El agente la vuelve a interceptar. Y otra vez. Y otra.
5. El bucle se repite hasta que la bola se cuele por debajo (recompensa -1.0, perder la bola) o hasta que el reloj llega a 600 pasos y la partida se corta.

El patrón es nitido: el ciego **mantiene la bola en juego** con soltura, y por eso va acumulando recompensa. Pero en ningún momento *apunta*. No puede. Para apuntar a un ladrillo concreto hace falta saber donde están los ladrillos, y esa información no está en su observación. Devuelve la bola de maravilla; el muro, simplemente, no existe para él.

LO QUE VE EL HUMANO VS. LO QUE VE EL AGENTE

Aspecto	Un humano jugando	El agente ciego
La bola	La ve y la sigue	La «ve»: x , y , v_x , v_y
La pala	La ve y la mueve	La «ve»: posición horizontal
El muro de ladrillos	Lo ve entero: cuáles quedan y donde	No lo ve. Cero números sobre ladrillos
«Voy a tirar por el hueco de la izquierda»	Planifica el ángulo a propósito	Imposible: no sabe que hay un hueco

Esta tabla es el corazón del capítulo. El humano y el agente comparten la mitad de la información (bola y pala), pero el humano ve **además** el objetivo. Pedirle al ciego que «limpie el nivel» es como pedirle a alguien que apague velas concretas de una tarta con los ojos vendados: puede soplar, e incluso apagar

alguna por suerte, pero no puede elegir *cual*. En terminos del juego: puede rebotar, e incluso tumbar algún ladrillo por azar, pero no puede dirigir la bola contra un ladrillo elegido.

Los resultados del ciego, sin maquillaje

Los números confirman la historia, y conviene leerlos con cuidado porque uno de ellos es una trampa.

QUE MIRAR

Fijate en como el mismo agente da resultados opuestos según el tipo de nivel. No mires el número alto en aislado: comparalo con los bajos y pregunta *por que* el muro lleno se le da «bien».

Niveles dispersos: 0%

Cuando los ladrillos estan repartidos con huecos, el ciego limpia el nivel el **0%** de las veces. No acierta nunca, porque limpiar exige apuntar y el no puede apuntar.

Unos 2 de 28 ladrillos

De media rompe alrededor de **2 ladrillos de 28**. Los pocos que caen son los que la bola toca de pasada, por trayectoria, no por decisión.

Rejilla llena: ~56% de «éxito»

Solo cuando el muro esta **completamente lleno**, el ciego «tiene éxito» en torno al **56%** de las partidas. Este es el número tramposo.

ANTES DE LA FÓRMULA

Para ver lo lejos que esta el ciego de «limpiar», basta con poner en proporción los ladrillos que tumba frente a los que hay. La fracción de muro demolido en niveles dispersos es, simplemente, los ladrillos rotos entre los ladrillos totales.

$$\text{ladrillos rotos} / \text{ladrillos totales} \approx 2 / 28 \approx 7\%$$

Es decir: en niveles dispersos el ciego deja en pie alrededor del **93%** del muro. Rompe los ladrillos que caen solos por la trayectoria de la bola, y nada más. Ese ~7% demolido, además, casi nunca alcanza para limpiar el nivel completo: de ahí el **0%** de éxito. Romper dos ladrillos por física y limpiar el muro entero son cosas muy distintas.

QUE NO DEMUESTRA

Ese 56% en muro lleno **no demuestra que el agente sepa jugar**. Es lo que llamamos **supervivencia degenerada**. Si el muro esta lleno y la bola rebota durante muchisimos pasos (del orden de **2.200**), acaba tocando casi todos los ladrillos *por pura física*: con tantos rebotes y un muro sin huecos, es difícil **no** darle a algo. El agente no apunta a nada; simplemente mantiene la bola viva el tiempo suficiente para que el azar haga el trabajo. Por eso el número se desploma a 0% en cuanto aparecen huecos: ahí ya no vale rebotar a ciegas, hay que elegir, y elegir es justo lo que no sabe hacer.

Hay que distinguir también dos palabras que en este 56% se mezclan de forma enganosa. **Recompensa** es lo que el agente acumula a lo largo de la partida (los +0.2 de cada rebote con la pala, los +1.0 de los ladrillos que caen por azar); el ciego cobra recompensa sin problema, porque rebotar mucho es precisamente sumar muchos +0.2. **Éxito** debería ser limpiar el nivel, cumplir el objetivo. El muro lleno permite acumular recompensa *sin* que haya verdadera puntería, y por eso un «éxito» del 56% ahí no significa lo que parece. Mucha recompensa no es lo mismo que jugar bien.

ERROR COMÚN

Leer «56% de éxito en muro lleno» y concluir que el agente «casi lo tiene, solo le falta un empujon». Es justo al revés: el 0% en niveles dispersos es el dato honesto, y el 56% es el espejismo. El agente no está a medio camino de aprender a jugar; está resolviendo otro problema (sobrevivir) que solo *parece* el bueno cuando el nivel es trivialmente denso.

Por qué el reloj fijo agravaba la trampa

El **timeout constante** de 600 pasos echaba lina al fuego. Un reloj igual para todos los niveles trata por igual dos situaciones muy distintas: un muro de 28 ladrillos y un muro casi limpio. En el muro lleno, mantener la bola viva hasta esos ~2.200 pasos de rebotes da tiempo de sobra para que la física vaya tumbando ladrillos; el reloj fijo, sin querer, premia la supervivencia. La lección la recogeremos más adelante: el reloj tiene que ser **proporcional** al tamaño del muro, para que el tiempo concedido mida la tarea y no regale «éxitos» por aguantar.

PARA CURIOSOS

El fenómeno tiene nombre fuera de Arkanoid: *reward hacking* o explotación de la recompensa. El agente optimiza con honestidad **la señal que le diste**, no **la intención que tenías**. Si la señal premia acumular (rebotes a +0.2 más ladrillos por azar en muro denso, con tiempo de sobra), el agente aprende a acumular, no a limpiar. Aquí no fue malicia del algoritmo: fue una formulación que dejaba un atajo abierto. Tapar atajos —reloj justo, observación completa, meta limpia— es buena parte del trabajo de modelizar una tarea de aprendizaje por refuerzo.

El diagnóstico: no era el algoritmo, era la formulación

Aquí está el giro que organiza todo el libro. Ante un agente que no aprende, la reacción instintiva es cambiar de algoritmo o entrenar más tiempo. Pero el ciego no fallaba por usar un algoritmo flojo ni por falta de pasos: fallaba porque la **tarea estaba mal planteada**. Le pedíamos que limpiara el muro y a la vez le ocultábamos el muro. Ningún algoritmo, por potente que sea, puede aprender a apuntar a algo que no aparece en su observación.

De aquí salen las dos preguntas que guiarán los siguientes capítulos y la solución que ya se intuye:

- | | |
|--------------------------------------|--|
| ¿Qué tiene que ver el agente? | Los ladrillos. La observación debe incluir el estado del muro, no solo la cinemática de la bola. (Adelanto: en la versión actual se añade una matriz de ocupación 8x10 — 80 celdas, un 1 por ladrillo vivo y un 0 por hueco— junto a los seis cinemáticos.) |
| ¿Cuánto tiempo es justo? | El que corresponda al tamaño del muro. (Adelanto: el reloj pasa a ser proporcional , no constante, para que no premie sobrevivir.) |

La diferencia entre «devolver la bola» y «limpiar el muro» no es de potencia de cálculo: es de información y de objetivo. El ciego nos lo dejó clarísimo precisamente por fracasar.

Quédate con esto

El primer agente veía la bola y la pala, pero **no** los ladrillos (6 cinemáticos, rejilla 4x7 de 28 ladrillos, timeout fijo de 600). Resultado: **0%** en niveles dispersos y ~2 de 28 ladrillos rotos (apenas un ~7% del muro), con un enganoso **~56%** en muro lleno que era pura *supervivencia degenerada* —la bola rebotando ~2.200 pasos toca casi todo por física, no por puntería. Sabia **moverse**, no **decidir**. El problema no era el algoritmo ni el tiempo de entrenamiento: era una **formulación** que ocultaba el objetivo y dejaba un atajo abierto. Arreglar eso —dar a ver los ladrillos y un reloj justo— es lo que desbloquea el aprendizaje.

AUTOCOMPROBACIÓN

1. ¿Por qué el agente ciego es capaz de devolver la bola pero incapaz de elegir a que ladrillo apuntar? ¿Qué información concreta le falta en sus seis números?
2. El ciego «tiene éxito» el ~56% de las veces en muro lleno y el 0% en niveles dispersos. ¿Cuál de los dos números refleja mejor si sabe jugar, y por qué el otro es enganoso?
3. Distingue **recompensa de éxito** en este caso. ¿Cómo consigue el ciego acumular recompensa sin estar realmente limpiando el nivel?

EJERCICIO

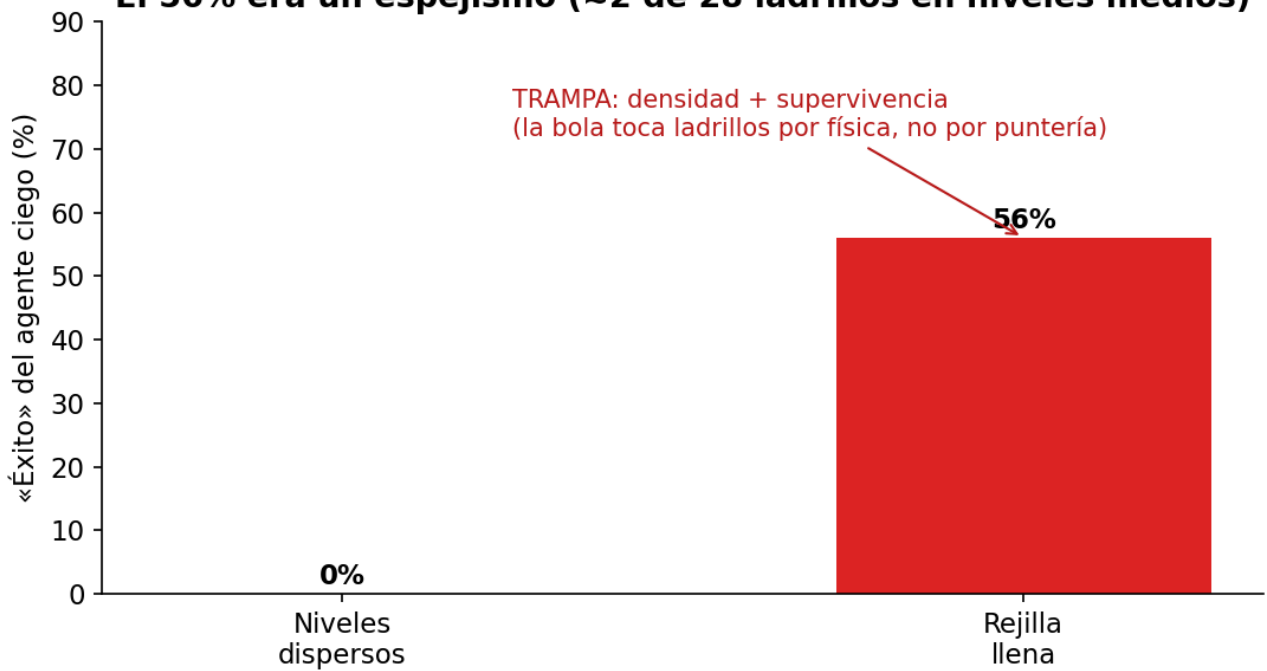
Imagina dos niveles con el mismo número de ladrillos: uno con el muro lleno en un bloque compacto y otro con los ladrillos dispersos y muchos huecos. Con un timeout **constante** de 600 pasos, ¿en cual de los dos esperarías que el agente ciego «tenga éxito» más a menudo y por qué?

Pista: piensa en cuantos ladrillos toca la bola por simple rebote en cada caso, sin que el agente apunte.

Respuesta: en el muro **lleno y compacto**. Sin huecos, casi cualquier trayectoria de la bola golpea un ladrillo, así que rebotar mucho tiempo (con el reloj fijo dando margen) tumba ladrillos por física. En el muro disperso, la mayoría de las trayectorias atraviesan huecos sin tocar nada, y como el agente no puede apuntar a los ladrillos que quedan, no los limpia.

Que significa para el agente: su «éxito» en muro lleno no mide puntería, mide supervivencia. Solo cuando le demos a ver los ladrillos y le pongamos un reloj proporcional dejara de depender del azar y empezara a **decidir**.

El 56% era un espejismo (≈ 2 de 28 ladrillos en niveles medios)



Los resultados del agente ciego: 0% en niveles dispersos y un 56% engañoso solo en la rejilla llena, donde la bola toca ladrillos por física. Un número alto que no medía puntería.



Y AHORA, PARA EXPERTOS NIVEL EXPERTO

«El agente ciego» es, técnicamente, un **POMDP** (proceso de decisión de Markov parcialmente observable): la observación que recibe la política no contiene el estado completo del entorno. Cuando la matriz de ladrillos no figura en la observación, ninguna política $\pi(a|o)$ definida sobre esa observación incompleta puede inferir lo que no se le da; el límite no es de cómputo sino de **información**.

Esta distinción es clave: más entrenamiento, más capas o más pasos no crean información que estaba ausente en la entrada. El techo lo fija el contenido de la observación, no el optimizador. La cura no es entrenar más, sino rediseñar la observación para que el estado relevante (la disposición de los ladrillos, la velocidad de la bola) sea observable; solo entonces el problema vuelve a ser resoluble.

Diagnostico: tres muros, ninguno el algoritmo

Reloj, recompensa y observación.

Si el agente no aprende a limpiar el nivel, la sospecha inmediata es «el algoritmo es malo, probemos otro». Pero, ¿y si el algoritmo no tiene la culpa? ¿Y si lo que falla es la forma en que le hemos planteado el problema: el tiempo que le damos, lo que premiamos y lo que le dejamos ver?

Diagnóstico: tres muros, ninguno el algoritmo

Cada muro: síntoma → hipótesis falsa → prueba → causa real → corrección.

MURO 1 · El reloj

Síntoma: nunca limpia.

Hipótesis falsa: «falta entrenar».

Prueba: 600 pasos < 7.200 que pide un 8x10.

Causa: timeout fijo demasiado corto.

Arreglo: timeout proporcional (90/ladrillo).

MURO 2 · La recompensa

Síntoma: sobrevive, no gana.

Hipótesis falsa: «mal algoritmo».

Prueba: quitar el shaping Φ sube +8.

Causa: Φ premiaba acercarse, no ganar.

Arreglo: meta limpia, sin shaping Φ .

MURO 3 · La observación

Síntoma: no apunta.

Hipótesis falsa: «más pasos».

Prueba: sin matriz de ladrillos, techo.

Causa: la observación no ve los ladrillos.

Arreglo: matriz de ocupación 8x10.

El diagnóstico de un vistazo: tres muros (reloj, recompensa, observación), cada uno con su síntoma, la hipótesis falsa, la prueba que lo destapa, la causa real y el arreglo.

En el capítulo anterior cerramos con una tesis provisional que ahora toca corregir. La intuición natural —y casi siempre equivocada— es tratar el algoritmo de aprendizaje por refuerzo como el sospechoso principal. Cuando el agente fracasa, la tentación es cambiar de DQN a PPO, de PPO a SAC, subir el número de pasos de entrenamiento o tocar la tasa de aprendizaje. Este capítulo es el diagnóstico clínico de por qué esa estrategia, en nuestro caso, no funcionaba: ninguno de los tres problemas de fondo era el algoritmo. Eran tres muros en la *formulación* de la tarea, y un buen algoritmo entrenado contra una tarea mal formulada sigue fracasando.

Conviene fijar una imagen antes de entrar en tecnicismos. Piensa en un alumno brillante al que suspendes en un examen: puede que el alumno sea malo, pero también puede que el examen estuviera cronometrado a la mitad de tiempo, que la pregunta premiara copiar al de al lado en vez de razonar, o que le tapasen medio enunciado. Tres causas que no son el alumno. En refuerzo pasa lo mismo, y la disciplina para distinguirlas es mirar siempre los mismos campos. Para cada muro seguimos esta plantilla clínica: **síntoma observado** (qué vimos en las métricas), **hipótesis falsa** (qué pensamos al principio), **prueba** (el experimento que distingue una explicación de otra), **causa real** (qué estaba pasando de verdad) y **corrección** (el cambio en la formulación que destrabó el muro).

ERROR COMÚN

Pensar que «no aprende» es siempre un problema de aprendizaje. En refuerzo, «no aprende» puede significar cuatro cosas muy distintas: no le da *tiempo* a aprender, le enseñamos a optimizar la *recompensa equivocada*, no *ve* lo que necesita para decidir, o el algoritmo en sí es inadecuado. Los tres primeros son fallos de formulación, no de algoritmo. Confundirlos lleva a «probar otro algoritmo» durante semanas sin tocar la causa real.

I Muro 1 — El reloj imposible

El primer muro es el más fácil de pasar por alto porque no parece un parámetro «del aprendizaje»: es el **timeout**, el número máximo de pasos que dura un episodio antes de cortarlo. En la era antigua del proyecto este límite era una constante, 600 pasos por episodio, daba igual el nivel.

Síntoma observado. El agente rompía unos pocos ladrillos y el episodio terminaba. Las curvas de recompensa subían un poco al principio y luego se aplanaban en un valor bajo. En la rejilla pequeña de entonces ($4 \times 7 = 28$ ladrillos) se rompían del orden de 2 ladrillos de media; en niveles dispersos el éxito era directamente del 0%. Daba la impresión de un agente que «se rinde» o que no es capaz de planificar.

ERROR COMÚN

La hipótesis falsa fue: «el agente no aprende a apuntar, necesita más capacidad de planificación o más pasos de entrenamiento». Es decir, leer un episodio corto y pobre como un problema de *inteligencia* del agente, cuando en realidad era un problema de *presupuesto temporal* del episodio.

La prueba. Aquí conviene hacer una cuenta antes de tocar nada. ¿Cuántos pasos necesita físicamente una bola para tener siquiera la oportunidad de romper 80 ladrillos? La bola tarda un cierto número de pasos en subir, golpear un ladrillo y volver a la pala, y solo rompe (como mucho) un ladrillo por viaje. Si el episodio se corta a los 600 pasos, el agente nunca llega a ver el final de un nivel grande, pase lo que pase con su política. La forma limpia de comprobarlo es desacoplar el reloj del tamaño del nivel: en lugar de un timeout fijo, dar un **timeout proporcional** al número de ladrillos.

ANTES DE LA FÓRMULA

La idea es sencilla: el tiempo que damos por episodio debe escalar con el trabajo que pedimos. Si cada ladrillo necesita del orden de unas decenas de pasos para ser alcanzado y derribado, el presupuesto total tiene que ser ese coste por ladrillo multiplicado por el número de ladrillos. Fijamos ese coste en 90 pasos por ladrillo.

$$\text{timeout} = 90 \times (\text{número de ladrillos})$$

Para la rejilla $8 \times 10 = 80$ ladrillos eso da $90 \times 80 = 7200$ pasos. Compara las dos cifras: 600 frente a 7200. El reloj antiguo daba menos de una décima parte del tiempo necesario para el nivel grande ($600 / 7200 \approx 0,083$, es decir, un 8% del presupuesto razonable).

TRADUCCIÓN AL JUEGO

Imagina que te dan una partida de Arkanoid pero te apagan la pantalla a los pocos segundos, antes de que la bola haya tenido tiempo de subir y bajar unas cuantas veces. Por muy bien que juegues, no puedes limpiar el nivel: no es que juegues mal, es que la partida se acaba antes de empezar. El timeout constante de 600 era ese apagado prematuro. El timeout proporcional ajusta la duración de la partida al tamaño real del nivel: a más ladrillos, más reloj.

La causa real. El episodio terminaba por *límite de tiempo*, no por incompetencia del agente. Un episodio cortado a destiempo contamina dos cosas a la vez. Primero, el agente casi nunca experimenta el evento de «nivel limpio», así que la señal de la gran recompensa de +5.0 por limpiar apenas aparece en su experiencia y no puede aprender a buscarla. Segundo, la señal de done que recibe en el corte por tiempo es engañosa: el episodio acaba sin que el agente haya hecho nada malo, lo que ensucia la estimación de los valores futuros —el agente puede acabar tratando un final perfectamente bueno como si fuera el final de perder la bola—.

La corrección. Sustituir el reloj fijo por el reloj proporcional. Y este muro no es retórico: lo medimos. En la ablación del capítulo siguiente, partiendo de la receta completa con DQN (que rinde un 77% de éxito), si *volvemos* al timeout constante el éxito cae al 54%, una pérdida de -23,5 puntos. El reloj, por sí solo, vale más de veinte puntos de éxito.

QUÉ NO DEMUESTRA

Que el timeout proporcional valga unos veinte puntos en *esta* tarea no significa que «más tiempo siempre es mejor». Un timeout demasiado largo desperdicia cómputo y permite episodios degenerados de pura supervivencia (lo veremos en el muro 2 y en el capítulo de la era ciega). La lección es de *proporción*: el reloj debe ajustarse al tamaño de la tarea, ni corto ni eterno.

I Muro 2 — La recompensa saboteadora

El segundo muro es el más sutil y el más instructivo, porque aquí el daño no lo hace lo que falta, sino lo que sobra: una recompensa auxiliar bienintencionada que, sin querer, enseñaba al agente a hacer lo que no queríamos.

Antes de la teoría, la intuición. Imagina que quieres que un niño aprenda a marcar goles y, para «ayudarlo», le das un caramelo cada vez que se acerca a la portería. Lo más probable es que acabe paseando junto a la portería comiendo caramelos sin chutar nunca: le has hecho fácil y goloso un comportamiento que no era el objetivo. Eso es, en esencia, lo que nos pasó. Para nombrarlo con precisión hay que introducir el **reward shaping potencial**, que llamaremos **shaping Φ** (Phi). La idea, muy popular en refuerzo, es añadir a la recompensa original una señal extra que «guíe» al agente hacia el objetivo, definida a partir de una función de potencial $\Phi(s)$ sobre los estados. En nuestro caso, esa potencial premiaba *acercar la pala a la bola*: cuanto más alineada estuviera la pala con la bola, mejor.

ANTES DE LA FÓRMULA

El shaping potencial no inventa premios de la nada: añade a cada transición la *diferencia* de potencial entre el estado nuevo y el viejo (con un factor de descuento γ). La gracia teórica es que, si la señal extra tiene exactamente esa forma de diferencia de potencial, la política óptima no cambia (resultado de Ng, Harada y Russell, 1999). La trampa práctica es que esa garantía vive en el límite ideal; con presupuesto finito de pasos y una potencial mal elegida, el agente puede quedarse enganchado a la señal guía antes de descubrir la meta.

$$r' = r + \gamma \cdot \Phi(s') - \Phi(s)$$

Donde r es la recompensa real de la tarea (romper ladrillos, rebotar, perder la bola, limpiar) y el término con Φ es la guía añadida. Sobre el papel, ese término extra debería ser inofensivo. En la práctica fue un sabotaje.

Síntoma observado. El agente aprendía a *seguir la bola con la pala* de forma muy diligente —mantenía la pala bien alineada— pero rompía pocos ladrillos. La recompensa total que reportaba el entorno subía; el éxito real (limpiar el nivel) no acompañaba. Esta es exactamente la señal de alarma que da nombre a una distinción que repetiremos en todo el libro: **recompensa** no es lo mismo que **éxito**. La recompensa es lo que el agente acumula; el éxito es la fracción de niveles que termina limpiando. Pueden ir en direcciones opuestas, y aquí lo hacían.

ERROR COMÚN

La hipótesis falsa fue: «la recompensa sube, luego el agente va por buen camino, solo necesita más entrenamiento para convertir esa habilidad en victorias». Leer una curva de recompensa creciente como progreso hacia el objetivo. El problema es que la recompensa que subía era en buena parte la guía de Φ (estar cerca de la bola), no la del objetivo (romper y limpiar).

La prueba. La forma de distinguir «buen camino» de «sabotaje» es separar las dos métricas y, sobre todo, hacer el experimento decisivo: *quitar* el shaping Φ y volver a entrenar con todo lo demás igual. Si Φ ayuda, quitarlo debería empeorar el éxito; si Φ estorba, quitarlo debería mejorarlo. No hace falta especular: el resultado de la ablación es tajante.

QUÉ MIRAR

En la tabla, la columna clave es el Δ de éxito al quitar el ingrediente. Un Δ negativo significa «el ingrediente ayudaba» (al retirarlo, el éxito baja); un Δ positivo significa «el ingrediente estorbaba» (al retirarlo, el éxito sube). Fija la vista en el signo del shaping Φ comparado con el de los otros dos muros: es el único positivo.

Cambio respecto a la receta completa (DQN, 77%)	Éxito	Δ
Reloj proporcional → constante (Muro 1)	54%	−23,5
Encoder convolucional → lista plana (Muro 3)	57%	−20,4
Quitar el shaping Φ (Muro 2)	85%	+8,0

La lectura es directa: los dos ingredientes que *añaden* formulación correcta (el reloj y la convolución) pierden unos veinte puntos cuando se retiran; el único que *sobra* es el shaping Φ , cuya retirada gana

ocho puntos.

La causa real. El shaping Φ creó un objetivo paralelo. Estar pegado a la bola es fácil y da recompensa continua; romper ladrillos es difícil y da recompensa esporádica. Con un presupuesto finito de pasos, el agente hace lo racional desde su punto de vista: maximiza la señal fácil y abundante. La política que sigue a la bola sin arriesgar para apuntar es un óptimo local muy cómodo. La guía que debía empujar hacia la meta acabó siendo un imán hacia un comportamiento mediocre. Este es el fenómeno clásico del refuerzo: el agente optimiza *exactamente* lo que le pones en la recompensa, no lo que tú querías decir.

La corrección. Quitar el shaping Φ y dejar la recompensa de la tarea limpia: +1.0 por ladrillo (con un bonus de +0.5 por cada ladrillo extra de un combo sin tocar la pala), +0.2 por rebote con la pala, -1.0 por perder la bola y +5.0 por limpiar el nivel. El rebote sigue teniendo un pequeño premio —sobrevivir es necesario— pero ya no hay una señal continua que recompense el mero hecho de acercarse. Resultado medido: el éxito sube de 77% a 85%, una mejora de +8,0 puntos por *quitar* algo. Es el único ingrediente de toda la ablación cuya eliminación mejora el resultado.

QUÉ NO DEMUESTRA

Que *este* shaping Φ sabotearse no significa que «todo reward shaping sea malo». El shaping potencial bien diseñado es una herramienta legítima y, en el límite teórico, no cambia la política óptima. Lo que demuestra el +8,0 es algo más específico: una potencial que premia un proxy fácil (la cercanía) en una tarea con recompensa escasa puede crear un óptimo local que el agente prefiera a la meta real. La meta debe ser limpia.

I Muro 3 — La observación ciega

El tercer muro es el más elemental de enunciar y el más difícil de aceptar: el agente no podía ver lo que necesitaba para decidir. Le pedíamos que apuntase a los ladrillos sin dejarle ver dónde estaban los ladrillos. Es como pedirle a alguien que dé en una diana con los ojos vendados: por mucha puntería que tenga, sin ver el blanco solo le queda la suerte.

Síntoma observado. En la era ciega, la observación eran solo 6 números cinemáticos: posición de la bola, su velocidad y la posición de la pala. Ni un solo dato sobre qué ladrillos seguían vivos. Con esa observación, el agente daba 0% de éxito en niveles dispersos. Pero había un dato desconcertante: en la rejilla *llena* reportaba un «¿éxito?» del 56%. Ese 56% es la trampa de este muro y merece una lectura cuidadosa.

TRADUCCIÓN AL JUEGO

Con la rejilla completamente llena, una bola que simplemente rebote durante muchos pasos acaba tocando casi todos los ladrillos por pura física: estén donde estén, hay un ladrillo en casi cualquier dirección. Combina eso con un episodio largo de pura supervivencia (en la era ciega, una bola podía sobrevivir del orden de un par de miles de pasos rebotando) y obtienes un nivel «limpiado» sin que el agente haya apuntado a nada. No es puntería; es geometría más tiempo. En cuanto el nivel tiene huecos —niveles dispersos— esa estrategia se desmorona y el éxito se va al 0%.

ERROR COMÚN

La hipótesis falsa fue leer ese 56% en la rejilla llena como evidencia de que «el agente sabe jugar y solo necesita generalizar a niveles más difíciles». Era justo lo contrario: ese 56% es **supervivencia degenerada**, un artefacto de evaluar solo sobre el caso más fácil (rejilla llena) donde rebotar mucho equivale a ganar. Promediado con el 0% de los niveles dispersos, el cuadro real es un agente que no apunta.

La prueba. Hay dos pruebas que se refuerzan. La primera es de evaluación: no medir solo en la rejilla llena, sino también en niveles dispersos y con huecos; el contraste 56% (llena) frente a 0% (dispersa) delata que el «éxito» dependía de la geometría, no de la habilidad. La segunda es de información: razonar qué necesita saber una política para apuntar. Para dirigir la bola hacia un ladrillo concreto hay que saber *dónde hay ladrillos*. Si esa información no está en la observación, ninguna red, por grande que sea, puede inferirla: no se puede aprender a leer un mapa que no se reparte.

PARA CURIOSOS

En términos formales, la era ciega convertía el problema en un proceso de decisión *parcialmente observable* (un POMDP): el estado real incluía qué ladrillos seguían vivos, pero la observación no lo exponía. La política óptima sobre esa observación recortada simplemente no puede contener información que la observación no transmite. Añadir la matriz de ladrillos no «ayuda» al agente en abstracto: le hace observable la parte del estado que importa para apuntar, devolviendo el problema a algo más cercano a un MDP.

La causa real. No era un fallo de aprendizaje ni de recompensa: era falta de información en la entrada. El agente no apuntaba porque no tenía delante el mapa al que apuntar.

La corrección. Ampliar la observación con una **matriz de ocupación** de la rejilla: una cuadrícula 8×10 de 80 celdas donde cada celda vale 1 si el ladrillo sigue vivo y 0 si ya es un hueco. La observación pasa de 6 números a 86: los 6 cinemáticos más las 80 celdas del mapa. Con eso el agente por fin ve a qué apuntar.

QUÉ MIRAR

La matriz por sí sola no basta: importa *cómo* se la das a la red. Si aplanas las 80 celdas en una lista y las metes por una capa densa, el agente pierde la estructura espacial (qué celda está al lado de cuál). En la tabla del muro 2 ya aparecía la cifra: cambiar el encoder convolucional por una lista plana hunde el éxito de 77% a 57%, un Δ de $-20,4$. El sesgo espacial de la convolución vale unos veinte puntos.

Por eso la arquitectura final tiene *dos ramas*: la matriz 8×10 entra por un encoder convolucional (filtros de 16 y 32, que respeta la vecindad de las celdas) y los 6 cinemáticos entran por una rama densa; luego se concatenan hacia las capas finales. Ver el mapa y verlo con su geometría son dos correcciones distintas, y las dos suman.

PARA CURIOSOS

Aquí hay una inversión histórica que conviene no olvidar. En la era antigua (rejilla pequeña, lista plana, sin convolución) bajar la escala de la señal de los ladrillos a 0.25 *desbloqueó* el aprendizaje. Con el encoder convolucional, esa *misma* escala 0.25 lo *mata*: el éxito se desploma al 1% con 100% de colapsos. El ingrediente óptimo no es absoluto: depende de la arquitectura con la que conviva. Lo veremos en detalle al hablar de la ablación.

La tabla de diagnóstico clínico

Reunidos los tres muros, el patrón es nítido. En los tres casos había un síntoma, una explicación incorrecta que apuntaba al agente o al algoritmo, y una explicación correcta que apuntaba a la formulación. Esta es la tabla que resume el diagnóstico.

QUÉ MIRAR

Lee la tabla por columnas, no por filas. Fija primero la columna del medio (la explicación incorrecta): en las tres filas señala al agente o al algoritmo. Luego la columna de la derecha (la correcta): en las tres señala la formulación. Esa migración sistemática de la culpa, del algoritmo a la formulación, es la tesis del capítulo.

Síntoma observado	Explicación incorrecta	Explicación correcta
Episodios cortos, pocos ladrillos rotos, curva que se aplana pronto	El agente no sabe planificar; necesita más entrenamiento o más capacidad	El timeout constante de 600 corta el episodio antes de poder limpiar 80 ladrillos (harían falta ~7200)
La recompensa sube pero el agente sigue la bola y rompe pocos ladrillos	Va por buen camino; con más pasos convertirá la recompensa en victorias	El shaping Φ premia el proxy fácil (acercarse) y crea un óptimo local; recompensa $\neq$ éxito
56% de «éxito» en rejilla llena, 0% en niveles dispersos	El agente sabe jugar y solo le falta generalizar a niveles difíciles	Supervivencia degenerada: sin la matriz de ladrillos no puede apuntar; el 56% es geometría, no puntería

Las tres explicaciones incorrectas comparten un sesgo: dan por hecho que la máquina de aprender es el cuello de botella. Las tres correctas dicen lo mismo desde tres ángulos: el cuello de botella estaba en cómo definíamos la tarea —el reloj, la meta y la vista—, no en el motor que la resuelve.

Por qué el algoritmo no puede compensar una mala formulación

Esta es la corrección explícita de la tesis provisional del capítulo anterior. La idea de «si no aprende, prueba otro algoritmo» falla porque cada muro impone un techo que el algoritmo no puede levantar:

Reloj imposible (Muro 1)	Si el episodio se corta antes de que sea físicamente posible limpiar el nivel, ningún algoritmo —ni DQN, ni PPO, ni SAC, ni un modelo del mundo— puede aprender a limpiarlo: nunca experimenta el evento que querría reforzar.
Recompensa saboteadora (Muro 2)	Todos los algoritmos de refuerzo optimizan la recompensa que les das. Si la recompensa premia un proxy equivocado, el algoritmo «bueno» lo optimiza mejor y se aleja más de la meta, no menos.
Observación ciega (Muro 3)	Ninguna red puede usar información que no está en su entrada. Si la observación no contiene los ladrillos, la mejor política posible sobre esa observación sigue siendo ciega.

La evidencia cuantitativa apoya la tesis por los dos lados. Por un lado, una vez bien formulada la tarea, el algoritmo sí importa —hay diferencias claras entre PPO, SAC, DQN y los modelos del mundo—, pero todos los algoritmos razonables aprenden *algo*: incluso el peor de la familia (el modelo del mundo con LSTM) ronda el 35% de éxito en TEST-ID, muy lejos del 0% de la era ciega. Por otro lado, la ablación muestra que romper un solo ingrediente de la formulación hunde al mejor agente: con DQN, quitar el reloj proporcional o el encoder convolucional cuesta del orden de veinte puntos cada uno, y degradar la escala de la señal de los ladrillos a 0.25 lo lleva al 1% con colapso total. La formulación pone el suelo y el techo; el algoritmo decide dónde caes dentro de ellos.

PARA CURIOSOS

Una calibración externa para no perder la perspectiva: el DQN original de DeepMind (Nature, 2015) aprendió 49 juegos de Atari —Breakout entre ellos— mirando los píxeles, con del orden de 50 millones de fotogramas por juego (equivalentes a casi 38 días de partidas continuas) y una red convolucional. Aquel resultado no es nuestra métrica ni es comparable directamente —nosotros no partimos de píxeles ni de ese presupuesto—, pero ilustra una idea: incluso con un algoritmo y un cómputo formidables, el problema solo se vuelve aprendible cuando la entrada (los píxeles, en su caso; la matriz de ladrillos, en el nuestro) contiene de verdad lo que hace falta para decidir.

QUÉ NO DEMUESTRA

Que la formulación domine en *esta* tarea no quiere decir que el algoritmo nunca importe ni que cualquier algoritmo valga. Las cifras de PPO (el más fiable, 91% en TEST-ID y ya competente a 700k pasos) frente a un modelo del mundo con techo ~55% demuestran lo contrario: con la tarea bien planteada, la elección de algoritmo cambia mucho el resultado. La tesis es de *prioridad*, no de irrelevancia: primero fórmula bien, luego elige algoritmo. Cambiar de algoritmo antes de arreglar la formulación es optimizar el motor de un coche al que le faltan las ruedas.

Quédate con esto

Tres muros frenaban al agente y ninguno era el algoritmo: un **reloj imposible** (timeout constante de 600 frente a los ~7200 que pide la rejilla 8×10), una **recompensa saboteadora** (el shaping Φ premiaba acercarse a la bola; quitarlo mejora +8 puntos) y una **observación ciega** (sin la matriz 8×10 de ladrillos no se puede apuntar; el 56% en rejilla llena era supervivencia degenerada). El algoritmo no puede compensar una mala formulación: corrige primero el reloj, la meta y la vista; después, elige algoritmo.

AUTOCOMPROBACIÓN

1. El timeout proporcional usa 90 pasos por ladrillo. ¿Cuántos pasos da en la rejilla 8×10, y cuántas veces más que el reloj constante de 600?
2. En el muro 2, ¿por qué una curva de recompensa creciente podía convivir con un éxito que no mejoraba? ¿Qué término estaba sumando esa recompensa?
3. El 56% de «éxito» de la era ciega aparecía solo en la rejilla llena y se desplomaba al 0% en niveles dispersos. ¿Por qué ese contraste demuestra que no era puntería sino «supervivencia degenerada»?

EJERCICIO

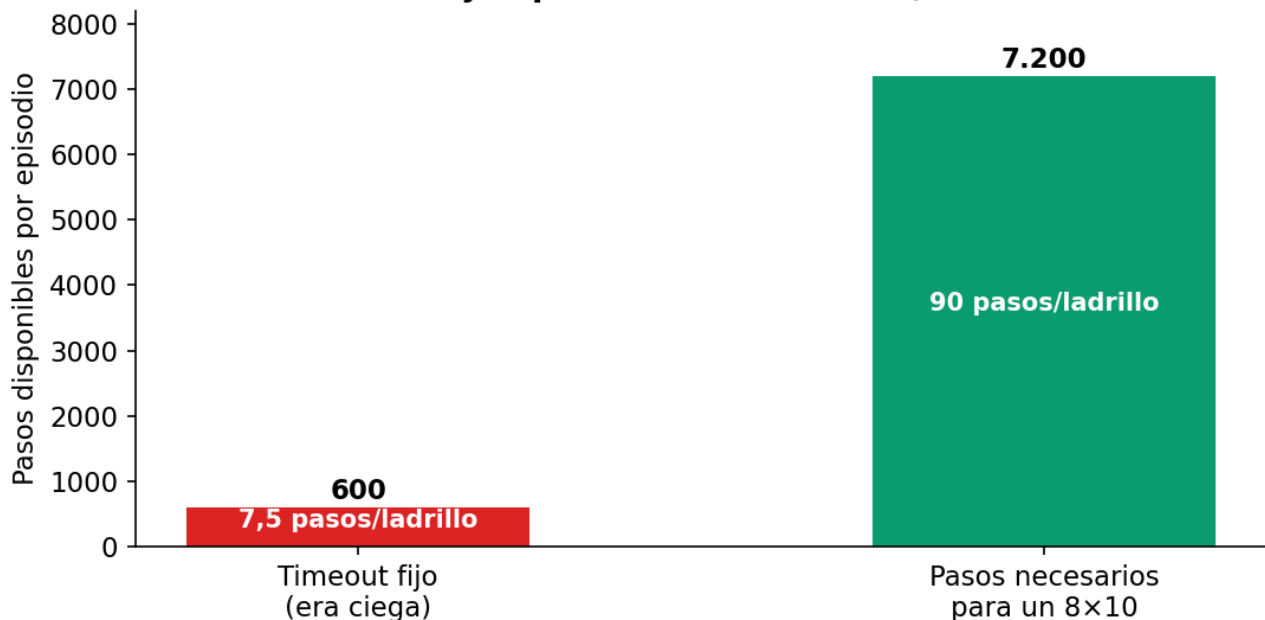
En la ablación, partiendo de DQN con la receta completa (77% de éxito), tres cambios dan estos resultados: volver al timeout constante → 54%; cambiar el encoder convolucional por una lista plana → 57%; quitar el shaping Φ → 85%. Calcula el Δ de cada uno e identifica cuál es el único ingrediente que conviene *quitar*.

Pista: el Δ es el éxito tras el cambio menos el 77% de partida. Un ingrediente «conviene quitarlo» si su eliminación da Δ positivo.

Respuesta: timeout constante $\Delta = -23,5$ (el reloj proporcional ayuda); lista plana $\Delta = -20,4$ (el encoder convolucional ayuda); quitar Φ $\Delta = +8,0$. El único que conviene quitar es el shaping Φ : es el único cuya eliminación mejora el éxito.

Para el agente esto significa que dos de los muros se arreglan *añadiendo* formulación correcta (reloj justo, vista con estructura) y uno se arregla *retirando* una señal que lo desviaba de la meta.

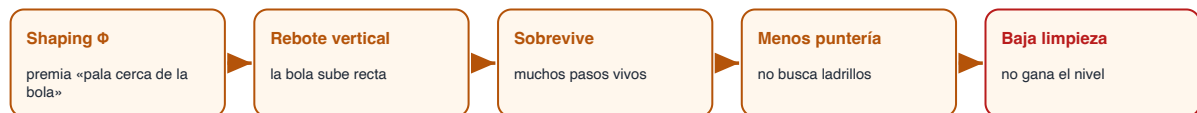
Muro 1 — el reloj imposible: no era difícil, era inalcanzable



Muro 1 cuantificado: con el reloj fijo de 600 pasos, un tablero de 80 ladrillos era inalcanzable ($\approx 7,5$ pasos/ladrillo). El arreglo —timeout proporcional— da 90 pasos por ladrillo.

Muro 2: cómo una «ayuda» saboteara el objetivo

El shaping Φ premiaba acercar la pala a la bola. El agente optimizó eso... no ganar.



Quitar $\Phi \rightarrow +8$ puntos de éxito. A veces, la mejor ayuda es no ayudar: deja que la recompensa real (ganar) guíe.

Muro 2: cómo una ayuda saboteara el objetivo. El shaping Φ premiaba acercar la pala a la bola; el agente aprendió a sobrevivir, no a apuntar. Quitarlo subió el éxito +8 puntos.

Y AHORA, PARA EXPERTOS NIVEL EXPERTO

Los «tres muros» del diagnóstico ilustran que **la formulación domina al algoritmo**: un MDP mal planteado no lo arregla ningún optimizador. El primer muro es la **asignación de crédito** (*credit assignment*): un *timeout* demasiado corto trunca el horizonte efectivo del episodio antes de que llegue la recompensa terminal, de modo que esa señal de logro se vuelve literalmente inalcanzable y el agente nunca aprende a perseguirla.

El segundo es el *reward shaping*. El teorema de **Ng, Harada y Russell (1999)** demuestra que un moldeado basado en potenciales,

$$F(s, s') = \gamma \Phi(s') - \Phi(s)$$

no altera la política óptima, sea cual sea Φ . Pero «no cambiar el óptimo» no significa «no hacer daño»: un Φ mal elegido sesga la dinámica de aprendizaje intermedia, ralentiza o desvía la optimización y puede inducir comportamientos espurios. Por eso, en la práctica, **quitar** un *shaping* mal calibrado mejoró el resultado.

La receta que desbloquea

Cinco cambios de formulación.

Si el algoritmo no era el culpable, ni tampoco la falta de entrenamiento, entonces, ¿qué convirtió exactamente este problema de imposible en aprendible?

La receta que desbloquea (5 ingredientes)

No fue un algoritmo mágico: fue formular bien la tarea. El número = puntos de la ablación.

1 · Reloj justo timeout proporcional -23 si falta	2 · Meta limpia quitar el shaping Φ +8 al quitarlo	3 · Ojos matriz de ocupación 8x10 crítico	4 · Encoder conv respeta la vecindad -20 si falta	5 · Variedad currículo + generador robustez
Resultado: de 0% (ciego) a 91% en niveles no vistos (PPO). Cada pieza ataca un muro distinto.				

Los cinco ingredientes que convierten el problema en aprendible, cada uno atacando un muro distinto. El número es su aporte medido en la ablación; juntos llevan del 0% al 91%.

En los capítulos anteriores vimos la **era ciega**: un agente que miraba solo seis números cinemáticos (posición y velocidad de la bola, posición de la pala), sobre una rejilla pequeña de 4x7, con un reloj de 600 pasos idéntico para todos los niveles. El resultado fue un **0% de éxito** en los niveles dispersos y unos **2 de 28 ladrillos** rotos de media. El único número que parecía bueno —un **56%** de «éxito»— aparecía *solo* en la rejilla completamente llena, y era un espejismo: una bola rebotando durante miles de pasos toca casi todos los ladrillos por pura física, no por puntería. Conviene fijar ya una distinción que recorrerá todo el capítulo: ese 56% es *supervivencia*, no *destreza*; medir bien empieza por no confundir lo uno con lo otro.

La tentación natural es pensar que faltaba potencia: un algoritmo más moderno, una red más grande, más horas de entrenamiento. Este capítulo defiende lo contrario. El salto no vino de cambiar el *motor*, sino de **reformular la tarea**. Vamos a desmontar la receta mínima en sus cinco ingredientes. Para cada uno responderemos a tres preguntas: por qué hacía falta, qué se cambió exactamente y qué habría pasado si no se cambiaba.

TRADUCCIÓN AL JUEGO

Piensa en un alumno al que pones un examen de geometría pero (a) le tapas el dibujo de la figura, (b) le puntuas con una rúbrica que premia cosas que no son la solución, (c) le das el mismo tiempo para un problema de 3 figuras que para uno de 80, y (d) le entregas los datos como una lista de números sueltos en vez de un plano. No es que el alumno sea torpe: es que el examen está mal puesto. La era ciega era ese examen mal puesto. La receta lo arregla, y como pasa con un examen, casi todo el mérito está en cómo se fórmula, no en lo lista que sea la persona que lo resuelve.

Ingrediente 1 — Un reloj justo: el timeout proporcional

Por qué hacía falta. En la era ciega, cada episodio se cortaba a los **600 pasos**, valiera el nivel 3 ladrillos o 80. Ese reloj constante tiene un efecto perverso: en un nivel grande es físicamente imposible limpiarlo en 600 pasos, así que el agente nunca llega a ver la señal de «nivel limpiado». Y si una señal nunca aparece en los datos, el agente no puede aprender a perseguirla: el aprendizaje por refuerzo solo puede optimizar recompensas que de hecho ocurren.

Qué se cambió. Se sustituyó el límite fijo por un **timeout proporcional**: 90 pasos por cada ladrillo. En la rejilla de 80 ladrillos eso da $90 \times 80 = 7200$ pasos. El reloj se adapta al tamaño del problema: un nivel grande recibe más tiempo, igual que un examen largo merece más minutos.

ANTES DE LA FÓRMULA

La regla es deliberadamente simple: no fijamos un número mágico de pasos, sino un *presupuesto por ladrillo*. Multiplicamos ese presupuesto por cuántos ladrillos haya vivos al empezar el nivel. Así el horizonte temporal escala con la dificultad real de cada nivel, en lugar de castigar a los niveles grandes con el mismo reloj que a los pequeños.

$$\text{pasos_máximos} = 90 \times (\text{número de ladrillos del nivel})$$

Un mini-cálculo para verlo. Imagina un nivel de 80 ladrillos. Con el reloj viejo de 600 pasos, el agente dispone de $600 / 80 = 7,5$ pasos por ladrillo: tendría que acertar un ladrillo distinto cada siete pasos y medio, sin fallar ni una vez, para limpiar a tiempo. Eso no es jugar bien, es un imposible físico, porque la bola tarda más que eso solo en ir y volver. Con el reloj proporcional sube a $7200 / 80 = 90$ pasos por ladrillo: ahora sí cabe ir colocando la pala, fallar algunos rebotes y aún así llegar a la señal de «nivel limpiado». La diferencia no es un ajuste fino; es la diferencia entre que la recompensa de +5.0 por limpiar exista o no exista en los datos de entrenamiento.

Qué habría pasado si no se cambiaba. Lo medimos en la ablación del capítulo siguiente. Partiendo de la receta completa con DQN (77% de éxito), si volvemos al timeout *constante* el éxito cae a 54%, una pérdida de **23,5 puntos**. No es el ingrediente más decisivo, pero quitarlo cuesta caro: sin tiempo suficiente, el agente aprende a sobrevivir, no a terminar.

Ingrediente 2 — Una meta limpia: quitar el shaping Phi

Por qué hacía falta. Una técnica clásica para acelerar el aprendizaje es el **reward shaping**: añadir pequeñas recompensas auxiliares que «guíen» al agente hacia el objetivo (por ejemplo, premiar acercar la pala a la bola). La idea es buena en teoría; el problema es que una señal auxiliar mal diseñada puede competir con el objetivo de verdad. El agente optimiza lo que le pagas, y si le pagas por moverse «bien» puede preferir cobrar esa propina antes que arriesgarse a romper ladrillos.

Qué se cambió. Se eliminó el término de shaping (lo llamamos Φ) y se dejó la recompensa *desnuda* con el objetivo del juego: +1.0 por ladrillo roto (con +0.5 de bonus por cada ladrillo extra de un combo sin tocar la pala), +0.2 por rebote con la pala, -1.0 por perder la bola y +5.0 por limpiar el nivel. Ninguna propina por «portarse bien»: solo el resultado.

ERROR COMÚN

Pensar que más señal de recompensa siempre ayuda. Es justo al revés cuando la señal auxiliar apunta en una dirección distinta a la meta. Aquí el shaping no era una ayuda débil: era un **saboteador**. El agente aprendía a maximizar Phi (acumular propina por «moverse bien») en lugar de a limpiar niveles, y como hay una cosa que cobrar a cada paso, prefería esa renta segura al riesgo de ir a por los ladrillos.

Qué habría pasado si no se cambiaba. Este es el resultado más contraintuitivo de toda la receta. En la ablación, quitar el shaping Phi **sube** el éxito de 77% a **85%**: una mejora de **+8,0 puntos**. Es el único «ingrediente» cuya ausencia mejora el sistema. Lo honesto no es esconderlo, sino aprender de él: una meta limpia vale más que una meta «ayudada» en la dirección equivocada.

PARA CURIOSOS

La teoría del *potential-based reward shaping* (Ng, Harada y Russell, 1999) demuestra que un shaping construido como diferencia de un potencial, de la forma $F(s, s') = \gamma \cdot \Phi(s') - \Phi(s)$, no cambia la política óptima. La clave está en ese «bien construido»: si el término no respeta esa forma exacta, o el potencial está mal elegido, las garantías se rompen y la señal puede desviar al agente. Nuestro hallazgo empírico es coherente con esa letra pequeña: un shaping que no cumple el teorema no es inofensivo «por si acaso».

Ingrediente 3 — Ojos: la matriz de ocupación 8x10

Por qué hacía falta. Este es el cambio de fondo. En la era ciega, el agente veía solo seis números cinemáticos. **No veía los ladrillos.** No sabía cuáles seguían vivos, cuáles ya había roto ni dónde estaban los huecos. Pedirle que limpie un nivel sin ver el nivel es como pedirle a alguien que recoja una habitación con los ojos vendados: puede tropezarse con cosas por azar, pero no puede planear.

Qué se cambió. Se añadió al estado una **matriz de ocupación** de $8 \times 10 = 80$ celdas, una por ladrillo. Cada celda vale 1 si el ladrillo sigue vivo y 0 si ya es un hueco. El estado pasó de 6 a **86 números**: los 6 cinemáticos de siempre más las 80 celdas de la matriz.

Cinemáticos (6) Bola x, bola y, velocidad vx, velocidad vy, pala x y distancia bola–pala. Dicen *dónde está* la acción ahora mismo.

Matriz 8x10 (80) El mapa de qué queda por romper. Dice *cuál es el trabajo pendiente* y su forma.

Las dos formas de dar ojos. Aquí hubo una decisión de diseño con dos caminos posibles, y conviene hacerla explícita:

QUE MIRAR

En la tabla siguiente, fija la atención en la columna «coste»: las dos opciones resuelven la ceguera, pero pagan precios muy distintos en datos y cómputo. La pregunta no es cuál ve mejor, sino cuál es coherente con lo que este libro quiere estudiar.

Opción	Qué ve el agente	Coste	Decisión
A: píxeles (estilo DeepMind)	La pantalla en bruto; la red infiere dónde hay ladrillos a partir de los píxeles	Muy alto: el DQN de DeepMind sobre Atari necesitó del orden de 50 millones de fotogramas por juego (~38 días de partidas continuas) y una red convolucional	Descartada aquí
B: matriz estructurada	Directamente qué celda está viva o muerta: la información útil, ya destilada	Bajo: 80 números que el simulador ya conoce, sin pasar por píxeles	Elegida

Lectura. DeepMind demostró en 2015 (Nature) que un DQN podía aprender 49 juegos de Atari — Breakout incluido— *mirando los píxeles*, con una red convolucional. Es un hito enorme, pero tiene un precio: del orden de 50 millones de fotogramas por juego. Ese es un dato externo, de contexto; **no es nuestro resultado**. Nosotros no queríamos demostrar que se puede aprender desde píxeles —ya está demostrado—, sino estudiar cómo se *fórmula* bien la tarea. Por eso elegimos la opción B: si nuestro simulador ya sabe qué ladrillos quedan vivos, dar esa información directamente como matriz nos ahorra que la red tenga que reaprender a «ver» desde cero a partir de manchas de color. La matriz 8x10 es la información útil ya destilada.

ERROR COMÚN

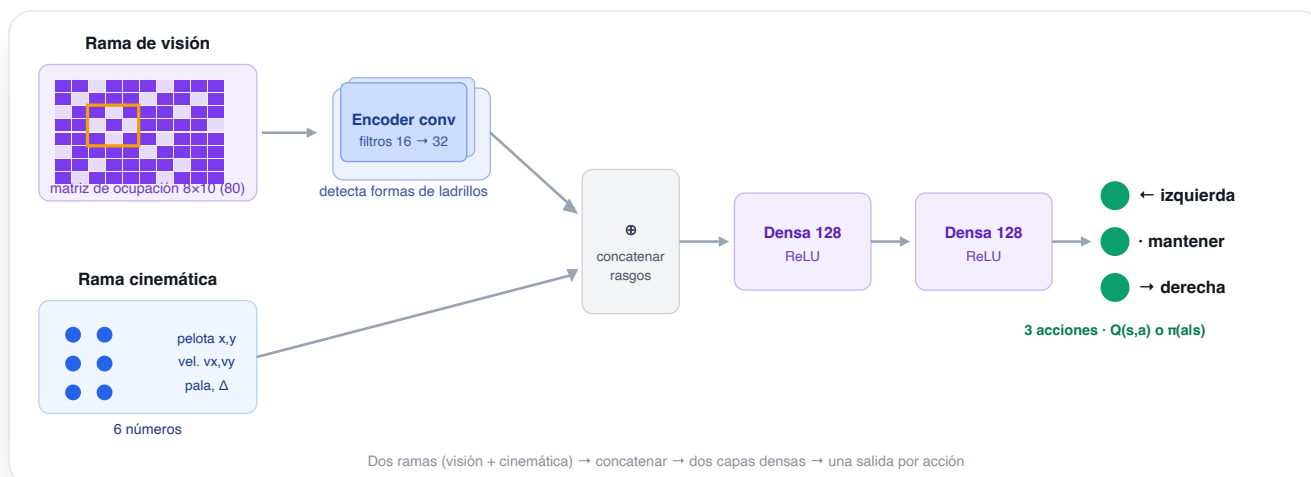
Crear que la opción B es «hacer trampa» frente a los píxeles. No lo es: es una decisión de *representación del estado* perfectamente legítima. La matriz no le dice al agente qué hacer ni hacia dónde mover la pala; solo le dice qué queda por romper. Sigue siendo él quien tiene que aprender la política. Lo que evitamos es quemar millones de fotogramas en el sub-problema de la visión, que no es el objeto de estudio de este libro.

Qué habría pasado si no se cambiaba. Sin ojos volvemos exactamente a la era ciega: 0% en niveles dispersos y un espejismo de supervivencia en la rejilla llena. Sin saber qué queda por romper, no hay puntería posible; solo azar.

Ingrediente 4 — Ojos *estructurados* : el encoder convolucional

Por qué hacía falta. Dar la matriz 8x10 no basta: importa *cómo* se la das a la red. Si aplanas las 80 celdas en una lista de 80 números y se la pasas a una red densa, estás tirando una información valiosísima: la **vecindad espacial**. Para una lista plana, dos celdas son «vecinas» solo si quedaron pegadas en la lista; pero el ladrillo que está *justo encima* de otro cae diez posiciones más adelante en la lista (una fila entera de 10), así que para la red densa esos dos ladrillos contiguos en el tablero parecen tan lejanos como dos cualesquiera. Esa estructura de rejilla —qué está al lado de qué— es justo lo que una **red convolucional** sabe aprovechar.

Qué se cambió. Se adoptó una **red de dos ramas**. La matriz 8x10 entra por un **encoder convolucional** (filtros 16 y 32), que mira parches locales de la rejilla y detecta patrones espaciales (bordes, columnas, bloques). Los 6 cinemáticos entran por una rama **densa** aparte. Las dos ramas se **concatenan** y pasan por dos capas de 128 con ReLU hasta la salida. La salida depende del algoritmo: DQN y el World Model dan 3 valores $Q(s, a)$; PPO y SAC dan un actor con 3 probabilidades más un crítico con 1 valor.



La red de dos ramas. Arriba, la matriz 8x10 atraviesa el encoder convolucional (filtros 16→32), que desliza un filtro por la rejilla y extrae patrones espaciales de los ladrillos; abajo, los 6 cinemáticos entran por una rama densa. Ambas representaciones se concatenan y se procesan en dos capas de 128 con ReLU hasta una salida por acción: el agente decide mirando a la vez *dónde está la bola* y *qué forma tiene el trabajo pendiente*. La convolución es lo que convierte 80 celdas sueltas en una imagen con estructura.

TRADUCCIÓN AL JUEGO

El filtro convolucional es como una plantilla pequeña que vas deslizando por encima de la rejilla buscando un patrón: «¿hay aquí una columna entera de ladrillos?», «¿hay un hueco abierto por el que colar la bola?». Como la misma plantilla se aplica en todas las posiciones, el agente reconoce una columna esté donde esté, sin tener que reaprenderla en cada rincón. Eso es el **sesgo espacial**: el supuesto, incorporado en la arquitectura, de que la vecindad importa y de que un patrón vale igual lo encuentres a la izquierda o a la derecha del tablero.

Qué habría pasado si no se cambiaba. La ablación lo cuantifica: si sustituimos el encoder convolucional por una lista plana procesada con una MLP, el éxito cae de 77% a **57%**, una pérdida de **20,4 puntos**. Dicho de otro modo: el sesgo espacial de la convolución vale unos **20 puntos** de éxito en esta tarea. Ver los ladrillos es necesario; verlos *con estructura* es lo que multiplica el rendimiento.

PARA CURIOSOS

Hay un giro histórico que merece atención. En la era antigua —rejilla pequeña, lista plana, sin convolución— bajar la *escala* de la señal de los ladrillos a 0.25 **desbloqueó** el aprendizaje. Con el encoder convolucional, esa misma escala 0.25 lo **mata**: en la ablación hunde el éxito al **1%** con **100% de colapsos**. No es contradicción: es la prueba de que el ingrediente óptimo *depende de la arquitectura*. Lo que cura en un diseño envenena en otro. Por eso la receta es un conjunto acoplado, no una suma de trucos independientes.

Ingrediente 5 — Variedad: currículo y generador de niveles

Por qué hacía falta. Si entrenas siempre con el mismo nivel, el agente puede *memorizarlo* en vez de aprender a jugar. Y si lo evalúas con ese mismo nivel, tu número de éxito miente: mide memoria, no habilidad. Hace falta variedad para entrenar y, sobre todo, niveles **nunca vistos** para evaluar con honestidad.

Qué se cambió. Se construyó un **generador procedural** de niveles con cinco familias de patrones: *dispersión, filas, columnas, bloque y simétrico*. Los niveles se reparten en conjuntos **disjuntos** de entrenamiento, validación y test, de modo que un nivel de test jamás aparece en el entrenamiento. Un

oráculo de limpiabilidad descarta de antemano los niveles imposibles, para no penalizar al agente por algo que la física no permite. Y se añadió un **currículo**: ordenar la exposición a los niveles para no soltar al agente directamente en lo más difícil.



Las cinco familias de niveles que produce el generador procedural: dispersión, filas, columnas, bloque y simétrico. Bajo cada puzzle se ve lo que recibe la red: la misma escena traducida a la matriz 8x10 de unos y ceros. Cada familia plantea un reto espacial distinto —huecos aislados, líneas, pasillos verticales, masas compactas, espejos—, y justamente por eso la matriz 8x10 y el encoder convolucional rinden: el agente debe leer la *forma* del trabajo pendiente, no memorizar un único tablero.

Para medir generalización de verdad, el test se separa en tres niveles de exigencia: **TEST-ID** (patrones del mismo tipo que vio en entrenamiento), **OOD-patrón** (formas de patrón no vistas) y **OOD-dificultad** (densidades de ladrillos no vistas). Así distinguimos «sabe repetir lo aprendido» de «sabe transferir a lo nuevo». Y aquí importa otra distinción honesta: la *validación* (conjunto disjunto que se usa durante el desarrollo) no es lo mismo que el *test*; las cifras de rendimiento que veremos en el libro son de test, sobre niveles que ni el entrenamiento ni el ajuste tocaron.

Qué habría pasado si no se cambiaba. Aquí conviene separar dos efectos. El *currículo* por sí solo aporta poco en estas condiciones: en la ablación, quitarlo deja el éxito en **75%** frente a 77% (apenas **-1,9 puntos**). Pero el *generador con conjuntos disjuntos* es lo que hace que cualquier número de este libro sea creíble: sin niveles no vistos, no podríamos afirmar que el agente generaliza en vez de memorizar. Su valor no es subir un número, sino **hacer honestos todos los demás**.

QUE NO DEMUESTRA

El generador procedural y los conjuntos disjuntos garantizan que la evaluación es honesta, pero no garantizan que el agente generalice a *cualquier* distribución imaginable. De hecho los propios resultados muestran que el rendimiento baja al pasar de TEST-ID a OOD-patrón y aún más a OOD-dificultad: el agente transfiere, pero no perfectamente. Que la evaluación sea honesta significa precisamente que *vemos* esa caída en lugar de esconderla.

La receta entera, de un vistazo

Estos cinco ingredientes no son trucos sueltos: son un sistema acoplado que reformula la tarea de cabo a rabo. La tabla resume el porqué de cada uno y, cuando lo medimos, cuánto pesa al quitarlo (siempre partiendo de la receta completa con DQN, 77% de éxito).

QUE MIRAR

En la columna «impacto al quitarlo», fija la atención en el signo. Cuatro ingredientes restan al quitarlos (hacen falta); uno —el shaping Phi— *suma* al quitarlo, porque era un saboteador. Y recuerda que la fila de la matriz 8x10 no tiene número de ablación porque sin ella sencillamente volvemos a la era ciega.

Ingrediente	Por qué hace falta	Impacto al quitarlo
1. Timeout proporcional (90/ladrillo)	Da tiempo a ver la señal de «nivel limpiado»	77% → 54% (−23,5)
2. Meta limpia (quitar Phi)	El objetivo desnudo, sin propinas que desvíen	77% → 85% (+8,0)
3. Ojos: matriz 8x10	Saber qué ladrillos quedan vivos	Sin ella, vuelta a la era ciega (0% disperso)
4. Encoder convolucional	Aprovechar la vecindad espacial de la rejilla	77% → 57% (−20,4)
5. Currículo + generador	Variedad para entrenar; niveles no vistos para evaluar honesto	Sin currículo: 77% → 75% (−1,9); el generador hace creíble todo lo demás

Lectura. Los dos pesos pesados de la receta son *ver los ladrillos con estructura* (matriz 8x10 + convolución, del orden de 20 puntos) y *el reloj justo* (timeout proporcional, ~23 puntos), seguidos del giro contraintuitivo de *quitar el shaping* (+8 puntos). El currículo aporta poco por sí mismo, pero el generador de niveles es el que sostiene la credibilidad de todas las cifras del libro. Ninguno de estos cambios es «cambiar de algoritmo» ni «entrenar más».

QUE NO DEMUESTRA

Que estos cinco ingredientes desbloqueen *esta* tarea no significa que sean la receta universal para cualquier problema de RL. Los números de la ablación son, además, sobre DQN y de una tanda concreta: el peso exacto de cada ingrediente podría variar con otro algoritmo. Lo que sí demuestra es la tesis del libro: en este caso, la palanca decisiva fue la **formulación** de la tarea, no la potencia del método.

Quédate con esto

El problema no se desbloqueó por cambiar de algoritmo ni por entrenar más, sino por **formular bien la tarea**: un reloj justo (timeout proporcional), una meta limpia (fuera el shaping Phi), ojos que ven los ladrillos (matriz 8x10), ojos *estructurados* (encoder convolucional) y variedad evaluada con honestidad (currículo y generador de niveles con conjuntos disjuntos). Entre las dos formas de dar vista —píxeles al estilo DeepMind o una matriz estructurada— elegimos la matriz: la información útil ya destilada, para estudiar la formulación y no el sub-problema de la visión.

AUTOCOMPROBACIÓN

1. ¿Por qué un timeout constante de 600 pasos impide aprender en los niveles grandes, y cómo lo arregla la regla de 90 pasos por ladrillo? (Pista: piensa en cuántos pasos por ladrillo deja cada reloj en un nivel de 80.)
2. De los cinco ingredientes, ¿cuál mejora el sistema al *quitarlo*, y por qué más recompensa puede empeorar el aprendizaje?
3. Había dos formas de dar «ojos» al agente (píxeles o matriz estructurada). ¿Cuál se eligió y cuál es el argumento principal a su favor en este libro?

EJERCICIO

Tu compañero propone una mejora «obvia»: como la convolución funciona tan bien, sugiere copiar también el truco de la era antigua y bajar la escala de la señal de los ladrillos a 0.25, «que ahí desbloqueó el aprendizaje». ¿Es buena idea con la arquitectura actual? **Pista:** mira qué le pasa a ese mismo cambio en la ablación cuando ya hay encoder convolucional. **Respuesta:** es mala idea. Con la red de dos ramas, bajar la escala a 0.25 hunde el éxito al 1% con 100% de colapsos; lo que desbloqueaba en la era antigua (lista plana, rejilla pequeña) *mata* con la convolución. El ingrediente óptimo depende de la arquitectura. Para el agente significa que la receta es un conjunto acoplado: no se pueden importar trucos de otra arquitectura sin volver a medir.

Y AHORA, PARA EXPERTOS NIVEL EXPERTO

«La receta que desbloquea» es una suma de **sesgos inductivos**, cada uno atacando un término distinto del problema. La **convolución** aporta *equivarianza a la traslación* y *weight sharing*: el mismo detector de patrones se aplica en toda la rejilla, lo que encaja con la estructura de la matriz **8×10** de ladrillos y reduce drásticamente los parámetros frente a tratar cada celda como independiente.

La **escala de la señal** (recompensas y entradas) no es cosmética: interactúa con la normalización y el **condicionamiento del gradiente**, de modo que una escala mal puesta degrada la optimización aunque la formulación sea correcta. Y el **currículo** ordena la dificultad de menos a más, lo que mejora la *exploración efectiva* al garantizar que el agente encuentra recompensa en las fases tempranas y construye sobre ella. Ninguno sustituye a los demás: arquitectura, escala y currículo operan sobre ejes ortogonales del mismo problema.

La primera conquista

De 0% a 91% en niveles no vistos.

Después de toda la Parte I hablando de formular bien la tarea, llega la pregunta que de verdad importa: ¿funciono? Y, sobre todo, ¿funciono donde cuenta, es decir, en niveles que el agente *no había visto nunca* durante el entrenamiento?

Este capítulo es el primero que pone una cifra encima de la mesa. Hasta ahora hemos defendido una tesis: que el cuello de botella no era el algoritmo ni el número de pasos de entrenamiento, sino la **formulación** de la tarea. Por formulación entendemos varias decisiones concretas que iremos repasando: un reloj justo (un **timeout proporcional** al tamaño del nivel en lugar de un límite fijo), una meta limpia (sin recompensas auxiliares que despisten), una observación que incluye los ladrillos (la matriz de ocupación 8x10), una escala de señal adecuada, un encoder convolucional y una evaluación honesta sobre niveles disjuntos de los de entrenamiento. Toca comprobar si esa tesis se sostiene con datos. Lo haremos sin que las tablas aplasten la lectura: primero una sola figura, después como leerla, después que *no* demuestra, y solo entonces presentamos a los cinco algoritmos que la Parte II va a desmontar pieza a pieza.

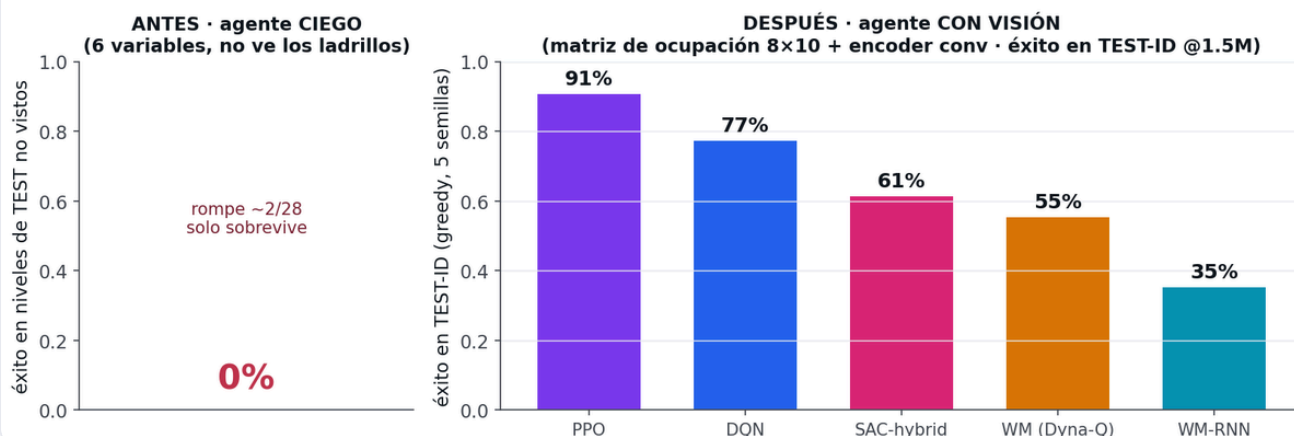
El antes y el después, en una imagen

La figura cuenta la historia entera con dos paneles. El de la izquierda es el punto de partida, lo que en el libro llamamos la **era ciega**: un agente cuya observación eran solo seis números cinemáticos (posición de la bola, su velocidad y la posición de la pala), **sin ningún dato de los ladrillos**. El de la derecha es el estado actual, el **agente con visión**: la misma cinemática más una matriz de ocupación 8x10 que le dice, celda a celda, que ladrillos siguen vivos (un 1 por ladrillo en pie, un 0 por hueco).

QUE MIRAR

Compara los dos paneles, pero fíjate en que **cada uno mide algo distinto** porque pertenecen a montajes históricos distintos. **Izquierda** (era ciega): sobre una rejilla pequeña de $4 \times 7 = 28$ ladrillos, el éxito en niveles *dispersos* no vistos es prácticamente nulo (0%); el agente rompía del orden de 2 ladrillos de 28 y básicamente se limitaba a *sobrevivir*. **Derecha** (agente con visión): sobre la rejilla $8 \times 10 = 80$ ladrillos, el eje mide éxito en el conjunto TEST-ID con evaluación greedy a 1,5 millones de pasos, promediado sobre 5 semillas. Las cinco barras están ordenadas de mayor a menor: PPO 91%, DQN 77%, SAC-critic-hybrid 61%, World Model Dyna-Q 55% y World Model RNN 35%. La lectura no es "que barra es la más alta", sino el salto entre paneles: de no resolver la tarea a resolverla, con la misma familia de algoritmos.

La conquista: de sobrevivir sin ver, a limpiar niveles NUNCA vistos apuntando



El salto que vertebra la Parte I. A la izquierda, el agente ciego sobre la rejilla 4x7: con solo seis números y sin ver los ladrillos, su éxito en niveles dispersos nuevos es nulo; lo único que aprende es a mantener la bola en juego. A la derecha, el mismo problema una vez bien formulado sobre la rejilla 8x10 (matriz de ocupación más encoder convolucional): cinco algoritmos distintos limpian niveles TEST-ID que nunca habían visto, con PPO a la cabeza. El mensaje no es la altura de cada barra, sino que el panel derecho *existe*.

Conviene detenerse en lo que separa los dos paneles, porque es la tesis del libro hecha imagen. Entre el ANTES y el DESPUÉS **no cambiamos de algoritmo buscando uno magico** ni dejamos el ordenador entrenando muchísimo más tiempo. Cambiamos como estaba planteada la tarea. El panel izquierdo no fracasa por falta de inteligencia del agente: fracasa porque le pedimos apuntar a unos ladrillos que su observación ni siquiera mencionaba. Es como pedirle a alguien que despeje una estantería con los ojos vendados; si por azar tira un par de objetos será por la física de sus manotazos, no por puntería. En cuanto la observación incluye donde están los ladrillos, la puntería se vuelve posible, y aparece el panel derecho.

ERROR COMÚN

Leer el 0% de la izquierda como "el agente no aprendía nada". Aprendía, pero aprendía lo que *podía* medir y premiar mal: a sobrevivir. En la era ciega histórica, sobre la rejilla pequeña 4x7 y completamente llena, una bola rebotando durante miles de pasos terminaba tocando casi todos los ladrillos por pura física, y eso inflaba un "éxito" aparente de en torno al 56%. Era una **supervivencia degenerada**: tocar mucho por rebote no es despejar el nivel con intención. Por eso la métrica honesta no es "sobrevivir", sino **limpiar niveles no vistos**, y ahí la era ciega marcaba cero en cuanto los ladrillos dejaban de cubrir toda la pared.

Qué no demuestra esta figura

Una sola imagen con cifras altas invita a sobreinterpretar, así que pongamos los límites por delante. Las barras del panel derecho no son la captura de una partida espectacular ni el mejor intento de una tarde afortunada.

QUE NO DEMUESTRA

Cada barra es la **media de 5 semillas** distintas (las semillas 0 a 4 del protocolo), no una única run brillante elegida a dedo. Es éxito (limpiar el nivel), **no recompensa acumulada**: medimos si el agente termina el trabajo, no cuantos puntos suma por el camino. Es evaluación **greedy** (sin exploración aleatoria, el agente elige siempre su mejor acción) sobre el conjunto **TEST-ID**, que contiene patrones del mismo *tipo* que vio en entrenamiento pero en niveles concretos disjuntos de los de train. Por tanto, la figura demuestra que la tarea quedo *resuelta y es reproducible* en su variante más amable de test; **no** demuestra todavía que aguante patrones radicalmente nuevos (OOD-patron) ni densidades nunca vistas (OOD-dificultad). Esa pregunta más dura, la generalización fuera de distribución, tiene sus propias columnas y la abordaremos más adelante; aquí solo afirmamos lo que la figura sostiene.

Esta distinción entre **validación** y **test**, y entre **recompensa** y **éxito**, va a ser una constante del libro. Recordemos la diferencia: la recompensa es la señal numerica que el agente acumula paso a paso (por ejemplo, +1 por ladrillo roto o -1 por perder la bola), mientras que el éxito es el resultado binario que de verdad nos importa, haber limpiado el nivel. Un agente puede sumar bastante recompensa rompiendo ladrillos sueltos y aun así fracasar en la meta. La mantenemos desde el primer resultado para no caer en la trampa habitual del aprendizaje por refuerzo: celebrar una curva de recompensa que sube mientras el agente, en realidad, no resuelve la tarea que nos importa.

Cinco algoritmos, cinco maneras de aprender

El panel derecho no es un único método: son cinco caminos distintos hacia el mismo objetivo de despejar el nivel. La Parte II los estudia uno a uno, pero conviene presentarlos ya como personajes, porque cada uno encarna una idea diferente de *que significa aprender a jugar*. Una linea para cada uno.

DQN	Valora acciones. Aprende, para cada estado, cuanto vale a la larga cada uno de los tres movimientos, y luego elige el de mayor valor. Es el clásico basado en valor: estable y solido, pero necesita su presupuesto de pasos para llegar arriba.
PPO	Cambia con prudencia. Ajusta directamente su política (su forma de decidir), pero se prohíbe a si mismo dar pasos demasiado grandes en cada actualización. Esa cautela es justo lo que lo hace el más fiable y el que antes aprende.
SAC	Gana sin perder variedad. Optimiza el éxito y, a la vez, premia mantener cierta aleatoriedad en sus decisiones, de modo que no se cierra en banda demasiado pronto. Lo estudiaremos en dos variantes; su actor por si solo (SAC-pure) funciona muy bien, mientras que la variante hibrida que aparece en la figura es más inestable.
World Model (Dyna-Q)	Practica en una maqueta. Además de jugar, construye un modelo interno de la dinámica del juego y entrena también sobre transiciones <i>imaginadas</i> en esa maqueta, para exprimir cada experiencia real.
World Model RNN	Imagina con memoria. Es la variante anterior, pero su maqueta lleva una memoria recurrente que arrastra lo ocurrido en los últimos pasos. La idea es seductora; si ayuda o no en esta tarea, lo veremos sin adornos (adelantamos que aquí no ayuda).

Las alturas relativas de las barras adelantan una conclusión que desarrollaremos despacio: aquí el campeón en fiabilidad es PPO, los métodos basados en valor (DQN) son competitivos, y los enfoques model-based de esta versión tienen un techo más bajo. Un matiz importante de honestidad: la barra de SAC que muestra la figura es la variante **SAC-critic-hybrid** (61%), que es bimodal y a veces colapsa; el actor puro de SAC rinde bastante más alto en esta misma prueba, como veremos en su capítulo. Pero

ese orden fino no es el tema de la Parte I. El tema de la Parte I es el salto entre los dos paneles, que es el mismo para los cinco.

TRADUCCIÓN AL JUEGO

Piensa en la figura como dos partidas filmadas del mismo jugador. En la primera, el jugador no ve los ladrillos: mueve la pala de lado a lado, mantiene la bola viva un buen rato y, casi sin querer, rompe alguno. En la segunda, ya ve el mapa completo de la pared y coloca la pala para dirigir la bola hacia donde aun quedan ladrillos, hasta vaciar el nivel. El cambio de jugador a jugador no fue más entrenamiento: fue quitarle la venda.

El puente hacia la Parte II

Con esto cerramos la Parte I. Hemos visto *que* paso (de cero a limpiar niveles no vistos) y *por que*, en lo esencial, paso (formular bien la tarea). Lo que aun no hemos hecho es abrir cada pieza: que es exactamente un valor $Q(s, a)$, que significa una política $\pi(a|s)$, como entra en juego el factor de descuento gamma, como aprende una red a estimar recompensas futuras o como se mide la calidad de una maqueta del mundo. Para entender **por que** funciona cada uno de los cinco personajes, primero necesitamos el vocabulario. Eso es justo lo que empieza en el siguiente capítulo: los fundamentos del aprendizaje por refuerzo, en el lenguaje concreto de nuestra pelota, nuestra pala y nuestra matriz 8x10.

Quédate con esto

El agente ciego no resolvía la tarea: con solo seis números y sin ver los ladrillos, su éxito en niveles dispersos nuevos era nulo y se limitaba a sobrevivir. Una vez bien formulada (matriz 8x10 más encoder convolucional), cinco algoritmos distintos limpian niveles de test no vistos del conjunto TEST-ID, con PPO a la cabeza con un 91% de media sobre 5 semillas. La conquista no vino de cambiar de algoritmo ni de entrenar más, sino de plantear bien el problema.

AUTOCOMPROBACIÓN

1. El panel izquierdo marca 0%. ¿Significa que el agente ciego no aprendía absolutamente nada, o que aprendía algo distinto de lo que queríamos medir? Explica que aprendía.
2. Las barras del panel derecho, ¿son la mejor run individual o la media de 5 semillas? ¿Y miden recompensa acumulada o éxito en limpiar el nivel?
3. ¿Qué conjunto de test usa la figura y por qué ese resultado no demuestra, por si solo, que el agente generalice a patrones radicalmente nuevos o a densidades no vistas?

EJERCICIO

En la era ciega historica, sobre la rejilla pequeña 4x7 y *llena*, se llevo a reportar en torno a un 56% de "éxito", muy por encima del 0% en niveles dispersos. **Pista:** piensa en una bola rebotando durante miles de pasos dentro de una pared completamente cubierta de ladrillos, y pregunta si tocarlos requiere punteria o basta la física. **Respuesta:** con la pared llena casi cualquier trayectoria larga acaba tocando casi todos los ladrillos, así que ese 56% mide supervivencia (bola viva mucho tiempo), no la capacidad de despejar a proposito; en cuanto los ladrillos se dispersan, la supervivencia ya no garantiza tocarlos y el éxito real se desploma a cero. Para el agente esto significa que la métrica equivocada premiaba sobrevivir, no resolver; cambiarla por "limpiar niveles no vistos" es parte de la formulación honesta que hizo posible el panel derecho.



Y AHORA, PARA EXPERTOS

NIVEL EXPERTO

«La primera conquista» exige **rigor estadístico**. Por la varianza inherente al RL —dependencia de la semilla, ruido de exploración, no estacionariedad—, reportamos la **media de 5 semillas** junto con su dispersión y el porcentaje de *colapsos* (runs que no convergen), nunca la mejor corrida. Una única ejecución brillante es ruido afortunado, no evidencia: con suficientes semillas, alguna parecerá excelente por azar.

Antes de afirmar que un método supera a otro conviene mirar la **distribución completa** de resultados y apoyarse en estadísticos robustos —como la **mediana** y el rango intercuartílico— que no se dejan arrastrar por *outliers*. Comparar máximos entre métodos es uno de los errores metodológicos más comunes en la literatura de RL; comparar distribuciones es la práctica honesta.

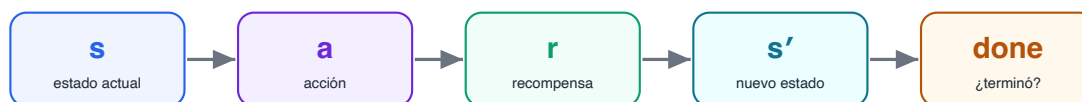
El bucle agente-entorno

La transición s, a, r, s' , done.

Un agente que aprende a jugar al Arkanoid no recibe un manual, ni un consejo de un maestro, ni una explicación de las reglas. Solo ve números, mueve la pala y, de vez en cuando, algo bueno o malo le ocurre. Entonces, ¿cuál es exactamente la unidad mínima de información de la que aprende? ¿Qué es, concretamente, "una experiencia"?

La transición: el átomo de la experiencia

El agente no aprende de «partidas»: aprende de millones de estas tuplas.



$(s, a, r, s', done)$ — en Arkanoid: «desde aquí, muevo la pala, gano +1, y queda este tablero»

La unidad mínima del aprendizaje: una tupla $(s, a, r, s', done)$. El agente no aprende de partidas enteras, sino de millones de estas transiciones.

TRANSICIÓN ACTUAL $(s, a, r, s', done)$ → $s (0.46, 0.58)$ → $a \leftarrow$ → $r (0.016)$ → $s' (0.47, 0.56)$ → $done false$

Una transición, la unidad mínima del aprendizaje por refuerzo: desde el estado s el agente toma la acción a , recibe la recompensa r y aparece el nuevo estado s' (y done si la partida acabó). Millones de estas tuplas son todo lo que el agente llega a saber del juego.

Antes de hablar de DQN, de PPO o de redes convolucionales, conviene detenerse en el latido que sostiene a todos ellos. Todo el aprendizaje por refuerzo, sin excepción, ocurre dentro de un bucle muy simple que se repite millones de veces. Este capítulo mira *una sola vuelta* de ese bucle con lupa, porque entender bien esa vuelta es entender de que material esta hecho todo lo demás. Cambia el algoritmo, cambia la red, cambia la estrategia de exploración; lo que no cambia nunca es el bucle. El resto del libro son maneras distintas de expresar esas mismas vueltas.

La escena: un instante de Arkanoid

Imagina la pantalla congelada en un fotograma. La bola baja en diagonal hacia la parte derecha del tablero. La pala esta un poco a la izquierda del punto donde caera la bola. Arriba quedan ladrillos por romper, repartidos en una rejilla de 8 filas por 10 columnas. En este instante el agente tiene que decidir una sola cosa: mover la pala a la izquierda, mantenerla quieta o moverla a la derecha. Nada más. No hay frases, no hay objetivos escritos, no hay nadie diciendole "cuidado, vas a fallar": hay una situación, una decisión y una consecuencia.

Esa terna —situación, decisión, consecuencia— es la intuición de todo lo que viene. El truco del aprendizaje por refuerzo consiste en ponerle un nombre preciso a cada parte y encadenarlas en un ciclo.

Una vez tienes esos nombres, casi todo el campo se reduce a la pregunta: ¿cómo uso estas piezas para mejorar mis decisiones?

Las cinco piezas del bucle

Formalmente, el agente y el entorno conversan a través de cinco objetos que reaparecen en cada instante de tiempo t . Vamos a definirlos primero de forma general y después los aterrizamos en el Arkanoid.

Estado s	La descripción de la situación actual que el agente recibe como entrada. Es <i>todo</i> lo que sabe del mundo en el instante t . Si algo no está en s , para el agente sencillamente no existe.
Acción a	La decisión que toma el agente a partir de s . En nuestra tarea es una de tres opciones discretas.
Recompensa r	Un número que el entorno devuelve como consecuencia de haber hecho a en s . Es la única señal de "esto ha estado bien o mal". No es un consejo sobre que hacer a continuación; es una nota sobre lo que ya pasó.
Siguiente estado s'	La nueva situación en la que queda el mundo después de la acción. Será el s del siguiente instante.
done	Una bandera (verdadero/falso) que indica si el episodio ha terminado en este paso, ya sea porque se perdió la bola, porque se limpió el nivel o porque se agotó el tiempo.

El bucle consiste en encadenar estas piezas una y otra vez. La **política** del agente, que escribimos $\pi(a | s)$, es precisamente la regla que, dado un estado, elige una acción. El entorno responde con la recompensa y el siguiente estado, y vuelta a empezar. Conviene fijar una asimetría desde ya: la política vive *dentro* del agente y es lo que el aprendizaje va a modificar; r y s' vienen *de fuera*, del entorno, y el agente no los controla, solo los recibe.

ANTES DE LA FÓRMULA

La fórmula que viene no calcula nada todavía: solo nombra el ciclo. Dice que, partiendo de un estado, el agente aplica su política para obtener una acción; que el entorno toma ese par estado-acción y produce una recompensa, un estado nuevo y la bandera de fin; y que ese estado nuevo se convierte en el punto de partida del siguiente paso. Léela como una flecha de tiempo, no como una ecuación numérica: es el guion del ciclo, no una cuenta.

$$s_t \rightarrow a_t \sim \pi(\cdot | s_t) \rightarrow (r_t, s_{t+1}, \text{done}_t) \rightarrow s_{t+1} \rightarrow a_{t+1} \rightarrow \dots$$

Cuando empaquetamos un solo eslabón de esta cadena en un objeto, obtenemos lo que en aprendizaje por refuerzo se llama una **transición** o tupla de experiencia: $(s, a, r, s', \text{done})$. Esta tupla es la célula de todo. Repito la idea porque es la tesis del capítulo: el agente no aprende de explicaciones, aprende de acumular estas tuplas.

Que es s en nuestro Arkanoid

En la versión actual del laboratorio, el estado no son los píxeles de la pantalla, sino un vector de 86 números. Conviene desglosarlo porque define literalmente lo que el agente puede percibir.

6 números cinemáticos	La posición de la bola (x , y), su velocidad (v_x , v_y), la posición de la pala (x) y la distancia entre la bola y la pala. Es la "física" de la jugada: donde esta cada cosa y hacia donde va.
80 números de ocupación	La matriz 8×10 de ladrillos, recorrida celda a celda: un 1 si el ladrillo sigue vivo y un 0 si ese hueco ya está roto. Es el "mapa" de lo que queda por hacer.

Sumados, $6 + 80 = 86$. Más adelante veremos que esos dos bloques no entran en la red por el mismo camino —la matriz pasa por un encoder convolucional y los seis cinemáticos por una rama densa, y luego se concatenan—, pero a nivel de la tupla todo eso es, sencillamente, s . La tupla no distingue por donde entrara cada número; solo guarda los 86.

ERROR COMÚN

Pensar que el agente "ve la pantalla" o que intuye el rebote como lo haría una persona. No. En esta versión solo recibe esos 86 números. Si la matriz 8×10 no estuviera en s —como pasaba en la era ciega del proyecto, cuando el estado eran solo los seis cinemáticos y no había ningún mapa de ladrillos— el agente sería literalmente incapaz de saber donde quedan ladrillos por romper. No es que le cueste deducirlo: es que esa información no le llega. El estado es la frontera exacta de su conocimiento, ni un bit más.

I Que es a y que es r

La acción a es trivial de enumerar: hay tres acciones discretas —izquierda, mantener, derecha—. La política elige una en cada paso, y solo una.

La recompensa r es más rica, porque es el único canal por el que el entorno comunica "valor". En esta tarea se compone así:

+1.0	por cada ladrillo roto.
+0.5 extra	de bonus por cada ladrillo adicional roto en un mismo combo, es decir, sin que la bola toque la pala entre medias.
+0.2	por un rebote en la pala.
-1.0	por perder la bola.
+5.0	por limpiar el nivel entero.

Fijate en que la recompensa de un paso es un *único* número: la suma de lo que haya ocurrido en ese instante. Si en un mismo paso la bola rebota en la pala y no rompe nada, $r = +0.2$. Si rompe un ladrillo, $r = +1.0$. Si pierde la bola, $r = -1.0$. La inmensa mayoría de los pasos, en realidad, tienen $r = 0$: la bola simplemente viaja por el aire sin tocar nada. Esa escasez de señal —muchos ceros y algún valor distinto de cero muy de vez en cuando— es uno de los problemas centrales del aprendizaje por refuerzo, y volveremos a él a lo largo del libro.

ERROR COMÚN

Confundir la recompensa con el éxito. r es la señal local de un paso, no la nota final de la partida ni el porcentaje de niveles que el agente termina limpiando. Un agente puede acumular bastante recompensa positiva a base de rebotes ($+0.2$) y aun así no limpiar casi ningún nivel; de hecho, más adelante veremos que perseguir ciertas recompensas intermedias mal diseñadas puede ir *en contra* del éxito real. Cuando en este libro hablemos de "éxito" nos referiremos a otra cosa: a la fracción de niveles completados en evaluación. La recompensa es el material del aprendizaje; el éxito es el resultado que medimos aparte.

Una transición, paso a paso

Vamos a seguir un instante completo con la escena del principio. La bola baja hacia la derecha, la pala esta un poco a su izquierda, y quedan, digamos, 62 ladrillos vivos en la rejilla (de los 80 que caben).

1) El estado s . El entorno entrega los 86 números: la x e y de la bola en su posición actual, su velocidad apuntando hacia abajo y a la derecha, la x de la pala, la distancia bola-pala (positiva, porque la pala esta a la izquierda) y la matriz 8×10 con 62 unos y 18 ceros.

2) La acción a . La política lee ese s y, como la bola caera a la derecha de la pala, decide *derecha* para ir a interceptarla. Esa es a .

3) El entorno avanza un paso. La pala se desplaza un poco a la derecha, la bola se mueve según su velocidad. Supongamos que en este instante todavía no llega a tocar la pala ni ningún ladrillo.

4) La recompensa r . Como no se rompió nada y no hubo rebote, $r = 0$. Un cero perfectamente normal y muy frecuente.

5) El siguiente estado s' . Casi idéntico a s , pero con la bola un poco más abajo, la pala un poco más a la derecha y la distancia bola-pala algo menor. La matriz 8×10 sigue con sus 62 unos: nada cambio en el mapa.

6) La bandera done. Falso. La bola sigue en juego, queda nivel por limpiar y no se agoto el tiempo.

Esa es *una* transición. Aparentemente no ha pasado nada interesante —recompensa cero, casi el mismo estado—, y sin embargo es exactamente el tipo de ladrillo (nunca mejor dicho) con el que se construye todo el conocimiento del agente. Unos pasos después, cuando la pala intercepte la bola, llegara una transición con $r = +0.2$; y si la bola sube y revienta un ladrillo, otra con $r = +1.0$. La gracia del aprendizaje sera conectar la decisión útil (irse a la derecha a tiempo, varios pasos antes) con la recompensa que llega más tarde. A ese problema —saber a que decisión pasada hay que atribuirle el merito de una recompensa posterior— se le llama **asignación de credito**, y es justo lo que cada algoritmo del libro intenta resolver a su manera.

TRADUCCIÓN AL JUEGO

La transición de arriba, escrita como tupla concreta, tiene esta forma:

$s = [\text{bola_x}, \text{bola_y}, \text{vx}>0, \text{vy}>0, \text{pala_x}, \text{dist}>0, \text{matriz } 8 \times 10 \text{ con } 62 \text{ unos}]$ (86 números)

$a = \text{derecha}$

$r = 0.0$

$s' = [\text{bola un poco más abajo}, \text{pala un poco más a la derecha}, \text{dist menor}, \text{misma matriz}, 62 \text{ unos}]$

$\text{done} = \text{falso}$

Cuando, pasos después, la bola por fin rompe un ladrillo, la tupla de ese instante llevara $r = +1.0$ y su s' tendrá un cero más en la matriz (61 unos): es la única huella visible, dentro de la tupla, de que el mapa ha cambiado.

De una tupla a millones: por que esto lo es todo

El agente repite este ciclo sin descanso. Cada vuelta produce una tupla, y todo lo que el agente llegara a saber sale de acumular y procesar esas tuplas. No hay otra fuente de conocimiento. En este proyecto, el orden de magnitud es muy concreto: el protocolo experimental entrena con presupuestos de 700 000, 1,5 millones y 3 millones de pasos, y el registro completo recoge 160 entrenamientos reales. Cada uno de esos pasos es, literalmente, una transición como la que acabamos de diseccionar.

¿De donde sale el aprendizaje, entonces? De buscar regularidades en ese torrente. Si miles de transiciones distintas coinciden en que "estar a la derecha de la bola que cae y moverse a la derecha" tiende a ir seguido, unos pasos más tarde, de un rebote ($+0.2$) en lugar de una bola pérdida (-1.0), el agente puede empezar a preferir esa acción en esa clase de estados. Los distintos algoritmos del libro — basados en valor como DQN, basados en política como PPO, o model-based como el World Model— no son más que estrategias distintas para destilar esa preferencia a partir del mismo material crudo: la tupla $(s, a, r, s', \text{done})$.

PARA CURIOSOS

El papel de done no es solo decorativo. Marca el corte entre episodios y, en las ecuaciones de aprendizaje, sirve para no "propagar" valor más allá del final de una partida: una bola pérdida no debe contaminar el estado inicial del siguiente episodio, que es independiente. Lo veremos de forma explícita en el objetivo de Bellman de DQN, donde aparece el factor $(1 - \text{done})$ multiplicando el valor del estado futuro — $r + \gamma \cdot (1 - \text{done}) \cdot Q_{\text{obj}}(s', \cdot)$ —: cuando done es verdadero ese termino se anula y el valor futuro deja de contar. Por eso la bandera viaja siempre dentro de la tupla, pegada a la transición concreta donde ocurrió el final.

QUE NO DEMUESTRA

Diseccionar una transición explica la *mecánica* del bucle, pero no demuestra por si misma que un agente aprenda. De una tupla aislada no se puede concluir casi nada: con $r = 0$ ni siquiera sabemos si la acción fue buena o mala, y aun con $r = +0.2$ no sabemos si fue la *mejor* acción posible en ese estado. El aprendizaje emerge solo del *volumen* de tuplas y de algoritmos que sepan asignar credito a lo largo del tiempo. Una transición es la unidad de información, no la unidad de conocimiento.

Quédate con esto

El aprendizaje por refuerzo es un bucle que repite $s \rightarrow a \rightarrow (r, s', \text{done}) \rightarrow s'$. Cada vuelta produce una tupla de experiencia $(s, a, r, s', \text{done})$, y absolutamente todo lo que el agente sabra procede de acumular millones de esas tuplas. En nuestro Arkanoid, s son 86 números (6 cinemáticos + matriz 8×10), a es una de tres acciones y r suele ser 0, con picos cuando se rompe un ladrillo $(+1.0)$, se rebota $(+0.2)$, se pierde la bola (-1.0) o se limpia el nivel $(+5.0)$. El estado es la frontera exacta de lo que el agente puede percibir; la recompensa es el material del aprendizaje, no la medida del éxito.

AUTOCOMPROBACIÓN

1. Enumera las cinco piezas de una transición y di, en una frase, que es cada una.
2. En nuestro Arkanoid, ¿cuántos números tiene s y como se reparten? ¿Qué pasaria si la matriz 8×10 no formara parte de s ?
3. Si en un paso la bola viaja por el aire sin tocar nada, ¿cuánto vale r ? ¿Por qué la mayoría de los pasos tienen ese valor?

EJERCICIO

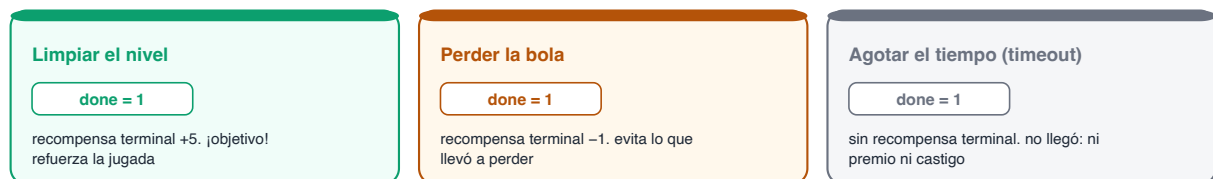
Escribe la tupla $(s, a, r, s', \text{done})$ del instante en que la bola, subiendo tras un rebote, rompe el último ladrillo que quedaba en un nivel.

Pista: piensa en dos cosas que cambian a la vez en ese paso: el número de unos en la matriz 8×10 y el hecho de que el nivel queda limpio. Recuerda las recompensas por romper ladrillo y por limpiar el nivel.

Respuesta: s tiene la matriz con un solo uno (un ladrillo vivo); a es la acción que la política eligiera en ese instante (cualquiera de las tres); la recompensa suma romper ese ladrillo $(+1.0)$ y limpiar el nivel $(+5.0)$, es decir $r = +6.0$ (más el bonus de $+0.5$ de combo si ese ladrillo formaba parte de una racha sin tocar la pala); s' tiene la matriz entera a ceros; y $\text{done} = \text{verdadero}$, porque limpiar el nivel termina el episodio.

Para el agente, esta es una de las transiciones más valiosas que existen: concentra la mayor recompensa de toda la tarea y, además, lleva la bandera de fin, así que marca el cierre limpio de un episodio. Aun así, recuerda el límite del capítulo: por si sola tampoco "enseña" nada; su valor solo se aprovecha cuando un algoritmo la combina con miles de transiciones más.

Cómo termina un episodio (y qué aprende el agente)



Un episodio puede terminar de tres formas; solo dos dan recompensa terminal. Cuando $\text{done}=1$, el futuro vale 0 y el agente aprende del desenlace.

Y AHORA, PARA EXPERTOS NIVEL EXPERTO

El bucle agente-entorno es, formalmente, el muestreo de un MDP (S, A, P, R, γ) . En cada paso el agente observa s_t , elige a_t , y el entorno responde con un nuevo estado y una recompensa según

$$s_{t+1} \sim P(\cdot | s_t, a_t), \quad r_t = R(s_t, a_t, s_{t+1})$$

Iterar este proceso genera la trayectoria $\tau = (s_0, a_0, r_0, s_1, \dots)$ sobre la que se define el retorno $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$.

El objetivo no es maximizar la recompensa **inmediata** r_t sino el **retorno esperado** $E[G_t]$: una acción puede tener recompensa instantánea baja y aun así ser óptima si conduce a estados de alto valor futuro. Esta diferencia entre lo inmediato y lo acumulado descontado es la esencia del control secuencial y lo que separa el RL de la optimización miope paso a paso.

MDP, observación y POMDP sin asustar

Por que el ciego tenía techo.

El primer agente entrenaba y entrenaba, pero se quedaba clavado en el mismo techo: rompía ~2 de 28 ladrillos en niveles dispersos. La tentación era obvia: dejarlo entrenar más tiempo. Pero, ¿y si su problema no fuera cuanto practicaba, sino que estaba mirando el juego con los ojos a medio abrir?

MDP vs POMDP, sin asustar

Si la observación contiene todo lo relevante es un MDP; si oculta algo necesario, un POMDP.

MDP — observable

la observación = el estado

todo lo relevante está a la vista

Arkanoid con matriz 8×10:

ve ladrillos y velocidad → casi markoviano

POMDP — parcial

la observación oculta información

el agente «ve con rendijas»

Agente ciego (sin ladrillos):

techo de INFORMACIÓN, no de cómputo

MDP frente a POMDP sin formalismo: si la observación contiene todo lo relevante (matriz 8×10), el problema es casi observable; si oculta los ladrillos (agente ciego), es parcial y tiene techo.

En el capítulo anterior describimos el bucle agente-entorno con palabras: el agente observa, actúa, recibe una recompensa y el juego avanza. Funciona como intuición, pero para razonar con rigor sobre *por que* un agente se atasca necesitamos un esqueleto matemático. Ese esqueleto se llama **Proceso de Decisión de Markov** (MDP). No te asustes: vamos a montarlo pieza a pieza, y cada pieza la veras enseguida dentro de Arkanoid. Al final tendras una herramienta para diagnosticar una clase de fallo que **ningún** volumen de entrenamiento puede arreglar.

Una escena para empezar

Antes de la matemática, una imagen. Imagina que juegas a Arkanoid con unas gafas empanadas: ves la pelota y la pala perfectamente, pero la pared de ladrillos es una mancha borrosa donde no distingues cuales siguen en pie y cuales ya rompiste. Sobrevives —devuelves la bola—, pero no puedes *apuntar* a nada concreto, porque no ves a que apuntar. Por mucho que practiques con esas gafas, no mejorarás la puntería: el límite no está en tu habilidad, sino en lo que entra por tus ojos. Esa escena es, casi literalmente, lo que le pasaba a nuestro primer agente. El MDP nos dará el lenguaje para nombrar ese problema con precisión y, sobre todo, para distinguirlo de un problema de entrenamiento.

Las cuatro piezas de un MDP

Un MDP es la forma estándar de describir un problema de decisión secuencial: alguien que toma decisiones, una tras otra, en un entorno que cambia. Se define con cuatro ingredientes. Vamos a nombrarlos primero y a traducirlos después.

Estado s	La descripción de la situación en un instante. Todo lo que el sistema "es" ahora mismo: donde esta cada cosa y como se mueve.
Acción a	Lo que el agente puede hacer desde un estado. El conjunto de acciones disponibles es su repertorio de decisiones.
Transición $P(s' s, a)$	La regla del entorno: dado que estoy en s y hago a , ¿a que estado s' paso? Puede ser determinista o tener algo de azar.
Recompensa $r(s, a)$	La señal numerica que el entorno devuelve tras la acción. Es la única definición de "jugar bien" que el agente conoce.

Hay una quinta pieza que ya viste en el capítulo anterior y que pega todo el marco: el factor de descuento γ , que decide cuanto pesa el futuro frente al presente. Con estas piezas, el objetivo del agente se enuncia de una sola forma: encontrar una **política** $\pi(a | s)$ —una forma de elegir acción en cada estado— que maximice el retorno descontado esperado. Nada más. Todo el aprendizaje por refuerzo es, en el fondo, resolver un MDP.

TRADUCCIÓN AL JUEGO

Pongamos cada pieza sobre la mesa de Arkanoid en su versión con visión:

- **Estado s** : los 86 números que recibe el agente. Seis son cinemáticos —posición de la pelota (x, y) , su velocidad (v_x, v_y) , la posición de la pala (x) y la distancia pelota-pala— y los otros 80 forman la matriz de ocupación 8×10 , con un 1 en cada celda donde sigue vivo un ladrillo y un 0 donde ya hay hueco.
- **Acción a** : tres opciones discretas. Mover la pala a la izquierda, mantenerla quieta o moverla a la derecha. Ese es todo el repertorio.
- **Transición P** : el motor de física del juego. Dada la posición y la velocidad de la pelota y la acción sobre la pala, el motor calcula el siguiente fotograma: la pelota avanza y rebota en paredes, ladrillos y pala según las reglas del juego.
- **Recompensa r** : +1.0 por cada ladrillo roto (con +0.5 de bonus por cada ladrillo extra encadenado en un combo sin tocar la pala), +0.2 por rebotar la pelota con la pala, -1.0 por perder la bola y +5.0 por limpiar el nivel entero.

I La propiedad de Markov: el presente basta

El nombre "de Markov" no es decorativo. Apunta a una hipótesis muy concreta sobre el estado, la **propiedad de Markov**: el futuro depende solo del estado actual, no de como llegaste hasta el. Si conoces s , el pasado no anade nada para predecir s' .

ANTES DE LA FÓRMULA

La fórmula de abajo dice algo sencillo con notación de probabilidad. A la izquierda: la probabilidad del próximo estado *sabiendo todo el historial* —todos los estados y acciones desde el principio de la partida—. A la derecha: la probabilidad del próximo estado *sabiendo solo el último estado y la última acción*. La propiedad de Markov afirma que ambas son iguales: el último estado ya resume todo lo que importa del pasado.

$$P(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0) = P(s_{t+1} | s_t, a_t)$$

¿Por qué importa tanto? Porque toda la maquinaria del aprendizaje por refuerzo —estimar el valor de un estado, mejorar la política, la propia ecuación de Bellman— da por hecho que el estado contiene la información necesaria para decidir bien. Si esa hipótesis se cumple, el agente puede aprender una buena política como función solo del estado actual. Si *no* se cumple, le estamos pidiendo que adivine con datos incompletos, por mucho que entrene.

TRADUCCIÓN AL JUEGO

¿Cumple Arkanoid la propiedad de Markov con el estado de 86 números? Casi. Sabiendo posición y velocidad de la pelota, posición de la pala y que ladrillos siguen vivos, puedes predecir el siguiente fotograma sin necesitar la historia previa de la partida. Por eso incluimos la *velocidad* y no solo la posición: con una sola foto de posiciones no sabrías hacia donde va la pelota —harían falta dos fotos, es decir, algo de pasado—. Meter la velocidad en el estado es justamente la jugada que devuelve la propiedad de Markov a una sola observación: la foto deja de necesitar un "antes".

EJERCICIO

Te dan una sola foto del juego en la que ves la pelota a media altura y la pala debajo, pero **sin** ninguna información de velocidad. ¿Puedes decidir bien la acción? ¿Por qué?

Pista: intenta predecir donde estará la pelota en el siguiente fotograma usando solo esa foto.

Respuesta: no puedes. La misma posición de la pelota es compatible con "subiendo" y con "bajando", y con ir hacia la izquierda o hacia la derecha; la acción correcta de la pala es distinta en cada caso. Sin velocidad, una sola foto no determina el futuro: la propiedad de Markov falla. Para el agente, esto significa que la velocidad no es un adorno del estado, sino el dato que convierte una foto ambigua en una situación decidable.

■ Cuando la observación no es el estado: el POMDP

Llegamos al corazón del capítulo. Hasta ahora hemos hablado como si el agente *viera* el estado completo s . Pero en muchos problemas reales el agente no accede al estado verdadero, sino solo a una **observación** o : una vista parcial, posiblemente incompleta, de lo que de verdad está pasando. Cuando eso ocurre, ya no tenemos un MDP, sino un **Proceso de Decisión de Markov Parcialmente Observable** (POMDP).

MDP El agente observa el estado completo. La observación es el estado: $o = s$.

POMDP El agente solo observa o , una función parcial del estado real s . Hay información del estado que la observación no contiene.

La diferencia es profunda. En un POMDP, dos estados verdaderos distintos pueden producir *la misma* observación. El agente los ve iguales, así que está obligado a tomar la misma decisión en ambos —aunque la acción óptima sea distinta—. No es que decida mal: es que **no tiene con que distinguirlos**. Su techo de rendimiento no lo fija el algoritmo ni el tiempo de entrenamiento, sino la información que entra por la observación. Volvemos a las gafas empanadas del principio: el límite está en los ojos, no en la habilidad.

TRADUCCIÓN AL JUEGO

Aquí esta la historia de nuestro primer agente, el de la era ciega. Su observación eran **solo los 6 cinemáticos**: posición y velocidad de la pelota, posición de la pala y distancia pelota-pala. **No incluía la matriz de ladrillos**. Es decir, el agente sabia donde estaba la pelota, pero no que ladrillos seguian en pie ni cuales ya estaban rotos.

Eso convierte la tarea en un POMDP de manual. Imagina dos situaciones identicas en cinemática — misma pelota, misma velocidad, misma pala— pero con disposiciones de ladrillos completamente distintas: en una quedan tres ladrillos arriba a la izquierda; en otra, tres abajo a la derecha. La acción correcta es opuesta (dirigir la pelota a un lado o al otro), pero el agente ciego **recibe exactamente la misma observación** en ambas. No puede apuntar a un objetivo que no ve.

Mini-ejemplo: dos estados, una observación

Hagamoslo concreto con dos transiciones leidas paso a paso, en la rejilla pequeña de la era ciega (4x7). Supongamos que en ambas la pelota esta en el centro subiendo y que el agente solo recibe esos números cinemáticos.

	Estado real A	Estado real B
Ladrillos vivos	solo la esquina sup. izquierda	solo la esquina sup. derecha
Pelota (x,y)	centro, subiendo	centro, subiendo
Observación o que recibe	centro, subiendo	centro, subiendo
Acción óptima	colocar la pala para desviar a la izquierda	colocar la pala para desviar a la derecha
Acción que puede elegir	la misma que en B: no los distingue	la misma que en A: no los distingue

Las dos primeras filas son el estado verdadero s ; difieren. La tercera es la observación o ; es identica. El agente colapsa A y B en un mismo caso y elige una única acción para los dos. Pongamosle número a la mejor estrategia posible que le queda: como las dos disposiciones son igual de probables y solo puede fijar una acción, acertara **como mucho la mitad de las veces** (una probabilidad de 0.5), y no por aprender, sino por azar de hacia donde caiga el patron. Ninguna cantidad de partidas sube ese 0.5: la información que necesita para decidir **nunca entra** por su observación.

ERROR COMÚN

"Si rinde poco, que entrene más pasos." Es el reflejo natural, y es justo el diagnostico equivocado para un POMDP. Más entrenamiento solo ayuda cuando el cuello de botella es *aprender* a partir de una observación suficiente. Si la observación es incompleta —si el estado verdadero no se puede reconstruir desde o — el agente choca contra un techo de **información**, no de optimización. Entrenar el doble de tiempo a un agente que no ve los ladrillos no le hace ver los ladrillos. La solución no esta en el algoritmo ni en el reloj, sino en **arreglar la observación**.

La prueba: un techo de información, no de entrenamiento

Los números de la era ciega cuentan esta historia sin ambigüedad. Aquel agente, con observación de 6 cinemáticos y rejilla 4x7 (28 ladrillos), lograba **0% en los niveles dispersos** y rompía de media **~2 de 28**

ladrillos. El detalle revelador esta en el único escenario donde "parecía" funcionar: en la rejilla **completamente llena** alcanzaba un **~56% de "éxito"**. Pero ese éxito era una ilusión estadística. Con todos los ladrillos presentes, una bola que rebota durante unos 2200 pasos toca casi cualquier celda por pura física, sin necesidad de apuntar. En cuanto la rejilla tenía huecos —es decir, en cuanto apuntar importaba de verdad— el rendimiento se hundía a cero.

QUE MIRAR

Fijate en el contraste entre las dos filas de la tabla: un mismo agente, mismo algoritmo, mismo entrenamiento. Lo único que cambia es cuanta información necesita la tarea para que el agente apunte. La columna "Lectura" traduce cada cifra a su significado; no leas el porcentaje suelto, leelo junto a lo que el escenario exige.

Escenario (era ciega, 4x7)	Resultado	Lectura
Niveles dispersos (hay que apuntar)	0% de éxito; ~2/28 ladrillos	sin la matriz, no puede dirigir la bola
Rejilla llena (basta sobrevivir)	~56% de "éxito"	supervivencia degenerada: la física toca casi todo

Esa caída del ~56% al 0% según pasamos de "sobrevivir" a "apuntar" es la firma de un POMDP. Un agente con techo de *entrenamiento* mejoraría en todos los escenarios con más pasos. Un agente con techo de *información* se desploma justo cuando la decisión exige datos que su observación no contiene. El primer agente era lo segundo. Y conviene no confundir dos cosas que aquí van separadas: ese ~56% es **éxito** (niveles superados), no **recompensa** acumulada; un agente que solo rebota la bola también suma muchos +0.2 sin romper apenas ladrillos, así que una recompensa decente puede convivir con una puntería nula.

QUE NO DEMUESTRA

Estos números demuestran que *aquella* observación ciega era insuficiente *para esta* tarea de apuntar. No demuestran que toda observación parcial sea inútil —muchos POMDP se resuelven razonablemente bien— ni que la cinemática sobre —sigue siendo necesaria—. Tampoco son comparables uno a uno con la era con visión: además de anadir la matriz, en la versión nueva cambiamos la rejilla a 8x10, el timeout (de constante a proporcional) y otros ingredientes, así que no se puede atribuir toda la mejora a un solo cambio. La lección limpia que **sí** se sostiene es de diagnóstico: un techo que no cede con más entrenamiento es candidato a ser un problema de observación.

¿ Cómo recupera Markov la versión con visión

La cura para un POMDP es enriquecer la observación hasta que vuelva a contener lo que la decisión necesita: convertirlo, en la practica, de nuevo en un MDP. Eso es exactamente lo que hace el paso a las 86 entradas. Al meter la matriz de ocupación 8x10 dentro del estado, los dos escenarios A y B del mini-ejemplo dejan de ser indistinguibles: ahora la observación incluye *donde estan* los ladrillos, así que el agente recibe vectores distintos y puede elegir acciones distintas. La acción óptima vuelve a ser una función del estado observado, que es justo lo que la propiedad de Markov pide. Quitate las gafas empanadas y la pared de ladrillos vuelve a estar enfocada.

PARA CURIOSOS

Formalmente, un MDP es una tupla (S, A, P, r, γ) : conjunto de estados, conjunto de acciones, función de transición, función de recompensa y descuento. Un POMDP anade dos piezas y pasa a $(S, A, P, r, \gamma, \Omega, O)$, donde Ω es el conjunto de observaciones posibles y $O(o|s', a)$ es la probabilidad de observar o dado el estado real. La vía clásica para atacar un POMDP es mantener una *creencia* —una distribución de probabilidad sobre en que estado real podrias estar— y decidir sobre ella; las arquitecturas con memoria (como el LSTM del World Model RNN que veras más adelante) son una forma de aproximar esa idea integrando varias observaciones en el tiempo. Pero existe una alternativa más directa y, para esta tarea, mucho más eficaz: si puedes **ampliar la propia observación** hasta que sea (casi) suficiente, el POMDP se vuelve un MDP y te ahorras toda la maquinaria de memoria. Eso es lo que conseguimos anadiendo la matriz 8×10 . De hecho, más adelante veremos que en esta tarea el LSTM del World Model RNN no mejora al World Model sin memoria: cuando la observación ya es casi suficiente, anadir memoria aporta poco.

Quédate con esto

Un MDP se define por estado, acción, transición y recompensa, y todo el aprendizaje por refuerzo asume que el estado contiene lo necesario para decidir (propiedad de Markov). Cuando la observación no contiene esa información, es un POMDP: el agente no puede distinguir situaciones que exigen acciones distintas, y su techo lo fija la información, no el entrenamiento. El primer agente fallaba por eso —era ciego a los ladrillos—, y la cura no fue entrenar más ni cambiar de algoritmo, sino arreglar la observación: meter la matriz de ocupación 8×10 en el estado.

AUTOCOMPROBACIÓN

1. Enuncia las cuatro piezas de un MDP y di que parte de Arkanoid es cada una.
2. ¿Por qué el estado incluye la *velocidad* de la pelota y no solo su posición? Relacionalo con la propiedad de Markov.
3. Un agente rinde mal y no mejora aunque lo entrenes el triple de pasos. ¿Qué dos hipótesis distintas podrían explicarlo y cual señala que el problema es un POMDP?

EJERCICIO

Diseña una observación *mínima* para una versión simplificada de Arkanoid con un solo ladrillo. ¿Qué números incluirías como mínimo para que la tarea sea un MDP y no un POMDP?

Pista: piensa que necesitas para predecir el siguiente fotograma y para saber a donde apuntar. Pregunta clave: ¿con una sola foto sabes hacia donde va la pelota?

Respuesta: necesitas al menos la posición de la pelota (x,y) , su velocidad (v_x,v_y) —sin ella no hay propiedad de Markov en una sola observación—, la posición de la pala (x) y la posición del ladrillo (o un bit de si sigue vivo y donde). Sin la posición del ladrillo, dos estados con el ladrillo en sitios distintos darian la misma observación: volverias a un POMDP. Para el agente, esto significa que cada número que anades a la observación no es "más datos" sin más, sino la diferencia entre poder distinguir dos situaciones que exigen acciones opuestas o quedarse ciego ante ellas.

¿Probar algo nuevo o ir a lo seguro? El agente lo echa a suertes



tira un dado
antes de mover

sale BAJO → prueba algo
nuevo al azar (EXPLORAR)

si no → lo que YA sabe que
funciona (EXPLOTAR)

Con el tiempo...

el dado pide explorar
cada vez MENOS (ya
ha aprendido).

Ese % de azar baja poco
a poco: «ε» que «decae».

La transición es **estocástica**: desde un estado y una acción, el siguiente estado no está fijado; sale de una distribución de probabilidad $P(s' | s, a)$, como tirar un dado cargado. El MDP es justo eso: las reglas del azar del entorno.



Y AHORA, PARA EXPERTOS NIVEL EXPERTO

Un **POMDP** se describe con la tupla $(S, A, O, T, Z, R, \gamma)$, donde O es el espacio de observaciones, T la dinámica de transición y $Z(o | s', a)$ la **función de observación** (probabilidad de observar o tras llegar a s' con la acción a). Como la observación no determina el estado, el agente actúa sobre un **estado de creencia** (*belief*) $b(s)$ —una distribución sobre estados que mantiene implícita o explícitamente— en lugar de sobre el estado real.

La **propiedad de Markov** dice que el futuro depende solo del estado presente, no de la historia. Un único fotograma de Arkanoid la **rompe**: la imagen muestra dónde está la bola pero no su **velocidad**, así que dos estados físicos distintos producen la misma observación y la dinámica deja de ser markoviana sobre O . **Apilar** fotogramas consecutivos —o medir explícitamente la velocidad— restaura la información cinemática y devuelve la propiedad de Markov a la observación.

Recompensa, retorno y descuento

G , gamma y el peso del futuro.

Si el agente rompe un ladrillo ahora mismo, gana $+1.0$ al instante. Pero, ¿y si ese ladrillo no fuera el mejor movimiento? ¿Cómo decidimos entre coger la recompensa que tenemos delante y renunciar a ella para colocar la bola y desencadenar un combo dentro de tres golpes? Para responder necesitamos dejar de mirar la recompensa de un solo paso y empezar a mirar la *suma* de todo lo que vendrá después.

Recompensa inmediata frente a lo que viene después

Hasta ahora hemos hablado de la **recompensa** r como la señal que el entorno devuelve tras cada acción. En nuestro Arkanoid esa señal es muy concreta: $+1.0$ por romper un ladrillo, $+0.5$ de bonus por cada ladrillo extra de un combo encadenado sin tocar la pala, $+0.2$ por un rebote en la pala, -1.0 por perder la bola y $+5.0$ por limpiar el nivel. Cada una de esas cifras es la recompensa *inmediata* de una transición: lo que cae en la mano justo en ese paso.

El problema es que una decisión casi nunca se agota en el paso en el que se toma. Mover la pala un píxel a la derecha ahora puede no romper nada (recompensa inmediata 0) y, sin embargo, ser la jugada que decenas de pasos después provoca un combo de cinco ladrillos. Si el agente solo persiguiera r del paso actual, sería un glotón sin memoria del futuro: incapaz de invertir un movimiento neutro hoy para cobrar mucho más mañana. Necesitamos una magnitud que mida no el premio de un paso, sino el premio acumulado de toda la trayectoria que arranca en ese momento.

Recompensa inmediata r_t La señal escalar que devuelve el entorno en el paso t , justo después de ejecutar la acción a_t . En nuestro juego, por ejemplo $+1.0$ si esa acción acabó rompiendo un ladrillo.

Retorno G_t La suma (descontada, como veremos enseguida) de todas las recompensas que el agente recibirá *desde* el paso t en adelante hasta el final del episodio. Es lo que de verdad queremos maximizar: no un premio suelto, sino la cosecha completa.

TRADUCCIÓN AL JUEGO

Imagina la bola subiendo hacia un racimo de ladrillos pegados. La acción "mantener" no rompe nada en este paso, así que su recompensa inmediata es 0 . Pero coloca la bola en el ángulo justo para que rebote dentro del racimo y encadene cuatro impactos seguidos sin volver a bajar a la pala. La recompensa inmediata, por sí sola, infravalora esa acción; el retorno —que incluye los $+1.0$ y los $+0.5$ del combo que viene después— refleja su valor real.

El retorno descontado: por qué el futuro pesa menos

La idea ingenua de retorno sería sumar a pelo todas las recompensas futuras: $r_t + r_{t+1} + r_{t+2} + \dots$. Funciona en episodios cortos, pero tiene dos pegas. La primera es práctica: si una partida pudiera alargarse mucho, esa suma podría dispararse y volverse difícil de manejar numéricamente. La segunda es conceptual y más importante: *no todas las recompensas futuras valen lo mismo desde donde estamos ahora*. Un ladrillo que tengo casi garantizado en el próximo golpe es más "mío" que un ladrillo hipotético dentro de doscientos pasos, cuando puede que ya haya perdido la bola. El futuro lejano es más incierto, así que es razonable que pese menos en la decisión de hoy.

Para capturar las dos cosas a la vez introducimos un único número, el **factor de descuento** gamma, con $0 \leq \gamma \leq 1$. La recompensa del paso siguiente se multiplica por gamma; la del paso de después por γ^2 ; y así sucesivamente. Cada paso que nos alejamos en el tiempo, la recompensa se encoge en un factor gamma. Es un interés compuesto al revés: en lugar de crecer hacia el futuro, el valor de un premio se va desinflando cuanto más tarde llega.

ANTES DE LA FÓRMULA

Lo que viene a continuación no es más que esto en símbolos: *coge la recompensa de este paso tal cual; suma la del paso siguiente pero rebajada por gamma; suma la del siguiente rebajada por gamma dos veces; y sigue hasta el final del episodio*. El resultado es el retorno descontado G_t , una sola cifra que resume "cuánto vale, visto desde el paso t , todo lo que está por venir". El exponente de gamma es simplemente cuántos pasos en el futuro cae cada recompensa.

$$G_t = r_t + \gamma \cdot r_{t+1} + \gamma^2 \cdot r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k \cdot r_{t+k}$$

Hay una segunda forma de leer la misma fórmula, y resulta ser la más útil para entrenar. Si sacas factor común gamma de todos los términos salvo el primero, el retorno de hoy se escribe en función del retorno de mañana:

$$G_t = r_t + \gamma \cdot G_{t+1}$$

Es decir: "lo que vale desde aquí es la recompensa de este paso más, descontado, lo que valdrá desde el paso siguiente". Esta forma *recursiva* es la semilla de la ecuación de Bellman que entrena la red, y por eso no hace falta esperar al final del episodio para empezar a aprender: basta con encadenar cada paso con el siguiente.

Conviene leer gamma como un dial entre dos personalidades del agente:

- | | |
|--|--|
| gamma cercano a 0 (cortoplacista) | El agente casi solo valora la recompensa inmediata. Con $\gamma = 0$ el retorno se reduce a $G_t = r_t$: vive el presente y es ciego al futuro. Rompería el ladrillo que tiene delante aunque eso arruine un combo mayor. |
| gamma cercano a 1 (previsor) | El agente trata las recompensas lejanas casi como las cercanas. Está dispuesto a sacrificar un premio ahora por una cosecha mayor más tarde. Es la personalidad que necesitamos para que planifique combos y, sobre todo, para que aprenda a no perder la bola: el -1.0 de fallar contamina hacia atrás muchas decisiones previas. |

En todos nuestros algoritmos basados en valor y en ventaja usamos $\gamma = 0.99$. Es un valor deliberadamente alto y cercano a 1: queremos un agente previsor, porque en Arkanoid el éxito (limpiar el

nivel, $+5.0$) y el gran fracaso (perder la bola, -1.0) son consecuencias diferidas de jugadas que ocurrieron muchos pasos antes. Con un timeout que llega hasta 7200 pasos en la rejilla 8×10 , necesitamos que la señal sepa viajar lejos en el tiempo.

El retorno $G = r_0 + \gamma \cdot r_1 + \gamma^2 \cdot r_2 + \dots$ (γ = factor de descuento, 0.99 aquí)



El peso γ^k de una recompensa según a cuántos pasos k cae en el futuro. La curva baja despacio cuando γ está cerca de 1 (el futuro lejano todavía cuenta) y se desploma cuando γ es pequeño (solo importa lo inmediato). En la práctica, ese ritmo de caída fija cuánto horizonte ve el agente: cuántos pasos hacia el futuro alcanza a tener en cuenta una jugada de hoy.

Mini-ejemplo: tres recompensas, dos valores de gamma

Hagamos el cálculo a mano para ver cómo γ cambia la decisión. Supongamos una secuencia corta de tres pasos con estas recompensas, en este orden: $r_0 = 1$ (rompo un ladrillo ya), luego $r_1 = 0$ (un paso neutro, coloco la bola) y luego $r_2 = 10$ (estalla un combo grande). El retorno desde el paso 0 es $G_0 = r_0 + \gamma \cdot r_1 + \gamma^2 \cdot r_2$. (El 10 es una cifra inventada solo para el ejemplo, escogida grande para que se vea el efecto; en el juego un combo así sumaría varios $+1.0$ y $+0.5$.)

QUÉ MIRAR

Fíjate en la última columna: con el mismo escenario de recompensas, dos valores de γ producen retornos muy distintos. La pregunta no es cuánto sale en cada celda, sino si el 10 diferido sigue dominando el 1 inmediato. Si domina, al agente le compensa esperar.

Caso	r_0	$+\gamma \cdot r_1$	$+\gamma^2 \cdot r_2$	G_0
Cortoplacista $\gamma = 0.5$	1	$0.5 \cdot 0 = 0$	$0.25 \cdot 10 = 2.5$	3.5
Previsor $\gamma = 0.99$	1	$0.99 \cdot 0 = 0$	$0.9801 \cdot 10 = 9.8$	10.8

Lectura: con $\gamma = 0.99$ el combo diferido aporta 9.8 al retorno, casi su valor íntegro de 10; la jugada paciente vale mucho más que el ladrillo suelto de $+1$ del principio. Con $\gamma = 0.5$ ese mismo combo, por estar dos pasos más allá, se ha encogido a 2.5: sigue siendo la mayor contribución, pero su ventaja se ha recortado drásticamente. Y si el combo estuviera mucho más lejos en el tiempo, con $\gamma = 0.5$ podría llegar a pesar menos que el $+1$ inmediato, y el agente cortoplacista preferiría romper ya. Ese es justo el comportamiento que queremos evitar en Arkanoid, y por eso elegimos un γ alto.

ERROR COMÚN

Pensar que gamma es "cuánto le gusta el premio al agente" o una preferencia ética por la paciencia. No lo es. gamma no toca el valor de ninguna recompensa *en el instante en que se cobra*: el +1.0 de un ladrillo siempre es +1.0 cuando cae. Lo que gamma hace es ponderar ese premio cuando lo miramos *desde un paso anterior*, para compararlo en igualdad de condiciones con premios más cercanos. Es una herramienta de comparación entre presente y futuro, no un recorte del premio en sí.

Por qué el retorno, y no la recompensa, es lo que aprendemos

Toda la maquinaria de los capítulos siguientes existe para estimar este retorno sin tener que esperar al final del episodio. La función de valor $V(s)$ es el retorno esperado partiendo de un estado s ; la función $Q(s, a)$ es el retorno esperado de tomar la acción a en s y luego seguir jugando bien. Ambas son, en el fondo, predicciones de G . Por eso el descuento gamma aparece dentro del objetivo de Bellman que entrena nuestra red DQN:

$$\text{objetivo} = r + \text{gamma} \cdot (1 - \text{done}) \cdot Q_{\text{objetivo}}(s', \text{argmax}_a Q(s', a))$$

Léelo a la luz de la forma recursiva $G_t = r_t + \text{gamma} \cdot G_{t+1}$ que vimos antes: aquí el papel del retorno de mañana G_{t+1} lo hace la mejor estimación disponible del valor del estado siguiente, $Q_{\text{objetivo}}(s', \text{argmax}_a Q(s', a))$. El valor de una acción es, pues, su recompensa inmediata r más el valor descontado de lo mejor que se puede hacer en s' . Es la definición del retorno, pero expresada paso a paso en vez de como una suma infinita. El factor $(1 - \text{done})$ apaga el término del futuro cuando el episodio termina ($\text{done} = \text{verdadero}$): si la bola se ha perdido o el nivel está limpio, no hay "paso siguiente" que descontar, y el objetivo se reduce a la recompensa final r .

PARA CURIOSOS

El descuento también garantiza que el retorno sea un número finito aunque el episodio fuera infinitamente largo. Si las recompensas estuvieran acotadas por un valor máximo R , la suma geométrica $R + \text{gamma} \cdot R + \text{gamma}^2 \cdot R + \dots$ converge a $R / (1 - \text{gamma})$ siempre que $\text{gamma} < 1$. Con $\text{gamma} = 0.99$ ese factor de amplificación es $1 / 0.01 = 100$: el "horizonte efectivo" del agente es del orden de cien pasos, una regla de bolsillo útil para intuir cuánto futuro pondera de verdad. Aun así, gracias a la propagación recursiva de Bellman, la señal de un acierto o de un fallo puede encadenarse a lo largo de toda la partida, mucho más allá de esos cien pasos.

QUÉ NO DEMUESTRA

Un gamma alto no garantiza por sí solo que el agente planifique bien. El descuento define *qué* queremos maximizar (un retorno previsor), pero no asegura que el agente *consiga* estimarlo: eso depende de la observación, de la arquitectura y de la formulación de la tarea. De hecho, en nuestra era ciega —cuando la observación eran solo seis números cinemáticos, sin los ladrillos— el agente también usaba descuento y aun así fracasaba, porque sin ver la rejilla no podía aprender la dinámica del combo que el retorno premia. El retorno descontado es la *vara de medir* correcta; no es, por sí mismo, la solución. Es un caso más de la tesis del libro: lo que desbloquea la tarea es formularla bien, no un único ajuste aislado.

Quédate con esto

La recompensa inmediata r es el premio de un paso; el retorno $G = \sum \gamma^k \cdot r_{t+k}$ es la suma descontada de todo lo que viene después, y es lo que de verdad maximizamos. Su forma recursiva $G_t = r_t + \gamma \cdot G_{t+1}$ es la que aparece en el objetivo de Bellman. El factor γ es un dial entre cortoplacista (≈ 0 , solo el presente) y previsor (≈ 1 , el futuro casi cuenta entero). Usamos $\gamma = 0.99$ porque en Arkanoid las consecuencias importantes —limpiar el nivel (+5.0) o perder la bola (-1.0)— llegan muchos pasos después de la jugada que las causó.

AUTOCOMPROBACIÓN

1. ¿Cuál es la diferencia entre la recompensa r_t y el retorno G_t ? ¿Cuál de los dos quiere maximizar el agente?
2. Con $\gamma = 0.99$, ¿cuánto pesa en el retorno una recompensa de +1.0 que llega dos pasos después? ¿Y con $\gamma = 0.5$?
3. En el objetivo de Bellman, ¿qué papel cumple el factor $(1 - \text{done})$ y por qué desactiva el término descontado al final del episodio?

EJERCICIO

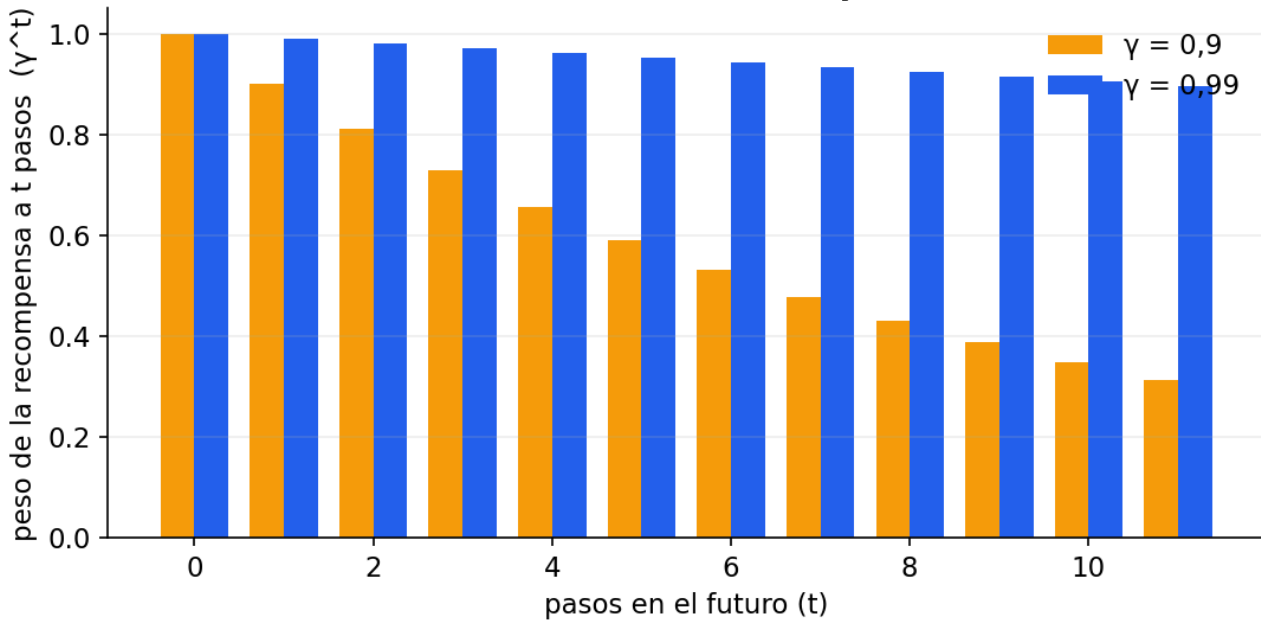
El agente está ante dos jugadas. La jugada A da $r_0 = 2$ y nada más ($r_1 = 0$, $r_2 = 0$). La jugada B da $r_0 = 0$, $r_1 = 0$ y $r_2 = 3$ (un combo dos pasos después). Calcula el retorno G_0 de cada jugada con $\gamma = 0.99$ y di cuál elige el agente. Repite con $\gamma = 0.5$.

Pista: aplica $G_0 = r_0 + \gamma \cdot r_1 + \gamma^2 \cdot r_2$ en cada caso.

Respuesta: Con $\gamma = 0.99$: A vale 2; B vale $0.9801 \cdot 3 = 2.94$. Gana B, la jugada paciente. Con $\gamma = 0.5$: A vale 2; B vale $0.25 \cdot 3 = 0.75$. Ahora gana A, la inmediata.

Qué significa para el agente: el mismo par de jugadas conduce a decisiones opuestas según el dial γ . Un agente previsor (0.99) invierte un paso neutro para cobrar el combo; uno cortoplacista (0.5) descarta el combo por estar demasiado lejos y se queda con el premio seguro.

El retorno descontado: cuánto pesa el futuro



El descuento, visto: cada recompensa futura pesa γ^t . Con $\gamma=0,99$ el futuro pesa casi tanto como el presente (agente previsor); con $\gamma=0,9$ se desvanece rápido (cortoplacista).

Y AHORA, PARA EXPERTOS NIVEL EXPERTO

El factor de descuento γ gobierna el compromiso **sesgo/varianza** y define el **horizonte efectivo** del agente, aproximadamente

$$1/(1-\gamma)$$

de modo que con $\gamma=0,99$ el agente «mira» del orden de ~ 100 pasos hacia delante. Bajar γ acorta ese horizonte (más sesgo, menos varianza, visión más miope); subirlo lo alarga (menos sesgo pero estimaciones más ruidosas).

Además, $\gamma < 1$ cumple una función matemática: garantiza que la serie del retorno $\sum \gamma^k r_{t+k}$ **converge** aunque la trayectoria sea infinita, siempre que las recompensas estén acotadas. El descuento es, pues, doblemente útil: regula cuánto futuro importa y asegura que el objetivo está bien definido, manteniendo además controlada la varianza del estimador del retorno.

Valor y Bellman

Q, V, target y TD-error.

El agente acaba de mover la pala a la derecha. La bola sube, todavía no ha tocado nada, y el premio que ha llegado en este paso es 0. Aun así, intuimos que esa acción ha sido buena: la bola va camino de un ladrillo. ¿Cómo ponemos número a "esa acción ha sido buena" cuando la recompensa de ahora mismo es cero y la consecuencia llegara dentro de muchos pasos?

Dos formas de medir «lo bueno»: $V(s)$ y $Q(s,a)$

$V(s)$ — valor del estado

¿Qué pinta tiene esta situación, juegue como juegue bien?

$Q(s,a)$ — valor de una acción

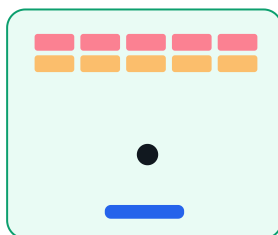
¿Qué pinta tiene HACER esta acción aquí?

← izquierda		0,42
· mantener		0,71
→ derecha		0,55

DQN aprende Q ; la política sale de elegir el Q mayor (argmax).

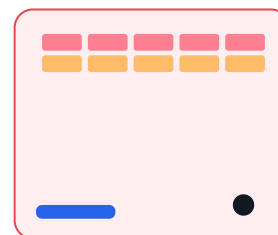
Dos maneras de medir «lo bueno»: $V(s)$ resume la situación; $Q(s,a)$ afina por acción. DQN aprende Q y actúa eligiendo el mayor (aquí, «mantener»).

El «valor» de una situación = ¿cómo de prometedora pinta?



BIEN: la pala está bajo la bola
valor alto ($\approx +4$)

La red aprende a ponerle un número a cada situación.



MAL: la bola se escapa por el lado
valor bajo (≈ -1)

Dos maneras de medir «lo bueno»: el valor del estado $V(s)$ resume cómo de prometedora es una situación, y el valor de acción $Q(s,a)$ afina esa medida para cada movimiento posible. DQN aprende Q ; la política sale de elegir el Q mayor.

Esta es la pregunta que sostiene casi toda la familia de algoritmos basados en valor (DQN y el World Model en nuestro laboratorio). La recompensa inmediata r nos dice lo que ha pasado *en este instante*, pero rara vez es lo que de verdad importa. En Arkanoid, mover la pala no rompe nada por sí mismo; rompe ladrillos varios pasos después, cuando la bola llega. Necesitamos una medida que mire más allá del paso actual y resuma **todo el futuro** que una decisión pone en marcha. Esa medida es el **valor**, y la ecuación que lo hace calculable es la de **Bellman**.

■ Dos formas de medir "lo bueno": $V(s)$ y $Q(s,a)$

Antes de la definición formal, la intuición. Imagina que pausas la partida y te preguntan dos cosas distintas. Primera: "vista esta pantalla, ¿pinta bien la cosa?" Segunda: "vista esta pantalla, ¿conviene mover a la derecha *ahora*?" La primera juzga la situación en bloque; la segunda juzga una jugada concreta dentro de esa situación. Reinforcement learning pone nombre y número a esas dos preguntas.

Valor del estado $V(s)$ El retorno que el agente espera acumular desde el estado s si a partir de ahí juega según su política π . Responde a "¿qué tan prometedora es esta situación?".

Valor de la acción $Q(s,a)$ El retorno que espera acumular si en el estado s ejecuta la acción a y luego sigue su política. Responde a "¿qué tan buena es *esta* jugada concreta aquí?".

La palabra clave en ambas definiciones es **retorno**: no la recompensa de un paso, sino la **suma descontada** de todas las recompensas futuras, $G = r + \gamma \cdot r' + \gamma^2 \cdot r'' + \dots$, con el factor de descuento $\gamma=0.99$ que ya conocemos del capítulo anterior. V y Q son *esperanzas* (medias) de ese retorno, porque el futuro tiene incertidumbre: la física de la bola y la propia política introducen variabilidad. Un mismo estado, jugado mil veces, no produce siempre el mismo retorno; el valor es la media de todos esos finales posibles.

La relación entre ambas cantidades es directa y conviene tenerla presente: el valor de un estado es el valor de la **mejor** acción disponible en el (cuando el agente actúa de forma codiciosa), es decir $V(s) = \max_a Q(s, a)$. Con tres acciones (izquierda, mantener, derecha), $V(s)$ es simplemente el mayor de los tres Q . Por eso, si tenemos buenas estimaciones de Q , jugar bien se reduce a una regla simple: en cada estado, elegir la acción con mayor Q . Toda la dificultad se traslada a *estimar* bien esos números.

TRADUCCIÓN AL JUEGO

En un tablero 8x10 con un solo ladrillo arriba a la izquierda y la bola subiendo hacia el, el estado s (los 6 cinemáticos + la matriz de ocupación) tiene un $V(s)$ alto: limpiar el nivel está cerca y trae el bonus de +5.0. Dentro de ese estado, $Q(s, \text{derecha})$ será mayor que $Q(s, \text{izquierda})$ si mover a la derecha coloca la pala bajo la trayectoria de caída. El estado es prometedor (V alto), pero no todas las acciones lo aprovechan igual (Q distintos): de eso va elegir bien.

■ El problema: no podemos simular todo el futuro

ANTES DE LA FÓRMULA

Calcular $Q(s, a)$ "por la definición" significaría, en teoría, desplegar **todas** las partidas posibles que arrancan tomando la acción a en s , sumar las recompensas descontadas de cada una y promediar. Con un timeout de hasta 7200 pasos (90 por ladrillo sobre la rejilla de 80) y una física continua, ese árbol de futuros es inabordable: no podemos simularlo. La idea de Bellman es no simular el futuro entero, sino **partirlo en dos**: la recompensa que recibimos *ahora* más el valor del estado al que *atterrizamos*. Así, estimar el valor de un estado se apoya en el valor de otro estado, un paso más adelante.

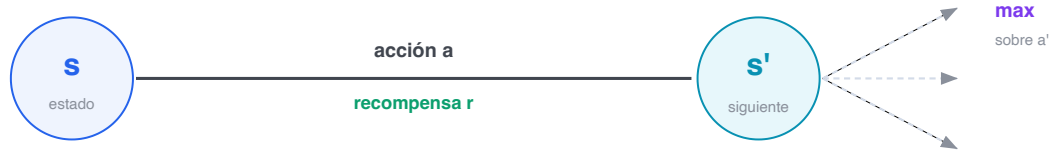
Esa descomposición recursiva es la **ecuación de Bellman**. Para el valor de acción óptimo se escribe así:

$$Q(s, a) = r + \gamma \cdot \max_{a'} Q(s', a')$$

En palabras: lo que vale tomar a en s es la recompensa inmediata r más, descontado por γ , lo mejor que se pueda hacer desde el estado siguiente s' . El valor de ahora se define con el valor del

futuro inmediato, no con el futuro completo. Hemos cambiado un problema imposible (sumar infinitos pasos) por uno tratable (mirar un paso y reutilizar una estimación ya disponible).

La idea de Bellman: descomponer el problema en 'recompensa de ahora + lo mejor que venga después'



$$Q(s, a) = r + \gamma \cdot \max_{a'} Q(s', a') \leftarrow \text{el valor de ahora se define con el valor del futuro}$$

Bellman parte el futuro en dos piezas que si sabemos manejar: la flecha del estado s al siguiente s' aporta la recompensa inmediata r ; el abanico de la derecha (el $\max$ sobre las acciones a') aporta el mejor valor alcanzable desde s' . La estimación de un estado se sostiene sobre la del estado vecino: por eso aprender valores es, en el fondo, hacer que esas estimaciones encajen entre si.

El target: lo que la red debería haber dicho

La red de DQN produce una estimación $Q(s, a)$. La parte derecha de Bellman nos da una versión **mejorada** de esa misma cantidad, porque incorpora una recompensa real r que de verdad ocurrió en el juego. A esa parte derecha la llamamos **objetivo** o **target**: es "lo que la red debería haber dicho" a la luz de lo que paso. En la practica, el target que usamos lleva dos matices respecto a la fórmula limpia de arriba:

$$\text{target} = r + \text{gamma} \cdot (1 - \text{done}) \cdot Q_{\text{objetivo}}(s', \text{argmax}_{a'} Q(s', a'))$$

El factor $(1 - \text{done})$	Apaga el futuro cuando el episodio termina. Si $\text{done}=1$ (la bola se perdio o el nivel quedo limpio), no hay s' con sentido y el termino del futuro debe desaparecer.
Q_{objetivo} (red objetivo)	Una copia estabilizada de la red que se actualiza despacio (soft update con $\text{tau}=0.01$). Evita que la red se persiga a si misma al calcular su propio target.
argmax separado del valor	Es el truco de Double DQN : la red "viva" <i>elige</i> que acción es la mejor en s' , pero la red objetivo <i>valora</i> esa acción. Separar quien elige de quien valora reduce la tendencia a sobrestimar.

No hace falta dominar todavía la red objetivo ni el Double DQN (volveremos a ellos en el capítulo de DQN). Lo esencial ahora es el esqueleto: **target = recompensa de ahora + futuro descontado, salvo que el episodio haya terminado.**

Mini-calculo: dos transiciones a mano

Pongamos números. Tomamos $\text{gamma}=0.99$ y supongamos que la estimación del mejor valor en el estado siguiente es $Q_{\text{futuro}} = 3.0$. Para aislar el efecto del factor terminal, las dos transiciones comparten la misma recompensa inmediata y solo se diferencian en done .

Caso 1 · la partida continua. El agente rompe un ladrillo, así que $r = +1.0$, y la bola sigue en juego ($\text{done} = 0$). El target es:

$$\text{target} = 1.0 + 0.99 \cdot (1 - 0) \cdot 3.0 = 1.0 + 2.97 = 3.97$$

El valor de esta jugada no es solo el $+1.0$ del ladrillo: arrastra 2.97 de "buen futuro" porque tras romperlo el agente queda en una posición prometedora. Esa es justamente la información que la recompensa inmediata, por si sola, no transmite.

Caso 2 · la bola se pierde. Mismo ladrillo roto en el último contacto útil, pero acto seguido la bola cae y se acaba la vida: $r = +1.0$ y $done = 1$. Ahora el factor $(1 - done)$ vale 0 y borra el futuro:

$$target = 1.0 + 0.99 \cdot (1 - 1) \cdot 3.0 = 1.0 + 0 = 1.0$$

Aunque la red creyera que "el futuro vale 3.0 ", ese futuro **no existe**: tras un estado terminal no hay s' que valorar. El $(1 - done)$ impide contar un futuro fantasma. Olvidar este factor es uno de los errores más costosos al implementar el bucle de aprendizaje: la red aprendería a valorar como prometedoras jugadas que en realidad acaban en perder la bola.

ERROR COMÚN

Pensar que $Q(s, a)$ es la recompensa de un paso. No: es el **retorno**, la suma descontada de todo lo que viene. En el caso 1, el target 3.97 es casi cuatro veces el $+1.0$ inmediato, precisamente porque Q mide el futuro entero, no la rodaja actual. Confundir ambos lleva a un agente que solo persigue premios inmediatos y nunca prepara jugadas.

El TD-error: la sorpresa que mueve el aprendizaje

Tenemos dos números para la misma cosa: lo que la red *dijo*, $Q(s, a)$, y lo que *debería haber dicho*, el target. Su diferencia es el **TD-error** (error de diferencia temporal):

$$\delta = target - Q(s, a) = [r + \gamma \cdot (1 - done) \cdot Q_{objetivo}(s', \cdot)] - Q(s, a)$$

La lectura intuitiva del δ es **sorpresa**. Si δ es grande y positivo, las cosas salieron mejor de lo que la red esperaba: subestimaba esa jugada. Si es grande y negativo, salieron peor: la sobrevaloraba. Si δ es casi 0 , la predicción encajaba con lo ocurrido y hay poco que aprender de esa transición. El entrenamiento consiste en empujar $Q(s, a)$ hacia el target para reducir $|\delta|$; usamos la **pérdida de Huber** sobre ese error, que se comporta como el cuadrado cerca de cero y de forma más suave (lineal) cuando δ es grande, para que una transición muy sorprendente no descontrola la actualización.

TRADUCCIÓN AL JUEGO

La red estima $Q(s, derecha) = 2.0$ para un estado en el que mover a la derecha encadena un combo de tres ladrillos sin tocar la pala. Por ese combo llega $r = +1.0 + 0.5 + 0.5 = 2.0$ (el primer ladrillo da $+1.0$ y los dos extra, $+0.5$ de bonus cada uno) y, además, el estado siguiente sigue siendo bueno. El target sale claramente por encima de 2.0 : $\delta > 0$, sorpresa agradable. La próxima vez, $Q(s, derecha)$ será mayor y esa jugada de combo se hará más atractiva. Así se propaga, transición a transición, la noción de "esto lleva a romper mucho".

PARA CURIOSOS

La ecuación de Bellman es un **punto fijo**: la función de valor verdadera es la única que satisface la igualdad para todos los estados a la vez. Aprender por TD es aplicar repetidamente el lado derecho como un *operador* que, bajo ciertas condiciones, contrae las estimaciones hacia ese punto fijo. En el caso tabular hay garantías de convergencia; con una red neuronal aproximando Q esas garantías se debilitan, y de ahí vienen parches prácticos como la red objetivo y el Double DQN que aparecen en nuestro target.

Del TD-error al replay (y a por que algunas transiciones valen más)

El delta no solo dirige *cuanto* se corrige una estimación: también sugiere *que* transiciones merece la pena repasar. Una transición con $|\delta|$ grande es una en la que el agente estaba muy equivocado, así que contiene mucha información por aprender; una con δ casi nulo ya estaba bien entendida y aporta poco al volver a verla.

Nuestro DQN guarda las transiciones en un **replay buffer** (capacidad 100.000) y entrena con lotes de 128 muestreados de él. En la versión base, ese muestreo es **uniforme**: todas las transiciones tienen la misma probabilidad de salir. La idea del **replay prioritario** es muestrear con probabilidad proporcional a $|\delta|$: repasar más a menudo lo sorprendente y menos lo ya aprendido. Aunque en nuestra receta de partida **no** lo activamos, el concepto cae por su propio peso una vez entendido el TD-error: la sorpresa es, a la vez, la señal que corrige una estimación y un criterio razonable de prioridad para decidir de que aprender primero.

QUE NO DEMUESTRA

Que el target sea "lo que la red debería haber dicho" no significa que sea la verdad. El target usa $Q_{\text{objetivo}}(s', \cdot)$, que es **otra estimación**, no un valor exacto: si esa estimación está sesgada, el target hereda el sesgo (es el bootstrapping, "aprender de tus propias conjeturas"). Por eso el aprendizaje por valor puede ser inestable y por eso existen la red objetivo y el Double DQN. Y nada de esto garantiza buen juego por sí solo: en nuestros resultados, el DQN llega al 77% en TEST-ID a 1.5M pasos (media de 5 semillas), por debajo del 91% de PPO. Bellman da el andamiaje para estimar valor; convertir buenos valores en buen juego depende, además, de toda la formulación de la tarea.

EJERCICIO

Una transición tiene $r = +0.2$ (rebote con la pala), $\text{done} = 0$, y el mejor valor estimado en s' es $Q_{\text{futuro}} = 4.0$. La red había predicho $Q(s, a) = 3.5$. Calcula el target y el TD-error con $\gamma=0.99$.

Pista: primero el target con la fórmula $r + \gamma \cdot (1 - \text{done}) \cdot Q_{\text{futuro}}$; luego resta la predicción.

Respuesta: $\text{target} = 0.2 + 0.99 \cdot 1 \cdot 4.0 = 0.2 + 3.96 = 4.16$; $\text{TD-error } \delta = 4.16 - 3.5 = +0.66$.

Para el agente: el rebote da poco premio inmediato (0.2), pero deja la bola en una posición muy prometedora (futuro 3.96); el δ positivo hará que la red valore más esa jugada la próxima vez.

Quédate con esto

$V(s)$ mide lo bueno de un estado y $Q(s, a)$ lo bueno de una acción, ambos como retorno descontado, no como premio de un paso. Bellman parte ese retorno en **recompensa de ahora + futuro descontado**, y de ahí sale el target $r + \gamma \cdot (1 - \text{done}) \cdot Q(s', \cdot)$, donde $(1 - \text{done})$ borra el futuro al terminar el episodio. La diferencia entre el target y la predicción es el **TD-error**, la sorpresa que el entrenamiento reduce y que, además, sugiere que transiciones priorizar.

AUTOCOMPROBACIÓN

1. ¿Por qué $Q(s, a)$ puede valer mucho más que la recompensa r de ese paso?
2. Con $r=+1.0$, $\gamma=0.99$ y $\text{done}=1$, ¿cuánto vale el target y por qué desaparece el termino del futuro?
3. ¿Qué indica un TD-error grande y positivo sobre la estimación previa de la red, y como conecta eso con el replay prioritario?

Bellman: un puente al futuro, no solo una fórmula

$$\text{objetivo} = r + \gamma \cdot (1 - \text{done}) \cdot \text{máx } Q(s', a')$$



El objetivo de Bellman es un puente: suma el premio de ahora (r) y lo que vale el futuro ($\gamma \cdot \text{máx } Q$). Si el episodio terminó ($\text{done}=1$), el futuro desaparece.

El TD-error: la «sorpresa» que mueve el aprendizaje

Es la diferencia entre lo que la red predijo y el objetivo de Bellman. Más sorpresa = más que aprender.



El TD-error es la sorpresa: cuánto se desvió la predicción de la red del objetivo de Bellman. Las transiciones más sorprendentes son las que más enseñan.



Y AHORA, PARA EXPERTOS NIVEL EXPERTO

El valor de un estado se rige por las **ecuaciones de Bellman**. La de esperanza relaciona el valor de un estado con el de sus sucesores bajo una política; la de **optimalidad** caracteriza el valor óptimo V^* y la función de acción-valor óptima Q^* tomando el máximo sobre acciones. El **operador de Bellman** es una **contracción** en la norma del supremo con factor γ : por el **teorema del punto fijo de Banach**, posee un único punto fijo — precisamente V^* — al que converge la iteración repetida, lo que fundamenta toda la programación dinámica y los métodos de valor.

Cuando no conocemos la dinámica, el aprendizaje por **diferencias temporales** TD(0) actualiza el valor con muestras usando el **TD-error**:

$$\delta = r + \gamma V(s') - V(s)$$

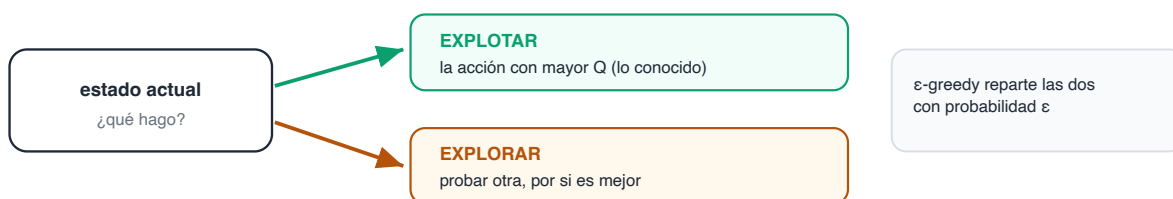
que mide la discrepancia entre la estimación actual y un blanco de un paso. Este δ es la señal de aprendizaje que reaparece, de una u otra forma, en casi todos los algoritmos modernos de RL.

Exploración y explotación

epsilon-greedy, política estocástica, entropía.

Si el agente ya tiene una opinion sobre que acción es la mejor, ¿por qué iba a probar otra cosa? Y al revés: si nunca prueba nada nuevo, ¿cómo va a descubrir que su opinion estaba equivocada?

Exploración vs explotación: el dilema de cada paso



En cada paso, el dilema: explotar la mejor acción conocida o explorar otra por si es mejor. ε-greedy reparte las dos con probabilidad ε.

Entropía = cuánta **VARIEDAD** (sorpresa) hay en sus decisiones



La entropía mide cuánta variedad guarda una política en sus decisiones: alta cuando reparte probabilidad entre varias acciones (explora), baja cuando apuesta casi todo a una (explota). Regularla es el corazón del equilibrio entre exploración y explotación.

Esta tensión recorre todo el aprendizaje por refuerzo y tiene nombre propio: el **dilema entre exploración y explotación**. **Explotar** es usar lo que ya crees que funciona, es decir, ejecutar la acción con mejor valor estimado. **Explorar** es renunciar a esa apuesta segura para probar algo distinto y obtener información. El conflicto es real porque no puedes hacer las dos cosas en el mismo paso: cada vez que el agente decide, elige entre cobrar lo conocido o investigar lo desconocido. Y como las estimaciones del principio son malas (el agente empieza sin saber nada), explotar demasiado pronto significa quedarse atrapado en una estrategia mediocre que parecía buena solo porque nunca se comparó con nada mejor.

Conviene fijar la intuición antes de cualquier fórmula: este capítulo trata de *como reparte* el agente sus decisiones entre esas dos actitudes, y de como ese reparto debe cambiar a lo largo del entrenamiento. Solo después le pondremos número (con epsilon y con la entropía).

La escena en Arkanoid

Imagina al agente en sus primeras partidas. Por puro azar, una vez movió la pala a la izquierda, reboto la bola y rompió un ladrillo: $+1.0$. Esa transición deja huella en sus estimaciones. Si a partir de ahí *solo* explotara, tendería a repetir ese movimiento que una vez salió bien, sin llegar a descubrir que mantener la pala quieta en cierta posición, o desviarla a la derecha, abrían combos mucho más largos (recuerda: un combo encadenado suma $+0.5$ extra por cada ladrillo roto sin tocar la pala). El agente no es tonto: es que **no tiene con qué comparar**. Para que sus valores $Q(s, a)$ sean fiables, cada acción tiene que haberse probado lo suficiente en cada tipo de situación. Y eso exige explorar de forma deliberada, sobre todo al principio.

TRADUCCIÓN AL JUEGO

Explotar = en el estado actual s (los 6 cinemáticos: pelota x, y , velocidad v_x, v_y , pala x y distancia pelota-pala; más la matriz 8×10), elegir la acción de las tres (izquierda, mantener, derecha) con mayor $Q(s, a)$ estimado. Explorar = elegir una de las tres al azar para ver que pasa, aunque ahora mismo no parezca la mejor. En una tarea con solo 3 acciones la exploración es barata, pero **imprescindible**: sin ella, el agente nunca sabría si la acción que descarta era en realidad la que abría el camino para limpiar el nivel ($+5.0$).

epsilon-greedy: una regla simple para repartir las dos cosas

La estrategia más común para resolver el dilema, y la que usan nuestros métodos basados en valor (DQN y el World Model), se llama **epsilon-greedy**. La idea cabe en una frase: la mayoría de las veces explota, pero de vez en cuando, con una probabilidad pequeña, explora al azar. Ese "de vez en cuando" lo controla un solo número, ϵ (epsilon), entre 0 y 1.

ANTES DE LA FÓRMULA

Lo que sigue no es más que un sorteo. Tiramos una moneda trucada que sale "explorar" con probabilidad ϵ y "explotar" con probabilidad $1 - \epsilon$. Si sale explorar, escogemos una de las 3 acciones por igual; si sale explotar, escogemos la de mayor valor estimado. Cuanto más alto sea ϵ , más a menudo el agente se desvía de su mejor apuesta para investigar.

$a = \text{acción al azar con prob. } \epsilon$; $a = \text{argmax}_a Q(s, a) \text{ con prob. } 1 - \epsilon$

El término $\text{argmax}_a Q(s, a)$ significa "la acción a que maximiza el valor estimado en el estado s ": la mejor según lo que el agente cree *ahora mismo*. La gracia de epsilon-greedy es que separa dos preguntas que conviene no mezclar: **cuanto** explorar (lo fija ϵ) y, cuando toca explotar, **qué** es lo mejor (lo decide el argmax sobre la red de valor).

TRADUCCIÓN AL JUEGO

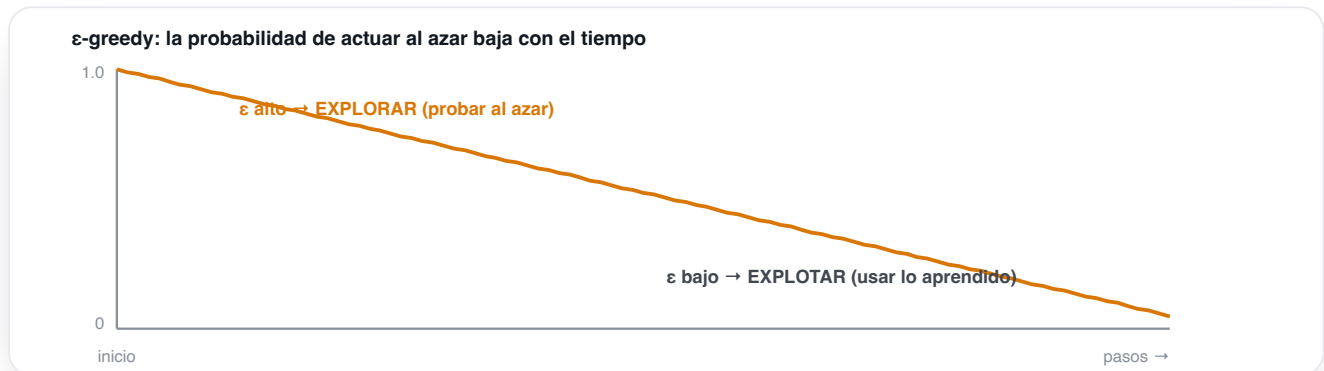
Un sorteo concreto con $\epsilon = 0.30$. El agente está en un estado s y su red estima $Q(\text{izquierda})=2.1$, $Q(\text{mantener})=0.8$, $Q(\text{derecha})=2.4$. Saca un número al azar entre 0 y 1. Si cae por debajo de 0.30, ignora esos valores y elige una de las tres acciones con probabilidad $1/3$ cada una (podría mover a la izquierda aunque su valor no sea el mayor). Si cae en 0.30 o por encima (el 70% del tiempo), aplica el argmax y juega "derecha", que es la de mayor Q . El ϵ no cambia *cual* es la mejor acción; solo decide con qué frecuencia el agente se permite no hacerle caso.

El decaimiento de epsilon: mucho al principio, casi nada al final

Un ϵ fijo seria un mal acuerdo. Si lo dejamos alto siempre, el agente nunca llega a explotar de verdad lo que aprendió: juega medio al azar para siempre y rinde por debajo de su nivel. Si lo dejamos bajo desde el principio, explota estimaciones que aun son basura y se atasca. La solución es hacer que ϵ **decaiga con el tiempo**: empezar explorando mucho, cuando no se sabe nada, e ir explotando cada vez más a medida que las estimaciones mejoran.

En nuestra configuración de DQN, ϵ arranca en 1.0 (exploración total: las primeras acciones son **todas** al azar, para llenar el replay con experiencia variada) y baja de forma lineal hasta 0.05 a lo largo de los primeros **25.000 pasos**. A partir de ahí se queda en ese suelo: el agente explota su política el 95% del tiempo, pero conserva un 5% de exploración residual para no cerrarse del todo a corregir errores.

Que esa bajada sea lenta o rápida no es un detalle menor, pero tampoco es el ingrediente que decide la tarea: en la ablación del laboratorio, pasar de un decaimiento de ϵ lento a uno rápido apenas movió el éxito (de 77% a 75%, un $\Delta -2,6$). Es un ajuste fino, no una palanca critica como si lo son, por ejemplo, la escala de la señal o el encoder convolucional, que veremos en otros capítulos.



El reparto entre explorar y explotar no es fijo: cambia durante el entrenamiento. Al inicio $\epsilon = 1.0$ y casi toda acción es un sondeo al azar (zona "explorar"); según pasan los pasos, la curva cae y el agente confía cada vez más en su mejor apuesta (zona "explotar"). En nuestra receta la bajada es lineal hasta $\epsilon = 0.05$ a los 25.000 pasos y ahí se mantiene: nunca llega a 0, para no perder ese 5% que le deja seguir descubriendo.

ERROR COMÚN

Pensar que cuando ϵ llega a su suelo el agente "ya no aprende". Son cosas distintas. ϵ solo gobierna **como elige acciones mientras entrena**; el aprendizaje ocurre aparte, ajustando la red con las transiciones del replay. De hecho, en la **evaluación** apagamos del todo la exploración: medimos con política *greedy* (siempre $\arg\max$, $\epsilon = 0$), porque ahí no queremos investigar, sino comprobar que ha aprendido. Por eso, cuando más adelante leas un porcentaje de éxito, recuerda que sale de jugar **sin** exploración.

Política determinista frente a política estocástica

ϵ -greedy nos lleva, casi sin querer, a una distinción más profunda sobre **como** una política elige acciones. Una **política determinista** asigna a cada estado *una* acción fija: dado s , siempre la misma a . El $\arg\max$ sobre $Q(s, a)$ es exactamente eso. En cambio, una **política estocástica** no devuelve una acción, sino una **distribución de probabilidad** sobre las acciones, que escribimos $\pi(a|s)$: "la probabilidad de tomar la acción a estando en s ". Al actuar, se muestrea de esa distribución, así que el agente puede elegir distinto en visitas distintas al mismo estado.

Política determinista	Función $s \rightarrow a$. Una sola acción por estado. La exploración, si se quiere, hay que añadirla por fuera (eso hace epsilon-greedy: envuelve una política determinista con un poco de azar).
Política estocástica $\pi(a s)$	Función $s \rightarrow$ distribución sobre acciones. La exploración está dentro de la propia política: si reparte algo de probabilidad entre varias acciones, ya explora al muestrear.

Esta diferencia no es académica: marca como explora cada familia de algoritmos del laboratorio. **DQN y el World Model** aprenden valores $Q(s, a)$, que son deterministas por naturaleza (el $\arg\max$), y por eso necesitan epsilon-greedy como muleta externa para explorar. **PPO y SAC**, en cambio, son métodos de política: su red *actor* emite directamente las 3 probabilidades $\pi(a|s)$, una distribución estocástica. No usan epsilon; exploran porque su política es, de salida, probabilística. Lo que necesitan es justo lo contrario: un mecanismo que **impida** que esa distribución se vuelva demasiado puntiaguda demasiado pronto. Y ese mecanismo se apoya en una medida que conviene definir con cuidado: la entropía.

La entropía: cuanta variedad hay en una decisión

La **entropía** de una política mide cuanto de **impredecible** o **variada** es su elección en un estado. Es alta cuando la probabilidad está repartida entre varias acciones (el agente "duda", explora) y baja cuando se concentra en una sola (el agente está "decidido", explota). Es la traducción numérica de la intuición de antes: una política con mucha entropía explora sola; una con poca entropía es casi determinista.

ANTES DE LA FÓRMULA

La entropía suma, sobre todas las acciones, "lo sorprendente que es cada una" ponderado por lo probable que es. Una acción casi segura (probabilidad cerca de 1) aporta poca sorpresa; una distribución repartida acumula sorpresa de varias acciones a la vez y da un total alto. El máximo se alcanza cuando todas las acciones son igual de probables: máxima indecisión, máxima variedad.

$$H(\pi) = - \sum_a \pi(a|s) \cdot \log \pi(a|s)$$

Con nuestras **3 acciones**, el valor máximo posible de la entropía es $\log(3) \approx 1,10$ (en logaritmo natural), y se da cuando las tres comparten exactamente la misma probabilidad, $1/3$ cada una. En el otro extremo, si la política pone toda la probabilidad en una sola acción, la entropía vale 0: no hay ninguna variedad, la decisión es perfectamente predecible.

PARA CURIOSOS

Esa cota $\log(3)$ es la que da sentido al objetivo de entropía de nuestro SAC discreto, fijado en $0,55 \cdot \log(3) \approx 0,60$: no le pedimos al actor que sea maximamente aleatorio (eso sería $\log(3)$ entero), sino que mantenga algo más de la mitad de esa variedad. SAC va incluso más allá y trata su peso de exploración, la temperatura α , como un parámetro **aprendible** (arranca en 0.20) que se ajusta solo para acercar la entropía real a ese objetivo. PPO usa una versión más sencilla: un **coeficiente de entropía** fijo de 0.01 que premia, en la función de pérdida, no colapsar la distribución; además, su *clip* (que mantiene el ratio de probabilidades en la banda $[0.8, 1.2]$) frena que la política cambie a saltos bruscos de una actualización a otra. Dos formas distintas de comprar exploración a cambio de un poco de recompensa inmediata.

Mini-ejemplo: dos políticas, misma mejor acción, distinta entropía

Veámoslo con números. Estamos en un estado s con las 3 acciones de siempre. Comparamos dos políticas estocásticas que coinciden en cuál creen que es la mejor acción (la **derecha**), pero difieren en cuánta probabilidad le reservan al resto.

QUE MIRAR

No mires solo cuál es la acción preferida: en las dos es "derecha". Mira el **reparto**. La política A deja casi un tercio de probabilidad a cada acción: su entropía $\approx 1,09$ roza el máximo $\log(3) \approx 1,10$, así que explora muchísimo. La política B vuelca el 90% en "derecha": su entropía cae a $\approx 0,39$, es casi determinista y apenas explora.

Política	P(izquierda)	P(mantener)	P(derecha)	Entropía H
A — exploradora	0,30	0,30	0,40	$\approx 1,09$
B — decidida	0,05	0,05	0,90	$\approx 0,39$

El cálculo de B, paso a paso, para que se vea de dónde sale el número: $H = -(0,05 \cdot \log 0,05 + 0,05 \cdot \log 0,05 + 0,90 \cdot \log 0,90)$. Con $\log 0,05 \approx -3,00$ y $\log 0,90 \approx -0,105$, los dos primeros términos valen $-0,05 \cdot (-3,00) = 0,150$ cada uno; el tercero, $-0,90 \cdot (-0,105) = 0,095$. Sumando: $0,150 + 0,150 + 0,095 \approx 0,39$. Números bajos = decisión concentrada = poca exploración.

La lectura que importa: **la entropía no te dice si la política es buena, solo cuanto varia**. La política B podría ser la correcta al final del entrenamiento (ya sabe que "derecha" es lo mejor y lo explota); o podría haberse cerrado en falso demasiado pronto, antes de comprobar las otras dos. Lo que queremos es justo la trayectoria que dibujan el decaimiento de ϵ y el objetivo de entropía: **empezar pareciendonos a A y terminar pareciendonos a B**, pero solo después de haber explorado lo suficiente como para que esa concentración este justificada.

QUE NO DEMUESTRA

Que más exploración sea siempre mejor. Explorar tiene un coste: cada acción al azar es un paso que no aprovechas para puntuar, y en Arkanoid un movimiento aleatorio en mal momento puede costarte la bola (-1.0). El arte no está en explorar mucho ni poco, sino en explorar **cuando hace falta** (al principio, cuando no sabes nada) y dejar de hacerlo cuando ya tienes estimaciones fiables. Ni ϵ alto eterno ni $\epsilon = 0$ desde el primer paso: el valor está en el **calendario** de la transición. Y ojo, este capítulo trata del mecanismo de exploración, no de cuál algoritmo gana: que PPO o SAC exploren "por dentro" no implica por sí solo que rindan más; eso depende de muchos otros ingredientes.

Quédate con esto

Todo agente vive el dilema explorar/explotar. Los métodos de valor (DQN, World Model) lo resuelven **por fuera** con epsilon-greedy: actúan al azar con probabilidad ϵ y, si no, eligen el $\arg\max$ de $Q(s, a)$; ese ϵ baja de 1.0 a 0.05 en 25.000 pasos. Los métodos de política (PPO, SAC) lo resuelven **por dentro**, con una política estocástica $\pi(a|s)$ a la que la **entropía** (máximo $\log(3) \approx 1,10$ con 3 acciones) le impide volverse determinista antes de tiempo. Distinto mecanismo, misma meta: explorar mucho cuando no se sabe nada y explotar cuando ya se sabe.

AUTOCOMPROBACIÓN

1. Con $\epsilon = 0.05$, ¿qué fracción de las acciones son al azar y cuál es por $\arg\max$? ¿Por qué no bajamos ϵ hasta 0 durante el entrenamiento?
2. Una política reparte $(1/3, 1/3, 1/3)$ entre las 3 acciones. ¿Cuánto vale su entropía y por qué es el máximo posible aquí?
3. ¿Por qué PPO y SAC no necesitan un parámetro ϵ para explorar, mientras que DQN sí?

EJERCICIO

Una política estocástica asigna $P(\text{izquierda})=0,1$, $P(\text{mantener})=0,1$, $P(\text{derecha})=0,8$. Calcula su entropía y di si esta más cerca de la política A (exploradora) o de la B (decidida) del mini-ejemplo.

Pista: usa $H = - \sum \pi_i \cdot \log \pi_i$ con logaritmo natural; $\log 0,1 \approx -2,30$ y $\log 0,8 \approx -0,22$.

Respuesta: $H = -(0,1 \cdot (-2,30) + 0,1 \cdot (-2,30) + 0,8 \cdot (-0,22)) = 0,230 + 0,230 + 0,179 \approx 0,64$. Esta del lado de la política **decidida** (B, $\approx 0,39$) y lejos de la exploradora (A, $\approx 1,09$): aunque deja una rendija a las otras dos acciones, vuelca el grueso en "derecha". Para el agente significa que ya casi ha tomado partido en este estado y apenas explora; útil al final del entrenamiento, arriesgado si todavía es pronto.

Dos formas de que el «cerebro» elija acción

DETERMINISTA: siempre la misma

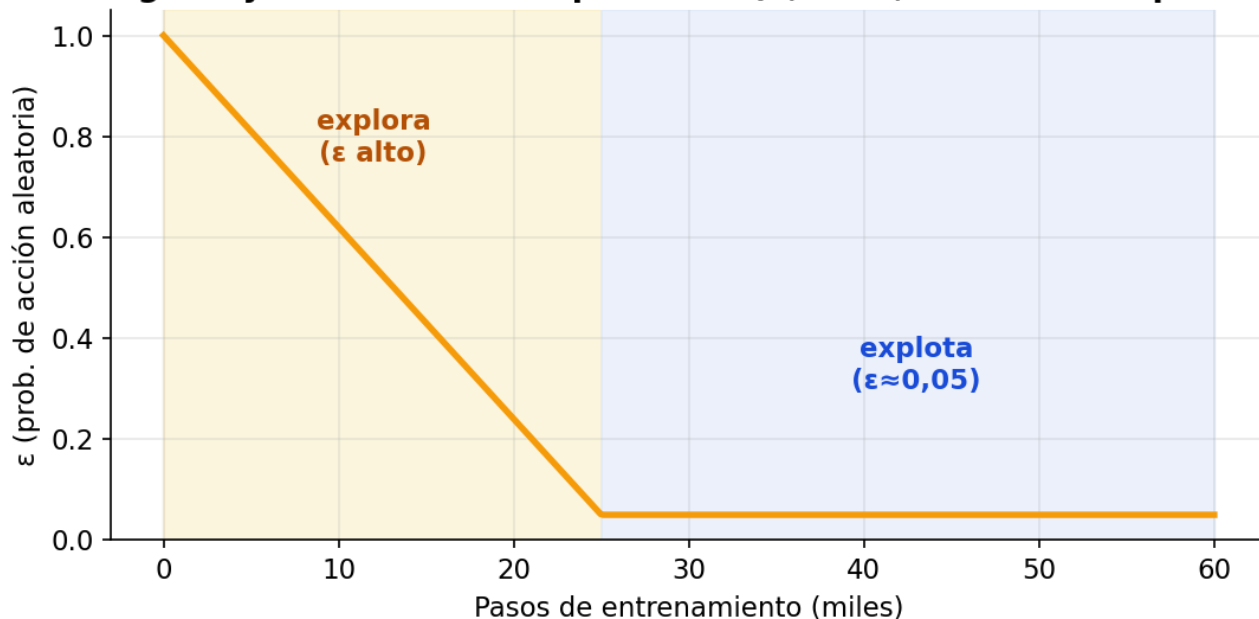


ESTOCÁSTICA: con un puntito de azar



Una política **determinista** elige siempre la misma acción en un estado; una política **estocástica** devuelve una distribución de probabilidad sobre las acciones y muestrea de ella. PPO y SAC aprenden políticas estocásticas: explorar forma parte de su propia definición.

ϵ -greedy: el horario de exploración (1,0 $\rightarrow$ 0,05 en 25.000 pasos)



El horario de exploración de ϵ -greedy: empieza explorando casi siempre ($\epsilon=1,0$) y baja a $\epsilon=0,05$ en 25.000 pasos. Mucho azar al principio, casi nada al final.



Y AHORA, PARA EXPERTOS NIVEL EXPERTO

El dilema exploración/explotación tiene formalizaciones precisas. La política **ϵ -greedy** explota la mejor acción con probabilidad $1-\epsilon$ y explora al azar con probabilidad ϵ ; su calidad se mide por el **regret** (pérdida acumulada frente a la política óptima). Para garantizar convergencia al óptimo se exige la condición **GLIE** (*greedy in the limit with infinite exploration*): explorar infinitamente pero volverse cada vez más codicioso, de modo que $\epsilon \rightarrow 0$ sin dejar de visitar todo el espacio.

Una alternativa es la **regularización por entropía**, que añade al objetivo el término

$$H(\pi) = -\sum_a \pi(a|s) \log \pi(a|s)$$

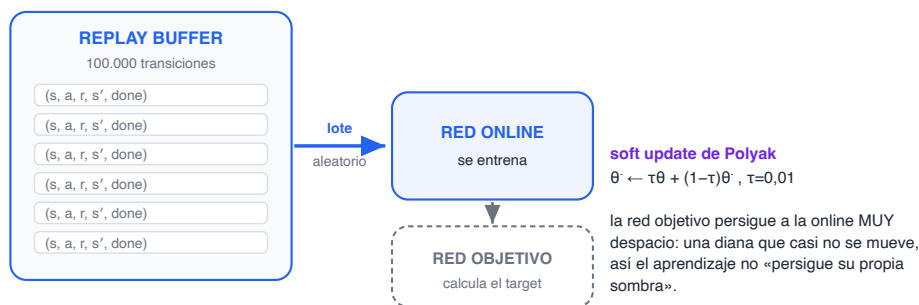
máximo cuando todas las acciones son equiprobables, lo que empuja a la política a no colapsar prematuramente. Otras estrategias clásicas son el **muestreo de Boltzmann/softmax** —probabilidad proporcional a $\exp(Q/T)$ con temperatura T — y **UCB**, que añade un bono de optimismo a las acciones poco probadas.

Replay y redes objetivo

Por que repasar estabiliza.

Un agente DQN juega una partida de Arkanoid y, paso a paso, va viendo estados muy parecidos entre si: la bola apenas se mueve un pixel, la pala sigue casi donde estaba, el mismo ladrillo sigue intacto. Si le enseñamos a la red exactamente esa secuencia, en el mismo orden en que ocurre, ¿qué pasa? Y, además, cuando calculamos el objetivo de aprendizaje con la propia red que estamos modificando, ¿no estamos disparando a una diana que se mueve cada vez que apretamos el gatillo? Este capítulo trata las dos piezas que resuelven esos dos problemas: el replay buffer y la red objetivo.

Replay buffer + red objetivo: estudiar bien y a una diana quieta



Dos piezas de estabilidad: el replay buffer baraja experiencias para romper la correlación temporal; la red objetivo fija un blanco lento (soft update $\tau=0,01$) para no perseguir la propia sombra.

REPLAY BUFFER (CONCEPTUAL) — experiencias y su recompensa
Cada experiencia es una transición (estado, acción, recompensa, siguiente). Las más informativas —una bola perdida o un ladrillo roto— tienen mayor **TD-error** (más "sorpresa" para la red) y, con replay prioritario, se muestrean con más frecuencia. **replay prioritario disponible**

	(s, a, +1, s') Ladrillo roto - TD-error 0.82	
	(s, a, +0.2, s') Rebote correcto - TD-error 0.34	
	(s, a, -1, done) Bola perdida - TD-error 0.93	
	(s, a, +5, done) Nivel completado - TD-error 0.76	

El *replay buffer* guarda las transiciones vividas y las vuelve a muestrear al azar para entrenar. Así rompe la correlación entre experiencias consecutivas y reaprovecha cada jugada muchas veces, en lugar de usarla una sola vez y tirarla.

La intuición: estudiar bien y disparar a una diana quieta

Antes de definir nada, dos escenas. La primera: imagina que preparas un examen leyendo cien veces seguidas el mismo parrafo antes de pasar al siguiente tema; te saturas de ese parrafo y, cuando llegas al final del temario, los primeros temas se te han borrado. La segunda: imagina disparar a una diana que se desplaza un poco cada vez que aprietas el gatillo, justo porque tu propio disparo la empuja. En ambos

casos el esfuerzo no cuaja. El aprendizaje DQN, sin las dos piezas de este capítulo, cae exactamente en esas dos trampas. Veamos por que.

El problema: datos correlacionados y una diana móvil

En los métodos basados en valor que aprenden *fuera de política* (DQN, SAC, World Model), la red se entrena minimizando la diferencia entre lo que predice ahora y un **objetivo** calculado con la ecuación de Bellman. La actualización es, en el fondo, un descenso de gradiente: muchos pasos pequeños sobre lotes de datos. Y el descenso de gradiente tiene dos supuestos implícitos que el aprendizaje por refuerzo, tal cual, rompe.

El primer supuesto es que los datos de cada lote son aproximadamente *independientes*. Cuando entrenas un clasificador de imágenes, barajas el conjunto y cada lote es una muestra variada. Pero las transiciones de una partida llegan **en orden temporal** y son fuertemente **correlacionadas**: el estado s de este paso se parece muchísimo al del paso anterior. Si alimentamos a la red con esa correa de fotogramas casi idénticos, durante decenas de pasos solo ve "bola arriba a la izquierda, ladrillo intacto", luego decenas de "bola bajando por el centro", y así. La red se especializa en lo último que vio y olvida lo anterior, igual que el párrafo leído cien veces.

El segundo supuesto es que el **objetivo** hacia el que ajustamos es estable, al menos durante un rato. En supervisión, la etiqueta de una imagen es fija: un gato es un gato. En DQN no: el objetivo de cada transición se calcula *con la misma red Q que estamos entrenando*. Cada vez que damos un paso de gradiente para acercar $Q(s, a)$ a su objetivo, movemos los pesos de la red y, con ellos, movemos el propio objetivo de la siguiente actualización. Es la diana móvil: cada vez que nos acercamos, se ha desplazado.

TRADUCCIÓN AL JUEGO

Imagina que el agente acaba de descubrir una secuencia que rompe un ladrillo. Sin las dos piezas de este capítulo pasarían dos cosas malas. Una: durante los siguientes cientos de pasos la red solo vería variaciones de esa misma jugada y ajustaría sus pesos hacia ese trozo concreto de partida, degradando lo que sabía de otras zonas de la rejilla 8×10 . Dos: al calcular cuanto vale el estado siguiente s' , usaría una estimación que cambia a la vez que la predicción que intenta corregir, de modo que el error nunca termina de "asentarse".

Pieza 1: el replay buffer

La idea es sencilla y potente: **no aprendas de la experiencia en el momento en que ocurre**. En lugar de eso, guárdala en una memoria grande y, para cada paso de entrenamiento, saca un lote *al azar* de esa memoria. Esa memoria es el **replay buffer** (memoria de repetición).

- | | |
|----------------------|---|
| Transición | la unidad que guardamos: la tupla $(s, a, r, s', done)$. Estado, acción tomada, recompensa recibida, estado siguiente y si el episodio terminó. |
| Replay buffer | una cola de capacidad fija (en nuestra configuración, 100.000 transiciones) que almacena las últimas experiencias. Cuando se llena, cada nueva transición expulsa a la más antigua (FIFO). |
| Muestreo | en cada paso de aprendizaje se extrae un lote de 128 transiciones elegidas al azar y uniformemente del buffer. |

El muestreo aleatorio resuelve el primer problema de raíz. Un lote de 128 transiciones sacadas al azar de un depósito de hasta 100.000 mezcla momentos de partidas muy distintas: un rebote con la pala de hace miles de pasos, un ladrillo roto reciente, una bola perdida, el arranque de un nivel casi lleno. La **correlación temporal queda rota**: el lote vuelve a parecerse a una muestra variada, que es justo lo que el descenso de gradiente necesita.

Y trae un segundo beneficio, este de eficiencia. Cada transición real cuesta interacción con el entorno (es cara), pero una vez guardada se puede **reutilizar muchas veces** en distintos lotes a lo largo del entrenamiento. Por eso decimos que estos métodos exprimen cada dato: un mismo rebote afortunado contribuye a muchas actualizaciones, no solo a una. Esto es exactamente lo que *permite* aprender fuera de política: estamos entrenando con datos generados por versiones *antiguas* de la política, no por la actual.

ERROR COMÚN

"Para que complicarse con un buffer: entreno con la última transición que acabo de vivir y listo." Eso es justo lo que NO debe hacerse en off-policy. Entrenar paso a paso con la experiencia recién generada devuelve la máxima correlación posible (cada lote es un único fotograma, casi idéntico al anterior) y tira cada dato tras usarlo una vez. La red se vuelve inestable, oscila y olvida. El lote aleatorio del buffer existe precisamente para evitar las dos cosas: la correlación y el desperdicio.

■ Pieza 2: la red objetivo

Para la diana móvil, la solución es desacoplar quien predice de quien fija el objetivo. Mantenemos **dos copias** de la red: la red **en línea** (la que entrenamos y que decide acciones) y una red **objetivo**, una copia que cambia mucho más despacio y que usamos solo para calcular el objetivo de Bellman. Así, mientras damos pasos de gradiente sobre la red en línea, el objetivo se mantiene casi quieto: la diana deja de huir a cada disparo.

ANTES DE LA FÓRMULA

El objetivo de una transición dice "cuanto debería valer hacer la acción a en el estado s ": la recompensa inmediata más, si el episodio no ha terminado, el valor descontado del mejor futuro a partir de s' . En Double DQN separamos dos roles para no sobrevalorar: la red *en línea* elige cual es la mejor acción en s' , y la red *objetivo* dice cuanto vale esa acción elegida.

$$\text{objetivo} = r + \gamma \cdot (1 - \text{done}) \cdot Q_{\text{objetivo}}(s', \arg\max_a Q_{\text{línea}}(s', a))$$

El factor $(1 - \text{done})$ apaga el término futuro cuando el episodio acaba: si en esa transición se pierde la bola o se limpia el nivel, no hay s' que valorar. Con $\gamma = 0.99$, el agente valora el futuro casi tanto como el presente, que es lo que necesita una tarea donde la recompensa grande (limpiar el nivel, +5.0) llega muchos pasos después de las decisiones que la hacen posible.

■ Cómo se actualiza la red objetivo: soft update de Polyak

¿Cada cuanto refrescamos la copia objetivo? Hay dos escuelas. La *hard update*: copiar de golpe los pesos de la red en línea a la objetivo cada N pasos. Nosotros usamos la otra, la **soft update** (actualización suave) de Polyak: en cada paso, la red objetivo da un pasito diminuto hacia la red en línea.

ANTES DE LA FÓRMULA

En lugar de copiar entero de vez en cuando, mezclamos: los pesos nuevos de la red objetivo son una media ponderada entre lo que ya tenía (casi todo) y los pesos actuales de la red en línea (una pizca). El parámetro tau es el tamaño de esa pizca.

$$\theta_{\text{objetivo}} \leftarrow \tau \cdot \theta_{\text{línea}} + (1 - \tau) \cdot \theta_{\text{objetivo}}$$

Con $\tau = 0.01$, en cada paso la red objetivo se queda con el 99% de lo que era y absorbe solo el 1% de la red en línea. Es un seguimiento lento y continuo: la diana se mueve, sí, pero tan despacio y de forma tan suave que, para la red en línea, durante muchos pasos es prácticamente fija. Obtenemos lo mejor de ambos mundos: un objetivo estable (no perseguimos la sombra) que aun así termina reflejando lo aprendido (no se queda congelado para siempre).

TRADUCCIÓN AL JUEGO

La red en línea es el jugador que está aprendiendo a leer la rejilla 8x10 y mejora a cada lote. La red objetivo es como su entrenador, que se forma una opinión serena de "cuanto vale esta posición" y solo la revisa muy poco a poco. El jugador se corrige contra el criterio tranquilo del entrenador en vez de contra su propia opinión del instante, que cambia a cada momento. El entrenador no se queda anclado: cada paso adopta un 1% de lo que el jugador va descubriendo.

Mini-ejemplo: un paso de soft update

Pongamos un único peso para ver el mecanismo con números. Supongamos que un peso de la red objetivo vale 2,00 y el mismo peso en la red en línea, tras entrenar, vale 3,00. Con $\tau = 0.01$:

$$\text{nuevo} = 0,01 \cdot 3,00 + 0,99 \cdot 2,00 = 0,03 + 1,98 = 2,01$$

Después de un paso, la red objetivo ha pasado de 2,00 a 2,01: se ha movido una centésima del camino. Si la red en línea se quedase quieta en 3,00, harían falta del orden de centenares de pasos para que la red objetivo se acercase de verdad a 3,00. Esa *inercia* es exactamente lo que buscamos: el objetivo cambia, pero a cámara lenta. Y observa que $(1 - \text{done})$ ni aparece aquí; la soft update solo mueve pesos, mientras que done actúa dentro del cálculo del objetivo de Bellman. Son dos engranajes distintos.

Idea de implementación

El bucle off-policy entrelaza tres acciones en cada paso real: *vivir* una transición y guardarla, *aprender* de un lote del buffer, y *arrastrar* la red objetivo un pelín hacia la red en línea.

PARA CURIOSOS

Esquema del paso de entrenamiento (DQN):

```
a = politica_epsilon(Q_linea, s) # acción con exploración epsilon
s', r, done = entorno.paso(a)
buffer.guardar(s, a, r, s', done)
lote = buffer.muestrear(128) # 128 transiciones al azar de hasta 100.000
y = r + gamma*(1-done)*Q_objetivo(s', argmax Q_linea(s', .))
pérdida = Huber(Q_linea(s,a), y) # Huber: robusta a errores grandes
optimizar(pérdida) # mueve theta_linea
theta_objetivo = 0.01*theta_linea + 0.99*theta_objetivo # soft update
```

En SAC el esquema es el mismo, pero con **dos** criticos y sus **dos** redes objetivo (se toma el mínimo de ambos para frenar la sobreestimación); el World Model lo reutiliza tal cual para su Q-net y le anade un modelo de dinámica aparte.

¿ Quién las usa y quién no

Estas dos piezas son el sello de los métodos **off-policy** (fuera de política), que aprenden de experiencias generadas por políticas pasadas. En nuestro banco de pruebas las usan el **DQN**, el **SAC** (con su buffer de 100.000 y sus dos redes objetivo) y el **World Model** (cuyo Q-net es identico al de DQN). No las usa el **PPO**, que es **on-policy** (sobre política): solo aprende de datos recién generados por la política actual.

QUE MIRAR

En la tabla, fijate en la columna "Buffer y red objetivo": el SI marca a los off-policy. La razón del NO de PPO no es un olvido, sino su naturaleza: tras cada rollout de 256 pasos, esos datos se usan unas pocas épocas (4 en nuestra receta) y se tiran. No hay experiencias antiguas que reaprovechar, así que un buffer no encaja; y como reentrena del todo cada poco con datos frescos, no sufre el problema de la diana móvil de la misma manera.

Algoritmo	Tipo	Buffer y red objetivo	Por que
DQN	off-policy	Si (buffer 100k, lote 128, tau 0.01)	Reutiliza datos antiguos y necesita objetivo estable.
SAC	off-policy	Si (buffer 100k, 2 redes objetivo, tau 0.01)	Igual que DQN, con doble critico para no sobreestimar.
World Model	off-policy	Si (Q-net identico a DQN)	Hereda el esquema y le suma transiciones imaginadas.
PPO	on-policy	No	Solo aprende del rollout reciente; lo descarta tras usarlo.

La lectura: replay y red objetivo no son "siempre buenos", son la respuesta concreta a dos patologías del aprendizaje off-policy. PPO esquiva esas patologías por diseño y, de hecho, en nuestros resultados a 1.5M pasos es el más fiable (91% de éxito en TEST-ID, 0% de colapsos, media de 5 semillas). Es decir: que un algoritmo no use buffer ni red objetivo no lo hace peor; simplemente resuelve la estabilidad por otra vía.

QUE NO DEMUESTRA

Estas dos piezas estabilizan el entrenamiento, pero no garantizan que el agente aprenda bien la tarea. Nuestro World Model usa exactamente el mismo Q-net con buffer y red objetivo que el DQN y aun así se queda en un techo de ~55% de éxito (TEST-ID, media de 5 semillas a 1.5M pasos): la limitación esta en otra parte (la calidad del modelo de dinámica), no en estas piezas. Buffer y red objetivo evitan que el aprendizaje *se rompa*; no deciden cuanto llega a aprender. Recuerda la tesis del libro: lo que desbloquee el problema fue formular bien la tarea, no afinar estos mecanismos.

Quédate con esto

El **replay buffer** (hasta 100.000 transiciones, lotes aleatorios de 128) rompe la correlación temporal y reutiliza cada dato. La **red objetivo** con **soft update $\tau = 0.01$** fija una diana casi inmovil para que la red en linea no persiga su propia sombra. Son el motor de estabilidad de los off-policy (DQN, SAC, World Model); PPO, al ser on-policy, no los necesita. Estabilizan el "como aprende", no el "cuanto aprende".

AUTOCOMPROBACIÓN

1. ¿Por qué entrenar con la última transición vivida, en vez de con un lote aleatorio del buffer, es mala idea en un método off-policy?
2. ¿Qué papel juega $\tau = 0.01$ en la red objetivo y que pasaria, intuitivamente, si lo subieramos mucho (cerca de 1)?
3. ¿Por qué PPO no lleva replay buffer ni red objetivo, y que tiene eso que ver con que sea on-policy?

EJERCICIO

Un peso de la red objetivo vale 5,0 y el de la red en linea vale 5,0 también (han convergido). En el siguiente paso de gradiente, el de la red en linea salta a 9,0 por una transición ruidosa. ¿Cuánto vale el peso de la red objetivo tras una soft update con $\tau = 0.01$? **Pista:** aplica $\tau \cdot \text{linea} + (1-\tau) \cdot \text{objetivo}$. **Respuesta:** $0,01 \cdot 9,0 + 0,99 \cdot 5,0 = 0,09 + 4,95 = 5,04$. La red objetivo apenas se inmuta ante el salto a 9,0: solo absorbe cuatro centesimas. *Que significa para el agente:* un golpe de ruido en la red en linea no contamina el objetivo, así que el aprendizaje no se desestabiliza por una transición atípica; la red objetivo actua de amortiguador.



Y AHORA, PARA EXPERTOS

NIVEL EXPERTO

Combinar **aproximación de funciones + bootstrapping + aprendizaje off-policy** forma la «**tríada letal**» (Sutton y Barto): juntos pueden hacer divergir las estimaciones de valor. Dos mecanismos la domestican. El **replay buffer** almacena transiciones y las muestrea al azar, rompiendo la correlación temporal entre muestras consecutivas; el **prioritized replay** (Schaul et al., 2016) las muestrea con probabilidad $\propto p_i^\alpha$ según la magnitud de su TD-error, y corrige el sesgo introducido con pesos de *importance sampling*.

El segundo mecanismo es la **red objetivo**: una copia retrasada de los parámetros que proporciona un blanco estable para el bootstrapping. Con *soft update* de Polyak,

$$\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$$

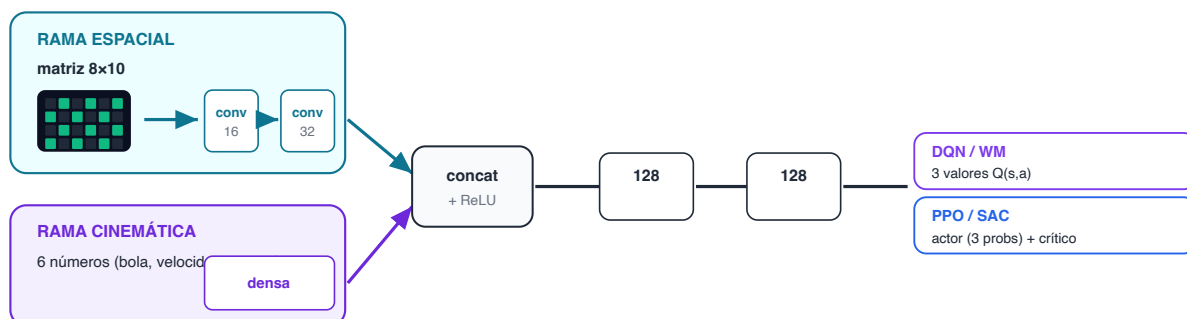
el blanco sigue a la red en línea muy despacio ($\tau \ll 1$), evitando el bucle de retroalimentación en el que la red persigue un objetivo que ella misma desplaza. Juntos, replay y red objetivo convierten un esquema potencialmente divergente en uno estable.

Redes neuronales para jugar

La red de dos ramas.

Tenemos un estado de 86 números (6 cinemáticos y una rejilla de 80 celdas) y necesitamos convertirlo en una decisión entre tres acciones. Que clase de función puede hacer ese trabajo, y por qué no nos vale tratar la rejilla de ladrillos como una simple lista de 80 cifras sueltas?

La red de dos ramas (arquitectura real)



El sesgo espacial de la convolución vale ~20 puntos de éxito (ablación).

La red de dos ramas: la matriz 8x10 entra por un encoder convolucional y los 6 cinemáticos por una rama densa; se concatenan y la cabeza cambia según el algoritmo. El sesgo espacial de la conv vale ~20 puntos.

En los capítulos anteriores hemos hablado de $Q(s, a)$, de $V(s)$ y de $\pi(a|s)$ como si fueran funciones que ya tuviéramos a mano. Pero ninguna de esas funciones nos la regalan: hay que *construirla*. El estado de Arkanoid es un vector de 86 números y la salida que queremos es pequeña, tres valores, uno por acción. Entre la entrada y la salida hace falta una máquina que aprenda a transformar la una en la otra ajustando parámetros con los datos del entrenamiento. Esa máquina es una **red neuronal**. En este capítulo la montamos desde la pieza más pequeña, la neurona, hasta la arquitectura concreta de dos ramas que usamos en el proyecto.

Conviene fijar el objetivo antes de entrar en los detalles. Una red neuronal no es más que una función con muchísimos parámetros ajustables que, partiendo de la entrada, produce una salida. Lo que la hace útil es que esos parámetros se pueden afinar automáticamente para que la salida se acerque a lo que queremos. En nuestro caso, la salida será $Q(s, a)$ para DQN y el World Model (tres valores), o un actor con tres probabilidades más un crítico con un valor para PPO y SAC. La forma de la entrada y la de la salida cambian poco entre algoritmos; lo que cambia es como entrenamos la red, y eso lo veremos algoritmo por algoritmo más adelante. Aquí nos ocupamos solo de la *arquitectura*: la forma de la máquina, no su entrenamiento.

La neurona: suma ponderada, sesgo y activación

Empecemos por la intuición. Imagina una pequeña unidad de decisión a la que le llegan varias pistas sobre la jugada y que tiene que producir un único veredicto numerico. No todas las pistas le importan

igual: a algunas les hace mucho caso, a otras poco, y alguna incluso le frena. La unidad pesa cada pista según su importancia, las junta, aplica un cierto umbral propio y emite su veredicto. Esa unidad mínima es la **neurona**. Ahora pongámosle nombres técnicos a cada paso.

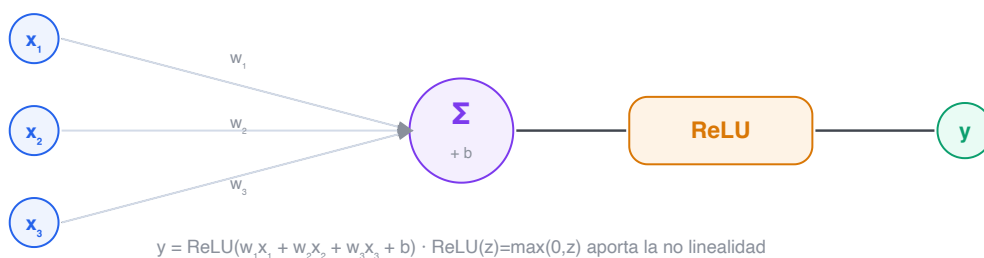
ANTES DE LA FÓRMULA

Una neurona hace tres cosas en orden. Primero, multiplica cada entrada x_i por un **peso** w_i , que mide cuanto le importa esa entrada (un peso grande y positivo: "esta entrada me empuja a activarme"; un peso negativo: "me frena"; un peso cerca de cero: "casi me da igual"). Segundo, suma todos esos productos y le anade un **sesgo** b , que desplaza el resultado hacia arriba o hacia abajo con independencia de las entradas (es el umbral propio de la neurona). Tercero, pasa esa suma por una **activación**, que es la que aporta la no linealidad y permite que la red represente relaciones que no son una simple recta.

$$y = \text{ReLU}(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

La activación que usamos en todo el proyecto es **ReLU** (de *rectified linear unit*). Su definición es de una sencillez casi decepcionante: $\text{ReLU}(z) = \max(0, z)$. Si la suma ponderada es positiva, la deja pasar tal cual; si es negativa, la corta a cero. Eso es todo. Pero ese pequeño doblez en cero es justo lo que da poder a la red, como veremos enseguida.

Una neurona: suma ponderada de sus entradas, más un sesgo, pasada por una activación



El camino de una neurona, de izquierda a derecha: las entradas x_1 , x_2 , x_3 se ponderan con w_1 , w_2 , w_3 , se suman junto al sesgo b en el nodo central, y el resultado pasa por ReLU antes de salir como y . Lo que la red *aprende* son precisamente los pesos y el sesgo: la forma del cálculo nunca cambia, solo cambian esos números. Por eso "entrenar" una red es, al pie de la letra, buscar buenos valores para todos sus pesos y sesgos.

ANTES DE LA FÓRMULA

Por que importa tanto que la activación no sea una línea recta. Si encadenamos neuronas que solo hacen sumas ponderadas (sin activación), el conjunto sigue siendo una suma ponderada: dos rectas en cascada dan otra recta. Da igual cuantas capas pongas, no ganas capacidad expresiva. La ReLU rompe ese hechizo: al cortar los negativos a cero, cada neurona se "enciende" solo en una región del espacio de entradas, y al combinar muchas neuronas que se encienden en regiones distintas la red puede aproximar funciones curvas y con esquinas. Esa es la diferencia entre poder representar "si la pelota viene por la izquierda y además queda un hueco arriba, entonces conviene moverse" y no poder pasar de una mezcla lineal de las entradas.

Hay motivos prácticos para elegir ReLU frente a activaciones más suaves: es baratísima de calcular (un simple max), su pendiente vale 1 en toda la zona positiva y eso ayuda a que la señal de entrenamiento no se desvanezca al atravesar varias capas. No es la única activación posible, pero es la que usamos en todas las redes del proyecto, tanto en las capas densas como tras cada etapa convolucional.

I De una neurona a una capa, y de una capa a un MLP

Una sola neurona da un solo número. Si ponemos muchas neuronas en paralelo, todas mirando las mismas entradas pero con sus propios pesos y sesgos, obtenemos una **capa**: un vector de salidas, cada una atenta a un aspecto distinto de la entrada. Y si apilamos capas, de modo que la salida de una sea la entrada de la siguiente, tenemos una red con varias capas. Cuando todas las capas son densas (cada neurona conectada a todas las entradas de la capa anterior) la estructura se llama **perceptron multicapa** o **MLP** (de *multi-layer perceptron*).

Capa densa	Conjunto de neuronas que reciben <i>todas</i> las entradas de la capa previa. Cada conexión tiene su peso, así que una capa densa de N neuronas sobre M entradas tiene $N \times M$ pesos más N sesgos.
Capa oculta	Cualquier capa intermedia entre la entrada y la salida. Es "oculta" porque su salida no la observamos directamente: es una representación interna que la red construye a su conveniencia.
Profundidad y anchura	La profundidad es el número de capas; la anchura, el número de neuronas por capa. En nuestra red, las dos capas densas finales tienen anchura 128.

El MLP es la herramienta natural cuando la entrada es una lista de números sin estructura espacial: precios, sensores, los seis cinemáticos de la pelota y la pala. Cada número es lo que es y su posición en la lista no significa nada geométrico. Para esa parte de nuestro estado, una capa densa es exactamente lo que toca. El problema empieza cuando intentamos meter por la misma puerta la rejilla de ladrillos.

I Por qué la matriz 8x10 no puede entrar como una lista plana

Recordemos como es esa parte del estado: una matriz de ocupación de 8 filas por 10 columnas, 80 celdas, donde un 1 es ladrillo vivo y un 0 es hueco. La tentación obvia es **aplanar** la matriz: leerla fila a fila, ponerla en una sola tira de 80 números y enchufarla a un MLP. Funciona, en el sentido de que el código corre y la red entrena. Pero tira a la basura algo valioso.

ERROR COMÚN

Pensar que aplanar la matriz "no pierde información porque están los 80 números ahí". Los números están, sí, pero se pierde la *vecindad*. Para un MLP que recibe una tira de 80 valores, dos celdas que en el tablero son contiguas (por ejemplo la primera y la segunda de una misma fila) están a priori tan relacionadas como dos celdas en esquinas opuestas: para la red, ninguna de las dos parejas tiene relación especial hasta que la aprende. La red no sabe que una celda está justo encima de otra, ni que un hueco que recorre una columna es un pasillo por donde la pelota puede colarse. Tendría que *reaprender* toda la geometría del tablero desde cero, a base de ejemplos, gastando capacidad y datos en algo que nosotros ya sabemos de antemano.

Lo importante de un tablero de ladrillos no son las celdas aisladas, sino los *patrones locales*: un borde donde acaba un grupo de ladrillos, un hueco abierto en una fila, una columna entera que sigue en pie. Esos patrones son relaciones entre celdas *vecinas*. Una representación que conserve la vecindad puede detectarlos con facilidad; una que la destruya tiene que adivinarla. Ahí es donde entra la convolución.

Qué es una convolución

ANTES DE LA FÓRMULA

Una convolución aplica un **filtro** (también llamado *kernel*): una ventanita pequeña de pesos, por ejemplo de 3×3, que se desliza por toda la rejilla. En cada posición, el filtro mira solo las celdas que tiene debajo, multiplica cada una por su peso correspondiente, lo suma todo, anade un sesgo y pasa por ReLU. El resultado es un número que dice "cuanto se parece este trocito del tablero al patron que busca este filtro". Al barrer toda la rejilla con la misma ventana, se obtiene un **mapa de rasgos**: una nueva rejilla donde cada celda indica si ese patron aparece en esa zona. La clave es que el mismo filtro, con los mismos pesos, se usa en todas las posiciones.

Formalmente, para un filtro de 3×3 con pesos w_{ij} aplicado sobre el trozo de tablero centrado en una celda, la salida en esa posición es:

$$y = \text{ReLU}(\sum_{i,j} w_{ij} \cdot (\text{celda vecina})_{ij} + b)$$

Es, fíjate, la misma operación de la neurona (suma ponderada, sesgo, ReLU), pero con dos diferencias que lo cambian todo. Primera: el filtro mira solo un *vecindario*, no toda la rejilla, así que está obligado a captar estructura local. Segunda: el filtro es el *mismo* en cada posición donde se aplica; sus pesos se comparten. Eso significa que un detector de "borde de ladrillos" aprendido en la esquina superior izquierda sirve igual de bien en el centro o en la esquina inferior derecha. La red no tiene que aprender el mismo patron 80 veces, una por celda; lo aprende una vez y lo reutiliza por todo el tablero. Esto, además de ahorrar parámetros, hace que lo aprendido en una zona valga para zonas donde durante el entrenamiento se hayan visto menos ejemplos.

TRADUCCIÓN AL JUEGO

Imagina un filtro 3×3 que ha aprendido a responder fuerte cuando ve "ladrillos a la izquierda, hueco a la derecha". Al deslizarlo por la rejilla, se enciende justo en los bordes verticales donde termina un grupo de ladrillos. Otro filtro puede especializarse en "columna vertical de ladrillos en pie", útil para detectar esos pasillos por los que conviene meter la pelota. Otro, en "hueco rodeado de ladrillos", que es una entrada a explotar. Cada filtro recorre el tablero entero y marca donde aparece su patron. Esa es justo la clase de información que el agente necesita para decidir hacia donde dirigir la pelota, y la convolución la extrae respetando la geometría del tablero.

En nuestra rama de visión encadenamos filtros en dos etapas: primero un grupo de **16** filtros sobre la matriz 8×10, y luego un grupo de **32** sobre los mapas de rasgos que produce la primera etapa. La idea de apilar es la de costumbre en redes convolucionales: la primera etapa detecta rasgos simples (bordes, huecos sueltos) y la segunda combina esos rasgos en estructuras más ricas (esquinas de un bloque, pasillos, formas del patron). Llamamos a este conjunto el **encoder convolucional**: su trabajo es resumir la rejilla de 80 celdas en un vector de rasgos que ya tiene en cuenta la vecindad.

PARA CURIOSOS

La idea de compartir pesos por toda la imagen y mirar solo vecindarios locales es la misma que hizo funcionar a la red convolucional del DQN de DeepMind (Nature, 2015), que aprendió a jugar a 49 juegos de Atari, Breakout incluido, mirando directamente los píxeles. Aquel sistema necesitó del orden de 50 millones de fotogramas por juego para llegar a su nivel. Nosotros no partimos de píxeles, sino de una matriz de ocupación ya construida (mucho más pequeña y limpia que una pantalla), pero el motivo de usar convoluciones es idéntico: en una rejilla, lo que importa es la estructura local y conviene detectarla con un detector que valga en cualquier posición.

El sesgo espacial no es gratis: vale puntos medibles

Que la convolución respete la vecindad no es un adorno teórico. En el proyecto lo medimos directamente con un experimento de ablación. Una **ablación** consiste en partir de la receta completa y quitar un solo ingrediente para ver cuanto cuesta perderlo, dejando todo lo demás igual. Partimos de la receta completa con DQN, que rinde un **77%** de éxito en el conjunto TEST-ID (evaluación greedy, media de 5 semillas), y sustituimos el encoder convolucional por una lista plana de 80 números procesada con un MLP. El resultado bajó a un **57%**: una caída de **20,4 puntos**.

QUE MIRAR

Compara las dos filas de la tabla: misma tarea, mismo algoritmo (DQN), mismo presupuesto y mismo protocolo de evaluación; la única diferencia es como entra la rejilla de ladrillos. Fíjate en el tamaño de la caída de la última columna y recuerda que NO hemos cambiado nada del aprendizaje, solo la forma de la entrada visual.

Tratamiento de la matriz 8x10	Éxito (TEST-ID)	Delta
Encoder convolucional (receta completa)	77%	—
Lista plana de 80 números (MLP)	57%	−20,4

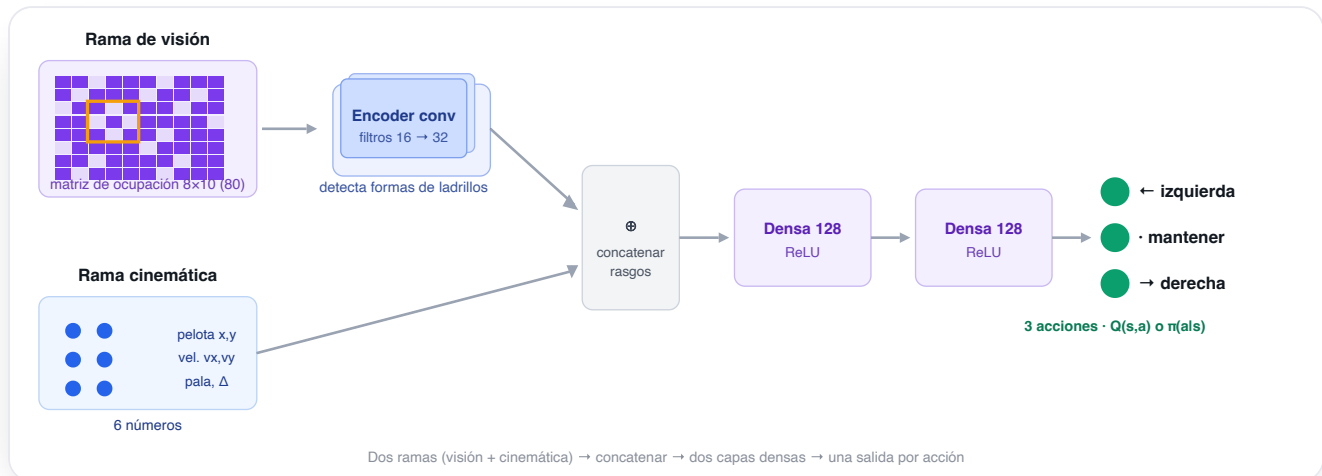
Esos veinte puntos largos son el valor del **sesgo espacial**: la información sobre la geometría del tablero que le regalamos a la red al elegir una arquitectura convolucional en lugar de una lista plana. No se la enseñamos con datos; se la damos de entrada por la forma de la red. Es uno de los ejemplos más limpios de la tesis del proyecto: el avance no vino de cambiar de algoritmo ni de entrenar más, sino de *formular* mejor la entrada.

QUE NO DEMUESTRA

Que un MLP sea "malo" o "inútil". El MLP plano sigue llegando al 57%, lejos del fracaso: aprende bastante de la tarea. Lo que el dato demuestra es algo más fino: para una entrada con estructura de rejilla, conviene una arquitectura que respete esa estructura. Tampoco demuestra que la convolución sea siempre superior; para los seis cinemáticos, que no tienen geometría, seguimos usando una rama densa, no convolucional. Y ojo: el 77% es la media de 5 semillas en TEST-ID, no una garantía sobre patrones o dificultades nunca vistos, que se miden aparte. Cada tipo de entrada pide la arquitectura que encaja con su estructura.

La red de dos ramas, pieza a pieza

Ya tenemos las dos mitades del argumento: la rejilla quiere convolución, los cinemáticos quieren densa. La solución es no forzar a ambos por el mismo camino, sino darle a cada uno la rama que le conviene y juntarlos después. Esa es la **red de dos ramas** que usamos en todos los algoritmos del proyecto.



La arquitectura completa. Arriba, la rama de visión: la matriz de ocupación 8x10 pasa por el encoder convolucional (filtros 16 → 32), con un filtro 3x3 resaltado en naranja para recordar que se desliza por la rejilla detectando formas de ladrillos. Abajo, la rama cinemática: los 6 números entran por una vía densa. Ambas se concatenan (el símbolo $\oplus$) en un solo vector de rasgos, que atraviesa dos capas densas de 128 con ReLU y termina en una salida con tres unidades, una por acción.

Lo que esa salida representa, $Q(s, a)$ o $\pi(a | s)$, depende del algoritmo; pero el cuerpo de la red es el mismo. Esa simetría es la que permite comparar algoritmos con los mismos ojos.

Sigamos el camino de un estado por la red, de la entrada a la decisión:

- 1. Rama de visión**
La matriz 8x10 entra en el encoder convolucional. Sus filtros (16 y luego 32) la recorren y producen un conjunto de mapas de rasgos que codifican que estructuras de ladrillos hay y donde. El resultado se resume en un vector de rasgos visuales.
- 2. Rama cinemática**
Los 6 números (pelota x,y; velocidad vx,vy; pala x; distancia pelota-pala) entran por una rama densa. No hay convolución aquí: son magnitudes sin geometría de rejilla, y una capa densa las procesa bien.
- 3. Concatenación**
El vector de rasgos visuales y el de la rama cinemática se pegan uno detrás de otro ($\oplus$) en un único vector. A partir de aquí, la red razona sobre los dos tipos de información a la vez: donde está la pelota y que forma tiene el tablero.
- 4. Dos capas densas de 128**
El vector concatenado pasa por una capa densa de 128 con ReLU, y luego por otra capa densa de 128 con ReLU. Aquí es donde se mezclan e interpretan conjuntamente los rasgos de las dos ramas.
- 5. Salida (3 unidades)**
Una unidad por acción: izquierda, mantener, derecha. En DQN y el World Model son los tres valores $Q(s, a)$. En PPO y SAC, el actor produce tres probabilidades $\pi(a | s)$ y, aparte, un crítico produce un valor. El tronco (ramas + concatenación + dos densas) es común; solo cambia la cabeza final según el algoritmo.

ERROR COMÚN

Creer que la red "ve la pantalla" como una persona. No ve píxeles ni colores: recibe 86 números, de los cuales 80 son ceros y unos de la matriz de ocupación y 6 son magnitudes cinemáticas. La convolución no procesa una imagen, sino esa rejilla de ocupación. Es una abstracción mucho más barata y limpia que una pantalla de juego, y por eso nos basta con una red modesta (dos etapas convolucionales y dos capas densas de 128) en lugar de las redes grandes que hacen falta para aprender de píxeles en bruto, como las del DQN de Atari.

Mini-ejemplo: un filtro 3x3 sobre un trozo de tablero

Hagamos pasar un trozo de rejilla por un filtro concreto, a mano, para ver de donde sale el número. Tomemos un filtro de 3x3 que pretende detectar "borde vertical: ladrillos a la izquierda, hueco a la derecha". Le damos estos pesos (positivos a la izquierda, negativos a la derecha) y sesgo $b = 0$:

```
w = [ +1 , 0 , -1 ; +1 , 0 , -1 ; +1 , 0 , -1 ]
```

Ahora aplicamoslo a dos trocitos distintos del tablero (recuerda: 1 = ladrillo, 0 = hueco).

Caso A, un borde claro (columna izquierda llena, columna derecha vacía):

```
parche = [ 1,1,0 ; 1,1,0 ; 1,1,0 ]
```

Multiplicamos celda a celda por los pesos y sumamos. La columna central (pesos 0) no aporta nada. La izquierda aporta $+1 \cdot 1$ tres veces = +3; la derecha aporta $-1 \cdot 0$ tres veces = 0. Total antes de la activación: +3. Y $\text{ReLU}(3) = 3$: el filtro se enciende con fuerza, justo lo que queríamos para un borde.

Caso B, sin borde (todo lleno, no hay transición):

```
parche = [ 1,1,1 ; 1,1,1 ; 1,1,1 ]
```

Ahora la izquierda aporta +3 y la derecha aporta -3; la central, 0. La suma es 0, y $\text{ReLU}(0) = 0$: el filtro no se activa, porque aquí no hay ningún borde que detectar. El mismo filtro, sin cambiar un solo peso, distingue un borde de una zona uniforme solo por como están colocados los ladrillos a su alrededor. Repetido por todas las posiciones de la rejilla, este filtro construye un mapa que marca exactamente donde están los bordes verticales del tablero. Y aquí se ve, en miniatura, por que la ReLU importa: sin ella, el caso B daría 0 igual, pero un parche con más hueco que ladrillo daría un número negativo que se mezclaría con el resto; al cortar en cero, la neurona deja de hablar cuando su patrón no está, en lugar de aportar ruido.

QUE NO DEMUESTRA

Que la red "decida" usar este filtro concreto. Los pesos de arriba los hemos elegido a mano para que el ejemplo sea legible. En el entrenamiento real, la red ajusta sola los pesos de sus 16 y luego 32 filtros para minimizar su pérdida; puede que aprenda detectores parecidos a este, o combinaciones que a nosotros no se nos ocurrirían. El ejemplo solo ilustra el mecanismo (suma ponderada local, compartida y rectificada), no el contenido exacto de lo que la red acaba aprendiendo.

La idea en pseudocodigo

Puesto en palabras de código, el paso hacia delante de la red es directo. Primero cada rama por su lado, luego la unión, luego el tronco compartido y la cabeza según el algoritmo:

```
rasgos_vision = encoder_conv(matriz_8x10) # filtros 16 -> 32, con ReLU
rasgos_cinem = densa(seis_cinematicos) # rama densa, con ReLU
z = concatenar(rasgos_vision, rasgos_cinem)
h = relu(densa128(z))
h = relu(densa128(h))
salida = cabeza(h) # 3 valores  $Q(s,a)$  o 3 probabilidades  $\pi(a|s)$  (+ critico)
```

Observa que la estructura es la misma para todos los algoritmos del proyecto; lo único que cambia es la última línea, la *cabeza*. Esa separación limpia entre un tronco común y una cabeza por algoritmo es lo que nos permite comparar DQN, PPO, SAC y el World Model en igualdad de condiciones: todos miran el estado con los mismos ojos (la misma red de dos ramas) y se diferencian en como aprenden y que producen al final.

Quédate con esto

Una neurona es una suma ponderada de sus entradas, más un sesgo, pasada por ReLU; apilando neuronas en capas densas se construye un MLP, bueno para entradas sin geometría como los 6 cinemáticos. La rejilla 8×10, en cambio, tiene estructura espacial: aplanarla a 80 números destruye la vecindad y obliga a la red a reaprender la geometría. La convolución lo evita deslizando filtros que detectan patrones locales (bordes, huecos, columnas) con pesos compartidos por todo el tablero. En la ablación, ese sesgo espacial vale unos 20 puntos (57% con MLP plano frente a 77% con encoder convolucional, en TEST-ID, media de 5 semillas). De ahí la arquitectura de dos ramas: conv sobre la matriz 8×10, densa sobre los 6 cinemáticos, concatenar, dos capas densas de 128 y una salida de tres unidades cuyo significado ($Q(s,a)$ o $\pi(a|s)$) depende del algoritmo.

AUTOCOMPROBACIÓN

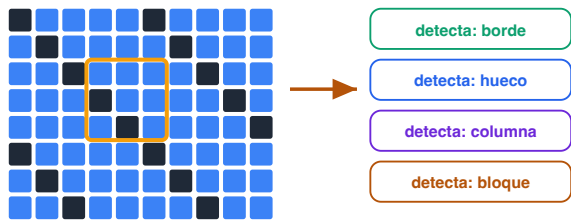
1. Que tres operaciones, y en que orden, realiza una neurona sobre sus entradas? Que aporta exactamente la ReLU que una suma ponderada sola no aporta?
2. Que información se pierde al aplanar la matriz 8×10 a una lista de 80 números, y por qué una convolución la conserva?
3. Por qué los 6 cinemáticos van por una rama densa y no por una convolucional? Que dato concreto de la ablación respalda usar convolución para la rejilla, y sobre que conjunto de evaluación se midió?

EJERCICIO

Aplica el filtro detector de borde del mini-ejemplo (pesos $[+1, 0, -1]$ en cada fila, sesgo 0) a este parche con el borde al revés, ladrillos a la *derecha* y hueco a la izquierda: $\begin{bmatrix} 0, 1, 1 \\ 0, 1, 1 \\ 0, 1, 1 \end{bmatrix}$. Cuanto vale la salida tras la ReLU? **Pista:** la columna izquierda (pesos +1) son ceros; la derecha (pesos -1) son unos; calcula la suma antes de aplicar $\max(0, z)$. **Respuesta:** la suma da $(+1) \cdot 0 \times 3 + 0 + (-1) \cdot 1 \times 3 = -3$, y $\text{ReLU}(-3) = 0$: el filtro no se enciende. Para el agente esto significa que un detector aprende una *orientación* concreta del patron; capturar el borde contrario requiere otro filtro distinto, y por eso la red dispone de varios (16, luego 32) en lugar de uno solo.

Qué es una convolución: una ventana que se desliza

Un filtro 3×3 recorre la matriz 8×10 detectando estructuras locales de ladrillos.



mismos pesos en toda la rejilla → respeta la vecindad espacial

Una convolución es un filtro 3×3 que recorre la matriz 8×10 detectando estructuras locales (bordes, huecos, columnas).
Comparte pesos por todo el tablero: por eso respeta la vecindad espacial.

Y AHORA, PARA EXPERTOS **NIVEL EXPERTO**

El **sesgo inductivo** de la convolución descansa en tres pilares: **equivarianza a la traslación** (desplazar la entrada desplaza la salida de igual modo), **weight sharing** (un mismo filtro recorre toda la imagen) y **localidad** a través de un *campo receptivo* acotado que captura vecindades espaciales. Un MLP que recibe la matriz **aplanada a 80 números** destruye esa vecindad —dos celdas adyacentes en la rejilla quedan arbitrariamente lejos en el vector— y, además, multiplica el número de parámetros al conectar todo con todo.

Esa diferencia no es teórica: en la ablación, el **encoder convolucional vale ~20 puntos de éxito** frente al MLP. La arquitectura final es de **dos ramas**: una rama convolucional procesa la disposición espacial de los ladrillos y una rama **cinemática** procesa las magnitudes continuas (posición y velocidad de la bola y la pala); sus representaciones se **concatenan** antes de la cabeza de decisión, combinando «dónde está cada cosa» con «cómo se mueve».

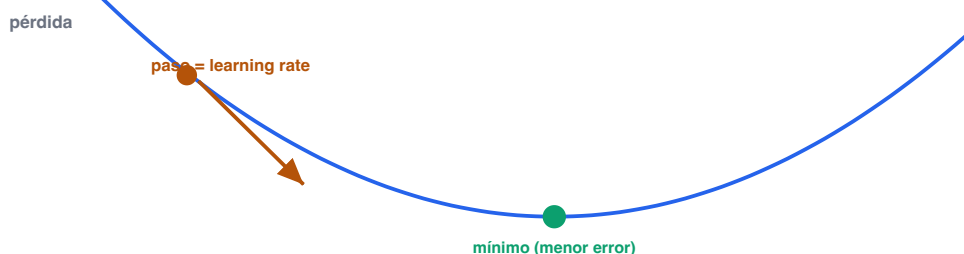
Entrenar una red

Pérdida, gradiente, learning rate.

Tenemos una red con miles de parámetros y un montón de partidas grabadas. Pero, ¿qué significa exactamente *entrenar*? ¿Cómo pasa la red de decidir al azar a apuntar la pala donde toca, si nadie le dice nunca cual era la jugada correcta?

Entrenar = bajar una cuesta a pasitos

La pérdida mide el error; el gradiente apunta cuesta abajo; el learning rate fija el tamaño del paso.



Entrenar es bajar una cuesta: la pérdida mide el error, el gradiente apunta cuesta abajo y el learning rate fija el tamaño del paso (rápido pero inestable vs. lento pero seguro).

En los capítulos anteriores montamos la red: dos ramas (la matriz 8x10 por un encoder convolucional, los 6 cinemáticos por una rama densa) que se concatenan, pasan por dos capas de 128 con ReLU y acaban en una salida de 3 valores $Q(s, a)$, uno por acción (izquierda, mantener, derecha). Esa red es, por dentro, una función gigante con miles de números ajustables: los **pesos**. Entrenar es, simplemente, **buscar los pesos que hacen que la red se equivoque lo menos posible**. Este capítulo explica el mecanismo concreto de esa búsqueda y, sobre todo, una rareza importante: en aprendizaje por refuerzo la señal de "cuanto me equivoco" no baja siempre, y eso no significa que algo este roto.

La pérdida: poner número al error

Para mejorar hace falta primero medir. Imagina que el agente predice un valor para "rebotar hacia el racimo de ladrillos" y resulta que ese valor estaba muy por debajo de lo que la jugada merecía: necesitamos un número que capture ese desajuste. La **pérdida** (en ingles, *loss*) es ese único número que resume cuanto se equivoca la red **en este momento**: cuanto más alta, peor lo esta haciendo; cuanto más baja, mejor. Es la brujula de todo el entrenamiento. La red no sabe "jugar bien" en abstracto; solo sabe empujar ese número hacia abajo.

ANTES DE LA FÓRMULA

La forma más sencilla de medir el error es comparar dos cosas: lo que la red *predice* y lo que *debería* haber predicho (el objetivo, y). Si predice 3 y el objetivo era 5, se ha quedado corta por 2. Para que los errores por exceso y por defecto no se cancelen entre sí al promediar, se eleva la diferencia al cuadrado. La media de esos cuadrados sobre varios ejemplos es la pérdida.

$$L = \text{media de } (\text{predicción} - \text{objetivo})^2$$

Esta es la pérdida cuadrática, la más intuitiva. En nuestro DQN no usamos exactamente esta, sino la **pérdida de Huber**, que se comporta como la cuadrática cuando el error es pequeño pero crece de forma más suave (lineal) cuando el error es grande. La idea de fondo es la misma —medir la distancia a un objetivo— pero Huber evita que una sola transición muy rara, con un error enorme, pegue un tirón desmesurado a los pesos. En una tarea con recompensas esporádicas como Arkanoid (+1, 0 por romper un ladrillo, -1, 0 por perder la bola, casi siempre cero entre medias), donde de vez en cuando aparece un valor atípico, esa robustez ayuda a no desestabilizar la red.

Descenso de gradiente: bajar la cuesta a pasitos

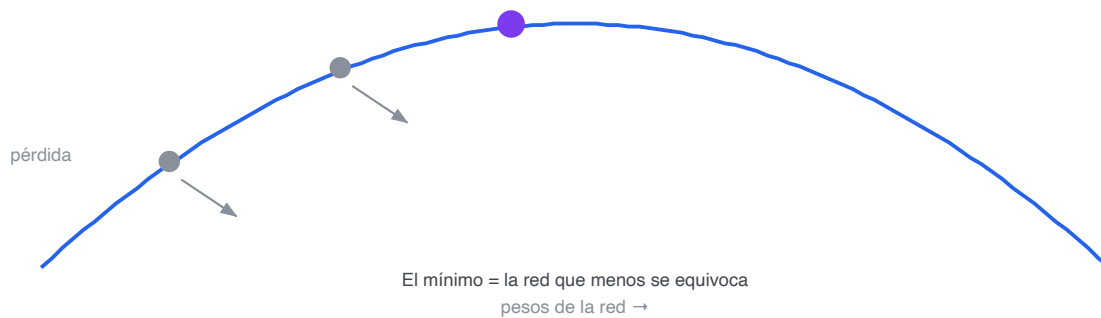
Ya tenemos un número que queremos minimizar. ¿Cómo? Imagina la pérdida como un paisaje montañoso: cada punto del suelo es una combinación concreta de pesos, y la altura en ese punto es lo que se equivoca la red con esos pesos. Entrenar es **bajar a un valle**: encontrar la zona baja donde el error es mínimo.

El truco es que no necesitamos ver el mapa entero (sería imposible: hay miles de dimensiones, una por peso). Basta con una información local: estando de pie en un punto, ¿hacia donde sube más la pendiente? Esa dirección de máxima subida la da el **gradiente**; nosotros caminamos justo en sentido contrario, cuesta abajo. Repetir ese gesto —mirar la pendiente, dar un paso hacia abajo, volver a mirar— es el **descenso de gradiente**, el motor de prácticamente todo el aprendizaje profundo.

QUE MIRAR

En la figura, el eje horizontal representa los pesos de la red y el vertical la pérdida; la curva es el "paisaje" de error. Fijate en los dos puntos grises, situados arriba en la curva (pesos malos, error alto): de cada uno sale una flecha que apunta *cuesta abajo*, en sentido contrario al gradiente. Sigue mentalmente esas flechas hasta el punto morado del fondo del valle: es el mínimo, los pesos con los que la red menos se equivoca. La clave está en que la red nunca ve la curva entera; en cada punto solo conoce la pendiente local y decide su siguiente paso con ella.

Entrenar = descenso de gradiente: cada paso ajusta los pesos para bajar un poco la pérdida



Entrenar como descenso por una cuesta. Cada punto del eje horizontal es una configuración de los pesos; la altura es la pérdida. Partiendo de posiciones malas (puntos grises, arriba en la curva), cada paso ajusta los pesos en la dirección que baja la pendiente, acercándose al mínimo (punto morado): la red que menos se equivoca. La red nunca ve la curva entera; en cada punto solo conoce la pendiente local y da un paso hacia abajo. El dibujo simplifica a un solo eje de pesos; en la red real son miles de ejes a la vez.

TRADUCCIÓN AL JUEGO

Un paso de gradiente en nuestro DQN no es magia: la red mira un lote de transiciones grabadas, calcula que valor $Q(s, a)$ predice para cada una, lo compara con el objetivo de Bellman $r + \gamma * (1 - \text{done}) * Q - \text{objetivo}(s', \arg\max_a Q(s', a))$, mide la pérdida de Huber y empuja un poquito todos los pesos para que la próxima vez su $Q(s, a)$ quede algo más cerca de ese objetivo. Tras millones de pasitos así, "rebotar la bola hacia el racimo de ladrillos" acaba teniendo un Q más alto que "irse al rincón vacío".

Learning rate: el tamaño del paso

Saber *hacia donde* bajar no basta; falta decidir *cuanto* avanzar en cada paso. Ese tamaño de paso es el **learning rate** o tasa de aprendizaje, y es uno de los ajustes más delicados de todo el entrenamiento.

Learning rate grande Pasos largos. Se avanza rápido al principio, pero se corre el riesgo de pasarse de largo el valle, rebotar de una ladera a la otra y no asentarse nunca. En el caso extremo, el error crece sin freno: el entrenamiento **diverge**.

Learning rate pequeño Pasos cortos. Mucho más estable y controlado, pero lento: pueden hacer falta muchísimas más actualizaciones para llegar al valle, y a veces uno se queda atascado antes de tiempo en una hondonada que no es la más profunda.

No hay un valor "correcto" universal: el bueno depende de la tarea, de la arquitectura y de los demás ingredientes. La lección práctica es que el learning rate gobierna el equilibrio entre **velocidad y estabilidad**, y conviene tratarlo con respeto. En el capítulo de DQN veremos, con resultados reales de nuestro propio barrido, como un ritmo demasiado rápido puede alcanzar el pico más alto y a la vez producir el peor resultado de la tanda: justo el síntoma de pasos demasiado largos que aciertan un instante y luego se desestabilizan.

Batch: aprender de un punado, no de uno solo

¿Cuántos ejemplos miramos para calcular cada paso? Si usamos una sola transición, la pendiente que estimamos es muy ruidosa: esa transición concreta puede ser un caso raro y mandarnos en una dirección engañosa. Si usáramos todas las transiciones guardadas a la vez, la estimación sería muy precisa pero carísima de calcular en cada paso.

El termino medio es el **batch** o lote: un grupo de transiciones que se procesan juntas para estimar el gradiente promedio. En nuestro DQN el lote es de **128 transiciones**, muestreadas al azar del *replay buffer* de **100.000**. Promediar sobre 128 ejemplos suaviza el ruido —los casos raros se diluyen— sin que cada paso cueste una fortuna. Además, al muestrear aleatoriamente del buffer rompemos la correlación entre experiencias consecutivas (dos pasos seguidos de una misma partida se parecen demasiado), lo que estabiliza aun más el aprendizaje.

La rareza del RL: la pérdida no tiene por que bajar siempre

Aquí llega la parte contraintuitiva, y la más importante de este capítulo. En el aprendizaje supervisado clásico —reconocer una foto de un gato— el objetivo y esta fijo: la etiqueta "gato" no cambia nunca. La pérdida debería, salvo ruido, ir bajando de forma más o menos sostenida; si sube y se queda arriba, algo va mal.

En nuestro DQN **el objetivo no esta fijo: se mueve**. Mira otra vez el objetivo de Bellman $r + \gamma * (1 - \text{done}) * Q - \text{objetivo}(s', \dots)$. Ese objetivo se calcula usando la *propia* estimación de valor de la red (a través de la red objetivo). A medida que la red aprende y cambia sus pesos, cambia también lo que considera el objetivo. Es como intentar acertar a una diana que se mueve cada vez que disparas: aunque dispaes mejor, la distancia al centro puede oscilar, porque el centro se ha desplazado. Por eso usamos una **red objetivo** con *soft update* ($\tau = 0.01$): mover la diana despacio, no congelarla del todo. Pero por mucho que la frenemos, la diana sigue moviendose.

ERROR COMÚN

Crear que "la pérdida sube = entrenamiento roto". En aprendizaje por refuerzo, la curva de pérdida sube, baja, da picos y se mueve constantemente, **y eso es normal**: el objetivo de Bellman se desplaza con la propia red. La pérdida del RL no mide "lo bien que juega el agente"; mide "lo lejos que esta la predicción de un objetivo que además se esta moviendo". Una pérdida con altibajos puede convivir perfectamente con un agente que cada vez rompe más ladrillos.

Entonces, ¿qué *si* debe preocuparnos? No los altibajos, sino la **divergencia**: que la pérdida crezca y crezca sin freno, hacia el infinito, sin volver nunca. Eso si señala que algo se ha desbocado (a menudo, un learning rate demasiado alto o un objetivo inestable). La salud del entrenamiento en RL no se lee en que la pérdida baje siempre, sino en dos cosas: que la pérdida **no diverja** (se mantenga acotada, aunque oscile) y que la **recompensa o el éxito por episodio** tiendan a subir. Esa segunda señal —cuantos ladrillos rompe, si limpia el nivel— es la que de verdad nos dice si el agente esta aprendiendo a jugar.

ERROR COMÚN

Confundir **pérdida** con **éxito**. Son cosas distintas y a veces van en direcciones contrarias durante tramos del entrenamiento. La pérdida vive en el mundo interno de la red (distancia a su objetivo móvil); el éxito vive en el juego (¿ha limpiado el nivel?). Cuando este libro reporta resultados, lo hace siempre en terminos de éxito y recompensa sobre partidas, evaluadas en modo *greedy* y sobre niveles no vistos en entrenamiento, no en terminos de pérdida.

PARA CURIOSOS

El gradiente se calcula con *retropropagación* (backpropagation): la regla de la cadena del calculo, aplicada capa a capa de la salida hacia la entrada, reparte la "culpa" del error entre todos los pesos de la red. En la practica no programamos esas derivadas a mano: el framework (en nuestro caso PyTorch sobre MPS, Apple M3 Ultra) las calcula automáticamente por diferenciación automática. Y rara vez se usa el descenso de gradiente "puro": en su lugar van optimizadores como Adam, que adaptan el tamaño de paso de cada peso por separado según el historial de gradientes, lo que suele hacer el entrenamiento más robusto a la elección del learning rate inicial.

Mini-ejemplo: un paso a mano

Hagamos un paso de descenso de gradiente con números de juguete para verlo concreto. Supongamos una transición en la que la red predice un valor $Q(s, a) = 2,0$, y el objetivo de Bellman para esa transición sale $y = 3,0$ (por ejemplo, porque en ese paso se rompio un ladrillo y eso aporta su $+1,0$).

1. **Error** predicción – objetivo = $2,0 - 3,0 = -1,0$. La red se ha quedado *corta*: valora menos esa acción de lo que el objetivo sugiere.
2. **Pérdida** $(-1,0)^2 = 1,0$.
(cuadratica,
para el
ejemplo)
3. **Dirección** como nos hemos quedado cortos, el gradiente nos dice que hay que **subir** ese $Q(s, a)$ para acercarlo a $3,0$.
4. **Paso** con un learning rate pequeño, ajustamos los pesos de forma que el nuevo $Q(s, a)$ pase de $2,0$ a, digamos, $2,1$. No saltamos directamente a $3,0$: damos un *pasito* en la buena dirección.

¿Por qué no saltar de golpe a $3,0$? Por dos razones que ya conocemos. Primera, ese $3,0$ es un objetivo **móvil**: la siguiente vez, cuando la red objetivo se haya desplazado un poco con el *soft update*, puede valer otra cosa, así que perseguirlo de golpe seria perseguir ruido. Segunda, los mismos pesos afectan a la predicción de *muchisimas* otras transiciones del lote; un salto brusco para acertar esta arruinaría las demás. Por eso se avanza a pasitos pequeños y promediando sobre las 128 transiciones del lote: cada actualización es un consenso prudente, no una corrección fanatica de un solo caso.

QUE NO DEMUESTRA

Que la pérdida baje (o se mantenga acotada) **no demuestra** que el agente juegue bien. Demuestra que la red es coherente consigo misma: que sus predicciones encajan con sus propios objetivos. Una red puede tener una pérdida bajisima y aun así jugar mal, si lo que ha aprendido a predecir de forma consistente es una política mediocre. La prueba real del aprendizaje no esta en la curva de pérdida, sino en la evaluación sobre niveles —y, para que cuente de verdad, sobre niveles **no vistos** en entrenamiento.

Quédate con esto

Entrenar una red es bajar una cuesta: la **pérdida** mide el error, el **gradiente** indica la dirección cuesta abajo, el **learning rate** fija el tamaño del paso (rápido pero inestable vs. lento pero estable) y el **batch** promedia varios ejemplos para que el paso no sea ruidoso. En aprendizaje por refuerzo el objetivo se mueve con la propia red, así que la pérdida **no tiene por que bajar siempre**: lo importante es que **no diverja** y que la recompensa por episodio tienda a subir. Pérdida no es lo mismo que éxito.

AUTOCOMPROBACIÓN

1. ¿Qué mide la pérdida y que mide el gradiente? ¿Por qué necesitamos los dos para dar un paso de entrenamiento?
2. ¿Por qué en nuestro DQN la pérdida puede subir sin que el entrenamiento este roto? ¿Qué señal SI indicaría que algo va mal de verdad?
3. Un compañero sube un learning rate muy alto y ve que la recompensa empeora y la pérdida se dispara hacia arriba sin volver. ¿Qué esta pasando y como lo llamariamos?

EJERCICIO

Una transición tiene predicción $Q(s, a) = 0,5$ y objetivo de Bellman $y = -0,5$ (en ese paso se perdió la bola, con su $-1,0$). Calcula el error y la pérdida cuadrática, y di en que dirección debe moverse $Q(s, a)$ tras el paso. **Pista:** error = predicción – objetivo; pérdida = error al cuadrado. **Respuesta:** error = $0,5 - (-0,5) = 1,0$; pérdida = $1,0^2 = 1,0$; como la red se ha pasado *por arriba* (predice más valor del que el objetivo justifica), el paso debe **bajar** ese $Q(s, a)$ hacia $-0,5$, sin saltar de golpe. Para el agente esto significa aprender que la acción que llevo a perder la bola en ese estado vale menos de lo que creía, y por tanto la elegira con menos ganas en situaciones parecidas.



Las cuatro señales que el laboratorio vigila durante el entrenamiento: **recompensa/éxito** por episodio, la **pérdida** (Huber), el tamaño del **replay buffer** y el decaimiento de ϵ . Aquí están vacías —antes de entrenar—; se van llenando paso a paso.



Y AHORA, PARA EXPERTOS NIVEL EXPERTO

Entrenar la red es **descenso de gradiente estocástico** sobre minilotes, normalmente con **Adam**, que adapta el paso por parámetro a partir de estimaciones de los **momentos** primero y segundo del gradiente (media y varianza), lo que estabiliza y acelera la optimización. Los gradientes se obtienen por **backpropagation**, es decir, la regla de la cadena aplicada capa a capa hacia atrás.

Hay, sin embargo, una diferencia esencial respecto al aprendizaje supervisado: en RL el **blanco no es estacionario** —se mueve con la red objetivo y con la propia política que cambia los datos que recoge—, de modo que la **pérdida no es un proxy fiable del rendimiento**: puede subir, bajar u oscilar sin que ello indique que el agente mejora o empeora. Lo que de verdad importa monitorizar es doble: que el entrenamiento **no diverja** y que el **éxito por episodio suba**. La curva de pérdida informa de la estabilidad numérica; la curva de éxito informa del aprendizaje real.

DQN — Deep Q-Network

Aprender el valor de cada acción.

¿Cómo decide una máquina, mirando la pantalla de Arkanoid, si le conviene más moverse a la izquierda, quedarse quieta o ir a la derecha? ¿Y cómo aprende a hacerlo bien sin que nadie le diga nunca cuál era la jugada correcta?

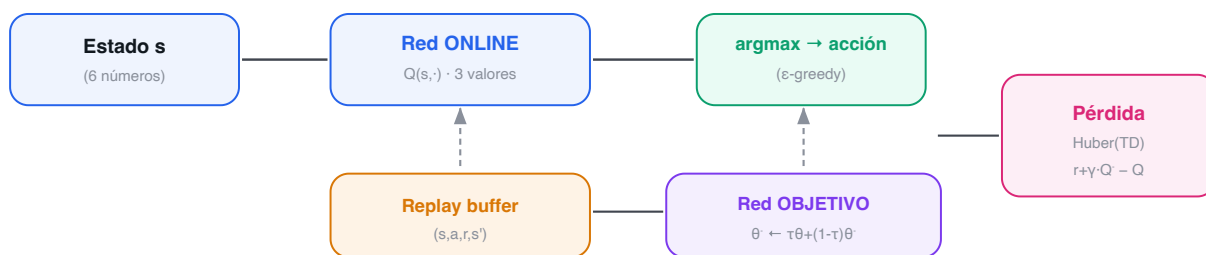
DQN: el ciclo de entrenamiento, paso a paso



El ciclo de DQN: jugar, guardar la transición, muestrear un lote, calcular el objetivo de Bellman y la pérdida, actualizar la red online y, despacio, la objetivo. Aquí encajan replay y red objetivo.

Empezamos el recorrido por los algoritmos con el más antiguo de los tres y, a la vez, el más instructivo: **DQN**, abreviatura de *Deep Q-Network*. Es el cimiento. Si entiendes a fondo cómo piensa DQN, los otros dos métodos que veremos después serán variaciones sobre un tema que ya te resultará familiar. Por eso este capítulo es deliberadamente lento y detallado: vamos a abrir el motor y mirar cada pieza.

DQN: la red online decide y se entrena; la red objetivo da un blanco estable



Esquema general de DQN: el estado entra por la izquierda, la red lo procesa y a la salida aparecen tres números, uno por cada acción posible. El agente se queda con la acción del número más alto.

La escena: tres puertas y ninguna etiqueta

Imagina que paras el juego en un fotograma cualquiera. La bola baja por la zona derecha del tablero, con cierta velocidad, y tu pala está más bien a la izquierda. Tienes tres puertas delante: *mover a la izquierda*, *mantener*, *mover a la derecha*. Una de ellas conduce, varios segundos después, a interceptar la bola y seguir rompiendo ladrillos. Las otras dos, probablemente, a perder la bola por abajo.

Aquí está la dificultad de fondo, la que hace que esto no sea un problema de los que se resuelven con una receta fija. **Nadie te dice cuál era la puerta buena.** No hay un profesor que, fotograma a fotograma, susurre «la correcta era la derecha». El único mensaje que recibe el agente es la recompensa, y además llega tarde: rompes un ladrillo varias jugadas *después* de haber empezado a mover la pala en la dirección acertada. El agente tiene que aprender a repartir el mérito de esa recompensa entre todas las decisiones que la hicieron posible. A eso lo llamamos el **problema de la asignación de crédito**, y resolverlo es justo lo que hace especial al aprendizaje por refuerzo frente a un clasificador corriente.

TRADUCCIÓN AL JUEGO

En Arkanoid no existe el «movimiento correcto» en abstracto. Mover a la derecha es magnífico si la bola va hacia la derecha y nefasto si va hacia la izquierda. El acierto depende del *estado*: de dónde está la bola, hacia dónde se mueve y dónde tienes la pala. DQN no aprende una regla del tipo «si pasa X, haz Y», sino algo más sutil: aprende a *valorar* cada puerta en cada situación.

■ Aprender por valor: la función Q

DQN pertenece a la familia de los métodos **basados en valor**. La idea central es que, en lugar de aprender directamente qué hacer, el agente aprende a estimar **cuánto vale** cada cosa que podría hacer. Si sabes poner precio a tus opciones, decidir es trivial: eliges la más cara.

Ese «precio» tiene nombre propio. Lo llamamos la **función de valor de acción**, o función Q, y se escribe $Q(s, a)$. Conviene leerla con calma, despacio, porque toda la asignatura cuelga de ella.

Estado s	La fotografía de la situación actual del juego: dónde está la bola, su velocidad, la posición de la pala y qué ladrillos siguen vivos. Es lo que el agente «ve».
Acción a	Una de las decisiones disponibles. En nuestro Arkanoid hay exactamente tres: izquierda, mantener, derecha.
Función Q $Q(s, a)$	Una estimación del <i>retorno</i> total que cabe esperar si, estando en el estado s , tomamos la acción a y a partir de ahí seguimos jugando lo mejor que sabemos. No es la recompensa inmediata: es la suma de todas las recompensas futuras, descontadas en el tiempo.

Subrayemos lo importante de esa definición, porque es donde tropieza casi todo el mundo al principio. $Q(s, a)$ **no** es «cuántos puntos me da esta acción ahora mismo». Es una promesa sobre el futuro: «si tomo esta acción y luego sigo jugando bien, terminaré acumulando aproximadamente *esto*». Por eso una acción puede tener un valor Q alto aunque su recompensa inmediata sea cero: mover la pala hacia la bola no da puntos en ese instante, pero abre la puerta a romper ladrillos dentro de unas jugadas, y ese valor futuro se refleja en su Q.

ANTES DE LA FÓRMULA

El «retorno» no es la suma pelada de recompensas futuras, sino una suma *descontada*: las recompensas lejanas pesan menos que las cercanas. El factor que las encoge se llama γ (gamma) y en nuestro laboratorio vale 0,99. Con ese valor, una recompensa dentro de cien pasos todavía cuenta bastante, pero una que llegue dentro de mil ya casi se ha desvanecido. Gamma fija el horizonte: cuán lejos mira el agente.

Con esto ya podemos escribir la definición formal del retorno G_t a partir del instante t :

$$G_t = r_t + \gamma \cdot r_{t+1} + \gamma^2 \cdot r_{t+2} + \gamma^3 \cdot r_{t+3} + \dots$$

Y la función Q óptima es, sencillamente, el retorno que esperamos obtener empezando por la acción a en el estado s y comportándonos de forma óptima después. El verbo clave es «esperamos»: como el juego tiene incertidumbre, hablamos de valor *esperado*, un promedio sobre todo lo que podría pasar.

Por qué «la mejor acción es $\operatorname{argmax} Q$ »

Aquí llega la parte elegante. Si de verdad tuviéramos la función Q perfecta —una que conociera el valor real de cada acción en cada estado—, jugar de forma óptima sería casi insultantemente fácil. En cada estado s , calculamos los tres valores Q (uno por acción) y elegimos el más alto. Esa operación, «dame la acción cuyo valor es máximo», se escribe con el operador argmax :

$$a^* = \operatorname{argmax}_a Q(s, a)$$

Léelo así: argmax_a no devuelve el valor máximo, sino el *argumento* que lo produce, es decir, **qué acción** es la mejor. Si en un estado los tres valores son $Q(s, \text{izquierda})=2,1$, $Q(s, \text{mantener})=0,4$ y $Q(s, \text{derecha})=-0,3$, entonces argmax devuelve «izquierda», porque $2,1$ es el mayor. La política — la forma de jugar — se deduce directamente de los valores. No hace falta aprenderla aparte: *está implícita en Q* .

ERROR COMÚN

Confundir $\max Q$ con $\operatorname{argmax} Q$. El primero es el *número* más alto (en el ejemplo, $2,1$); el segundo es la *acción* que lo consigue («izquierda»). DQN usa los dos en momentos distintos: $\max$ para calcular el valor objetivo durante el aprendizaje, y argmax para decidir qué hacer cuando juega de verdad.

Por eso decimos que DQN es un método *basado en valor* y no *basado en política*: no tiene una red que diga «haz esto», tiene una red que dice «esto vale tanto», y la decisión sale de comparar valores. Es una diferencia que será importante cuando, más adelante, conozcamos algoritmos que sí modelan la política directamente.

El truco profundo: una red neuronal en lugar de una tabla

Si el número de estados fuera pequeño, podríamos guardar todos los valores Q en una tabla gigante: una fila por estado, una columna por acción. Eso es el Q-learning clásico, anterior a la era del aprendizaje profundo. Pero en nuestro Arkanoid el estado incluye una rejilla de $8 \times 10 = 80$ ladrillos que pueden estar vivos o rotos en cualquier combinación, más seis números continuos con posiciones y velocidades. El número de estados posibles es astronómico. Una tabla es inviable.

La «D» de DQN —*Deep*— es exactamente la respuesta a ese problema. En vez de una tabla, usamos una **red neuronal profunda** que aproxima la función Q . Le damos el estado a la entrada y nos devuelve los valores Q a la salida. La red no memoriza estados uno a uno: aprende patrones que *generalizan*, de modo que ante una configuración de ladrillos que no ha visto nunca puede, aun así, dar una estimación razonable. Esa capacidad de generalizar es la que nos permitirá, al final, evaluar al agente en niveles que jamás vio durante el entrenamiento.

PARA CURIOSOS

El salto histórico que dio fama a DQN fue precisamente este. En 2015, el equipo de DeepMind publicó en *Nature* un sistema que aprendió a jugar a 49 juegos distintos de Atari directamente desde los píxeles de la pantalla, con la misma red convolucional y sin conocimiento específico de cada juego, a base de ver del orden de cincuenta millones de fotogramas. Nuestro laboratorio no juega desde píxeles ni a esa escala — es un banco de pruebas mucho más modesto y controlado —, pero hereda la misma idea: aproximar Q con una red profunda.

Dos piezas heredadas: replay y red objetivo

Aproximar Q con una red trae un problema serio de estabilidad, y DQN lo domestica con dos mecanismos que ya presentamos en los fundamentos y que aquí solo recordamos brevemente, porque son imprescindibles para entender el algoritmo.

Memoria de repetición (replay)

Un gran almacén —en nuestro caso de 100.000 transiciones— donde el agente guarda sus experiencias (s, a, r, s') a medida que juega. Para entrenar, no usa la última jugada, sino un lote aleatorio de 128 experiencias sacadas de ese almacén. Mezclar el pasado rompe la correlación entre fotogramas consecutivos y estabiliza el aprendizaje.

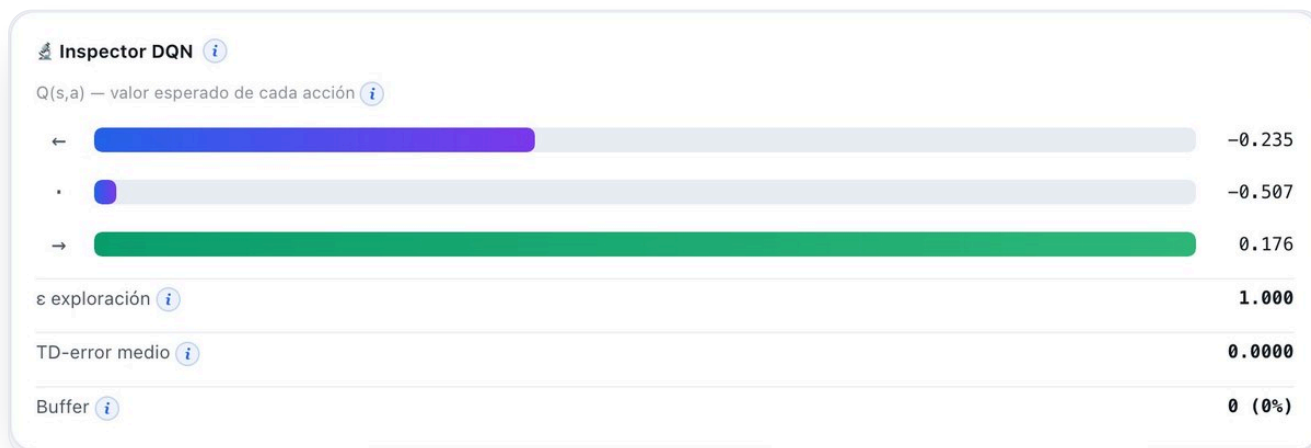
Red objetivo (target network)

Una segunda copia de la red, que cambia muy despacio, y que se usa para calcular hacia dónde debe moverse la red principal. Sin ella, la red perseguiría un objetivo que ella misma modifica a cada paso —como pintar una pared mientras alguien la repinta detrás de ti—, y el entrenamiento se vuelve inestable.

En nuestra implementación la red objetivo no se copia de golpe, sino que se actualiza con un **soft update** de Polyak: en cada paso, la copia se acerca un poquito a la red principal, con un factor $\tau = 0,01$. Es como mover un termostato un 1% hacia la temperatura deseada en cada lectura: nunca da bandazos, siempre converge suavemente. Volveremos sobre esto en el bloque para expertos.

Cómo se ve DQN en nuestro Arkanoid

Toca aterrizar todo lo anterior en la arquitectura concreta del laboratorio. Nuestra observación tiene 86 números, y no todos son del mismo tipo, así que la red los trata por separado antes de juntarlos. Tiene **dos ramas**.



El inspector de la red muestra las dos ramas: la rejilla de ladrillos entra por el encoder convolucional, los seis valores cinemáticos por la rama densa, y ambas se fusionan antes de la salida de tres valores Q.

Por un lado están los **6 valores cinemáticos**: la posición de la bola (x, y), su velocidad (v_x, v_y), la posición de la pala (x) y la distancia entre bola y pala. Son magnitudes continuas, una lista corta de números; van a una **rama densa** normal.

Por otro lado está la **matriz de ocupación 8×10** : una cuadrícula con un 1 donde queda un ladrillo vivo y un 0 donde ya está roto. Esto no es una lista cualquiera: es una *imagen* pequeña, con estructura espacial. Aquí está cerca de allí, los huecos forman formas. Para aprovechar esa estructura, la matriz pasa por un **encoder convolucional** con dos capas de filtros (16 y luego 32), igual que se hace con imágenes. La convolución reconoce patrones locales —un hueco abierto, una columna casi limpia— sin importar en qué parte del tablero aparezcan.

Las dos ramas **se concatenan** en un único vector, que atraviesa dos capas densas de 128 neuronas con activación ReLU, y finalmente desemboca en la **capa de salida con 3 valores Q**: uno por cada acción. Eso es todo. Tres números a la salida, y el agente elige el mayor.

QUÉ MIRAR

Cuando observes el inspector de la red, fíjate en que la salida tiene exactamente tres casillas, no ochenta ni una. Las ochenta entran (la rejilla); las tres salen (las acciones). Y mira la concatenación: es el punto donde «lo que veo del tablero» y «cómo se mueve la bola» se funden en una sola decisión.

Que esa rama convolucional importe de verdad no es una suposición: lo medimos. Cuando, en un experimento de ablación, sustituimos el encoder convolucional por una simple lista plana procesada con una red densa corriente, el éxito se desplomó de 77% a 57%. El sesgo espacial de la convolución vale, en nuestra tarea, unos 20 puntos. Ver el tablero *como un tablero*, y no como una ristra de ochenta números sueltos, es decisivo.

El corazón del aprendizaje: el objetivo de Bellman

Llegamos al mecanismo por el que DQN realmente aprende. La pregunta es: si la red da una estimación $Q(s, a)$ y no tenemos la etiqueta correcta, ¿hacia qué número la corregimos? La respuesta es una de las ideas más bonitas del campo, la **ecuación de Bellman**, y dice algo casi de sentido común:

$$y = r + \gamma \cdot \max_{a'} Q(s', a')$$

En palabras: el valor de tomar la acción a en el estado s debería ser igual a **la recompensa que recibo ahora (r) más el valor descontado de lo mejor que puedo hacer en el estado siguiente s'** . El futuro se descompone en «un paso ahora + todo el resto», y «todo el resto» es, otra vez, un valor Q . La función Q se define en términos de sí misma un paso más adelante. A ese morderse la cola controladamente lo llamamos **bootstrapping**: la red aprende de sus propias estimaciones.

Con ese objetivo y —que llamamos *target*— ya tenemos contra qué comparar. El error es la diferencia entre lo que la red predijo y ese objetivo, y entrenar consiste en empujar la predicción hacia y mediante descenso de gradiente. Cuando s' es un estado terminal (perdimos la bola o limpiamos el nivel), no hay futuro que valorar, así que el término del futuro se anula y el objetivo es solo r .

Mini-ejemplo numérico: una actualización paso a paso

Hagamos las cuentas con un caso inventado pero plausible, para ver el engranaje girar. Supongamos esta transición:

Estado s	La bola baja hacia la pala; quedan ladrillos.
Acción tomada a	«derecha».
Recompensa r	+0,2 (la pala interceptó la bola: bonus por rebote en pala).
Estado siguiente s'	La bola sube de nuevo hacia los ladrillos.

Supongamos que la red *actual* estima para la acción tomada un valor $Q(s, \text{derecha}) = 1,50$. Y supongamos que, evaluando el estado siguiente s' , los tres valores Q son:

Acción en s'	$Q(s', a')$
izquierda	0,90
mantener	2,40
derecha	1,10

El término del futuro usa $\max_{a'} Q(s', a')$, que es el mayor de los tres: 2,40 (la acción «mantener»). Aplicamos Bellman con $\gamma = 0,99$:

$$y = 0,2 + 0,99 \cdot 2,40 = 0,2 + 2,376 = 2,576$$

Ese es el objetivo. La red había predicho 1,50 para la acción que tomó, pero Bellman dice que debería valer en torno a 2,576. El error (la diferencia) es:

$$\text{error} = y - Q(s, \text{derecha}) = 2,576 - 1,50 = +1,076$$

El signo positivo significa que la red *infravaloraba* esa acción: en realidad llevaba a un futuro mejor de lo que pensaba. El entrenamiento ajustará los pesos para que, la próxima vez que vea un estado parecido, $Q(s, \text{derecha})$ suba acercándose a 2,576. Importante: solo se corrige la salida correspondiente a la acción *que de verdad se tomó*; las otras dos no entran en esta actualización, porque para ellas no observamos qué habría pasado.

TRADUCCIÓN AL JUEGO

Esta única actualización no «enseña a jugar». Solo refina un poquito el precio de una jugada concreta. Pero multiplica esto por millones de transiciones distintas, sacadas en lotes del replay, y la estimación de Q se va volviendo cada vez más fiel a la realidad del juego. La política óptima emerge sola, como subproducto de saber valorar bien.

Una mejora necesaria: Double DQN

El max de la fórmula de Bellman tiene un defecto conocido: tiende a **sobreestimar** los valores. Al elegir siempre el máximo de unas estimaciones que tienen ruido, sistemáticamente nos quedamos con las que el ruido infló por casualidad, y el optimismo se va acumulando. Nuestro DQN incorpora la corrección estándar, **Double DQN**: una red elige cuál es la mejor acción en s' y *otra* red dice cuánto vale esa acción. Al separar «quién elige» de «quién evalúa», se rompe el sesgo optimista. En el bloque de expertos verás la fórmula exacta.

¿Y cómo explora? El ϵ -greedy

Queda un cabo suelto. Si el agente eligiera *siempre* el argmax , al principio —cuando sus valores Q son basura, recién inicializados al azar— se quedaría atascado repitiendo tonterías sin descubrir nada mejor. Necesita explorar. La receta clásica es **ϵ -greedy**: con probabilidad ϵ el agente actúa al azar (explora), y con probabilidad $1-\epsilon$ hace caso a su $\text{argmax } Q$ (explota lo que sabe).

En nuestro laboratorio, ϵ arranca en $1,0$ (al comienzo el agente actúa totalmente al azar) y desciende hasta $0,05$ a lo largo de 25.000 pasos. Es decir: mucha curiosidad al principio, casi pura explotación al final. Conviene no confundir esto con la evaluación: cuando *medimos* el rendimiento del agente lo hacemos en modo **greedy** puro ($\epsilon = 0$), sin ninguna jugada al azar, para ver lo que de verdad ha aprendido.

ANTES DE LA FÓRMULA

Reúne todos los hiperparámetros en la cabeza antes de mirar las curvas, porque dan forma a todo lo que verás: replay de 100.000 , lote de 128 , $\gamma = 0,99$, soft update $\tau = 0,01$ y ϵ de $1,0$ a $0,05$ en 25.000 pasos. No son cifras mágicas: son las que fijamos en nuestro protocolo y mantenemos congeladas para que las comparaciones sean justas.

El resultado honesto de DQN

Ahora la parte que de verdad importa: ¿funciona? Sí, y de forma notablemente fiable. Las cifras que siguen son la **media de cinco semillas** distintas (no un golpe de suerte de una ejecución afortunada) y se midieron en modo **greedy**, con el éxito definido como *limpiar niveles que el agente nunca vio durante el entrenamiento*. Esa última condición es exigente y honesta: no premiamos memorizar, premiamos generalizar.



Curvas de entrenamiento del laboratorio. La tendencia ascendente y la ausencia de derrumbes reflejan el carácter estable de DQN; el techo lo marca el presupuesto de pasos.

Banco de pruebas	Éxito DQN (greedy, media de 5 semillas)
TEST-ID (niveles del mismo tipo)	77%
OOD-patrón (patrones nuevos)	74%
OOD-dificultad (más difíciles)	64%

Hay dos virtudes que merece la pena destacar. La primera es la **robustez**: 0% de colapsos (ninguna semilla se vino abajo durante el entrenamiento) y 80% de las semillas superaron el 80% de éxito. DQN no es brillante, pero es de fiar; rara vez te deja tirado. La segunda es que **mejora de forma monótona con el presupuesto** de pasos. Si lo dejamos entrenar más, sube de manera ordenada:

Presupuesto de pasos	Éxito TEST-ID
700.000	67%
1.500.000	77%
3.000.000	85%

Esa escalera tan limpia —67, 77, 85— es la firma de un método sano: no se atasca ni se desestabiliza, simplemente quiere tiempo. La frase que mejor resume a DQN es: **el cimiento simple y fiable, que necesita presupuesto**. Cuando un nivel limpia, lo hace en torno a los 2.125 pasos.

QUÉ NO DEMUESTRA

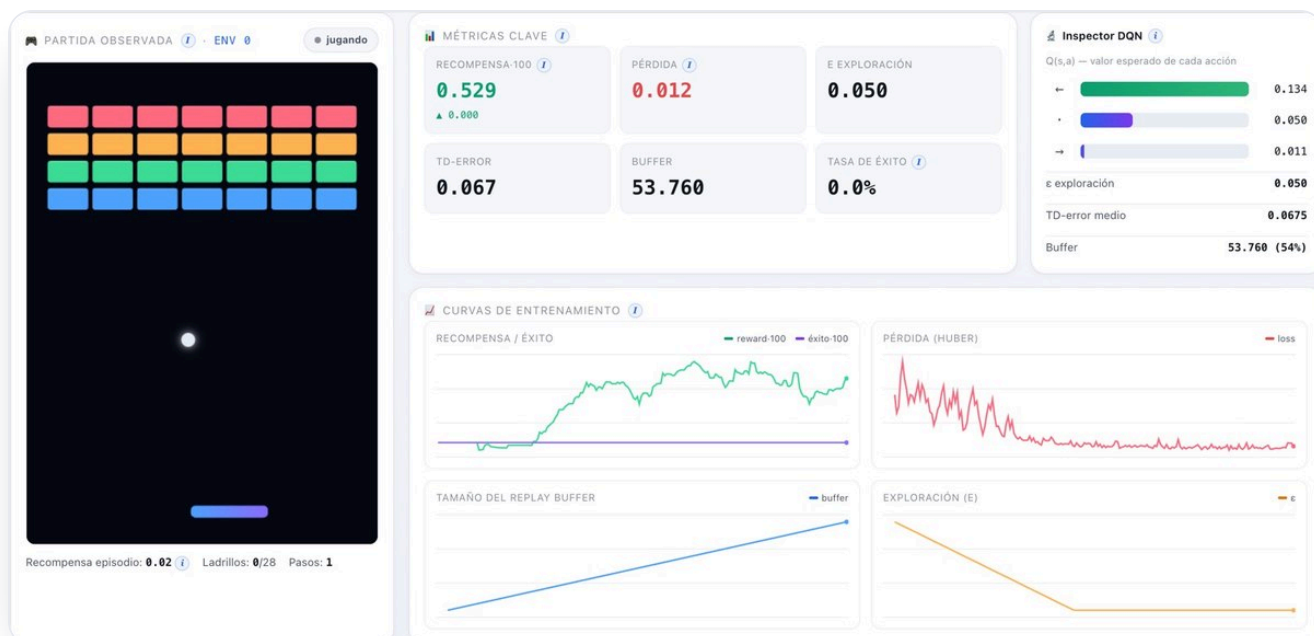
Que DQN saque 77% en TEST-ID no prueba que sea el mejor algoritmo, ni que 3M de pasos sean «suficientes». Tampoco la cifra de OOD-dificultad (64%) es comparable directamente con la de TEST-ID: son bancos de prueba distintos, con niveles de dificultad distintos. Y todo esto es *nuestro* Arkanoid con *nuestra* formulación; no son resultados de Atari ni cifras de DeepMind. Lo único que estas cifras demuestran es que, **en esta tarea bien planteada**, DQN aprende a generalizar a niveles nuevos de forma estable.

Lectura crítica: lo que de verdad desbloqueó el problema

Es tentador pensar que estos resultados vienen «de DQN». Pero la evidencia de nuestras ablaciones apunta a otra cosa, y es la tesis que recorre todo el libro. Mira qué pasa cuando, partiendo del DQN que logra 77%, estropeamos un solo ingrediente de la *formulación* de la tarea:

Qué cambiamos	Éxito resultante
DQN completo (referencia)	77%
Escala de la señal de recompensa demasiado baja	1% (cae 76 puntos, 100% colapsos)
Encoder convolucional → lista plana (MLP)	57%
Timeout proporcional → constante	54%

El dato más elocuente es el primero: bajar la escala de la recompensa hunde el éxito al 1% con colapsos totales. El algoritmo es el mismo DQN; lo único que cambió fue *cómo está formulada* la señal. La conclusión es contundente y va contra la intuición habitual: el problema no se desbloqueó por elegir un algoritmo más listo ni por entrenar más horas, sino por **formular bien la tarea** —un reloj justo (timeout proporcional), una recompensa con la escala adecuada, una observación que incluye los ladrillos como rejilla 8x10, un encoder que respeta el espacio, y una evaluación honesta en niveles no vistos—. DQN es un buen motor, pero estos números son, sobre todo, el premio a un buen planteamiento.



DQN jugando en la aplicación del laboratorio. El agente intercepta la bola y desgasta el muro de ladrillos con un juego sobrio y consistente, fiel a su perfil: nada espectacular, pero rara vez falla.

DQN

rejilla: ritmo (filas) × γ (columnas) · color = recompensa

		γ →		
		0.97	0.99	0.995
ritmo ↓	3e-4	0.09	0.19	0.20 ✓
	8e-4	0.08	0.19	0.01
	1.5e-3	0.02	-0.07	0.12

Mejor: ritmo 3e-4 · γ 0.995 → recompensa 0.20

Ficha-resumen de DQN: familia (basado en valor, off-policy), piezas clave (replay, red objetivo, Double DQN, Huber) e hiperparámetros del laboratorio de un vistazo.

Quédate con esto

DQN aprende a *valorar* cada acción en cada estado con la función $Q(s, a)$; jugar es entonces tan fácil como elegir $\arg\max Q$. Una red profunda de dos ramas (convolucional para los ladrillos, densa para la cinemática) aproxima esa Q , y el objetivo de Bellman —recompensa de ahora más lo mejor del estado siguiente— la va corrigiendo. El replay y la red objetivo lo estabilizan. Resultado: 77% en TEST-ID (media de 5 semillas, greedy), 0% de colapsos y mejora monótona con el presupuesto (67→77→85). Un cimiento simple y fiable. Y, sobre todo: lo que desbloqueó el problema fue formular bien la tarea, no el algoritmo.

AUTOCOMPROBACIÓN

1. ¿Qué representa $Q(s, a)$ y por qué no es lo mismo que la recompensa inmediata r ?
2. Si en un estado los valores son $Q = (1, 2; 0, 5; 1, 9)$ para (izquierda; mantener; derecha), ¿qué devuelve $\arg\max_a Q$ y qué devuelve $\max_a Q$?
3. Con $r = -1, 0$ (perdiste la bola, estado terminal) y $\gamma = 0, 99$, ¿cuánto vale el objetivo de Bellman y? (Pista: si el estado es terminal, no hay futuro.)

EJERCICIO

Repite el mini-ejemplo numérico, pero ahora la recompensa inmediata es $r = +1, 0$ (rompiste un ladrillo) y los valores en el estado siguiente s' son $Q(s') = (0, 5; 1, 8; 3, 2)$ para (izquierda; mantener; derecha). La red predijo $Q(s, a) = 2, 0$ para la acción tomada. Calcula el objetivo de Bellman y el error de la predicción, y di si la red estaba infravalorando o sobrevalorando la acción.

Pista: usa $y = r + \gamma \cdot \max_a Q(s', a')$ con $\gamma = 0, 99$; el máximo de los tres valores de s' es 3, 2.

Respuesta: $y = 1, 0 + 0, 99 \cdot 3, 2 = 1, 0 + 3, 168 = 4, 168$. El error es $4, 168 - 2, 0 = +2, 168$: positivo, luego la red *infravaloraba* la acción, y el entrenamiento empujará $Q(s, a)$ hacia arriba.

Y AHORA, PARA EXPERTOS NIVEL EXPERTO

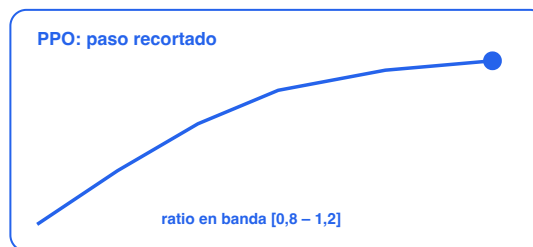
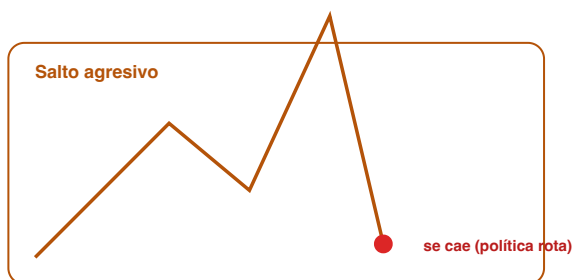
Formalicemos. DQN ajusta $Q_\theta(s, a)$ minimizando, sobre lotes de 128 muestras extraídas i.i.d. del *replay buffer* (capacidad 10^5), la **pérdida de Huber** entre la predicción y el objetivo, en lugar del error cuadrático: la Huber es cuadrática para residuos pequeños y lineal para los grandes, lo que acota el gradiente ante *outliers* de TD y mejora la estabilidad. El objetivo emplea la corrección **Double DQN** (van Hasselt et al., 2016), que desacopla selección y evaluación de la acción: $y = r + \gamma \cdot (1 - \text{done}) \cdot Q_{\text{obj}}(s', \arg\max_a Q_\theta(s', a))$. La red online Q_θ elige el **argmax** en s' ; la red objetivo Q_{obj} lo cuantifica; el factor **(1-done)** anula el bootstrapping en estados terminales. La red objetivo se mantiene con **soft update de Polyak**, $\theta_{\text{obj}} \leftarrow \tau \cdot \theta + (1 - \tau) \cdot \theta_{\text{obj}}$ con $\tau = 0, 01$ por paso, evitando los saltos de la copia dura periódica. Restantes: $\gamma = 0, 99$; exploración ϵ -greedy 1, 0 $\rightarrow$ 0, 05 linealmente en 25.000 pasos; evaluación greedy. La razón de ser de *replay* y red objetivo es contener la **tríada letal** (Sutton & Barto): la combinación de aproximación de funciones, *bootstrapping* y aprendizaje *off-policy* puede divergir; el *replay* rompe la correlación temporal y aproxima muestras independientes, y la red objetivo congela parcialmente el blanco del operador de Bellman, transformando un objetivo móvil y autorreferente en uno casi estacionario que el descenso de gradiente sí puede perseguir con garantías prácticas. La sobreestimación residual del **max** —sesgo de Jensen sobre estimadores ruidosos— la mitiga, precisamente, el desacoplo de Double DQN.

PPO — Proximal Policy Optimization

Mejorar la política a pasos prudentes.

¿Y si en lugar de aprender cuánto vale cada jugada, aprendiéramos directamente a jugar?

PPO: mejorar la política a pasos prudentes («proximal»)



Por qué «proximal»: una actualización agresiva puede romper la política; PPO da pasos pequeños y recorta el cambio para mantener el ratio en la banda $[0,8-1,2]$. Prudencia que paga (el mejor resultado).

Hasta ahora la familia de algoritmos que hemos visto razonaba de forma indirecta. Estimaba un número —el valor de cada acción en cada estado— y la conducta salía de ahí como un subproducto: «si esta acción vale más que las otras, la tomo». PPO da un giro distinto. No pregunta «¿cuánto vale moverse a la derecha?»; pregunta «¿con qué probabilidad debería moverme a la derecha?», y ajusta esa probabilidad directamente. Es la diferencia entre llevar una tabla de precios en la cabeza y, sencillamente, tener una intuición afinada de qué hacer en cada momento.

Este capítulo importa porque PPO fue, en nuestros experimentos, **el algoritmo que mejor rindió y el más fiable**: 91 % de éxito en niveles de test con la misma distribución que el entrenamiento, cero colapsos y, además, el más eficiente en datos. Pero conviene decirlo desde el principio para no confundir el mérito: como veremos al final, lo que de verdad desbloqueó el problema no fue elegir PPO, sino haber *formulado* bien la tarea. PPO brilla porque el terreno ya estaba bien preparado.

PPO

rejilla: ritmo (filas) × recorte ϵ (columnas) · color = recompensa

		recorte $\epsilon \rightarrow$		
		0.1	0.2	0.3
ritmo $\downarrow$	3e-4	0.06	0.07	0.20
	6e-4	0.15	0.10	0.56 ✓
	1.2e-3	0.05	0.06	0.06

Mejor: ritmo 6e-4 · recorte ϵ 0.3 → recompensa 0.56

Ficha resumen de PPO: familia (por política), tipo de aprendizaje (on-policy), y sus piezas características —actor, crítico y el recorte— que iremos desgranando en este capítulo.

■ Aprender la política, no el valor

Llamamos **política** a la función que decide qué hace el agente. En su versión más útil no es una regla rígida sino una *distribución de probabilidad*: dado un estado s , devuelve con qué probabilidad tomar cada acción a . La escribimos así:

$$\pi_{\theta}(a \mid s)$$

- π la política: el «cerebro» que reparte probabilidad entre las acciones.
- θ los parámetros (los pesos de la red neuronal) que definen esa política. Aprender es ajustar θ .
- $a \mid s$ la acción a condicionada al estado s en el que estamos.

Un método **por política** (en inglés, *policy-based*) modifica θ para que las acciones que han salido bien ganen probabilidad y las que han salido mal la pierdan. No hay ninguna tabla de valores intermedia que consultar: la decisión es la salida de la red. Frente al enfoque por valor, esto tiene dos ventajas que nos interesan. Primero, maneja con naturalidad la incertidumbre: una política puede decir «70 % izquierda, 30 % derecha» en una situación ambigua, en vez de comprometerse en seco. Segundo, el objetivo que optimiza está mucho más cerca de lo que de verdad queremos —jugar bien— que de un número auxiliar.

ANTES DE LA FÓRMULA

La idea de fondo del aprendizaje por política es el llamado *gradiente de política*: empuja los parámetros en la dirección que aumenta la probabilidad de las acciones que obtuvieron buenos resultados, en proporción a lo buenos que fueron. El problema clásico de esa receta es que da pasos descontrolados: una racha afortunada puede disparar una probabilidad hasta romper todo lo aprendido. PPO es, esencialmente, **ese gradiente de política con un freno de seguridad**. Ese freno es el «recorte» que da nombre al método (*Proximal* = próximo: no te alejes demasiado de donde estabas).

Actor y crítico: dos voces en la misma red

PPO es un método **actor-crítico**. Conviven dos roles. El **actor** es la política: mira el estado y propone probabilidades de acción. El **crítico** estima un valor $V(s)$: «desde este estado, ¿qué cabe esperar de aquí al final?». El actor decide; el crítico le dice si lo que está pasando es mejor o peor de lo previsto, y esa comparación es justamente la señal que afina al actor. Sin crítico, el actor solo sabría el resultado final de la partida, una señal pobre y ruidosa; con crítico, recibe en cada paso una valoración de si va por buen camino.

En nuestra implementación, ambas voces salen del **mismo tronco** de red. Recordemos la arquitectura de dos ramas que ya describimos: una rama convolucional (capas de 16 y 32 filtros) lee la matriz 8×10 de ladrillos, una rama densa procesa los 6 valores cinemáticos (posiciones y velocidades), ambas se concatenan y pasan por dos capas de 128 unidades con activación ReLU. La novedad de PPO está en la **salida**, que se bifurca:

TRADUCCIÓN AL JUEGO

La cabeza del actor produce **3 números** —una probabilidad para cada acción: mover la pala a la izquierda, quedarse quieta, mover a la derecha— que suman 1. La cabeza del crítico produce **1 número**: el valor estimado $V(s)$ del estado actual. El mismo «cuerpo» de la red sirve a las dos cabezas, de modo que las características visuales aprendidas (dónde quedan ladrillos, por dónde va la bola) las comparten quien decide y quien evalúa.

On-policy: por qué PPO no recicla experiencias

Aquí aparece una diferencia de fondo con los métodos por valor. PPO es **on-policy**: aprende **solo** de datos recogidos con la política actual. Cada vez que mejora θ , las experiencias viejas —generadas por una versión anterior del agente— quedan obsoletas y se descartan. No hay memoria de repetición (*replay buffer*) que vuelva a usar partidas antiguas una y otra vez.

¿Por qué esta «austeridad»? Porque el recorte de PPO razona sobre *cuánto ha cambiado la política respecto a la que generó los datos*. Esa comparación solo tiene sentido si los datos son recientes. Reutilizar trayectorias muy viejas mediría un cambio enorme y carente de significado. La consecuencia práctica es un ciclo muy ordenado: **(1)** jugar un tramo con la política actual —el *rollout*—, **(2)** exprimir ese tramo con varias pasadas de aprendizaje, **(3)** tirar los datos y volver a jugar.

ERROR COMÚN

Pensar que «on-policy» significa que PPO desperdicia datos y por eso debería ser menos eficiente. En la práctica ocurrió lo contrario: fue el algoritmo **más eficiente en datos** de los que probamos (ya alcanzaba 90 % de éxito con solo 700 000 pasos). La clave es que, dentro de cada rollout, PPO reaprovecha el mismo lote varias veces (4 épocas) antes de descartarlo. No es despilfarro; es exprimir bien lo fresco y no aferrarse a lo rancio.

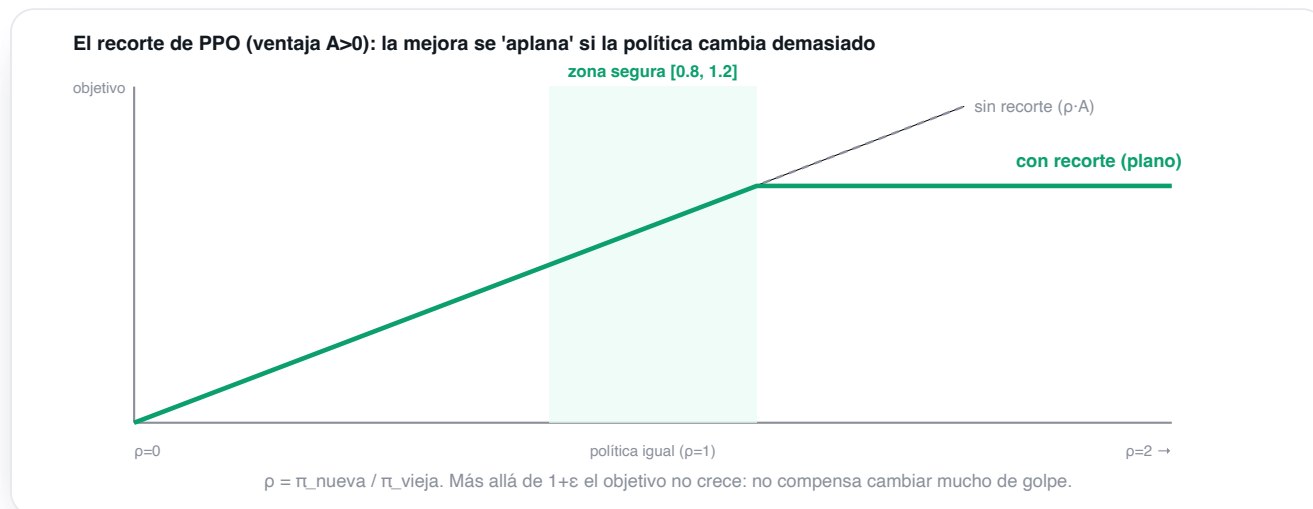
El corazón de PPO: el ratio y el recorte

Toda la maquinaria gira en torno a una pregunta sencilla: *tras mirar lo que pasó, ¿cuánto debería cambiar la probabilidad de una acción?* PPO mide ese cambio con un cociente, el **ratio de probabilidad**:

$$r_t = \pi_{\theta}(a_t \mid s_t) / \pi_{\theta_{\text{old}}}(a_t \mid s_t)$$

π_θ	la política nueva, la que estamos ajustando ahora mismo.
π_{θ_old}	la política con la que se jugó el rollout (la «vieja», congelada para este lote de datos).
r_t	el ratio. Si vale 1, no ha cambiado nada; 1,2 significa que la acción es ahora un 20 % más probable; 0,8, un 20 % menos.

Ese ratio se multiplica por la **ventaja** A_t , que indica si la acción tomada salió mejor ($A > 0$) o peor ($A < 0$) de lo que el crítico esperaba. Si una acción tuvo ventaja positiva, queremos subir su probabilidad; si fue negativa, bajarla. Hasta aquí, gradiente de política clásico. El problema: nada impide que r_t se dispare y la política pegue un volantazo. La solución de PPO es **recortar el ratio** a una banda estrecha, en nuestro caso $[0,8 ; 1,2]$ (es decir, $\epsilon = 0,2$), y quedarse con el resultado más conservador.



El recorte de PPO. Mientras el ratio se mantiene dentro de la banda $[0,8 ; 1,2]$ el aprendizaje fluye normal; en cuanto la política intenta alejarse demasiado de la versión que generó los datos, la función se aplana y deja de premiar ese movimiento. Es un tope, no un empujón.

TRADUCCIÓN AL JUEGO

Imagina que en cierto estado la pala estaba quieta justo cuando la bola venía hacia ese lado, y rebotó rompiendo varios ladrillos: la ventaja fue claramente positiva. El gradiente quiere subir mucho la probabilidad de «quedarse quieta» en estados parecidos. El recorte responde: «sube, pero como mucho hasta un 20 % más que antes». Así, una sola jugada afortunada no convierte al agente en un fanático de la quietud que ignora todo lo demás. Aprende sin sobre-reaccionar.

Mini-ejemplo: el recorte en tres números

Pongamos cifras concretas. Supongamos una acción con ventaja positiva $A = +1$.

Caso moderado. La política nueva quiere hacer la acción un 15 % más probable: $r = 1,15$. Está dentro de la banda $[0,8 ; 1,2]$. El objetivo vale $\min(1,15 \cdot 1 ; 1,2 \cdot 1) = 1,15$. Se permite el cambio íntegro: el aprendizaje fluye.

Caso exagerado. La política nueva se entusiasma y quiere un $r = 1,8$ (un 80 % más probable). Fuera de banda. El objetivo vale $\min(1,8 \cdot 1 ; 1,2 \cdot 1) = 1,2$. El recorte ha mordido: por mucho que el ratio crezca, la recompensa al cambio se queda topada en 1,2, así que el gradiente deja de empujar en esa dirección. **La política no puede dar el volantazo.**

El efecto es el mismo, simétrico, para acciones con ventaja negativa: PPO permite bajar la probabilidad hasta cierto punto, pero no hundirla de golpe. En ambos casos la palabra clave es *proximal*: nos

movemos, pero cerca de donde estábamos.

PARA CURIOSOS

¿De dónde sale la ventaja A_t ? La calculamos con **GAE** (*Generalized Advantage Estimation*) y un parámetro $\lambda = 0,95$. GAE mezcla las valoraciones del crítico a lo largo de varios pasos para estimar la ventaja con poco ruido sin dejar de ser bastante fiel a lo que pasó. Un λ alto (cercano a 1) da más peso a lo que realmente ocurrió a largo plazo; bajarlo confía más en la predicción del crítico paso a paso. 0,95 es un punto intermedio que funciona bien.

❖ Cómo se ve esto en el código (idea, no listado)

No hace falta el código completo para captar la mecánica. Un paso de PPO encadena estas operaciones: se juega un rollout de 256 pasos por entorno y se guardan estados, acciones, recompensas y las probabilidades de la política vieja; se calculan las ventajas con GAE; y entonces, durante **4 épocas**, se recorre ese lote en *minibatches* de 1024 muestras calculando el ratio, aplicándole el recorte y dando un paso de descenso de gradiente. La pérdida total combina tres términos: el objetivo recortado del actor, el error del crítico al predecir el valor (con peso 0,5) y un pequeño premio a la exploración (la entropía, con peso 0,01). Tras las 4 épocas, se descartan los datos y vuelta a empezar.

ERROR COMÚN

Confundir las **épocas** de PPO con reutilizar datos viejos. Las 4 épocas repasan *el mismo* rollout recién recogido, no rollouts de hace mil pasos. Sigue siendo on-policy: en cuanto la política se actualiza demasiado respecto a la que generó el lote, el recorte frena, y por eso no se hacen 40 épocas sino unas pocas.

❖ Resultado: el mejor y el más fiable

Con el protocolo de evaluación congelado —5 semillas, evaluación *greedy* (sin azar, eligiendo siempre la acción más probable) y éxito medido en niveles **no vistos** durante el entrenamiento— PPO obtuvo los mejores números del estudio. La siguiente tabla recoge la media de las 5 semillas.

Métrica (media de 5 semillas)	PPO
Éxito en test con la misma distribución (TEST-ID)	91 %
Éxito fuera de distribución, por patrón (OOD-patrón)	89 %
Éxito fuera de distribución, por dificultad (OOD-dificultad)	86 %
Colapsos de entrenamiento	0 %
Semillas que superan el 80 % de éxito	100 %
Pasos para limpiar un nivel	1923

Dos cosas merecen subrayarse. La primera es la **fiabilidad**: cero colapsos y las 5 semillas por encima del 80 %. Esto no es un detalle estético. Significa que el resultado no depende de tener suerte con la inicialización: arranques el experimento las veces que lo arranques, sale bien. La segunda es la **eficiencia en datos**. Mirando solo TEST-ID por presupuesto de entrenamiento:

Presupuesto de pasos	Éxito TEST-ID
700 000	90 %
1,5 millones	91 %
3 millones	87 %

PPO **ya aprende a 700k**: con la quinta parte del presupuesto mayor alcanza el 90 %. Fue el método más eficiente en datos del estudio.



Curvas de entrenamiento de PPO. La métrica sube pronto y se estabiliza, sin los desplomes bruscos típicos de un colapso. La estabilidad de la curva es coherente con el dato de 0 % de colapsos y con que las 5 semillas terminarían por encima del 80 %.

QUÉ MIRAR

En el inspector, fíjate en las **3 probabilidades** del actor en estados claros frente a estados ambiguos. Cuando la bola cae limpia hacia un lado, una acción debería dominar (probabilidad alta); en situaciones dudosas, las tres se reparten más. Esa modulación es la firma de una buena política: decidida cuando toca, prudente cuando no está claro.

Inspector PPO

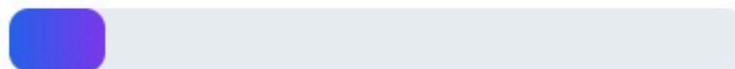
$\pi(a|s)$ — probabilidad de cada acción

←



0.244

·



0.129

→



0.627

Valor $V(s)$

0.724

Entropía $H(\pi)$

0.901

Rollout

64%

Inspector de PPO: a la izquierda el estado (cinemática + matriz de ladrillos), a la derecha la distribución del actor sobre las 3 acciones y el valor estimado por el crítico. Permite ver, paso a paso, en qué se apoya cada decisión.

Lectura crítica: qué demuestra y qué no

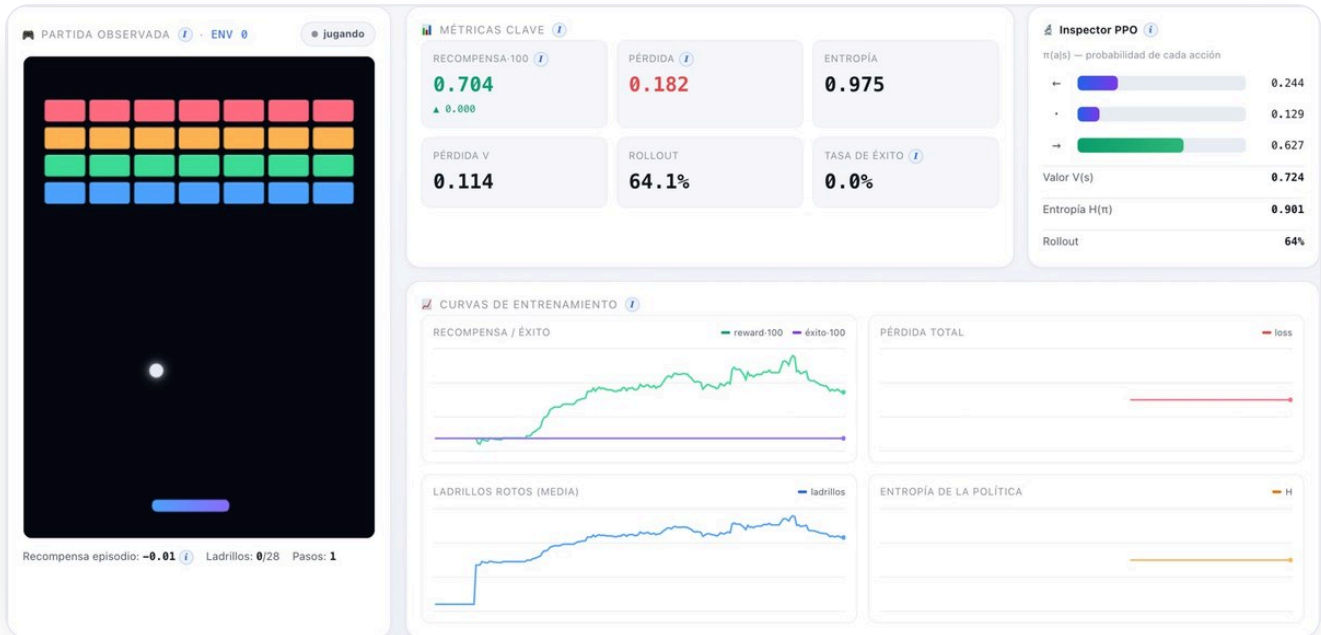
El 87 % a 3 millones de pasos, por debajo del 91 % a 1,5M, no es un error de medición: es una señal honesta de que **más entrenamiento no siempre es mejor**. Con un presupuesto grande, una política on-policy puede sobreajustarse a los niveles de entrenamiento y perder algo de generalización. El dato útil no es «PPO mejora indefinidamente», sino «PPO alcanza su mejor punto pronto».

QUÉ NO DEMUESTRA

Que PPO sea «el mejor algoritmo» en abstracto. Demuestra que, **sobre esta formulación concreta del Arkanoid** (reloj interno, recompensa sin *shaping*, visión 8×10, escalado de entradas, ramas convolucionales, currículo y un test honesto con niveles no vistos), PPO fue el que mejor explotó ese terreno. Cambiar la formulación podría cambiar el ranking. Y todos los números son la media de 5 semillas con evaluación greedy: no confundas esta media con el resultado de una única ejecución afortunada, ni el éxito (limpiar el nivel) con la recompensa acumulada, que son cosas distintas.

Quédate con esto

PPO aprende **la política directamente** (no el valor), es **actor-crítico** y **on-policy**, y su sello es el **recorte**: deja que la política se mueva pero solo a pasos pequeños, dentro de la banda $[0,8 ; 1,2]$, para no romper lo aprendido. En el Arkanoid fue **el mejor y el más fiable** (91 % TEST-ID, 0 % de colapsos, 100 % de semillas >80 %) y **el más eficiente en datos** (90 % ya a 700k). Pero lo que de verdad desbloqueó el problema fue *formular bien la tarea*, no escoger el algoritmo: PPO destacó porque el terreno estaba bien preparado.



PPO jugando en la aplicación. La pala intercepta con limpieza y el nivel se va vaciando; es el comportamiento esperable de la política con mejor éxito y mayor fiabilidad del estudio.

AUTOCOMPROBACIÓN

1. ¿Qué aprende exactamente PPO: el valor de cada acción o la probabilidad de tomarla? ¿Por qué se le llama método «por política»?
2. Explica con tus palabras qué hace el recorte cuando el ratio vale 1,8 y la ventaja es positiva. ¿Por qué impide un «volantazo»?
3. ¿Por qué PPO no usa memoria de repetición? ¿Qué relación tiene eso con ser on-policy?
4. El éxito a 3M (87 %) es menor que a 1,5M (91 %). ¿Significa esto que PPO empeora con más datos, o hay otra lectura?
5. ¿Por qué decimos que «el mérito no es del algoritmo»? ¿Qué hubiera hecho falta para que la frase fuera distinta?

EJERCICIO

Abre el inspector y registra, en 10 estados distintos, las 3 probabilidades del actor y el valor del crítico. Clasifica cada estado como «claro» o «ambiguo» según la posición de la bola respecto a la pala. ¿Se cumple que en los estados claros una acción domina y en los ambiguos la probabilidad se reparte más? Después, repite el mismo recorrido con la evaluación greedy y comprueba que la acción elegida coincide siempre con la de mayor probabilidad.



Y AHORA, PARA EXPERTOS

NIVEL EXPERTO

PPO optimiza un **objetivo sustituto recortado** en lugar del gradiente de política directo:

$$L^{\text{CLIP}} = E[\min(r_t \cdot A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \cdot A_t)]$$

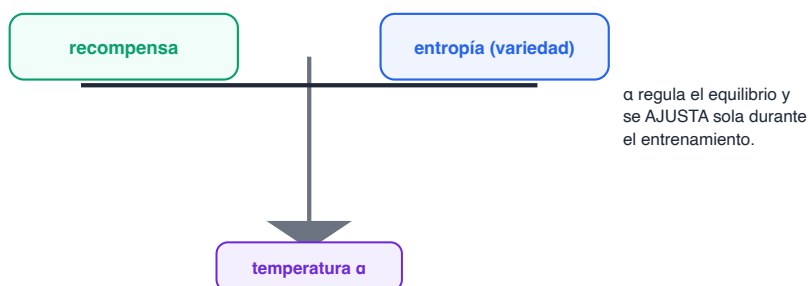
con $r_t = \pi_\theta / \pi_{\theta_{\text{old}}}$ y $\epsilon = 0,2$ (banda $[0,8 ; 1,2]$). El **min** toma siempre la rama más pesimista, de modo que el recorte solo penaliza —nunca premia— alejarse de la política antigua: es un límite de confianza, no una zanahoria. La ventaja A_t se estima por **GAE($\lambda = 0,95$)**, que interpola entre la estimación de un paso (sesgada, poco ruidosa) y Monte Carlo (insesgada, ruidosa). Al ser **on-policy**, el lote pierde validez en cuanto θ se aleja de θ_{old} , lo que justifica las pocas épocas. La pérdida total suma el término del actor, un término de valor con **coef. 0,5** (error cuadrático del crítico) y un **coef. de entropía** que regula la exploración: su efecto fue muy marcado en nuestros experimentos —bajarlo de 0,01 a 0,003 casi duplicó el rendimiento, porque con 0,01 la política se quedaba demasiado cerca de uniforme y nunca llegaba a comprometerse con una estrategia—. Hiperparámetros del protocolo: **rollout 256** pasos/entorno, **4 épocas**, **minibatch 1024**, **clip $\epsilon = 0,2$** . La sample-efficiency observada (90 % a 700k, frente a la degradación hasta 87 % a 3M) sugiere que, con presupuestos altos, conviene vigilar el sobreajuste a los niveles de entrenamiento más que seguir entrenando: el óptimo de generalización llega antes que el de recompensa en entrenamiento.

SAC — Soft Actor-Critic (discreto)

Ganar sin dejar de explorar.

Los algoritmos que hemos visto hasta ahora se enfrentan a un dilema incómodo: cuanto mejor creen saber qué hacer, menos prueban cosas nuevas, y antes se cierran en una única forma de jugar. ¿Y si, en lugar de pelear contra esa tendencia con una muleta externa, le pidiéramos al agente que *maximice la recompensa y, a la vez, mantenga a propósito cierta variedad* en sus decisiones, regulando él mismo cuánta variedad conviene en cada momento? Ese es el giro de SAC.

SAC: maximizar recompensa SIN dejar de explorar



SAC busca dos cosas a la vez: recompensa y variedad (entropía). La temperatura α es el fiel de la balanza, y se ajusta sola durante el entrenamiento.

SAC (*Soft Actor-Critic*, «actor-crítico suave») es el algoritmo de **máxima entropía** de nuestro laboratorio. Su idea de fondo se resume en una frase: no basta con ganar, hay que ganar *sin perder variedad antes de tiempo*. Donde DQN persigue el valor y PPO ajusta la política con prudencia, SAC añade un objetivo explícito —la entropía de la política— al lado de la recompensa, y aprende a dosificarlo solo. Como DQN, es un método **off-policy** (fuera de política): aprende de un depósito de experiencias pasadas mediante *replay buffer* y redes objetivo, en vez de aprender solo de la jugada recién vivida.

Este capítulo tiene una particularidad de honestidad que conviene anunciar de entrada. SAC aparece en el libro en **dos variantes distintas**, y rinden muy diferente. La primera, **SAC-puro** (el actor puro), funciona muy bien: limpia el 87% de los niveles de test conocidos. La segunda, **SAC-híbrido** (la variante con crítico híbrido), se queda en el 61% y, sobre todo, es *bimodal*: a veces aprende bien y a veces colapsa. No vamos a maquillar esa diferencia con un número único: la contaremos entera, porque es justo el tipo de resultado matizado del que más se aprende.

TRADUCCIÓN AL JUEGO

Imagina dos jugadores que entrenan al Arkanoid. El primero, en cuanto encuentra una jugada que le funciona, la repite sin descanso y deja de probar ángulos nuevos: se vuelve rígido. El segundo se obliga a mantener un pequeño abanico de respuestas razonables ante cada situación —no una sola «correcta»— y solo va estrechando ese abanico a medida que de verdad sabe lo que hace. SAC es el segundo jugador: premia explícitamente conservar ese abanico de opciones, y tiene un mando interno (la temperatura) para abrirlo o cerrarlo según le convenga.

El objetivo de máxima entropía

En PPO y DQN, lo que se maximiza es, en esencia, el retorno: la suma descontada de recompensas. SAC cambia ese objetivo añadiéndole un segundo término. En cada paso, el agente no solo quiere recompensa r , sino también *incertidumbre saludable* en su decisión, medida por la **entropía** de la política $H(\pi(\cdot | s))$ —esa cantidad que ya definimos en el capítulo de exploración como $H = -\sum_a \pi(a | s) \cdot \log \pi(a | s)$, máxima cuando las tres acciones son igual de probables.

$$J(\pi) = \sum_t E[r_t + \alpha \cdot H(\pi(\cdot | s_t))]$$

r_t	La recompensa del paso (romper un ladrillo, limpiar el nivel, perder la bola). Lo de siempre.
$H(\pi(\cdot s_t))$	La entropía de la política en ese estado: cuánta variedad mantiene el actor entre izquierda, mantener y derecha. Alta si reparte; baja si apuesta casi todo a una acción.
α (alfa, la temperatura)	El peso que se le da a esa variedad frente a la recompensa. Es el mando del que hablábamos. Con α alto, el agente paga mucho por mantener opciones abiertas (explora más); con α bajo, prioriza la recompensa pura (explota).

La diferencia con PPO es de naturaleza, no de grado. PPO añade un *coeficiente de entropía* fijo (0,01) que solo actúa como un pequeño empujón dentro de la función de pérdida para que la distribución no colapse. SAC mete la entropía **dentro del propio objetivo a maximizar**: el agente la persigue igual que persigue la recompensa, paso a paso, en cada estimación de valor. Por eso se llama «suave» (*soft*): no busca la mejor acción a secas, sino la mejor acción *teniendo en cuenta que conservar variedad también vale*.

ANTES DE LA FÓRMULA

«Maximizar la entropía» no significa «jugar al azar». Significa: *entre dos formas de jugar que dan una recompensa parecida, prefiere la que reparte más sus apuestas*. Si una acción es claramente mejor, el agente la elegirá igualmente, porque su ventaja en recompensa supera con creces lo que ganaría repartiendo. La entropía solo decide los *empates* y frena que el actor descarte una acción para siempre por unas pocas malas experiencias tempranas.

La temperatura se aprende sola

El verdadero ingrediente fino de SAC no es la entropía en sí, sino que **no fijamos α a mano**. Elegir bien ese peso es difícil y cambia a lo largo del entrenamiento: al principio conviene explorar mucho; más tarde, casi nada. En vez de adivinar un calendario, SAC convierte α en un **parámetro aprendible** y le da una

entropía objetivo: «quiero que tu política mantenga *esta* cantidad de variedad». Entonces α se ajusta solo, como un termostato, para acercar la entropía real a esa meta.

α inicial Arranca en 0,20. Es el punto de partida del termostato, no un valor fijo.

Entropía objetivo Fijada en $0,55 \cdot \log(3) \approx 0,60$. Recuerda que con 3 acciones la entropía máxima posible es $\log(3) \approx 1,10$ (las tres equiprobables). No pedimos eso: pedimos algo más de la *mitad* de esa variedad. Ni rígido ni caótico.

El mecanismo es un lazo de realimentación sencillo: si la política se está volviendo demasiado puntiaguda (entropía real por debajo de 0,60), α sube y encarece cerrarse, empujando de vuelta hacia la variedad; si está demasiado dispersa (entropía por encima de 0,60), α baja y deja que el actor se concentre en la mejor acción. El agente regula su propia exploración sin que nadie le dicte el ritmo.

TRADUCCIÓN AL JUEGO

Es como un entrenador que ajusta la presión sobre el jugador según lo encorsetado que lo ve. Si nota que el jugador se ha obsesionado con un único golpe (poca variedad), le insiste en que pruebe alternativas. Si lo ve disperso y dubitativo (demasiada variedad), le deja decidirse. La meta «0,60» es el punto de equilibrio que el entrenador considera sano para esta tarea de tres acciones: con margen para sorprender, pero sin perder el rumbo.

Cinco redes: actor, dos críticos y dos copias objetivo

Para sostener este objetivo, SAC monta la maquinaria más cara del laboratorio. No tiene una red, ni dos: usa **cinco**. El cuerpo de todas ellas es la misma red de dos ramas que ya conocemos (la matriz 8×10 por el encoder convolucional, los 6 cinemáticos por la rama densa, concatenación y dos capas de 128 con ReLU); lo que cambia es la cabeza y el papel de cada una.

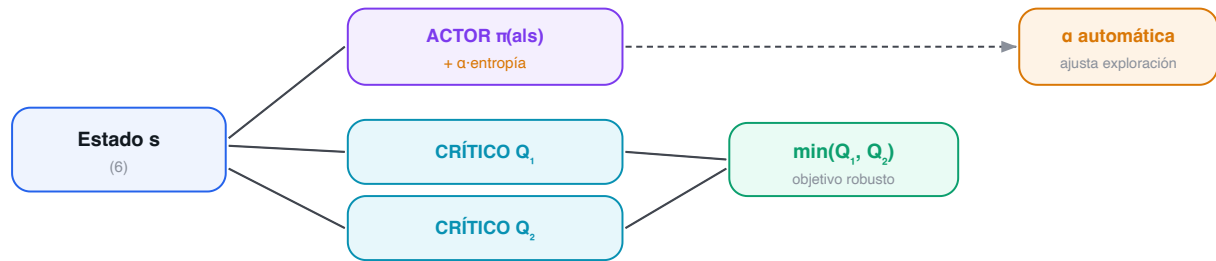
El actor (1 red) Emite las 3 probabilidades $\pi(a|s)$: izquierda, mantener, derecha. Es quien decide. Igual que el actor de PPO, su salida es una distribución estocástica.

Dos críticos (2 redes) Cada uno estima el valor $Q(s, a)$ de las acciones. Son *dos*, no uno, a propósito (lo explicamos enseguida): se toma el **mínimo** de los dos para no sobreestimar.

Dos redes objetivo (2 redes) Copias lentas de los dos críticos, que cambian despacio para fijar una diana estable al calcular el objetivo de Bellman. Son las mismas redes objetivo que vimos como motor de estabilidad de los métodos off-policy.

Las dos redes objetivo se actualizan con un *soft update* de $\tau = 0,01$: en cada paso copian solo una centésima de los críticos en línea, de modo que la diana se mueve casi imperceptiblemente. Es el mismo mecanismo, y el mismo valor de τ , que en DQN; SAC lo aplica por duplicado.

SAC: dos críticos (se usa el menor, evita sobreestimar) + entropía con temperatura α



El engranaje de SAC. El actor (arriba) decide la acción a partir del estado; las experiencias caen al *replay buffer*, de donde sale el lote de 128 que alimenta a los **dos críticos**. El objetivo de Bellman se calcula con el *mínimo* de las **dos redes objetivo** (las copias lentas, $\tau = 0,01$), y un término de entropía pondera la temperatura α , que a su vez se ajusta sola hacia la meta de 0,60. Cuenta las cajas: cinco redes en danza, frente a la una de DQN o las dos de PPO. Esa es, literalmente, la razón de que SAC sea el más caro del laboratorio.

ANTES DE LA FÓRMULA

¿Por qué **dos** críticos y luego el mínimo? Porque una red que estima valores tiende a ser optimista: el ruido de las estimaciones, al pasar por el max de Bellman, sesga los valores hacia arriba. Mantener dos críticos entrenados por separado y quedarse siempre con el *menor* de los dos (el **clipped double-Q**) es una forma barata de corregir ese optimismo: si uno de los dos se ha venido arriba con una acción, el otro probablemente no, y el mínimo lo contiene. A cambio se pagan más redes que entrenar.

Cómo aprende: el bucle off-policy

Al ser off-policy, SAC reutiliza exactamente la infraestructura de DQN. Cada paso jugado se guarda como una transición $(s, a, r, s', done)$ en un **replay buffer** de hasta **100.000** transiciones. Para aprender, se extrae un **lote de 128** al azar de ese depósito, rompiendo la correlación temporal. Los dos críticos se entrenan contra el objetivo de Bellman; el actor se entrena para elegir acciones que los críticos valoran alto *pero conservando entropía*; y la temperatura α se ajusta hacia su objetivo de 0,60. El factor de descuento es el de toda la casa, $\gamma = 0,99$.

Pieza	SAC (discreto)	Comparado con...
Familia	Off-policy (replay + redes objetivo)	Como DQN y World Model; al revés que PPO (on-policy)
Política	Actor estocástico $\pi(a s)$	Como PPO; al revés que DQN (valor + argmax)
Exploración	Entropía dentro del objetivo, α aprendible	PPO: coef. fijo 0,01; DQN: epsilon-greedy externo
Redes	5 (actor + 2 críticos + 2 objetivo)	Las más de todo el banco de pruebas
Replay / lote / τ	100.000 / 128 / 0,01	Idénticos a DQN

Inspector SAC

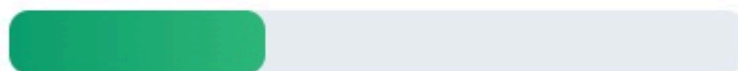
$\pi(a|s)$ — política estocástica

←



0.346

·



0.352

→



0.302

Temperatura α

0.1564

Entropía $H(\pi)$

1.087

$\min(Q1, Q2)$

← 0.59 · 0.56 → 0.56

El inspector mostrando una decisión de SAC paso a paso. Fíjate en las tres probabilidades $\pi(a|s)$ del actor (izquierda, mantener, derecha): rara vez una de ellas es 1 y las otras 0. SAC mantiene a propósito un reparto —esa es la huella visible de la entropía— en lugar de apostararlo todo a una acción. Cuando el agente lo tiene claro, la distribución se afila; cuando duda, se reparte. La temperatura α es lo que regula cuánto se permite afilar.

QUÉ MIRAR

Cuando leas los resultados, no juntes SAC-pure y SAC-híbrido en una sola cifra «de SAC». Son dos montajes distintos: el primero confía en el *actor* para decidir; el segundo apoya la decisión en un esquema de *crítico híbrido*. Comparten el marco de máxima entropía, pero su comportamiento difiere tanto que mezclarlos borraría justo lo interesante.

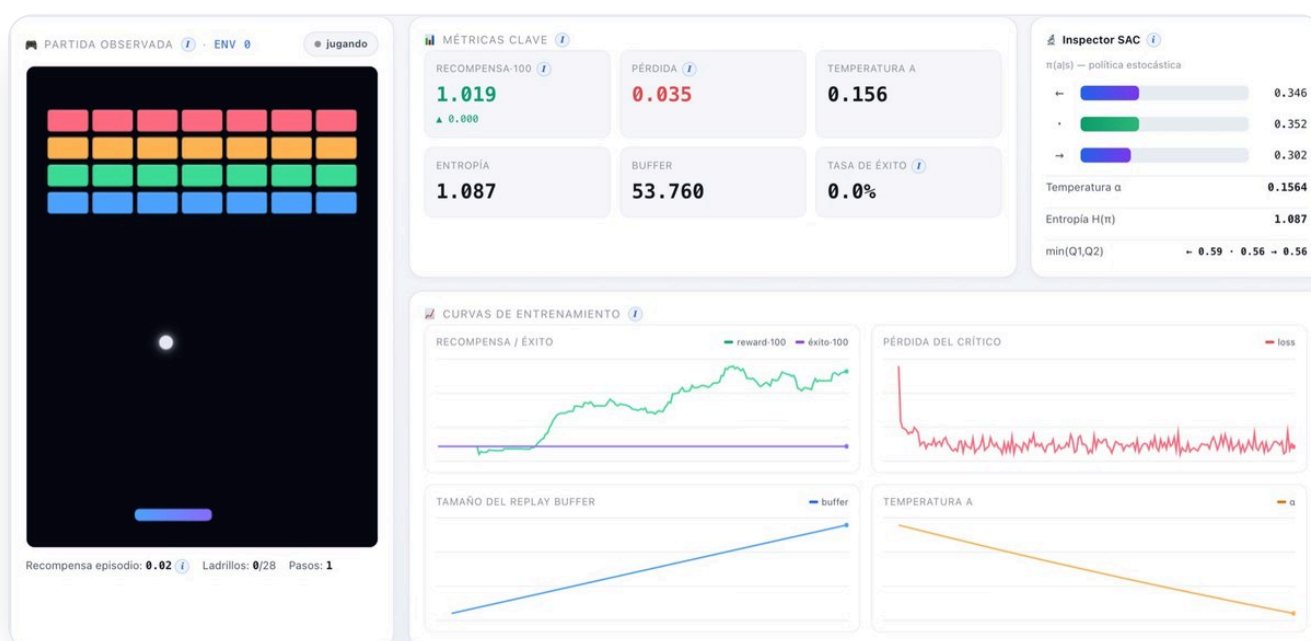
Resultados: el actor sí funciona

Bajo el protocolo congelado (5 semillas, evaluación *greedy*, niveles no vistos), **SAC-pure** da una cara sólida. Limpia el 87% de los niveles de test conocidos (TEST-ID) y aguanta bien al alejarse: 84% en patrones nuevos (OOD-patrón) y 68% en dificultad no vista (OOD-dificultad). No sufre **ningún colapso** (0% de las semillas se hundieron por debajo del 10% de éxito) y el 80% de sus semillas superan el 80% de éxito. Necesita unos 2.732 pasos para limpiar un nivel. La lectura es directa: **el actor de SAC sí funciona**.



Curvas de aprendizaje de SAC bajo el protocolo congelado. Interpreta dos cosas a la vez: el nivel que alcanza la curva (el techo de éxito) y, sobre todo, lo *parejas* que van las distintas semillas. En SAC-pure las trayectorias suben de forma suave y agrupada —señal de estabilidad—; en SAC-híbrido conviven trayectorias que despegan con otras que se quedan planas, la firma visual de un comportamiento bimodal.

La dependencia del presupuesto de pasos es reveladora. SAC-pure pasa de un 25% de éxito a 700k pasos a un 87% a 1,5M, y sube luego con suavidad hasta el 91% a 3M. Es decir: **despega entre 700k y 1,5M de pasos** y, a partir de ahí, mejora poco a poco sin sobresaltos. De toda la familia, es el que sube de forma *más estable a la larga*: no es el que antes arranca, pero su ascenso es limpio.



Un agente SAC-pure entrenado, jugando en la aplicación sobre la rejilla 8x10. Es la traducción en pantalla del 87% de la hoja: el agente dirige la bola hacia los ladrillos pendientes en niveles que no vio durante el entrenamiento. Recuerda que la evaluación es *greedy* (toma siempre la acción más probable del actor), así que aquí no hay azar: lo que ves es la política ya aprendida, no exploración.

PARA CURIOSOS

Que el 68% de OOD-dificultad caiga bastante por debajo del 87% de TEST-ID no es un fallo, sino información. Indica que SAC-pure capturó muy bien la *idea* de despejar los patrones conocidos, pero pierde fuelle cuando la densidad de ladrillos crece más allá de lo visto en entrenamiento. La pendiente entre las tres columnas es, en sí misma, una medida de cuánto entendió frente a cuánto se ajustó a lo familiar; aquí esa pendiente es más pronunciada que la de PPO y más suave que la de los modelos del mundo.

La otra cara: SAC-híbrido es bimodal

La variante **SAC-híbrido** (SAC-critic-hybrid) cuenta una historia distinta y menos amable. Su TEST-ID se queda en 61%, con 60% en OOD-patrón y 51% en OOD-dificultad. Pero el dato que de verdad la define no es la media, sino la **dispersión**: tiene un 20% de colapsos (una de cada cinco semillas se hundió por debajo del 10% de éxito) y solo el 40% de sus semillas superan el 80%. Es un comportamiento **bimodal**: a veces aprende razonablemente, a veces colapsa, y la media de 61% es en parte el promedio de esos dos destinos opuestos, no el rendimiento «típico» de una ejecución.

ERROR COMÚN

Citar «SAC-híbrido = 88%» a secas. Es media verdad. Ese 88% es *solo* a 3M de pasos; a 1,5M baja a 61% y a 700k se desploma a un 4%. La dependencia del presupuesto es brutal. Y aun a 3M, el riesgo de colapso no desaparece por ser bimodal. Un número así, sin sus condiciones (presupuesto, número de semillas, que es media), engaña sobre lo que cabe esperar de una ejecución cualquiera.

Comparada con la trayectoria suave de SAC-pure, la de SAC-híbrido es una rampa empinada y resbaladiza: 4% (700k) → 61% (1,5M) → 88% (3M). Puede llegar alto, pero su rendimiento *depende de forma brutal del presupuesto* y no garantiza llegar siempre. Por eso, cuando en la Parte I la figura comparativa mostraba «SAC = 61%», estaba mostrando precisamente esta variante híbrida y bimodal, no el actor puro.

Variante	TEST-ID	OOD-patrón	OOD-dificultad	% colapsos	% semillas >80	Pasos para limpiar
SAC-pure (actor)	87	84	68	0%	80%	2732
SAC-híbrido	61	60	51	20%	40%	1716

SAC

rejilla: ritmo actor (filas) × entropía obj. (columnas) · color = recompensa

		entropía obj. →		
		0.4	0.55	0.7
ritmo actor ↓	3e-4	0.01	-0.01	0.16
	6e-4	0.05	0.12	0.13
	1.2e-3	0.20	0.31 ✓	0.23

Mejor: ritmo actor 1.2e-3 · entropía obj. 0.55 → recompensa 0.31

Ficha-resumen de SAC sobre las tres pruebas y las dos variantes. Léela contrastando filas: SAC-pure ocupa la franja alta con cero colapsos; SAC-híbrido vive más abajo y, sobre todo, arrastra ese 20% de colapsos que ninguna otra variante del banco de pruebas tiene. La distancia entre ambas filas es el mensaje central del capítulo.

LÍMITES DE ESTA LECTURA

Las cifras son medias de 5 semillas sobre conjuntos concretos, con evaluación greedy y bajo un protocolo congelado. No dicen que SAC «sea peor o mejor» en abstracto: dicen cómo rinden *estas dos variantes*, con *esta* formulación de la tarea y *estos* presupuestos. Con 5 semillas, un 20% de colapsos significa literalmente «una semilla se hundió»: la muestra es pequeña, y por eso hablamos de tendencia bimodal, no de una probabilidad de colapso medida con precisión.

Lectura crítica: el más estable a la larga, el más caro de mantener

Dos conclusiones se sostienen sobre los datos de la hoja, sin adornos. La primera: **SAC-pure es el más estable a la larga** —sube suave, sin colapsos, y es el que mejor aguanta el paso del tiempo de entrenamiento—, aunque no sea el que antes despegue ni el que más alto llega (ese sigue siendo PPO, con 91% en TEST-ID y caída suave). La segunda: **SAC es el más caro** de todo el laboratorio, porque mantiene cinco redes. Más estabilidad potencial, pero a un coste de cómputo que ningún otro algoritmo del banco iguala.

Y la moraleja encaja con la tesis del libro entero. La diferencia entre el 87% de SAC-pure y el 61% de SAC-híbrido *no viene de cambiar de algoritmo* —ambos son SAC, comparten el marco de máxima entropía—, sino de cómo se monta la pieza que decide: confiar en el actor frente a apoyarse en un crítico híbrido. Es otra cara de la misma idea: **formular bien pesa más que cambiar de etiqueta de algoritmo**. Dentro de un mismo nombre, «SAC», caben un agente sólido y otro que a veces se hunde.

Quédate con esto

SAC es el algoritmo de **máxima entropía**: maximiza recompensa y variedad de decisiones a la vez, con una temperatura α **aprendible** (arranca en 0,20) que se autorregula hacia una entropía objetivo de $0,55 \cdot \log(3) \approx 0,60$. Es off-policy (replay 100.000, lote 128, redes objetivo con $\tau = 0,01$) y usa **cinco redes** (actor + 2 críticos + 2 objetivo), lo que lo hace el más caro. Hay que distinguir dos variantes: **SAC-pure** (el actor) limpia el 87% de TEST-ID, 0% de colapsos, sube suave y estable — el actor sí funciona —; **SAC-híbrido** se queda en 61%, es bimodal (20% de colapsos) y depende brutalmente del presupuesto (4% $\rightarrow$ 61% $\rightarrow$ 88%). La lección: dentro de un mismo algoritmo, *cómo* se fórmula la pieza decisoria cambia el resultado más que el nombre del método.

AUTOCOMPROBACIÓN

1. El objetivo de SAC suma $r_t + \alpha \cdot H(\pi)$. ¿Qué pasaría con la forma de jugar del agente si α se hiciera enorme? ¿Y si α se hiciera cero? Relaciona cada extremo con explorar o explotar.
2. SAC-pure marca 87 en TEST-ID y 68 en OOD-dificultad; SAC-híbrido marca 61 con un 20% de colapsos. ¿Por qué no basta con comparar solo las dos medias (87 frente a 61) para describir la diferencia entre las dos variantes? ¿Qué columna añade información que la media oculta?
3. ¿Por qué SAC usa dos críticos y se queda con el mínimo de ambos, en lugar de fiarse de uno solo? ¿Qué problema concreto evita ese mínimo?
4. SAC-híbrido pasa de 4% (700k) a 88% (3M). Si alguien te dice «SAC-híbrido alcanza el 88%», ¿qué tres condiciones le faltan a esa afirmación para ser honesta?

EJERCICIO

La entropía objetivo de SAC está fijada en $0,55 \cdot \log(3)$. Calcula su valor numérico y compáralo con la entropía máxima posible para 3 acciones. ¿Qué fracción de la variedad máxima le estamos pidiendo al actor que conserve? **Pista:** $\log(3) \approx 1,0986$ con logaritmo natural; la entropía máxima con 3 acciones equiprobables es justo $\log(3)$. **Respuesta:** $0,55 \cdot 1,0986 \approx 0,604$, de ahí el $\approx 0,60$ de la hoja. Como la máxima es 1,0986, le pedimos conservar el 55% de la variedad máxima posible: algo más de la mitad. *Qué significa para el agente:* no le exigimos jugar al azar (eso sería el 100%, las tres acciones siempre equiprobables), pero tampoco le dejamos cerrarse en una única acción (eso sería entropía 0); el termostato de α trabaja para mantenerlo en ese punto medio sano mientras aprende.

SAC: dos variantes que conviene no confundir

SAC-pure (actor)

el ACTOR decide

87% · sube a 91% a 3M

→ el actor SÍ funciona

SAC-híbrido (crítico)

el crítico guía la acción

61% a 1,5M · 20% colapsos

→ bimodal: a veces colapsa

Cuidado al leer SAC: el actor puro funciona (87%, sube a 91% a 3M); la variante híbrida es bimodal (61%, a veces colapsa). No son lo mismo.

Y AHORA, PARA EXPERTOS NIVEL EXPERTO

SAC se deriva del **marco de máxima entropía**: en lugar del objetivo estándar de RL, se maximiza el retorno aumentado con la entropía de la política,

$$J(\pi) = \sum_t E[r_t + \alpha \cdot H(\pi(\cdot | s_t))]$$

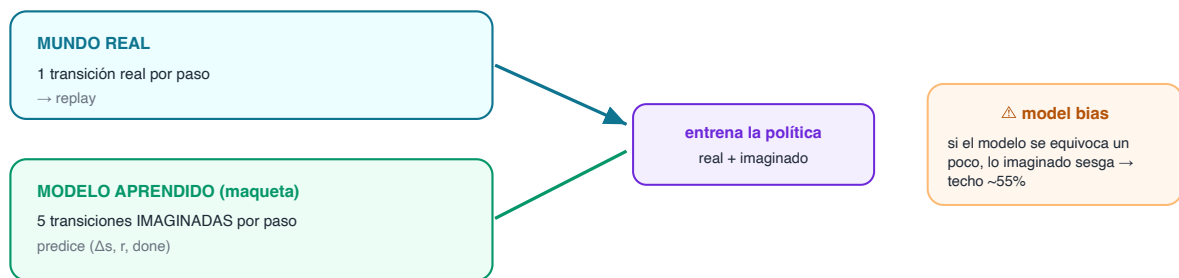
lo que equivale a definir un *soft Q-value* cuyo backup de Bellman incorpora el término entrópico. La implementación usa **dos críticos con clipped double-Q** (Fujimoto et al.) —se entrena cada crítico contra el objetivo formado con el *mínimo* de las dos **redes objetivo**, mitigando el sesgo de sobreestimación maximization bias— más **soft updates** de Polyak con $\tau = 0,01$. La **temperatura α es aprendible** (arranca en 0,20): se optimiza por descenso de gradiente sobre el dual lagrangiano $J(\alpha) = E[-\alpha \cdot (\log \pi(a|s) + \bar{H})]$, con entropía objetivo $\bar{H} = 0,55 \cdot \log(3) \approx 0,60$, de modo que α sube cuando la entropía real cae por debajo de $\bar{H}$ y baja cuando la supera. Trabajamos con la formulación de **SAC discreto** (Christodoulou, 2019): con espacio de acciones finito (3 acciones) no hace falta el truco de reparametrización del SAC continuo; en su lugar, como conocemos toda la distribución categórica $\pi(\cdot | s)$, las pérdidas del actor y de α se calculan tomando la **esperanza explícita sobre las probabilidades de acción** — $\sum_a \pi(a|s) \cdot [\alpha \cdot \log \pi(a|s) - Q(s, a)]$ para el actor—, lo que reduce la varianza del gradiente respecto a muestrear una sola acción. Por qué **SAC-pure $\neq$ SAC-híbrido**: la variante *pure* destila la decisión final en el actor estocástico entrenado contra el soft-Q, mientras que la *híbrida* (SAC-critic-hybrid) acopla la elección de acción al crítico de forma que reintroduce la fragilidad de los métodos basados en valor —de ahí su bimodalidad y su 20% de colapsos frente al 0% del actor puro—; mismo objetivo entrópico, distinto camino al control, distinto perfil de estabilidad. El precio del marco es estructural: cinco redes (actor + 2 críticos + 2 objetivo) lo convierten en el algoritmo de mayor coste de cómputo del banco de pruebas.

World Model — Dyna-Q

Practicar en una maqueta del juego.

Todos los agentes que hemos visto hasta ahora aprenden de la misma forma: juegan una partida de verdad, y de cada paso real extraen algo. Pero jugar de verdad es caro: cada transición cuesta una llamada al simulador. ¿Y si el agente, además de jugar, se construyera una *maqueta* del juego —un pequeño modelo de cómo se mueve la bola y cuándo cae un ladrillo— y entrenara también con partidas imaginadas dentro de esa maqueta? Esa es la apuesta del primer algoritmo *model-based* del libro. La pregunta es doble: ¿de verdad ayuda imaginar?, y si ayuda, ¿hasta dónde llega esa ayuda?

World Model (Dyna-Q): aprender de la realidad Y de lo imaginado



Dyna-Q entrena con la realidad y con la imaginación: por cada paso real, 5 transiciones imaginadas por un modelo aprendido. El precio es el model bias: si el modelo se equivoca, lo imaginado sesga (techo ~55%).

Hasta este punto, DQN, PPO y SAC compartían un rasgo: son **model-free**. No saben nada de la física de Arkanoid; solo saben que ciertas acciones, en ciertos estados, acabaron dando recompensa. El World Model rompe esa norma. Aprende, en paralelo a su política, una segunda red que **predice qué pasará**: dado un estado y una acción, ¿hacia dónde se moverá la bola, qué recompensa llegará, terminará la partida? Con esa predicción, el agente ya no necesita el simulador para practicar: puede generar experiencia él solo. La técnica se llama **Dyna-Q**, y es una de las ideas más elegantes del aprendizaje por refuerzo. También, como veremos con datos, una de las que más claramente choca con un techo.

World Model (Dyna-Q): el agente aprende de experiencia REAL y también de experiencia IMAGINADA



El bucle de Dyna-Q tiene dos fuentes de experiencia que alimentan al mismo agente. A la izquierda, el camino **real**: el agente actúa en el simulador de Arkanoid y recibe una transición verdadera, que guarda en el *replay* y usa para dos cosas a la vez: mejorar la política y mejorar el modelo. A la derecha, el camino **imaginado**: el modelo ya entrenado genera transiciones ficticias —partidas que nunca ocurrieron— con las que el agente también entrena su Q. Lo que interpreta la figura es que el aprendizaje no se limita a la velocidad del simulador: por cada paso real, el agente puede expresar varios pasos imaginados. La pega, que el dibujo no muestra, es que esas transiciones imaginadas son tan buenas como el modelo que las sueña.

La intuición: practicar en una maqueta

Imagina a alguien que aprende a jugar al billar. Una forma de mejorar es jugar partidas reales: tirar, ver qué pasa, corregir. Pero hay otra que multiplica la primera: cerrar los ojos y *visualizar* tiradas —"si golpeo esta bola con este ángulo, irá hacia allí y tocará aquella otra"— y razonar sobre el resultado sin tocar el taco. Esa visualización solo sirve si quien la hace tiene un modelo mental fiable de cómo ruedan las bolas. Si su intuición física es buena, imaginar le ahorra horas de mesa. Si es mala, imaginar le enseña tonterías con mucha convicción.

El World Model hace exactamente eso. Mientras juega partidas reales de Arkanoid, va ajustando una red que aprende la *física aproximada* del juego: cómo cambia el estado al mover la pala, cuándo llega un +1.0 por romper un ladrillo, cuándo se acaba la partida. Y entonces, en lugar de quedarse esperando a que el simulador le dé el siguiente paso, se pone a **soñar pasos**: parte de un estado que ya conoce, elige una acción, y le pregunta al modelo —no al simulador— qué pasaría. Cada uno de esos sueños es una transición más con la que entrenar la política. La promesa es seductora: aprender mucho con pocos datos reales. El riesgo, también claro: si la maqueta miente, el agente practica sobre un Arkanoid que no existe.

TRADUCCIÓN AL JUEGO

El simulador real de Arkanoid sabe la verdad: dada la posición y velocidad de la bola, calcula exactamente el siguiente fotograma, el rebote, el ladrillo que cae. El **modelo** del agente solo sabe una versión aprendida y aproximada de esa verdad. En estados fáciles —la bola subiendo por el centro hacia una pared llena— el modelo acierta casi siempre. En estados delicados —un rebote en una esquina, un combo encadenado, el instante en que se pierde la bola— el modelo se equivoca, y cada error suyo contamina las partidas imaginadas. El agente no distingue un sueño bueno de uno malo: entrena con ambos por igual.

El concepto técnico: Q-net más modelo de dinámica

El World Model no inventa una forma nueva de decidir. Su **Q-net** —la red que estima $Q(s, a)$, el valor de cada acción— es **idéntica a la de DQN**: las mismas dos ramas (encoder convolucional sobre la matriz 8×10 y rama densa sobre los 6 cinemáticos), la misma concatenación, las mismas dos capas de 128, la

misma salida de tres valores Q, el mismo objetivo de Bellman y la misma red objetivo. Todo lo que aprendiste sobre cómo decide y aprende DQN sigue valiendo aquí sin cambiar una coma. La novedad es una **segunda red** que vive al lado: el modelo de dinámica.

El modelo de dinámica es una red densa pequeña (dos capas de 200 unidades, 200×200) que recibe un par estado-acción y predice tres cosas a la vez:

- Δs** El **incremento** del estado: no el siguiente estado entero, sino *cuánto cambia* respecto al actual. El siguiente estado se reconstruye sumando: $s' = s + \Delta s$.
- r** La **recompensa** que el agente recibiría en esa transición (el +1.0 de un ladrillo, el -1.0 de perder la bola, etc.).
- done** Si la transición **termina** el episodio (1) o no (0).

ANTES DE LA FÓRMULA

Predecir el incremento Δs en lugar del estado completo s' es una decisión deliberada y muy común en modelos de dinámica. En Arkanoid, de un paso al siguiente casi todo se queda igual: la mayoría de los 80 ladrillos no cambian, la pala se mueve poco, la bola se desplaza un pasito. Pedirle a la red que prediga el estado entero la obligaría a "recordar" en cada paso las 86 cifras casi idénticas; pedirle solo el cambio la deja concentrarse en lo poco que de verdad se mueve. Es más fácil aprender "la bola subió media celda" que "la bola está exactamente en la coordenada 4,73".

$$\hat{s}' = s + \Delta \hat{s}, \hat{r}, \hat{done} = \text{modelo}(s, a)$$

La diferencia entre lo que el modelo predice y lo que el simulador realmente devolvió es el **error del modelo**, y se mide con el **RMSE** (la raíz del error cuadrático medio): un número que dice, de media, cuánto se desvía la maqueta de la realidad. Cuanto más bajo el RMSE, más fiables son los sueños. Ese número será la clave para entender el techo del algoritmo: el modelo nunca llega a RMSE cero, y ese resto de error no desaparece, se *propaga*.

World Model

rejilla: ritmo (filas) $\times$ planning (columnas) $\cdot$ color = recompensa

		planning $\rightarrow$		
		3	5	8
ritmo $\downarrow$	3e-4	0.07	0.15	0.01
	8e-4	0.07	0.21 ✓	-0.07
	1.5e-3	0.16	0.06	0.00

Mejor: ritmo 8e-4 $\cdot$ planning 5 $\rightarrow$ recompensa 0.21

Ficha técnica del World Model. Comparte con DQN toda la columna de decisión —Q-net de dos ramas, valor $Q(s, a)$, objetivo de Bellman, red objetivo, *replay*— y añade la pieza propia: el modelo de dinámica 200×200 que predice (Δs , r , *done*) y el bucle de *planning* con 5 pasos imaginados por cada paso real. La lectura de la ficha es que aquí no cambiamos el cerebro que decide, sino que le añadimos un simulador interno barato. Por eso el World Model es, ante todo, un DQN que además sueña.

El planning: cinco pasos imaginados por cada paso real

El corazón de Dyna-Q es el **planning**: el proceso de generar y aprovechar experiencia imaginada. La regla en este proyecto es concreta y fácil de recordar: por cada transición **real** que el agente vive en el simulador, genera **5 transiciones imaginadas** con el modelo y entrena la Q-net con todas ellas. Es decir, de cada seis actualizaciones de la política, una se apoya en realidad y cinco en imaginación. Esa proporción 1:5 es la palanca de eficiencia: el agente extrae seis veces más entrenamiento de Q por cada visita —cara— al simulador.

Importa el orden en que arrancan las piezas, porque ninguna de las dos redes sirve recién nacida. El modelo de dinámica **empieza a entrenarse a las 1.000 experiencias** reales: antes de eso no ha visto suficientes transiciones para predecir nada con sentido, y dejarle soñar sería sembrar ruido. La Q-net **empieza a entrenarse a las 2.000 experiencias**, el doble: se le da margen para que el modelo ya tenga una mínima calidad antes de alimentar la política con sueños. Este escalonado —primero recopilar, luego modelar, luego planificar— evita que el agente aprenda de un modelo todavía inútil.

QUÉ MIRAR

No confundas las dos redes ni los dos arranques. Hay **dos** aprendizajes ocurriendo en paralelo: el del modelo (aprender la física, arranca a las 1.000) y el de la política (aprender a jugar, arranca a las 2.000). Y hay **dos** fuentes de datos para la política: 1 transición real y 5 imaginadas por paso. Si lees una curva de este agente, pregúntate siempre de cuál de las dos redes habla: una Q que no mejora puede deberse a un modelo que se ha estancado, no a un problema de la política en sí.

Mini-ejemplo: una transición imaginada, paso a paso

Veamos un sueño concreto, para que "transición imaginada" deje de ser una abstracción. El agente toma un estado s que ya tiene en su *replay*: la bola subiendo hacia un racimo de ladrillos, con cierta velocidad, la pala a la izquierda. Elige una acción $a = \text{"mantener"}$. Y entonces, en lugar de pedirle el resultado al simulador, se lo pregunta al modelo:

TRADUCCIÓN AL JUEGO

El modelo recibe (s, a) y devuelve tres predicciones: un incremento $\Delta\hat{s}$ que dice "la bola sube media celda y se desplaza un poco a la derecha; un ladrillo del racimo desaparece", una recompensa $\hat{r} = +1.0$ (predice que ese ladrillo cae) y $\hat{done} = 0$ (la partida sigue). El agente reconstruye el siguiente estado imaginado $\hat{s}' = s + \Delta\hat{s}$ y arma la transición ficticia $(s, a, +1.0, \hat{s}', 0)$. Esa tupla entra en la Q-net *exactamente igual* que una transición real: se calcula el objetivo de Bellman $\hat{r} + \gamma \cdot (1 - \hat{done}) \cdot Q\text{-objetivo}(\hat{s}', \cdot)$ y se da un paso de gradiente. El agente acaba de aprender de una partida que nunca jugó.

El truco —y el peligro— está en ese $\hat{r} = +1.0$. Si en la realidad ese ladrillo *no* habría caído (porque la bola pasaba de largo), el modelo acaba de inventarse una recompensa, y la Q-net acaba de aprender que "mantener" en ese estado vale más de lo que vale. Un solo sueño erróneo no hace daño; millones de ellos, sesgados en la misma dirección, sí. Cuando además se encadenan —el modelo predice $\hat{s}'$, y desde $\hat{s}'$ se podría predecir $\hat{s}''$ —, los pequeños errores se **componen**: cada paso imaginado parte de un estado que ya era un poco falso, y añade su propia falsedad encima.

EL BUCLE, EN PSEUDOCÓDIGO

```
por cada paso real:
    s' , r , done = simulador.paso(s, a) # transición REAL
    replay.guardar(s, a, r, s', done)
    si experiencias > 1000: entrenar_modelo(replay) # aprende ( $\Delta s, r, done$ )
    si experiencias > 2000: entrenar_Q(s, a, r, s', done)

# PLANNING: 5 pasos IMAGINADOS
repetir 5 veces:
    s_i = replay.muestra_estado()
    a_i = política(s_i)
     $\Delta s, r_i, done_i$  = modelo(s_i, a_i) # NO toca el simulador
    entrenar_Q(s_i, a_i, r_i, s_i +  $\Delta s, done_i$ )
```

El resultado honesto: funciona, pero con techo

Y aquí toca ser claro, porque es el corazón de este capítulo. El World Model **aprende**: pasa de no resolver la tarea a limpiar más de la mitad de los niveles. Pero se topa con un **techo** que ni más cómputo ni más tiempo logran levantar. Los números, sobre el protocolo congelado (5 semillas, evaluación *greedy*, niveles no vistos), son estos:

Examen	Éxito (media de 5 semillas)
TEST-ID (mismo tipo de patrón)	55%
OOD-patrón (patrones no vistos)	53%
OOD-dificultad (densidades no vistas)	42%

Conviene leerlos con cuidado. Un **55%** en TEST-ID significa que, de media entre las cinco semillas, el agente limpia poco más de la mitad de los niveles del mismo tipo que vio entrenando. La caída a **53%** en patrones nunca vistos es pequeña —generaliza razonablemente—, pero el descenso a **42%** cuando cambian las densidades muestra que lo desconocido le pesa. Dos rasgos más, ambos honestos: **0% de colapsos** (ninguna de las cinco semillas se hunde por debajo del 10%, es un agente *estable*) y, a la vez, **0% de semillas por encima del 80%** (ni una sola de las cinco llega a jugar de verdad bien). El agente es fiable y mediocre al mismo tiempo: nunca falla estrepitosamente, pero nunca brilla. Limpia un nivel en unos **1977 pasos**.



Curvas de entrenamiento del World Model. Lo que hay que mirar no es solo la subida inicial —que existe: el agente aprende—, sino la **meseta**: a partir de cierto punto la curva se aplana en torno a su techo y deja de subir aunque se siga entrenando. Esa horizontalidad es la firma visual del *model bias*: el agente ya ha exprimido todo lo que su maqueta imperfecta puede enseñarle, y el error residual del modelo le impide pasar de ahí. No es falta de paciencia ni de datos; es un límite estructural del método.

La prueba más contundente de que el techo es real —y no falta de entrenamiento— está en el barrido por presupuesto. Si medimos el éxito a 700k, 1,5M y 3M de pasos, el World Model hace esto:

700k → 33% 1,5M → 55% 3M → 56%

El salto de 700k a 1,5M es grande (de 33 a 55): el agente está aprendiendo. Pero de 1,5M a 3M apenas se mueve, de **55 a 56**: *duplicar* el cómputo añade un mísero punto. Esa es la huella inconfundible de un techo. Y el cuello de botella tiene nombre: no es el cómputo, es el **sesgo del modelo**. Por mucho que entrenemos la Q-net, sigue aprendiendo de un simulador interno que no es perfecto; el error del modelo pone un suelo al error de la política, y ningún presupuesto extra lo perfora.

QUÉ NO DEMUESTRA

Que el World Model se quede en el 55% **no** demuestra que Dyna-Q sea un mal algoritmo, ni que la idea de imaginar partidas esté equivocada. Demuestra algo más específico: que *en esta tarea*, con *este* modelo de dinámica, la calidad alcanzable de la maqueta pone un techo. Con un modelo más expresivo, o en un entorno cuya física fuera más fácil de aprender, el resultado podría ser otro. Y un matiz importante de honestidad estadística: ese 55% es la **media de cinco semillas** evaluada en **test** sobre niveles no vistos, no el pico de una ejecución afortunada en validación. Es el número que de verdad importa, pero es un número modesto.

Inspector World Model

Q(s,a) del agente Dyna-Q

←		3.290
·		3.396
→		3.779

Estado real → predicho (acción greedy)

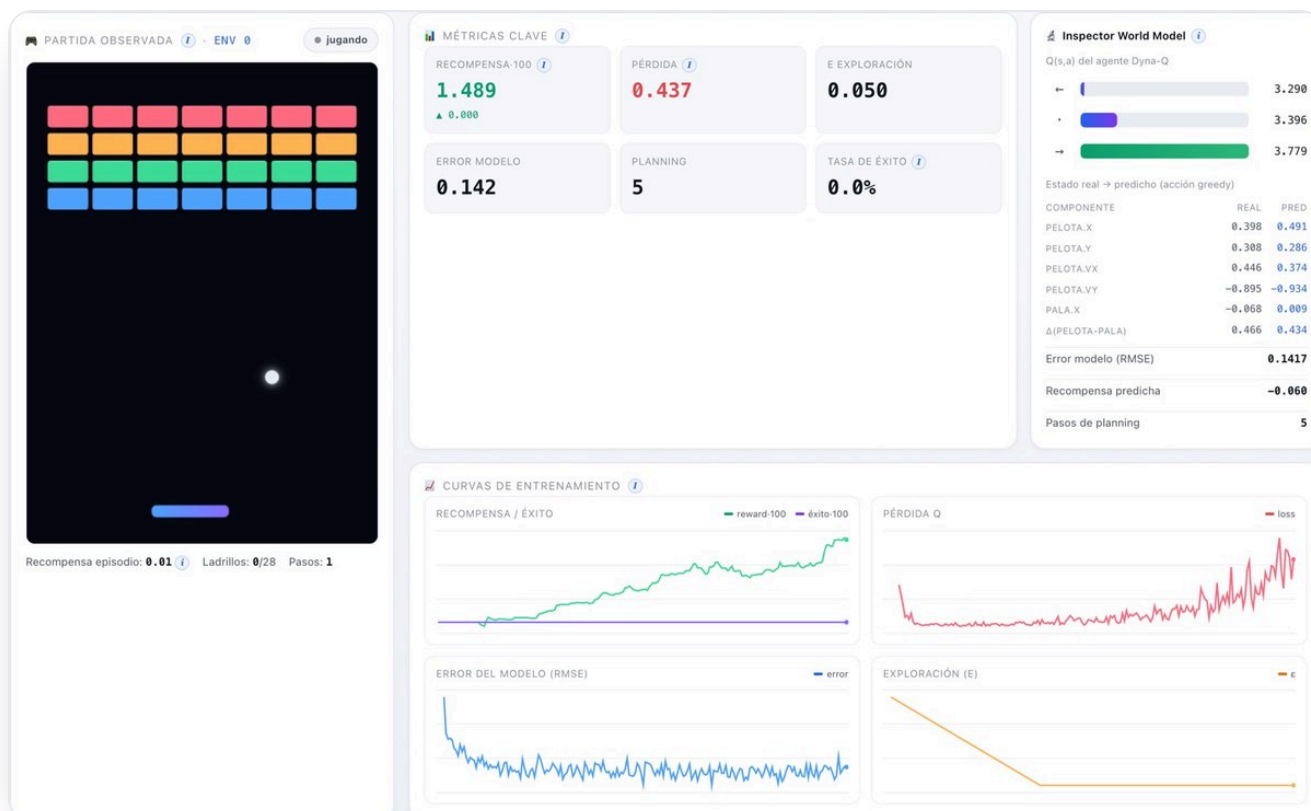
COMPONENTE	REAL	PRED
PELOTA.X	0.398	0.491
PELOTA.Y	0.308	0.286
PELOTA.VX	0.446	0.374
PELOTA.VY	-0.895	-0.934
PALA.X	-0.068	0.009
$\Delta(\text{PELOTA-PALA})$	0.466	0.434

Error modelo (RMSE) **0.1417**

Recompensa predicha **-0.060**

Pasos de planning **5**

El inspector permite abrir al agente por dentro mientras juega. Aquí lo interesante es contrastar lo que el modelo *predice* que pasará con lo que el simulador realmente hace: cuando la bola está en zonas físicamente sencillas, predicción y realidad casi coinciden; en rebotes complicados o en el instante de perder la bola, la predicción se separa. Esa separación, acumulada sobre millones de pasos imaginados, es el origen visible del techo: el agente entrena, en parte, sobre un Arkanoid ligeramente equivocado.



El World Model jugando en la aplicación, con política *greedy* (sin exploración, como en la evaluación). Se le ve competente y prudente: mantiene la bola en juego y rompe ladrillos con regularidad, sin los hundimientos de los agentes inestables. Pero también se nota el límite: no encadena los combos largos ni la puntería fina que llevarían a limpiar el nivel de forma sistemática. Es la imagen, en movimiento, de un agente que nunca colapsa y nunca despunta.

Lectura crítica: la idea más eficiente en datos, con un precio

¿Para qué sirve, entonces, el World Model, si su éxito se queda por debajo de DQN (77%), PPO (91%) o SAC-pure (87%)? Su virtud está en otra columna que las tablas de éxito final no muestran del todo: la **eficiencia en datos**. El planning 1:5 hace que cada transición real rinda mucho más, y por eso a 700k pasos el agente ya está en un 33% mientras arranca su curva. La idea de imaginar es, conceptualmente, la más prometedora de todo el libro para escenarios donde recoger datos reales sea costoso —un robot físico, una simulación lentísima—. El problema es que esa promesa viene atada a una condición dura: necesitas un modelo lo bastante bueno, y aprender la dinámica de un entorno es, a menudo, tan difícil como aprender a jugarlo.

ERROR COMÚN

Pensar que "model-based siempre es mejor porque usa más información". No es así. Añadir un modelo añade también una **nueva fuente de error**: ahora el agente puede equivocarse no solo al valorar acciones (como DQN), sino al imaginar el mundo. Si el modelo es bueno, ganas eficiencia; si no llega, le pones un techo a tu política sin darte cuenta. Lo que decide no es la etiqueta "model-based", sino la **calidad alcanzable del modelo** en esa tarea concreta. En Arkanoid, esa calidad se queda corta, y el resultado es un agente honesto pero con tope.

Y esto enlaza con la tesis del libro entero. El World Model no fracasa por elegir mal el algoritmo; fracasa —relativamente— por una limitación intrínseca de aprender la física de un entorno con física no trivial. La lección no es "Dyna-Q es peor que PPO", sino "imaginar solo vale lo que vale tu imaginación". Cuando la maqueta es fiel, este enfoque es imbatible en eficiencia; cuando no, el sesgo del modelo manda. Decirlo claro es parte del pacto: **este algoritmo tiene techo, y el techo es el sesgo del modelo**.

Quédate con esto

El World Model es el primer agente *model-based*: usa la misma Q-net que DQN y le añade un modelo de dinámica 200×200 que predice $(\Delta s, r, done)$, con $s' = s + \Delta s$. Con ese modelo practica en una maqueta interna —**5 pasos imaginados por cada paso real** (Dyna-Q)—, lo que lo hace muy **eficiente en datos**. El modelo arranca a las 1.000 experiencias y la Q-net a las 2.000. Resultado honesto: TEST-ID **55%**, OOD-patrón **53%**, OOD-dificultad **42%** (media de 5 semillas, test), **0% colapsos** y **0% de semillas >80**: estable pero sin brillo. Y un **techo** claro: 33% a 700k → 55% a 1,5M → 56% a 3M. Doblar el cómputo no sube nada porque el cuello no es el cómputo, sino el **sesgo del modelo**: el agente entrena, en parte, sobre un Arkanoid imperfecto.

AUTOCOMPROBACIÓN

1. ¿En qué se diferencia un agente *model-based* como el World Model de uno *model-free* como DQN? ¿Qué pieza tiene de más, y para qué la usa?
2. El agente entrena su Q-net con 1 transición real y 5 imaginadas por paso. ¿De dónde salen esas 5, y por qué son a la vez una ventaja (¿en qué?) y un riesgo (¿cuál?)?
3. El éxito va de 33% (700k) a 55% (1,5M) a 56% (3M). ¿Qué te dice el salto casi nulo de 1,5M a 3M sobre dónde está el cuello de botella? ¿Es cómputo o es otra cosa?

EJERCICIO

El planning del World Model usa una proporción de 1 transición real por cada 5 imaginadas. Supón que el agente vive 700.000 pasos reales en el simulador. ¿Cuántas *actualizaciones de la Q-net basadas en experiencia imaginada* habrá hecho (aproximadamente, ignorando el arranque a las 2.000), y cuántas basadas en experiencia real? ¿Qué fracción del total de su entrenamiento de Q proviene de partidas que nunca ocurrieron? **Pista:** por cada paso real hay 1 actualización real y 5 imaginadas; calcula ambos totales y divide. **Respuesta:** 700.000 actualizaciones reales y $700.000 \times 5 = 3.500.000$ imaginadas; sobre un total de 4.200.000, eso es $5/6 \approx 83\%$ del entrenamiento de la política procedente de sueños. **Qué significa para el agente:** cinco de cada seis lecciones que recibe la Q-net vienen de la maqueta, no del mundo real. Por eso el agente es tan eficiente en datos reales, y por eso, exactamente, la calidad de esa maqueta —su RMSE— le pone el techo: si el 83% de lo que aprende está teñido por el sesgo del modelo, su política no puede ser mejor que el modelo que la entrena.

Y AHORA, PARA EXPERTOS NIVEL EXPERTO

El World Model implementa **Dyna-Q** en su forma clásica (Sutton, 1991) sobre una Q-net de valor: el modelo de dinámica aprende $\hat{P}(\Delta s, r, \text{done} \mid s, a)$ mediante una red densa 200×200 , y el siguiente estado se obtiene como $s' = s + \Delta s$ (predicción de residuo, no de estado absoluto, que reduce la varianza del objetivo de regresión al aprovechar que la dinámica de Arkanoid es casi-identidad paso a paso). El *planning* ejecuta **5 transiciones imaginadas por paso real**, partiendo de estados muestreados del *replay*; el arranque escalonado (modelo a 1.000 experiencias, Q-net a 2.000) garantiza que la política no se actualice contra un modelo no calibrado. La calidad del modelo se monitoriza por **RMSE**, y es ahí donde aparece el límite teórico: el **model bias**. El error del modelo no es ruido de media cero que se cancele al promediar, sino sesgo sistemático en estados de baja visita (rebotes en esquina, transiciones de **done**, combos), y se **compone** a lo largo de los *rollouts* imaginados —cada paso parte de un $\hat{s}$ ya contaminado—, lo que acota por arriba la calidad de **Q** con independencia del presupuesto. El barrido $33 \rightarrow 55 \rightarrow 56$ (700k/1,5M/3M) es la firma empírica de ese acotamiento: la política satura no por falta de gradiente, sino porque el objetivo de Bellman imaginado, $\hat{r} + \gamma(1 - \text{done})Q\text{-objetivo}(\hat{s}', \cdot)$, está sesgado en una fracción no nula de las muestras. El **ratio de actualizaciones reales/imaginadas** (1:5) gobierna el compromiso: subirlo mejora la eficiencia en datos pero amplifica el *model bias*; bajarlo lo atenúa pero renuncia a la ventaja *model-based*. Aproximaciones modernas (Dyna con *rollouts* cortos tipo MBPO, o ensembles de modelos con cuantificación de incertidumbre que truncan el *rollout* donde el modelo deja de ser fiable) atacan precisamente este término; aquí, con un único modelo determinista, el techo **~55%** es la consecuencia directa y honesta de no acotar el error compuesto.

World Model RNN — Dyna-Q + LSTM

Imaginar con memoria (y un hallazgo honesto).

El World Model normal se construye una maqueta del juego y entrena también sobre jugadas imaginadas, pero su maqueta predice cada paso de forma aislada, sin recordar lo que venía pasando. La tentación es obvia: ¿y si le diéramos *memoria*, de modo que la maqueta tuviera en cuenta los últimos pasos antes de predecir el siguiente? Suena a mejora segura. Este capítulo cuenta qué ocurrió cuando lo probamos de verdad y por qué el resultado fue justo el que no esperábamos.

WM-RNN: añadir memoria... que aquí no compensa (hallazgo honesto)

World Model



55%

WM-RNN (+LSTM)



35%

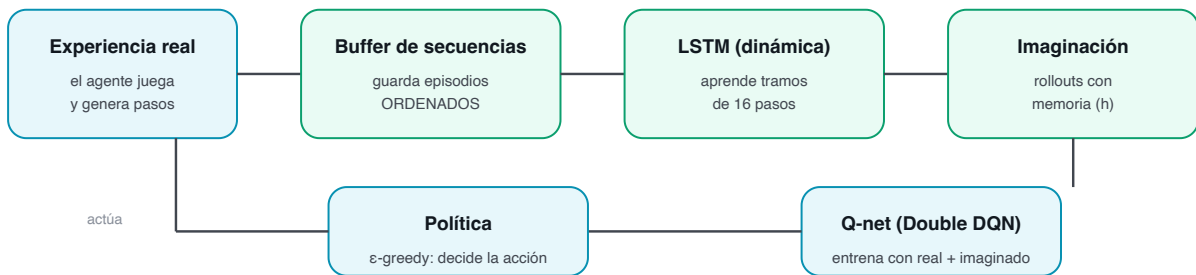
Más complejidad, peor resultado:

- + coste (LSTM, secuencias de 16 pasos)
- + varianza (resultados inestables)
- la observación 8×10 ya es casi markoviana
- → la memoria no tiene nada útil que recordar

El hallazgo honesto: añadir memoria (LSTM) al World Model no mejora aquí (55%→35%). Más coste y varianza, y la observación 8×10 ya es casi markoviana: la memoria no tiene nada útil que recordar.

En el capítulo anterior vimos el World Model con Dyna-Q: un agente que, además de jugar partidas reales, mantiene un modelo de la dinámica del entorno y genera con él transiciones *imaginadas* para exprimir cada experiencia. Aquel modelo era una red densa que, dado un estado y una acción, predecía el siguiente paso sin más contexto que ese par. Aquí cambiamos una sola pieza —el modelo de dinámica— y dejamos todo lo demás idéntico. La pregunta de fondo es si añadir memoria a esa pieza ayuda al agente a jugar mejor a Arkanoid.

World Model RNN: el modelo de dinámica es un LSTM entrenado con secuencias



El World Model recurrente. El esquema es el mismo que el del World Model normal —un Q-net que decide la acción y un modelo de dinámica que imagina transiciones para entrenar al Q-net—, pero el modelo de dinámica ya no es una red densa sin contexto: es una red recurrente que arrastra un estado oculto de un paso al siguiente. La flecha que vuelve sobre sí misma es justamente esa memoria: cada predicción se hace teniendo en cuenta un resumen de lo que acaba de ocurrir, no solo el último estado y la última acción.

La intuición: una maqueta que recuerda

Imagina dos maquetistas que intentan anticipar cómo se moverá la bola. El primero mira una foto fija — dónde está la bola, hacia dónde apunta su velocidad, dónde está la pala— y, a partir de esa única foto, dibuja el siguiente fotograma. El segundo hace lo mismo, pero además lleva en la cabeza un resumen de los últimos instantes: "la bola venía acelerando hacia la esquina, rebotó hace dos golpes, el combo sigue abierto". Si la foto fija no bastara para predecir bien —si faltara información que solo se entiende viendo la secuencia—, el segundo maquetista debería ganar. Esa es exactamente la apuesta de añadirle memoria al modelo de dinámica.

La memoria se materializa en una arquitectura recurrente. En lugar de una red densa que recibe un par (s, a) y escupe el siguiente paso, usamos una red que, además de la entrada, mantiene un **estado oculto**: un vector que se actualiza paso a paso y condensa lo que la red ha "visto" hasta ahora. Ese estado oculto viaja de un instante al siguiente y se convierte en el contexto sobre el que se hace cada nueva predicción. La red recurrente concreta que usamos es una **LSTM** (memoria larga y corta), diseñada precisamente para arrastrar información a lo largo de muchos pasos sin que se diluya.

TRADUCCIÓN AL JUEGO

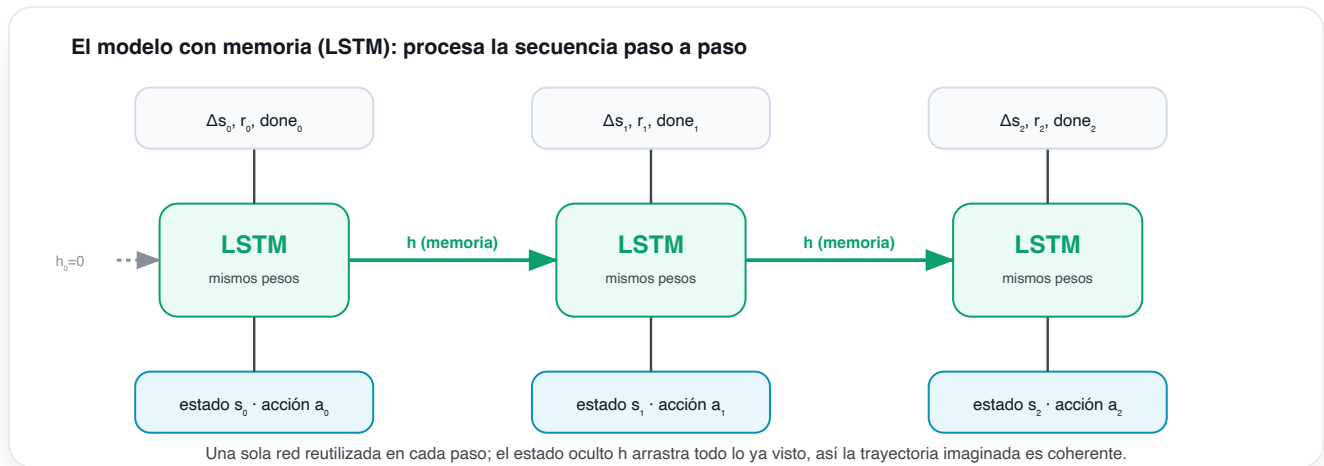
En el World Model normal, la maqueta predice así: «dado este estado s (las posiciones y velocidades de ahora mismo) y esta acción a , el siguiente estado será $s' = s + \Delta s$, con recompensa r y bandera done». En el World Model RNN la predicción es la misma, pero el modelo dispone además de un estado oculto que resume los pasos anteriores de la partida: la maqueta "recuerda" la trayectoria reciente de la bola, no solo su foto actual. Todo lo demás —el Q-net que decide, las recompensas, la rejilla 8x10, la evaluación greedy sobre niveles no vistos— es idéntico al capítulo anterior.

El concepto: la celda LSTM

Una neurona densa corriente calcula su salida solo a partir de la entrada del momento. Una **celda recurrente** añade una segunda entrada: su propio estado del paso anterior. Formalmente, en el instante t recibe la entrada x_t y el estado oculto previo h_{t-1} , y produce un nuevo estado oculto h_t :

$$h_t = f(x_t, h_{t-1})$$

El problema de una recurrencia ingenua es que, al encadenar muchos pasos, la información antigua o bien se desvanece o bien se dispara. La **LSTM** resuelve esto con dos vías de memoria —un estado oculto h_t y un estado de celda c_t — y un juego de **puertas**: pequeñas compuertas que deciden, en cada paso, qué parte de la memoria antigua se olvida, qué parte de la entrada nueva se incorpora y qué parte del estado interno se deja salir. Esa maquinaria de puertas es lo que le permite arrastrar contexto a lo largo de decenas de pasos sin degradarse.



El interior de una celda LSTM. Dos líneas de memoria la recorren —el estado de celda c_t arriba y el estado oculto h_t abajo— y tres puertas las regulan: una decide qué olvidar de la memoria vieja, otra qué escribir de la entrada nueva y otra qué dejar salir como contexto hacia el paso siguiente. El resultado es una memoria que puede conservar información útil durante muchos pasos o descartarla cuando deja de serlo; eso es lo que distingue a una LSTM de una recurrencia simple.

- x_t** La entrada del paso t . En nuestro modelo de dinámica, una representación del estado del juego y la acción tomada en ese instante.
- h_t** El estado oculto: un vector de tamaño 128 que resume la secuencia vista hasta t y que se pasa al paso siguiente. Es la "memoria de trabajo" de la maqueta.
- c_t** El estado de celda: la memoria de largo recorrido que las puertas protegen del desvanecimiento.
- Puertas** Compuertas que, paso a paso, regulan qué se olvida, qué se incorpora y qué se deja salir. Son lo que convierte la recurrencia en memoria gobernable.

Traducción: cómo se entrena con secuencias

Aquí está la diferencia operativa más importante respecto al World Model normal. Una red sin memoria se entrena con transiciones sueltas y barajadas: se toma un lote de tuplas $(s, a, r, s', done)$ al azar del buffer y se actualiza. Pero una red recurrente **no puede** aprender de transiciones sueltas y desordenadas, porque su salida depende del estado oculto, y el estado oculto solo tiene sentido si las transiciones llegan *en orden* y *en cadena*. Por eso el modelo de dinámica recurrente se entrena con **secuencias**: trozos consecutivos de una partida.

ANTES DE LA FÓRMULA

No hay una ecuación nueva que destacar aquí: la pérdida sigue siendo el error de predicción del modelo (cuánto se equivoca al anticipar $\Delta s, r$ y $done$). Lo que cambia es *sobre qué* se calcula esa pérdida. En vez de transiciones independientes, se procesa una secuencia entera arrastrando el estado oculto de principio a fin, y el gradiente se propaga hacia atrás a lo largo de toda la secuencia. A esa técnica se le llama retropropagación a través del tiempo.

Los números concretos de nuestra configuración: la LSTM tiene un estado oculto de tamaño **128**; se entrena con secuencias de **16 pasos** consecutivos; se procesan **32** secuencias por lote; y el buffer guarda los últimos **256 episodios** completos (no transiciones sueltas, sino partidas enteras, porque hacen falta tramos ordenados para formar las secuencias). El resto del agente —el Q-net que decide la acción, el esquema Dyna-Q de imaginar transiciones para entrenarlo, la recompensa, la observación 8x10— se hereda intacto del World Model.

ERROR COMÚN

Pensar que "más memoria es siempre mejor". La memoria recurrente no es gratis: introduce parámetros nuevos, exige entrenar con secuencias (más costoso y con más varianza que barajar transiciones) y solo aporta valor si en la observación falta información temporal que la secuencia sí contiene. Si el estado actual ya basta para predecir el siguiente —si la tarea es casi *markoviana*—, la memoria no tiene nada útil que recordar y se limita a añadir ruido. Que un mecanismo sea más sofisticado no implica que ayude en *esta* tarea.

Un mini-ejemplo: qué ve la maqueta paso a paso

Sigamos una secuencia corta para ver dónde *podría* ayudar la memoria. Supongamos cuatro pasos consecutivos de una partida:

- Paso 1** La bola sube hacia los ladrillos. La maqueta sin memoria predice el siguiente paso a partir de la foto; la maqueta con memoria empieza a construir su estado oculto h_1 .
- Paso 2** La bola rompe un ladrillo ($r = +1.0$) y sigue. El estado oculto pasa a h_2 arrastrando que veníamos de una subida limpia.
- Paso 3** La bola rompe un segundo ladrillo sin tocar la pala: es un combo ($+1.0$ más $+0.5$ de bonus). Aquí la memoria *podría* importar: saber que el combo sigue abierto.
- Paso 4** La bola cambia de dirección. La maqueta con memoria dispone de h_3 para predecir mejor el rebote; la maqueta sin memoria solo tiene la foto del paso 3.

En teoría, el paso 3 es el caso donde la memoria justifica su existencia: el bonus de combo depende de la *historia* reciente (no haber tocado la pala desde el último golpe), no solo del estado instantáneo. Si esa información no estuviera en la observación, la LSTM tendría algo real que aportar. La pregunta empírica es si, en nuestra formulación concreta, esa información *ya está* en el estado o no. Y la respuesta, como veremos, decide todo.

El resultado, sin adornos

Medimos el World Model RNN con el mismo protocolo congelado que todos los demás: 5 semillas, evaluación greedy, los tres conjuntos de test disjuntos. Estos son sus números a 1,5 millones de pasos, junto al World Model normal como referencia.

QUE MIRAR

No mires solo la primera columna. Compara la fila del World Model RNN con la del World Model normal: el añadido de memoria **baja** la nota en los tres conjuntos. Y fíjate especialmente en la última columna, los pasos para limpiar, junto con el dato de presupuesto que viene después: el RNN parece "rápido" (1370 pasos), pero esa rapidez no se traduce en más éxito, y su comportamiento con el presupuesto no es monótono. La lectura importante no es una barra alta, sino una mejora esperada que no aparece.

Agente	TEST-ID	OOD-patrón	OOD-dificultad	% colapsos	% semillas >80	Pasos limpiar
World Model (Dyna-Q)	55	53	42	0%	0%	1977
World Model RNN	35	29	27	0%	0%	1370

El veredicto es claro y es negativo: el World Model RNN logra un **35%** en TEST-ID, frente al **55%** del World Model normal. Cae también en OOD-patrón (29 frente a 53) y en OOD-dificultad (27 frente a 42). No sufre colapsos (0% de semillas por debajo del 10%) y ninguna de sus cinco semillas supera el 80% de éxito (0%), igual que el World Model normal. Es decir: **añadir memoria no solo no mejora; empeora**.

El comportamiento con el presupuesto de entrenamiento lo confirma y lo agrava. Si en lugar de mirar un solo punto seguimos el éxito en TEST-ID a medida que damos más pasos, el World Model normal sube y se estanca en un techo (33 a 700k → 55 a 1,5M → 56 a 3M): mejora de forma *monótona* hasta su límite. El World Model RNN, en cambio, hace esto:

Agente	700k pasos	1,5M pasos	3M pasos	Patrón
World Model (Dyna-Q)	33	55	56	monótono, techo ~55
World Model RNN	54	35	40	no-monótono, alta varianza

Léelo despacio: **54 → 35 → 40**. Más entrenamiento *empeora* el resultado entre 700k y 1,5M, y luego rebota a medias. Esto es lo contrario de lo que esperamos de un aprendizaje sano. Un agente que aprende debería mejorar, o como mucho estancarse, al recibir más datos; no *desaprender*. La no-monotonía y la varianza alta son la firma de un proceso de aprendizaje frágil, donde el resultado depende más del azar de la semilla y del punto exacto de corte que de una competencia que se consolida.

QUE NO DEMUESTRA

Que la memoria recurrente sea mala "en general" para el aprendizaje por refuerzo. No lo es: las arquitecturas recurrentes son la herramienta estándar precisamente cuando la observación es parcial y hay que integrar información en el tiempo. Lo que estos números demuestran es algo mucho más local y honesto: en **esta** tarea, con **esta** observación 8x10, la memoria recurrente no aporta y sí estorba. Cambiar de tarea —una con observación verdaderamente parcial— podría invertir el resultado por completo.

Lectura crítica: por qué la memoria no ayuda aquí

La explicación está en la naturaleza de nuestra observación. Recordemos qué ve el agente: 6 cinemáticos (posición de la bola, su velocidad en x e y, posición de la pala, distancia bola-pala) más la matriz de ocupación 8x10 con qué ladrillos siguen vivos. Crucialmente, **la velocidad ya forma parte del estado**. Y la velocidad es justamente la información que normalmente exige memoria: para saber hacia dónde va la bola con una foto fija haría falta recordar la foto anterior. Pero aquí no hace falta recordar nada, porque la velocidad *ya está dada*.

Dicho en el lenguaje de la Parte II: una observación es **markoviana** cuando el estado actual basta para predecir el futuro, sin necesidad de recordar el pasado. Cuando un problema es markoviano, es un MDP y la memoria es superflua; cuando no lo es, es un POMDP y la memoria —mantener una *creencia* integrando varias observaciones— se vuelve valiosa. Nuestra observación 8x10, con la velocidad incluida, es **casi markoviana**: contiene prácticamente toda la información necesaria para anticipar el siguiente paso. Y sobre una observación casi markoviana, la LSTM no encuentra nada útil que recordar.

Lo único que añade es complejidad: parámetros extra, entrenamiento con secuencias, más varianza. Memoria sin información que memorizar es ruido con otro nombre.

TRADUCCIÓN AL JUEGO

Volvamos al mini-ejemplo del combo. Parecía que el paso 3 (saber que el combo sigue abierto) era donde la memoria ganaría. Pero esa información, en la práctica, ya queda determinada por el estado y la dinámica del entorno que el modelo aprende de todos modos. El maquetista con buena foto —y nuestra foto incluye la velocidad— no necesita recordar la trayectoria: la lee directamente de lo que tiene delante. La memoria solo habría marcado la diferencia si le hubiéramos quitado la velocidad al estado, es decir, si hubiéramos roto la observación para volverla parcial a propósito.

Hay un segundo hallazgo que ata este capítulo con la tesis del libro, y es de los más reveladores. En los capítulos de DQN y del World Model normal vimos un ingrediente de la ablación: hacer que ϵ baje *rápido* (explorar poco, explotar pronto) apenas perjudica a DQN (cae solo 2,6 puntos). Es un cambio casi inocuo para los métodos de valor con observación markoviana. Pues bien: ese mismo cambio **perjudica** al World Model RNN. No es un detalle anecdótico: indica que su aprendizaje tiene *otra naturaleza*. Una red recurrente necesita ver secuencias variadas y largas para que su memoria aprenda a representar la dinámica; recortar la exploración demasiado pronto la deja sin el material que necesita. El mismo botón que para DQN es casi irrelevante, para el RNN es nocivo. Tocar un hiperparámetro sin entender qué algoritmo tienes delante es jugar a ciegas.

Y aquí está la lección que justifica dedicar un capítulo entero a algo que no funcionó. La hipótesis "la memoria ayudará" era razonable, elegante y muy citada en la literatura. De haberla dado por buena sin medir, habríamos publicado una mejora inexistente. Lo único que la desmontó fue el experimento: el mismo protocolo congelado, las mismas 5 semillas, la misma evaluación honesta sobre niveles no vistos. **Medir, no suponer.** Un resultado negativo bien medido enseña tanto como uno positivo, porque acota lo que de verdad importa —aquí, que la clave estaba en la observación, no en la arquitectura de la maqueta— y nos vacuna contra la sofisticación que no rinde.

World Model RNN

rejilla: ritmo (filas) × long. secuencia (columnas) · color = recompensa

		long. secuencia →		
		8	16	32
ritmo ↓	3e-4	-0.09	0.16 ✓	0.09
	8e-4	0.09	-0.05	0.01
	1.5e-3	0.03	-0.06	0.06

Mejor: ritmo 3e-4 · long. secuencia 16 → recompensa 0.16

Ficha-resumen del World Model recurrente: modelo de dinámica LSTM (estado oculto 128), entrenado con secuencias de 16 pasos, 32 por lote, sobre un buffer de 256 episodios; Q-net y esquema Dyna-Q heredados del World Model normal. Resultado: 35% en TEST-ID frente al 55% de la variante sin memoria, con un perfil de presupuesto no-monótono. La ficha registra un hallazgo negativo honesto, no una mejora.

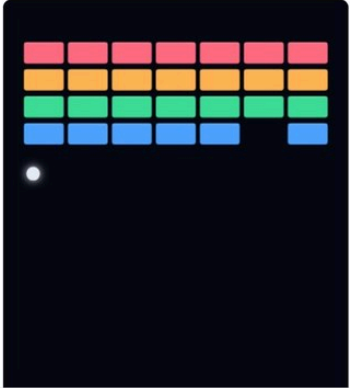
**Arkanoid DRL Learning Lab**
Estudia el bucle agente-entorno, los datos que genera y cómo aprende cada algoritmo · DQN · PPO · SAC · World Model · World Model RNN

backend: cpu entrenando Paso Reiniciar Pausar

Motivación del laboratorio. Esta interfaz está pensada como guía de estudio: no solo muestra al agente jugando, también el **mecanismo interno**. Por eso hay un icono de información (i) junto a cada grupo y cada dato importante. **Pulsa cualquier** i para abrir una ficha completa que explica qué es, por qué importa y, cuando aplica, su fórmula y sus datos.

Laboratorio Comparativa de modelos

PARTIDA OBSERVADA i ENV 0 jugando



MÉTRICAS CLAVE i

RECOMPENSA 100 i
0.095
▼ -0.004

PÉRDIDA i
0.012

E EXPLORACIÓN i
0.050

ERROR MODELO i
0.126

MEMORIA LSTM i
128

TASA DE ÉXITO i
0.0%

Inspector World Model i

Q(s,a) del agente Dyna-Q

← 0.867

· 0.735

→ 0.861

Estado real → predicho (acción greedy)

COMPONENTE	REAL	PRED
PELOTA.X	-0.826	-0.843
PELOTA.Y	-0.265	-0.236
PELOTA.VX	0.782	0.772
PELOTA.VY	-0.623	-0.558
PALA.X	0.620	0.576
Δ(PELOTA-PALA)	-1.000	-0.980

Error modelo (RMSE) **0.1264**

Recompensa predicha **0.050**

Pasos de planning **5**

El World Model RNN en la aplicación de demostración del laboratorio: a la izquierda, la partida real; a la derecha, lo que la maqueta recurrente "imagina" como siguiente paso. Sirve para ver el mecanismo en marcha —la maqueta arrastra su estado oculto a lo largo de la secuencia—, no para concluir nada sobre el rendimiento: las cifras salen del protocolo congelado, no de una partida suelta.

Quédate con esto

El World Model RNN es el World Model normal con una sola pieza cambiada: el modelo de dinámica es una **LSTM** (memoria recurrente) entrenada con **secuencias de 16 pasos**, en vez de una red densa sin contexto. La idea —dar memoria a la maqueta— suena a mejora segura, pero **no lo es aquí**: baja a un 35% en TEST-ID frente al 55% del World Model normal, y su éxito con el presupuesto es **no-monótono** ($54 \rightarrow 35 \rightarrow 40$) y de alta varianza. ¿La razón? Nuestra observación 8x10 incluye la velocidad y es **casi markoviana**: el estado actual ya basta para predecir el futuro, así que la memoria no tiene nada útil que recordar y solo añade ruido. La moraleja del capítulo es la tesis del libro en su forma más pura: **medir, no suponer**. Un resultado negativo, bien medido, enseña.

AUTOCOMPROBACIÓN

1. ¿Qué única pieza distingue al World Model RNN del World Model normal, y por qué obliga eso a entrenar con secuencias en vez de con transiciones sueltas?
2. El éxito del World Model RNN en TEST-ID va de 54 (700k) a 35 (1,5M) y a 40 (3M). ¿Por qué decimos que esa progresión "no-monótona" es señal de un aprendizaje frágil?
3. ¿Por qué una observación que incluye la velocidad de la bola hace que la memoria recurrente apenas tenga nada que aportar? Relaciona tu respuesta con la diferencia entre un MDP y un POMDP.

EJERCICIO

Imagina que rehacemos el experimento del World Model RNN pero **quitando la velocidad** del estado: el agente solo ve la posición de la bola y de la pala, sin v_x ni v_y . **Pista:** piensa qué información necesita el modelo para anticipar hacia dónde se moverá la bola, y dónde tendría que ir a buscarla ahora que la velocidad no está dada explícitamente. **Respuesta:** sin la velocidad, una sola foto ya no basta para saber la dirección de la bola; haría falta comparar la posición actual con la anterior, es decir, recordar el pasado. La observación pasaría de casi markoviana a claramente parcial (un POMDP), y ahí la memoria de la LSTM *sí* tendría información real que integrar. **Qué significa para el agente:** la utilidad de la memoria no es una propiedad del algoritmo, sino de la formulación de la tarea; el mismo LSTM que aquí estorba podría ser decisivo si rompiéramos la observación a propósito.



Y AHORA, PARA EXPERTOS NIVEL EXPERTO

El modelo de dinámica es una **LSTM con estado oculto de 128** entrenada con **secuencias de 16 pasos** mediante retropropagación a través del tiempo truncada (**truncated BPTT**), **32 secuencias por lote**, sobre un buffer de **256 episodios** completos —no transiciones sueltas, porque la recurrencia exige tramos ordenados—. La motivación teórica es impecable: el estado oculto h_t actúa como aproximación de un **belief state** bajo observación parcial, integrando la historia $(o_{1:t}, a_{1:t-1})$ en un resumen suficiente para decidir, que es exactamente la receta canónica para atacar un POMDP. El problema es que la premisa no se cumple: la observación 8×10 incluye las velocidades v_x, v_y y es **casi markoviana**, de modo que $P(s' | s, a)$ ya es casi independiente de la historia. Sobre un MDP, el belief state colapsa al propio estado y la recurrencia no añade información, solo capacidad y, con ella, varianza. Eso explica el perfil no-monótono (**54** → **35** → **40** por presupuesto) y la alta varianza entre semillas: la dinámica de optimización del BPTT sobre secuencias correlacionadas, con un buffer pequeño de episodios y la inestabilidad propia de propagar gradientes en el tiempo, hace que el aprendizaje del modelo y el del Q-net entren en regímenes que no se consolidan monótonamente. Es coherente, además, con la asimetría frente a **epsilon**: el **epsilon**-decay rápido que apenas afecta a DQN ($\Delta -2,6$) **perjudica** al WM-RNN, porque una LSTM necesita exposición a secuencias largas y variadas para que su memoria aprenda representaciones útiles; recortar la exploración temprano la deja sin cobertura de trayectorias. La conclusión de diseño es nítida: ante una tarea casi markoviana, ampliar la observación (la matriz 8×10) domina a añadir memoria recurrente; la capacidad extra sin información extra que explotar se paga en varianza, no en rendimiento.

Los cinco, cara a cara

Valor o política, on u off-policy, con o sin modelo.

Hemos recorrido cinco algoritmos por separado: DQN, PPO, SAC, el World Model y su versión con memoria, el WM-RNN. Cada uno con su capítulo, su intuición y su pseudocódigo. Pero quien los ve por primera vez tiene derecho a una pregunta sencilla y a la vez incómoda: ¿no hacen todos lo mismo? ¿En qué se diferencian de verdad, más allá del nombre? Y, sobre todo, ¿hay alguna forma de ponerlos en orden, de clasificarlos, para no tener que memorizar cinco recetas sueltas sino entender tres o cuatro ideas de las que todas ellas son variantes?

Los cinco, cara a cara: un mapa

Dos ejes: aprende valor↔política y sin modelo↔con modelo. Forma = on/off-policy.



Los cinco algoritmos en un mapa: el eje vertical es qué aprenden (valor↔política), el horizontal si usan modelo del mundo, y la forma distingue on-policy (PPO) de off-policy (el resto).

Este capítulo es de **síntesis**. No introduce ningún algoritmo nuevo ni vuelve a desmenuzar los que ya conocemos: pone orden antes de llegar a los resultados. La tesis del libro entero es que el cuello de botella nunca fue el algoritmo, sino la **formulación** de la tarea; pero para defender esa tesis con honestidad hay que entender primero qué familia de soluciones representa cada uno de los cinco, porque solo así se ve que ninguno de ellos era, por sí mismo, la pieza que faltaba. Aquí comparamos *filosofías*, no cifras. Los números duros (porcentajes de éxito, semillas, colapsos) llegan en el capítulo de resultados; este es el mapa que conviene tener en la cabeza antes de mirarlos.

TRADUCCIÓN AL JUEGO

Imagina que cinco jugadores distintos aprenden a jugar a nuestro Arkanoid de rejilla 8×10 , cada uno con un método propio. El primero lleva una libreta donde anota, para cada situación y cada movimiento de pala, cuánto cree que va a ganar a la larga; nunca decide directamente, siempre consulta la libreta y elige el movimiento con mejor nota. El segundo no lleva libreta de movimientos: lleva una *forma de jugar* que va afinando, y un segundo cuaderno aparte donde solo apunta lo buena que era cada situación, para saber si su última tanda de partidas fue mejor o peor de lo esperado. El tercero hace como el segundo, pero además se obliga a no jugar siempre igual: premia un punto la variedad, para no encasillarse. El cuarto, antes de jugar de verdad, construye en su cabeza una *maqueta* del juego y practica miles de partidas imaginarias contra esa maqueta. El quinto hace lo mismo, pero su maqueta tiene memoria: recuerda lo que pasó hace varios pasos. Son cinco filosofías de aprender, no cinco trucos. Clasificarlas es el objeto del capítulo.

Un marco para comparar: tres preguntas, no cinco fichas

La forma perezosa de estudiar estos algoritmos es memorizarlos uno a uno. La forma útil es darse cuenta de que las diferencias entre ellos se reducen, casi siempre, a la respuesta que cada uno da a **tres preguntas**. Una vez tienes esas tres preguntas en la cabeza, no necesitas recordar cinco recetas: necesitas situar cada algoritmo en tres ejes. Las preguntas son estas:

Eje 1 · ¿Qué aprende: el valor o la política?

¿El algoritmo aprende a **puntuar** situaciones y acciones (un valor $Q(s, a)$ del que luego deduce qué hacer) o aprende **directamente** una forma de actuar, la política $\pi(a | s)$, sin pasar por una tabla de notas explícita?

Eje 2 · ¿Reutiliza la experiencia o no? (on-policy / off-policy)

¿Puede aprender de partidas **viejas**, jugadas por versiones anteriores de sí mismo y guardadas en una memoria (un **replay buffer**), o necesita **datos frescos**, recién generados por su política actual, y los tira en cuanto cambia?

Eje 3 · ¿Construye un modelo del mundo o no? (sin modelo / con modelo)

¿Aprende a actuar directamente de la experiencia (**model-free**) o primero aprende a **predecir** cómo responde el entorno (un modelo de la dinámica) y luego usa esa maqueta para planificar o para generar experiencia imaginaria (**model-based**)?

Tres preguntas, y con ellas se ordena todo el zoo de algoritmos modernos, no solo nuestros cinco. Lo bonito es que los ejes son **independientes**: un algoritmo puede aprender la política y a la vez ser off-policy y a la vez tener modelo, o cualquier otra combinación. Veámoslos uno a uno, y luego los juntamos en una sola tabla.

QUÉ MIRAR

El error al leer estos tres ejes es tratarlos como sinónimos. No lo son. «Aprender el valor» (eje 1) no implica reutilizar experiencia (eje 2), aunque en la práctica DQN haga las dos cosas. «Tener modelo» (eje 3) no dice nada sobre si aprendes valor o política. Lee cada eje como una pregunta separada y resiste la tentación de fundirlos: justo esa independencia es lo que permite que existan tantos algoritmos distintos.

Eje 1 — Aprender el valor frente a aprender la política

Esta es la división más antigua y más profunda del campo. Hay dos maneras de que un agente sepa qué hacer.

La primera es **aprender a valorar**. El agente estima una función $Q(s, a)$: «si estoy en la situación s y muevo la pala así, ¿cuánta recompensa total espero a futuro?». Una vez tiene esa función, actuar es trivial: en cada estado mira las tres acciones (izquierda, mantener, derecha), consulta su nota y elige la mejor. La política no se aprende por separado; **emerge** de la función de valor, como el movimiento mejor puntuado. Es la filosofía de **DQN**, el cimiento de este libro: una sola red que aprende Q , sencilla, fiable y fácil de entender.

La segunda es **aprender a actuar directamente**. Aquí el agente lleva una red que es la política: recibe el estado y devuelve, de una vez, las probabilidades de cada acción. No hay que comparar notas; la red dice ya qué hacer. El problema es que entrenar esa política a ciegas tiene mucho ruido, así que casi siempre se le añade un segundo cuaderno, una red de valor que estima lo buena que era cada situación y sirve de referencia para saber si la última jugada fue mejor o peor de lo esperado. A esa pareja —**actor** que decide, **crítico** que evalúa— se la llama **actor-crítico**, y es la familia de **PPO** y de **SAC**.



El esquema actor-crítico. El **actor** (la política) decide qué acción tomar a partir del estado; el **crítico** (la función de valor) juzga lo buena que era esa situación y devuelve al actor una señal —la **ventaja**— que le dice si lo que hizo fue mejor o peor de lo esperado. Dos redes que se ayudan: una actúa, la otra corrige.

ANTES DE LA FÓRMULA

El crítico no le dice al actor qué acción era la correcta (eso sería aprendizaje supervisado, y no lo tenemos). Le dice algo más sutil: si la recompensa que de verdad llegó fue **mejor o peor** de lo que el propio crítico esperaba. Esa diferencia, la **ventaja**, es la señal con la que el actor empuja su política: «esto salió mejor de lo previsto, hazlo más a menudo; esto salió peor, hazlo menos». No es una instrucción, es una corrección relativa.

¿Cuál conviene? El enfoque de valor brilla cuando las acciones son pocas y discretas —y las nuestras lo son: solo tres— porque comparar tres notas es barato. El enfoque de política gana terreno cuando hay muchísimas acciones o son continuas (un volante que gira cualquier ángulo, un brazo robótico), porque ahí comparar notas una a una es imposible y conviene que la red proponga la acción directamente. En nuestro Arkanoid de tres botones, las dos familias compiten de tú a tú; por eso el laboratorio prueba ambas.

Eje 2 — On-policy frente a off-policy: ¿se puede reciclar el pasado?

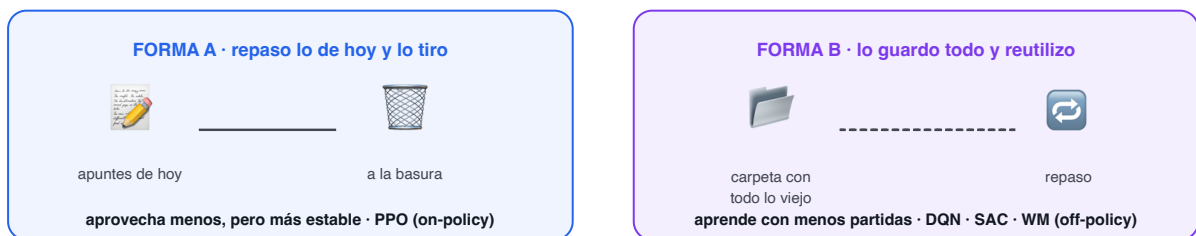
Este eje suena técnico y es, en la práctica, el que más decide el **coste** y la **estabilidad** de un entrenamiento. La pregunta es: ¿las partidas que el agente ya jugó sirven para seguir aprendiendo, o

caducan?

Un algoritmo **off-policy** puede aprender de experiencia ajena a su política actual. Guarda en una memoria —el **replay buffer**— millones de transiciones (s, a, r, s') de partidas pasadas, y reentrena una y otra vez sobre ellas, aunque las jugara una versión vieja de sí mismo. Reutiliza. Es el caso de **DQN**, de **SAC** y de los dos World Models. La ventaja es enorme: cada partida jugada se expresa muchas veces, así que se aprende con *menos* interacción con el entorno (más **eficiencia en datos**).

Un algoritmo **on-policy**, en cambio, solo puede aprender de datos **frescos**, generados por su política *actual*. En cuanto la política cambia un poco, las partidas viejas dejan de ser representativas y hay que tirarlas. No usa replay. Es el caso de **PPO**: juega una tanda, aprende de ella, la descarta y vuelve a jugar. Suena derrochador, y lo es en cantidad de partidas; a cambio, evita un problema sutil de los métodos off-policy (aprender de datos que ya no se parecen a lo que el agente haría hoy), lo que suele traducirse en entrenamientos más **estables** y predecibles.

¿Aprovecha solo lo último o también lo viejo? Dos formas de estudiar



On-policy frente a off-policy. A la izquierda, el flujo de PPO: generar, aprender, **descartar**. A la derecha, el flujo de DQN, SAC y los World Models: cada transición cae en un **replay buffer** y se reutiliza muchas veces. Reciclar el pasado da eficiencia en datos; exigir datos frescos da estabilidad.

ERROR COMÚN

Creer que «más eficiente en datos» es siempre «mejor». No lo es. Eficiencia en datos significa que aprende con menos partidas, lo cual importa cuando interactuar con el entorno es caro (un robot real, un coche). Pero en un simulador como el nuestro generar partidas es **barato**, y lo que de verdad escasea es la estabilidad y el no tener que reajustar mil veces. Por eso un método on-policy y «derrochador» como PPO puede ganar a uno off-policy y «ahorrador» como el World Model: gastar más partidas no cuesta casi nada, mientras que un techo de rendimiento o un colapso sí cuestan. La eficiencia en datos es una virtud, no la virtud.

Eje 3 — Sin modelo frente a con modelo: ¿aprender a jugar o aprender a soñar el juego?

El tercer eje separa a los que aprenden a actuar *directamente* de los que primero aprenden a **imaginar** el entorno.

Un método **sin modelo** (model-free) no intenta entender cómo funciona el juego. Solo aprende, de la experiencia bruta, qué conviene hacer: o una función de valor (DQN), o una política (PPO, SAC). El mundo es una caja negra que da recompensas; el agente no la modela, la explota. La mayoría de los algoritmos clásicos son de esta familia.

Un método **con modelo** (model-based) hace un rodeo: primero aprende una red que **predice la dinámica** —dado s y a , qué siguiente estado s' y qué recompensa r saldrán— y luego usa esa maqueta para entrenar *sin jugar partidas reales*, generando experiencia imaginaria. Es la idea de **Dyna-Q**

que siguen nuestros dos World Models: una parte aprende el modelo del Arkanoid, otra entrena la política dentro de ese modelo, soñando partidas. El **WM-RNN** añade a la maqueta una memoria LSTM, para que el modelo pueda recordar lo que pasó varios pasos atrás y no solo el instante presente.

TRADUCCIÓN AL JUEGO

Un agente sin modelo aprende a jugar al Arkanoid jugando al Arkanoid. Un agente con modelo aprende primero a *predecir* el Arkanoid —«si la bola va aquí con esta velocidad y muevo la pala, rebotará así»— y después practica miles de partidas contra esa predicción, su «Arkanoid de juguete». Si la maqueta es buena, aprende rapidísimo y con pocas partidas reales. Pero si la maqueta se equivoca —y siempre se equivoca un poco—, el agente aprende a jugar de maravilla a un juego que no existe. A ese fallo lo llamamos **model bias**: el agente queda limitado por la calidad de su propio sueño, y de ahí surge un *techo* de rendimiento que ningún entrenamiento extra rompe.

Los cinco, en una sola tabla

Con los tres ejes definidos, cada uno de nuestros algoritmos queda situado por su combinación de respuestas. Esta es la fotografía completa. Las dos últimas columnas no son cifras de éxito (esas van en el capítulo de resultados), sino el **rasgo cualitativo** de cada familia y su coste relativo de cómputo.

Algoritmo	Eje 1 · Qué aprende	Eje 2 · On/off-policy	Eje 3 · ¿Modelo?	Rasgo dominante (laboratorio)	Coste de cómputo
DQN	Valor $Q(s, a)$	Off-policy (replay)	Sin modelo	Cimiento simple y fiable; el punto de partida	Bajo (1 red + objetivo)
PPO	Política (actor-crítico)	On-policy (datos frescos, sin replay)	Sin modelo	La convergencia más rápida tras afinar; la más fiable	Medio (actor + crítico)
SAC	Política (actor-crítico, máxima entropía)	Off-policy (replay)	Sin modelo	La más estable, pero la más cara	Alto (hasta 5 redes)
World Model	Política sobre un modelo (Dyna-Q)	Off-policy	Con modelo	Eficiente en datos, pero con techo (<i>model bias</i>)	Medio-alto (modelo + política)
WM-RNN	Política sobre un modelo con memoria	Off-policy	Con modelo (+ LSTM)	A veces competitivo, pero exige explorar más	Alto (modelo recurrente + política)

QUÉ MIRAR

Fíjate en que PPO es el único **on-policy** de los cinco, y el único que no usa replay buffer. Esa casilla aislada en la columna del eje 2 no es un detalle menor: es la raíz de buena parte de su comportamiento (estabilidad, sencillez del flujo) y de su «derroche» de partidas. Cuando en el capítulo de resultados veas su curva, recuerda esta columna: muchas de sus virtudes se explican aquí.

■ Cuándo conviene cada familia

Clasificar no es un fin en sí mismo: sirve para **elegir**. Resumido en reglas prácticas, sabiendo que toda regla tiene excepciones:

Acciones pocas y discretas, quieres algo simple	Empieza por un método de valor tipo DQN. Es el cimiento: pocas piezas, fácil de depurar, un buen punto de referencia contra el que medir todo lo demás.
Buscas estabilidad y entrenamientos predecibles, y generar datos es barato	Un on-policy actor-crítico como PPO . Gasta más partidas, pero en un simulador eso casi no cuesta, y a cambio evita muchos dolores de cabeza de los métodos off-policy.
Interactuar con el entorno es caro y necesitas exprimir cada partida	Un off-policy con replay como SAC (si puedes pagar su cómputo) o, llevándolo al extremo, un método con modelo que aprenda de experiencia imaginaria. Máxima eficiencia en datos, a cambio de más complejidad y del riesgo del <i>model bias</i> .
Sospechas que el presente no basta para decidir (hace falta memoria)	Una arquitectura recurrente como el WM-RNN , que recuerda varios pasos atrás. Aporta memoria, pero pide explorar más y entrenar con más cuidado.

Y aquí conviene volver a la tesis del libro. Estos cinco algoritmos cubren tres familias filosóficas distintas —valor, política y modelo—, on-policy y off-policy, con y sin maqueta del mundo. Es un abanico amplio. Si el problema hubiera sido «elegir el algoritmo correcto», alguna de estas familias habría resuelto la tarea de un plumazo y las demás habrían fracasado de forma clara. No fue así. Lo que de verdad movió la aguja —y lo veremos con números en el siguiente capítulo— no fue saltar de una familia a otra, sino arreglar **cómo le planteábamos la tarea al agente**: qué veía en el estado, cómo repartíamos la recompensa, cuánto tiempo le dábamos por partida. El algoritmo importa; la formulación importa más.

Quédate con esto

Los cinco algoritmos no son cinco recetas sueltas, sino combinaciones de respuestas a **tres preguntas**: ¿aprendo el *valor* o la *política*? (DQN aprende valor; PPO y SAC, política, con su pareja actor-crítico). ¿Reutilizo experiencia (*off-policy*, con replay: DQN, SAC, los World Models) o exijo datos frescos (*on-policy*: solo PPO)? ¿Aprendo a actuar directamente (*sin modelo*) o construyo una maqueta del mundo para soñar partidas (*con modelo*: los dos World Models)? Cada eje trae un compromiso —eficiencia en datos frente a estabilidad frente a coste de cómputo—, pero la lección de fondo es que ninguna de estas tres familias fue, por sí sola, la pieza que faltaba: lo decisivo fue formular bien la tarea.

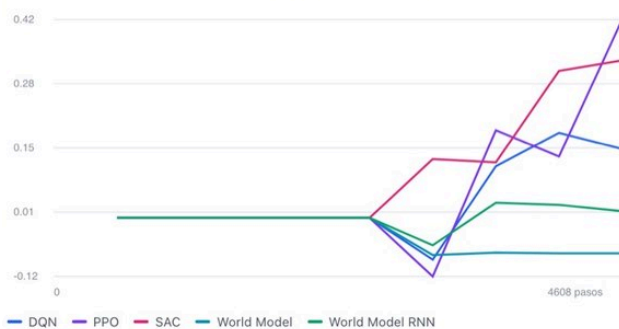
AUTOCOMPROBACIÓN

1. Nombra los tres ejes para clasificar algoritmos de RL y, para cada uno, di qué pregunta responde y por qué es independiente de los otros dos.
2. De los cinco algoritmos del libro, ¿cuál es el único on-policy? ¿Qué consecuencia práctica tiene eso respecto al replay buffer y a la eficiencia en datos?
3. Explica con tus palabras qué es el *model bias* y por qué produce un techo de rendimiento en los métodos con modelo, usando la imagen del «Arkanoid de juguete».
4. ¿Por qué «más eficiente en datos» no es lo mismo que «mejor» en un entorno simulado y barato como el nuestro? Relaciónalo con por qué PPO puede superar a un World Model.

RESULTADOS

🏆 **World Model RNN** es el mejor en esta corrida (éxito 0.0%, recompensa greedy 0.510). El más estable: **World Model**.

CURVAS DE APRENDIZAJE (RECOMPENSA DE ENTRENAMIENTO VS PASOS)



EVALUACIÓN GREEDY (POLÍTICA FINAL, SIN EXPLORACIÓN)

MODELO	RECOMPENSA GREEDY	TASA DE ÉXITO	LADRILLOS	ESTABILIDAD (Σ)	EXP/S
DQN	-0.072	0.0% ✓	1.1	0.017	480
PPO	0.045	0.0% ✓	1.1	0.144	1027 ✓
SAC	0.435	0.0% ✓	1.4 ✓	0.011	220
World Model	0.065	0.0% ✓	1.1	0.000 ✓	56
World Model RNN	0.510 ✓	0.0% ✓	1.3	0.007	56

Cada modelo se entrena una vez (corrida corta y ruidosa): léelo como una **tendencia**. ✓ = mejor de su columna.

El panel comparativo del laboratorio, con los cinco algoritmos enfrentados. Es el puente hacia el capítulo de resultados: aquí solo lo enseñamos como anticipo: las cifras concretas, sus condiciones (cinco semillas, niveles no vistos, validación frente a test) y su lectura crítica se desarrollan allí.

Los cinco, cara a cara: ficha funcional

Algoritmo	Qué aprende	Replay	Política	Modelo	Rasgo principal
DQN	valor $Q(s,a)$	sí	off-policy	no	cimiento simple y fiable
PPO	política (actor)	no	on-policy	no	el mejor y más fiable (91%)
SAC	política + entropía	sí	off-policy	no	estable, pero el más caro
World Model	Q + modelo	sí	off-policy	sí	eficiente, techo ~55%
WM-RNN	Q + modelo LSTM	sí	off-policy	sí	memoria que no ayuda aquí

Los cinco, en una ficha funcional: qué aprende cada uno, si usa replay, on/off-policy, si tiene modelo del mundo y su rasgo principal. Diferencias de fondo, no de nombre.

¿Qué algoritmo elegir? (criterio práctico)



Un criterio práctico: según qué busques, qué familia conviene. La regla del WM-RNN: si crees que hay historia parcial importante, considéralo... pero mídelo.

Y AHORA, PARA EXPERTOS NIVEL EXPERTO

En la taxonomía estándar, nuestros cinco caen así: **value-based** (DQN, que aprende $Q(s, a)$ y deriva la política como $\arg\max$); **policy-gradient / actor-crítico** (PPO y SAC, que parametrizan $\pi_\theta(a|s)$ y la entrenan por ascenso de gradiente con un crítico que reduce la varianza del estimador); y **model-based** (World Model y WM-RNN, estilo Dyna, que aprenden la dinámica $p(s', r | s, a)$ y entrenan la política con *rollouts* imaginarios; el WM-RNN modela la dinámica con un estado latente recurrente **LSTM**). En el eje de uso de datos, **on-policy** (PPO) exige muestras de la política vigente y no puede reutilizar un replay sin correcciones de *importance sampling*; **off-policy** (DQN, SAC y los WM) reutilizan el **replay buffer**, lo que aumenta la eficiencia en datos a costa de la *deadly triad* (aproximación de función + bootstrapping + datos off-policy) y de su inestabilidad asociada. El hilo conductor es el **compromiso sesgo/varianza del estimador de gradiente**: el crítico baja varianza introduciendo sesgo; el bootstrapping de TD baja varianza pero sesga; Monte Carlo no sesga pero dispara la varianza; el modelo aprendido recorta la varianza muestral a cambio del sesgo sistemático del *model bias*. Y, al final, todo se resume en un triángulo de tensiones: **eficiencia en datos** frente a **estabilidad** frente a **coste de cómputo**. SAC es el más caro —llega a coordinar hasta cinco redes (dos críticos Q , sus dos objetivos y el actor, más el ajuste automático del coeficiente de entropía α)— y el más estable; PPO es el más eficiente en datos por iteración de cómputo y el más fiable tras afinar; DQN es el cimiento barato; y los métodos con modelo apuestan la eficiencia muestral contra el techo que impone su propia maqueta. Saber en qué celda de estos tres ejes vive cada algoritmo predice, antes de entrenar, casi todo su comportamiento.

Qué dicen los datos: el veredicto

Éxito en niveles no vistos, media de 5 semillas.

Después de seis algoritmos, miles de partidas y un protocolo congelado, ¿qué dicen realmente los datos? ¿Quién gana, por cuánto, y —sobre todo— por qué?

Este es el capítulo del veredicto, así que conviene fijar las reglas antes de mirar una sola cifra. No medimos *recompensa*: medimos **éxito**, entendido como el porcentaje de niveles que el agente **limpia por completo**. No lo medimos en los niveles con los que entrenó, sino en niveles **nunca vistos**. Lo medimos con la política en modo **GREEDY** (sin exploración, la mejor jugada que el agente cree conocer en cada paso). Y no reportamos el mejor intento afortunado, sino la **media de cinco semillas** independientes. Cualquier número de este capítulo que no cumpla esas cuatro condiciones a la vez no es un resultado: es una anécdota.

Qué mirar

Antes de leer la tabla, conviene saber qué columna pesa y cuál engaña. **TEST-ID** es el éxito sobre niveles con el patrón de disposición ya visto en entrenamiento, pero en configuraciones nuevas: mide si el agente aprendió a *jugar*. **OOD-patrón** usa disposiciones de ladrillos que el agente jamás vio: mide si generaliza la *forma*. **OOD-dificultad** sube la densidad de ladrillos a niveles fuera del rango de entrenamiento: mide si generaliza la *presión*. **% colapsos** es el porcentaje de semillas que se hunden por debajo del 10% de éxito (una sola semilla rota basta para delatar inestabilidad). **% semillas >80** es cuántas de las cinco superan el 80%: distingue un algoritmo fiable de uno que «a veces sale bien».

Pasos para limpiar es la eficiencia *en partida*: cuántos pasos tarda en vaciar un nivel cuando lo consigue.

La trampa clásica es leer solo TEST-ID. Un número alto en la primera columna con colapsos en la cuarta es un agente que memoriza y se rompe, no uno que aprende.

Algoritmo	TEST-ID	OOD-patrón	OOD-dificultad	% colapsos	% semillas >80	Pasos para limpiar
PPO	91	89	86	0%	100%	1923
SAC-pure	87	84	68	0%	80%	2732
DQN	77	74	64	0%	80%	2125
SAC-híbrido	61	60	51	20%	40%	1716
World Model	55	53	42	0%	0%	1977
WM-RNN	35	29	27	0%	0%	1370

Lectura fila a fila

PPO es el veredicto en una línea: **91%** de niveles no vistos limpiados a 1,5M de pasos, **0% de colapsos** y **100% de semillas por encima del 80%**. Las cinco semillas funcionan, y funcionan bien. Y lo más importante: la degradación al salir del dominio es mínima —de 91 a 89 en patrones nuevos, a 86 en dificultades nuevas—. Eso no es memorización: es una política que aprendió a jugar al Arkanoid, no a recitar diez niveles concretos.

SAC-pure queda segundo y honroso: **87** en TEST-ID, sin colapsos, 4 de 5 semillas por encima de 80. Pero la grieta aparece en la última columna OOD: cae a **68** en dificultad no vista. Generaliza bien la forma (84 en OOD-patrón) y peor la presión. Además paga su precio en *pasos para limpiar*: 2732, el más lento del cuadro. Resuelve, pero da más vueltas.

DQN rinde un sólido **77** sin un solo colapso. Su virtud no está en esta tabla (capturada a 1,5M) sino en su *trayectoria*: es el único cuyo éxito crece de forma **monótona** con el presupuesto, y a 3M de pasos sube hasta el **85%**. Es el clásico fiable: lento, estable, sin sorpresas desagradables.

SAC-híbrido es la advertencia del capítulo. Un **61** de media que esconde lo peor de todo: **20% de colapsos** y solo **40% de semillas por encima de 80**. Es un resultado **bimodal**: unas semillas vuelan y otras se estrellan. La media no describe a ningún agente real; describe el promedio entre triunfos y catástrofes. Curiosamente, es el más rápido en limpiar (1716 pasos) *cuando* lo consigue.

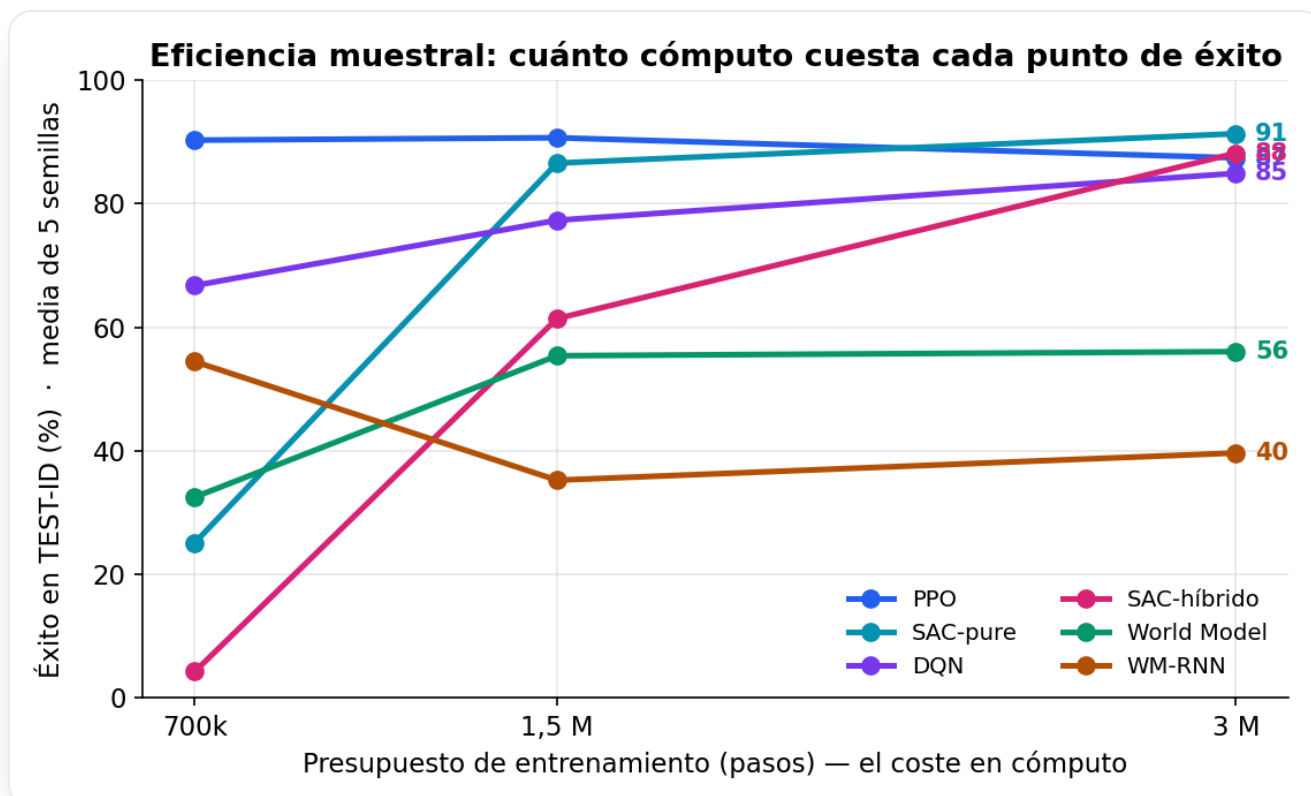
World Model y **WM-RNN** cierran la tabla con **55** y **35**. Ninguna de sus semillas supera el 80% (**0%** en esa columna), y aunque no colapsan, su techo es bajo y su degradación OOD es pronunciada (World Model cae de 55 a 42; WM-RNN de 35 a 27). Modelar el mundo para luego planificar sobre el modelo es elegante, pero aquí el error del modelo se acumula y pone un tope al rendimiento.

Quédate con esto

PPO no gana por poco: gana por mucho y, sobre todo, gana **fiable** —0% de colapsos, 100% de semillas por encima de 80% y una degradación de apenas 5 puntos al salir del dominio de entrenamiento—. El resto del cuadro ordena el resto: SAC-pure resuelve pero generaliza peor la dificultad, DQN es el lento-fiable que mejora con presupuesto, y SAC-híbrido es la lección de que una media decente puede esconder un 20% de catástrofes.

La eficiencia muestral: el eje del coste

La tabla maestra es una foto a 1,5M de pasos. Pero un agente no solo se juzga por dónde llega, sino por **cuánto cómputo** necesita para llegar. Esa es la pregunta de la eficiencia muestral, y aquí el eje no es el rendimiento sino el **coste**: el éxito en TEST-ID frente a tres presupuestos —700k, 1,5M y 3M de pasos—.



Eficiencia muestral: éxito en TEST-ID de los seis algoritmos frente al presupuesto de cómputo (700k / 1,5M / 3M de pasos). El eje horizontal es el coste; lo que importa es la **forma** de cada curva, no solo su altura final.

Las trayectorias cuentan seis historias distintas:

PPO: 90 → 91 → 87 | SAC-pure: 25 → 87 → 91 | DQN: 67 → 77 → 85

SAC-híbrido: 4 → 61 → 88 | World Model: 33 → 55 → 56 | WM-RNN: 54 → 35 → 40

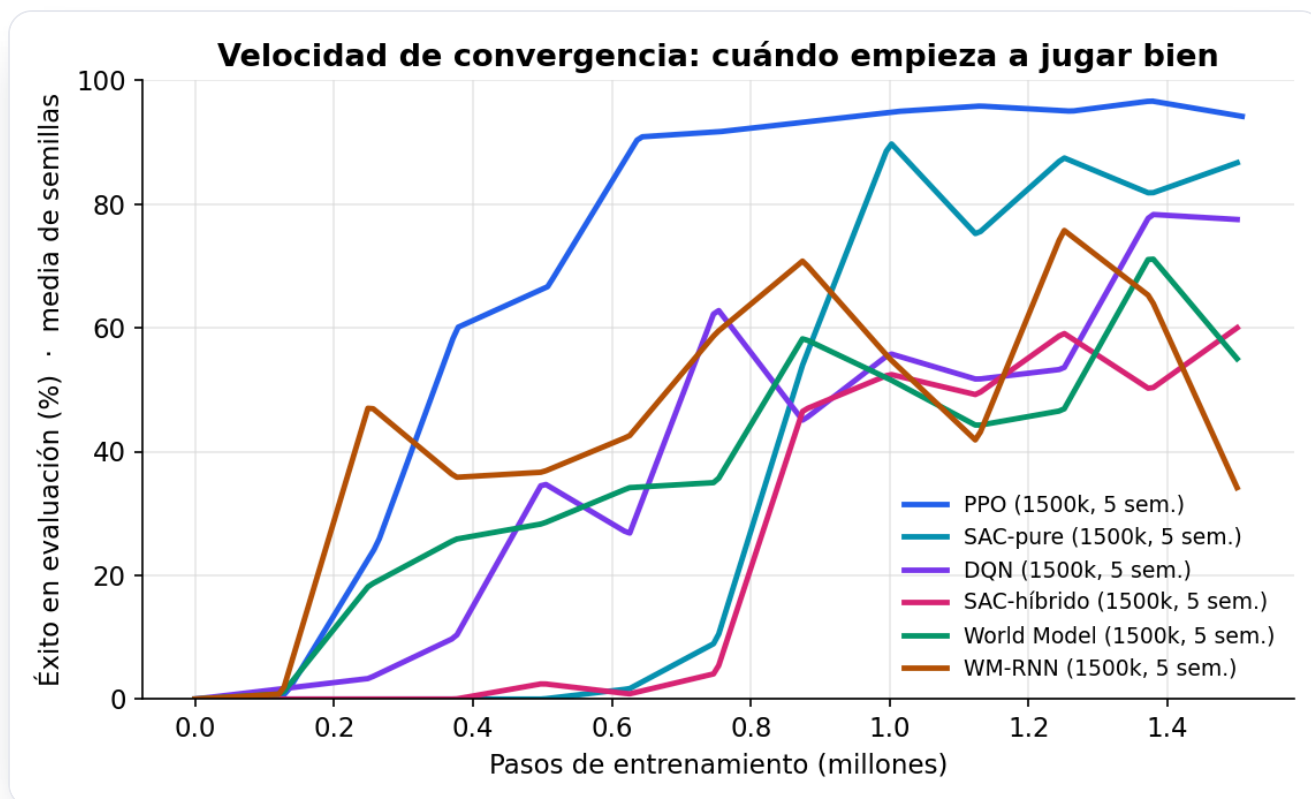
PPO es el más eficiente, y con diferencia. Ya con 700k pasos limpia el **90%**: prácticamente todo lo que va a saber lo sabe pronto. El presupuesto extra apenas mueve la aguja (91 a 1,5M, 87 a 3M). Si el cómputo es caro, PPO es la elección obvia.

SAC-pure y SAC-híbrido tienen una dependencia brutal del presupuesto. SAC-pure arranca en un pobre **25** a 700k y termina en **91** a 3M —alcanza a PPO, pero le cuesta el doble de cómputo llegar—. SAC-híbrido es el caso extremo: **4 → 61 → 88**. Empieza prácticamente sin saber jugar y necesita los 3M completos para volverse competitivo. Esa pendiente tan pronunciada es la otra cara de su inestabilidad: tarda en arrancar y, cuando arranca, no todas las semillas lo hacen igual.

DQN es el único monótono limpio: 67 → 77 → 85. Cada euro de cómputo compra rendimiento, sin retrocesos. Es la curva que un ingeniero quiere ver cuando no puede permitirse sorpresas.

World Model choca contra un techo: 33 → 55 → 56. Sube hasta 1,5M y luego se estanca: darle el doble de pasos (de 1,5M a 3M) le aporta un mísero punto. Ese plató es la firma del error de modelo acumulado. **WM-RNN es directamente no-monótono** (54 → 35 → 40): más cómputo no garantiza más éxito, una señal de inestabilidad en el aprendizaje del modelo recurrente.

La velocidad de convergencia



Velocidad de convergencia: éxito de evaluación frente a pasos de entrenamiento. Una curva que sube pronto y se aplana arriba (PPO) indica aprendizaje rápido y estable; curvas que despegan tarde (los SAC) o que oscilan (los World Model) delatan, respectivamente, alta dependencia del presupuesto e inestabilidad.

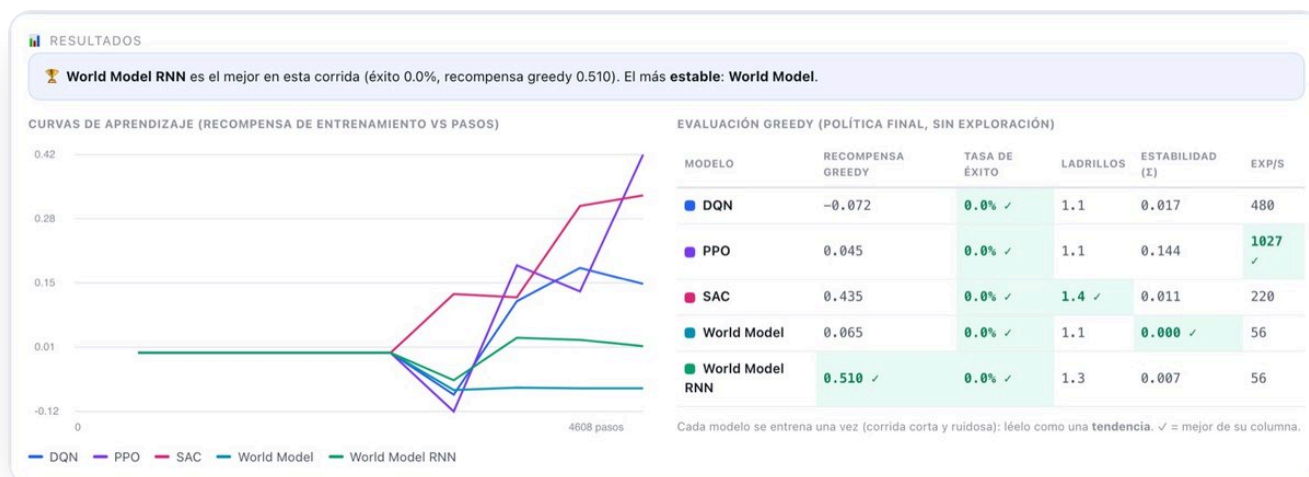
La convergencia confirma desde otro ángulo lo que dijo la eficiencia: PPO no solo termina arriba, sino que **llega arriba antes** y se queda. Los SAC despegan tarde. Los World Model se aplanan en un techo bajo. La forma de la curva, más que su punto final, es lo que distingue a un algoritmo robusto de uno frágil.

Error común

Confundir «converge rápido» con «converge alto», o al revés. SAC-pure converge **lento** pero **alto** (acaba en 91). WM-RNN converge **rápido** a 700k pero a un techo **bajo** y encima retrocede después. Son dos ejes independientes: la velocidad de aprendizaje y el rendimiento asintótico. Un buen algoritmo gana en ambos; la mayoría hace un trade-off.

Qué no demuestra

Nada de este capítulo se apoya en las cifras de **recompensa** que muestra el laboratorio web. La recompensa es el *instrumento* con el que entrenamos —una señal interna que el agente intenta maximizar—, no el *logro*. Un agente puede acumular recompensa altísima explotando un atajo del shaping y aun así limpiar pocos niveles nuevos. Por eso el veredicto se mide en una sola moneda: **éxito sobre niveles no vistos, en GREEDY, promediado sobre 5 semillas**. Si alguien te enseña una recompensa espectacular como prueba de que «el agente aprendió», pídele la columna de éxito OOD. Ahí se cae casi todo.



Panel comparativo del laboratorio web. Útil para inspeccionar el comportamiento de cada agente en vivo, pero recuerda: las curvas de recompensa que aquí aparecen son el instrumento de entrenamiento, no el veredicto. El veredicto está en la tabla maestra de arriba.

La tesis: el cuello de botella nunca fue el algoritmo

Si se ordenan los seis algoritmos por éxito, es tentador concluir «PPO es mejor que SAC, que es mejor que los World Model». Sería una lectura pobre y, peor, falsa en su moraleja. La verdadera enseñanza del experimento es otra, y va a contracorriente de la intuición.

El cuello de botella nunca fue el tiempo de entrenamiento ni la elección del algoritmo. Fue la FORMULACIÓN del problema.

Cada gran salto de rendimiento de este laboratorio no vino de cambiar de optimizador ni de regalar más pasos de cómputo, sino de arreglar *cómo estaba planteada la tarea*: el **reloj proporcional** (que el tiempo límite escale con el tamaño del nivel), la **meta sin shaping Φ** mal puesto (recompensar el objetivo real, no proxies que el agente puede reventar), la **observación 8x10** (darle al agente una vista del tablero que contenga la información necesaria), la **escala de la señal** (que las recompensas vivan en un rango que el aprendizaje pueda digerir), el **encoder convolucional** (procesar la rejilla con la arquitectura adecuada a su estructura espacial), el **currículo** (ordenar la dificultad de menos a más) y, transversal a todo, la **evaluación honesta** (medir en niveles no vistos, en GREEDY, con varias semillas).

Traducción al juego

Imagina enseñar a un amigo a jugar al Arkanoid. Puedes elegir al «alumno más listo» (el mejor algoritmo) y darle «más horas de práctica» (más cómputo). Pero si le tapas media pantalla (observación pobre), le pones un cronómetro que no escala con el tamaño del nivel (reloj fijo), y le premias por mover la pala en vez de por romper ladrillos (shaping mal puesto)... ni el alumno más brillante con horas infinitas aprenderá a jugar bien. Y al revés: con la pantalla entera, un reloj justo y un premio claro, hasta un alumno modesto (DQN) aprende decentemente. Eso es exactamente lo que cuentan estos datos.

Los World Model que vemos en el fondo de la tabla no «fracasaron por ser World Model». Rindieron lo que su formulación les permitió. Y PPO no «ganó por ser PPO»: ganó porque, sobre una formulación bien construida, es el algoritmo que mejor aprovecha esa buena formulación con menos cómputo. Cambia el orden de las causas y entenderás mal todo el experimento.

Qué no demuestra

Que PPO sea «el mejor algoritmo de RL» en abstracto. Demuestra que, **en esta tarea, con esta formulación, este presupuesto y esta evaluación honesta**, PPO es la elección más eficiente y fiable. En otra tarea —con horizonte largo, recompensas muy dispersas o necesidad de planificación explícita—

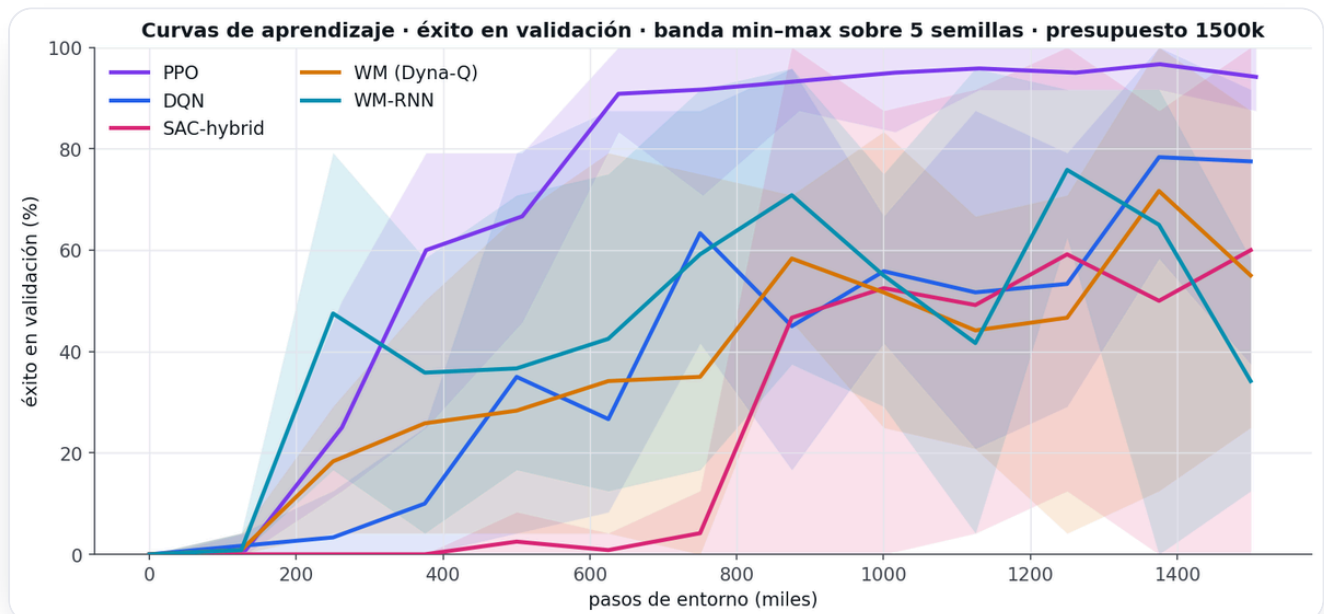
un World Model bien formulado podría darle la vuelta a la tabla. La lección portable no es «usa PPO»: es «**arregla la formulación antes de culpar al algoritmo**».

Quédate con esto

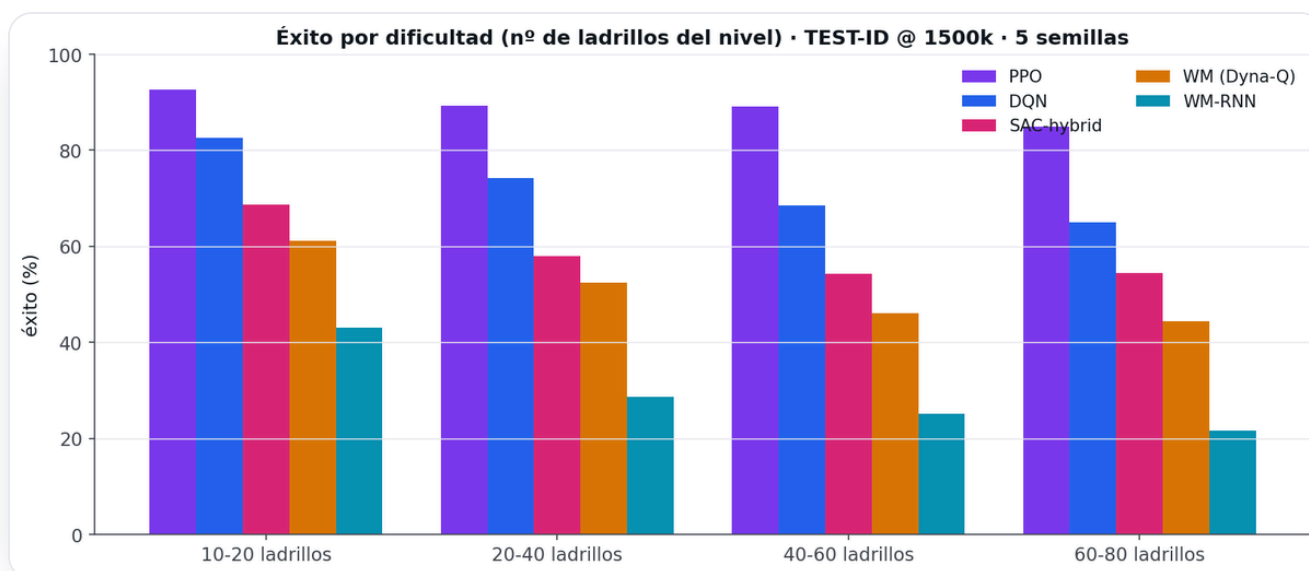
El veredicto tiene dos capas. En la superficie: PPO limpia el 91% de niveles no vistos a 1,5M, sin colapsos y con todas sus semillas por encima del 80%, siendo además el más eficiente en cómputo. En el fondo, y más importante: el ranking de algoritmos es secundario frente a la tesis central —el cuello de botella fue siempre la **formulación del problema** (reloj, meta, observación, escala, encoder, currículo y evaluación honesta), no el algoritmo ni los pasos de entrenamiento—. Arregla la pregunta y casi cualquier algoritmo responde; deja la pregunta rota y ninguno lo hará.

AUTOCOMPROBACIÓN

1. El éxito que reporta este capítulo, ¿se mide sobre los niveles de entrenamiento o sobre niveles no vistos? ¿Y con qué política y cuántas semillas?
2. SAC-híbrido tiene un 61% de éxito medio pero un 20% de colapsos y solo el 40% de semillas por encima de 80. ¿Por qué decimos que su media «no describe a ningún agente real»?
3. PPO hace 90 → 91 → 87 al aumentar el presupuesto; SAC-híbrido hace 4 → 61 → 88. ¿Qué dice cada trayectoria sobre la dependencia del cómputo de cada algoritmo?
4. Un compañero presume de una recompensa altísima en el laboratorio web. ¿Qué columna le pedirías para saber si su agente de verdad aprendió a jugar?
5. Enuncia con tus palabras la tesis central del capítulo. ¿Por qué «PPO es mejor que SAC» es una lectura insuficiente de los datos?



Curvas de aprendizaje reales: **éxito en validación** frente a pasos de entorno, con banda mín-máx sobre las 5 semillas (presupuesto 1,5 M). PPO (morado) despegó antes y se mantiene arriba; los *model-based* (WM, WM-RNN) se quedan atrás. Una banda ancha = semillas dispares, es decir, más riesgo.



Éxito desglosado por **dificultad del nivel** (número de ladrillos), en TEST-ID a 1,5 M. Todos rinden peor cuanto más lleno está el nivel, pero el orden se mantiene: PPO aguanta mejor; WM-RNN se desploma. Generalizar a niveles más densos es lo más difícil.



Y AHORA, PARA EXPERTOS NIVEL EXPERTO

El rigor de estas cifras no está en los números, sino en el **protocolo** que los produjo. Cada celda es la **media de 5 semillas** [0..4], no el mejor run; y junto a la media vigilamos la **dispersión** y los **colapsos** (de ahí las columnas «% colapsos» y «% semillas >80», que delatan la bimodalidad de SAC-híbrido que una media aislada ocultaría). Aun así, reportar la media es el eslabón débil: en regímenes de pocas semillas la media es sensible a las colas y a los outliers. Agarwal et al. (2021), en «Deep Reinforcement Learning at the Edge of the Statistical Precipice», muestran que conviene reportar la **mediana intercuartílica (IQM)** —robusta a colapsos y a runs afortunados— acompañada de **intervalos de confianza por bootstrap estratificado**, en lugar de la media puntual; con 5 semillas, una media sin barras de error es estadísticamente frágil. La segunda mitad del rigor es el **protocolo congelado**: `frozen_hash = a1ab7ce18d7bad6b` sella el entorno completo (torch 2.12.0 + numpy 2.4.4 + Python 3.13 sobre MPS en Apple M3 Ultra), los presupuestos (700k/1,5M/3M), los conjuntos **disjuntos** de train/validación/test y la evaluación GREEDY. Ese sellado no es burocracia: es el antídoto contra los *researcher degrees of freedom* —la libertad de ir ajustando hiperparámetros, criterios de parada o conjuntos de evaluación hasta que el número favorito sale bonito— y contra el **p-hacking** implícito de elegir *a posteriori* qué reportar. Congelar el protocolo *antes* de mirar los resultados (y no volver a re-congelarlo nunca) convierte un experimento en una prueba, no en una narración.

La ablación: qué ingrediente desbloquea

Se quita una pieza y se mide el daño.

Llevamos todo el libro defendiendo una tesis incómoda: que el cuello de botella nunca fue el algoritmo, sino la formulación de la tarea. Es una afirmación bonita, pero las afirmaciones bonitas no valen nada en ingeniería si no se pueden poner a prueba. Así que toca la pregunta dura: ¿cómo se demuestra eso con números, y no con relato? ¿Cómo se mide cuánto aporta de verdad *cada* ingrediente de la receta —la escala de la señal, el reloj del episodio, la convolución, el currículo, el shaping— sin engañarnos? Y, sobre todo, ¿qué pasaría si alguno de esos ingredientes que damos por buenos resultara, al medirlo, un estorbo?

La respuesta del método científico aplicado al aprendizaje por refuerzo tiene nombre y es de una simplicidad brutal: la **ablación**. Este capítulo es la prueba empírica de la tesis del libro. No introduce ningún algoritmo nuevo; coge la receta completa, la del agente que funciona, y le va quitando piezas de una en una para ver cuánto duele cada amputación. Lo que duele mucho era crítico; lo que no duele apenas era decorativo; y —esto es lo que nadie espera— lo que *mejora* al quitarlo era un saboteador disfrazado de ayuda.

Qué es una ablación (y por qué se hace de una pieza en una)

La palabra viene de la medicina y de la cirugía: *ablación* es la extirpación de una parte. En ciencia experimental, una **ablación** es un experimento en el que se **elimina o desactiva un solo componente** de un sistema que funciona y se mide cuánto cambia su rendimiento, dejando absolutamente todo lo demás igual. La lógica es la del control de variables que aprendimos en el colegio: si cambias diez cosas a la vez y el resultado mejora, no sabes *cuál* de las diez lo hizo. Si cambias una sola, el cambio en el resultado es atribuible a esa sola pieza. Eso, y solo eso, permite hablar de **causalidad** y no de mera correlación.

TRADUCCIÓN AL JUEGO

Imagina que tienes una receta de tarta que sale perfecta y quieres saber qué papel juega cada ingrediente. No tiene sentido hornear una tarta cambiando a la vez la harina, el azúcar y el horno: si sale mal, no sabrás a quién culpar. Lo que haces es hornear una tarta **sin la levadura** y todo lo demás idéntico; sale plana, luego la levadura era crítica. Horneas otra **sin la pizca de sal**: sale casi igual, luego la sal era cosmética. Y horneas una más **sin esa cucharada de vinagre que añadiste por costumbre**: resulta que sale *mejor* sin ella, porque cortaba la masa. Eso es exactamente lo que hacemos con el agente: una receta base que funciona y, para cada ingrediente, una versión idéntica salvo por esa pieza. El daño (o la mejora) medido es la contribución de esa pieza.

Aquí hay que ser muy precisos con el punto de partida, porque toda ablación se mide **respecto a una base**. Nuestra base es el agente **DQN con la receta completa**, evaluado en el conjunto TEST-ID (niveles no vistos en entrenamiento, evaluación *greedy*, media de 5 semillas, presupuesto de 1,5M de pasos). Esa configuración rinde un **77%** de éxito. Cada fila de la ablación parte de ese 77% y le quita *un* ingrediente; la diferencia Δ respecto a 77 es, por definición, lo que ese ingrediente aportaba.

ERROR COMÚN

Confundir la base de la ablación con «el mejor número del libro». No lo es. El mejor agente del estudio no es necesariamente este DQN, y el propio 77% no es el techo de DQN (con más presupuesto sube). El 77% es, simplemente, el **punto de referencia fijo** sobre el que medimos las amputaciones, todas con el mismo algoritmo y la misma tanda para que los Δ sean comparables entre sí. Mezclar números de algoritmos distintos o de presupuestos distintos rompería la única virtud de la ablación: que cambia una sola cosa cada vez.

Por qué **una** pieza y no varias a la vez tiene un nombre técnico: ablación de *una sola variable*, o en la jerga, **one-factor-at-a-time**. Es la forma más limpia de atribuir causalidad. Tiene una limitación que veremos al final —no captura bien las *interacciones* entre ingredientes—, pero como primer barrido es insustituible: te dice, sin ambigüedad, qué piezas mueven la aguja y cuáles no.

Los seis resultados, de un vistazo

El laboratorio midió seis ablaciones sobre la base DQN del 77%. Antes de mirar la figura conviene tener la fotografía completa en una tabla, ordenada de la amputación más dolorosa a la que, paradójicamente, cura. La columna Δ es el cambio en puntos de éxito respecto a la base: negativo significa que *quitar* ese ingrediente **empeora** (luego hacía falta); positivo, que quitarlo **mejora** (luego estorbaba).

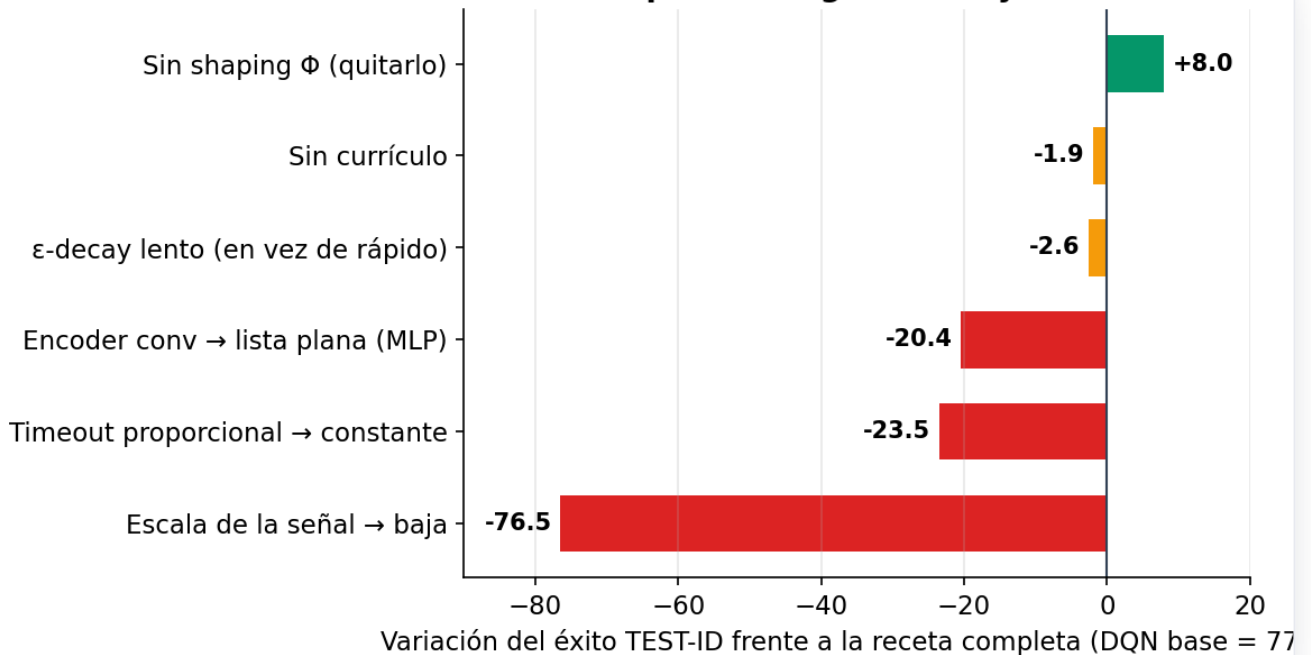
Ingrediente que se quita	Se sustituye por...	Éxito TEST-ID	Δ vs. 77%	Lectura
Escala de la señal de ladrillos	Escala baja (0, 25)	~1%	-76,5	Catastrófico. 100% de colapsos. El ingrediente más crítico .
Timeout proporcional	Timeout constante	54%	-23,5	El reloj del episodio pesa mucho.
Encoder convolucional	Lista plana + MLP	57%	-20,4	El sesgo espacial vale unos 20 puntos.
ϵ-decay rápido	ϵ -decay lento	75%	-2,6	Ajuste fino; efecto pequeño en DQN.
Currículo	Sin currículo	75%	-1,9	Efecto pequeño en estas condiciones.
Shaping potencial Φ	Quitarlo (meta limpia)	85%	+8,0	Quitarlo mejora : el shaping sabotaba el objetivo.

Dos cosas saltan a la vista incluso antes de razonarlas. La primera: las contribuciones **no son parejas**. No es que cada ingrediente aporte «un poquito»; hay una jerarquía abismal entre el primero (-76 puntos) y los del medio (-2 puntos). La segunda, y la más importante para la honestidad del libro: el signo de la última fila es **positivo**. Hay un ingrediente que metimos con buena intención y que, medido en frío, restaba. Vamos a recorrerlos por grupos.

QUÉ MIRAR

En la figura que viene, fíjate antes que nada en el **signo y la longitud** de cada barra, no solo en el orden. Son barras horizontales de la variación del éxito al quitar cada ingrediente, partiendo de la base DQN = 77%. Las barras que apuntan hacia el lado negativo (las largas a la izquierda) son ingredientes que **hacían falta**: cuanto más larga, más crítico. La única barra que apunta al lado positivo es el shaping Φ : su mera existencia, apuntando «al revés», es el resultado más contraintuitivo del capítulo. No leas la figura como «cuánto aporta cada cosa»; léela como «cuánto duele —o alivia— quitar cada cosa».

Ablación: se quita un ingrediente y se mide el daño



La ablación de la receta completa, medida sobre la base DQN del 77% en TEST-ID (media de 5 semillas, *greedy*, 1,5M de pasos). Cada barra es el Δ de éxito al *quitar* un ingrediente. La escala de la señal de ladrillos domina con diferencia el lado del daño (de 77% a ~1%, con colapso total); el timeout proporcional y el encoder convolucional pesan del orden de veinte puntos cada uno; el ϵ -decay y el currículo apenas mueven la aguja. La única barra que apunta al lado contrario es el **shaping Φ** : quitarlo mejora el resultado en +8 puntos. La figura es, en una imagen, la demostración de la tesis del libro: lo que desbloquea el problema es la formulación.

El ingrediente crítico: la escala de la señal de ladrillos (–76)

Empezamos por el más espectacular. La señal de los ladrillos —la matriz 8x10 que el agente recibe como parte del estado— entra en la red con una cierta **escala** numérica. En la receta completa esa escala está calibrada para convivir con el encoder convolucional. La ablación prueba qué pasa si la bajamos a 0,25: el éxito se desploma de 77% a un 1% escaso, con un **100% de colapsos** en las cinco semillas. No es que el agente rinda peor; es que **deja de aprender por completo**. Es, con muchísima diferencia, la amputación más letal de las seis: por sí sola explica la barra que se sale del gráfico.

La lección es contundente: cómo de fuerte le llega al agente la información de los ladrillos no es un detalle de implementación, es la diferencia entre un agente que juega y un agente muerto. Una señal mal escalada en la entrada de una red puede, literalmente, hacer que el aprendizaje no arranque. Y guarda esta pieza en la memoria, porque tiene una historia detrás —la **inversión histórica**— que cierra el capítulo y que es, quizá, su moraleja más fina.

■ Dos ingredientes que pesan de verdad: el reloj (–23) y la convolución (–20)

El siguiente escalón lo forman dos ingredientes que, sin ser catastróficos como la escala, valen del orden de **veinte puntos de éxito cada uno**. Son la prueba de que la formulación de la tarea se juega en varios frentes a la vez.

El primero es el **timeout proporcional**. Cada episodio tiene un límite de pasos —un reloj—, y en la receta completa ese límite es *proporcional* al número de ladrillos que quedan: cuantos más ladrillos, más tiempo se concede para limpiarlos. La ablación lo sustituye por un **timeout constante**, el mismo para todos los niveles, y el éxito cae a **54%** (Δ –23, 5). La razón es sutil y profunda: con un reloj fijo demasiado corto para los niveles cargados, el agente descubre que no le da tiempo a terminar y, racionalmente, **aprende a sobrevivir en lugar de a ganar**. El reloj no es un parámetro administrativo; define qué significa «éxito» para el agente, y por tanto qué aprende a optimizar.

El segundo es el **encoder convolucional**. La matriz 8x10 de ladrillos tiene **estructura espacial**: un hueco está al lado de otro, una columna entera importa como columna. El encoder convolucional respeta esa vecindad deslizando filtros por el tablero. La ablación lo sustituye por una **lista plana** de 80 números procesada con una MLP densa —aplanar la rejilla destruye la geometría— y el éxito baja a **57%** (Δ –20, 4). Es la medida directa de lo que vale el **sesgo espacial**: ver los ladrillos es necesario, pero verlos *con estructura* vale unos veinte puntos. No es lo mismo darle al agente la información que dársela en una forma que le permite explotar la geometría del problema.

PARA CURIOSOS

Que estos dos ingredientes pesen parecido (–23 y –20) y la escala pese cuatro veces más (–76) no es casualidad numérica, sino una jerarquía de fallos: la escala mal puesta **impide aprender** (colapso total, 1%); el reloj y la convolución **limitan cuánto se aprende** (el agente sí aprende algo, pero rinde la mitad). Son dos categorías distintas de daño: «no arranca» frente a «arranca cojo». Por eso la barra de la escala no está «un poco más a la izquierda» que las otras, sino en otra liga.

■ Dos ingredientes casi cosméticos: ϵ -decay y currículo (–2 cada uno)

No todo lo que metimos en la receta resultó decisivo, y es de honestos decirlo. Dos ingredientes que en la literatura se presentan a menudo como importantes apenas mueven la aguja *en este problema y con DQN*.

El **ϵ -decay** regula la exploración: al principio el agente actúa muy al azar (ϵ alto) y poco a poco confía más en lo aprendido (ϵ bajo). La receta usa un decaimiento *rápido*; la ablación lo cambia por uno *lento* y el éxito pasa de 77% a **75%** (Δ –2, 6). El **currículo** —presentar primero niveles fáciles y luego difíciles— es la otra: quitarlo deja el éxito en **75%** (Δ –1, 9). En ambos casos hablamos de ajustes finos, no de palancas. Conviene tenerlos, pero no son lo que separa el éxito del fracaso.

QUÉ NO DEMUESTRA

Que el currículo y el ϵ -decay rápido aporten poco *aquí* no significa que sean inútiles en general. La ablación está hecha sobre **DQN**, en *esta* tarea y con *esta* tanda; en otro algoritmo (PPO, SAC) o en un problema con exploración más dura, su peso podría ser muy distinto. Lo que la ablación demuestra no es que estos ingredientes sean malos, sino que en este montaje no son la pieza que desbloquea. Además, el currículo tiene un valor que la cifra de éxito no captura: el generador de niveles con conjuntos disjuntos es lo que hace **creíbles** todos los demás números del libro, porque garantiza que el agente generaliza a niveles no vistos en vez de memorizar.

El resultado incómodo: quitar el shaping Φ MEJORA (+8)

Y llegamos a la única barra que apunta al otro lado. El **shaping potencial Φ** es una señal de recompensa extra que añadimos para «ayudar» al agente: una pista continua que premiaba, paso a paso, acercarse a una situación que creíamos buena. La intuición era razonable —dar más realimentación suele acelerar el aprendizaje—. La ablación la desmiente sin piedad: **quitar Φ** y dejar la recompensa de la tarea limpia sube el éxito de 77% a **85%** ($\Delta +8, 0$). Es el *único* ingrediente de los seis cuya **eliminación mejora** el sistema.

¿Cómo puede ser que ayudar perjudique? Porque el potencial Φ estaba, sin que lo viéramos, **saboteando el objetivo real**. La señal continua premiaba un comportamiento (acercarse) que no siempre coincidía con ganar la partida; el agente, que optimiza lo que le pagas y no lo que querías pagarle, aprendía a maximizar la pista en lugar del objetivo. Quitarla deja una meta limpia —ladrillos, rebotes, perder la bola, limpiar el nivel— y el agente vuelve a optimizar lo que de verdad importa. Lo honesto no es esconder este resultado, sino aprender de él: una meta *limpia* vale más que una meta «ayudada» en la dirección equivocada.

ERROR COMÚN

Pensar que «más señal de recompensa siempre ayuda». Es una de las trampas más caras del diseño de recompensas. Toda señal extra que añades es una **hipótesis** sobre qué conduce al objetivo; si la hipótesis es ligeramente errónea, el agente la explotará al pie de la letra y se desviará del objetivo real. El **reward shaping** puede acelerar o sabotear, y la única forma de saber cuál de las dos es —lo dice esta misma ablación— es *medirlo* quitándolo. La buena intención no es evidencia.

La inversión histórica de la escala: el ingrediente óptimo depende de la arquitectura

Volvamos al ingrediente más crítico, la escala de la señal, porque esconde la moraleja más sutil del capítulo. Acabamos de ver que bajar la escala a **0,25 mata** el aprendizaje con la arquitectura actual (1% y colapso total). Pues bien: en la **era antigua** del proyecto —cuando la rejilla era pequeña (4x7), el estado era una lista plana y no había convolución— bajar la escala a ese mismo **0,25** hacía justo lo contrario: **desbloqueaba** el aprendizaje. El mismísimo valor numérico, **0,25**, que en un diseño *curaba*, en el otro *envenena*.

PARA CURIOSOS

No es una contradicción ni un error de medida: es la prueba palpable de que **el ingrediente óptimo no es absoluto, depende de la arquitectura con la que conviva**. La escala correcta de una señal de entrada está acoplada a cómo la procesa la red: una rejilla pequeña aplanada y una rejilla grande con encoder convolucional «quieren» escalas distintas. Por eso la receta no es una suma de trucos independientes que se puedan copiar de un proyecto a otro, sino un **conjunto acoplado**: cambiar la arquitectura obliga a recalibrar la entrada. Lo que funcionó ayer puede matar hoy, y solo medir —no la intuición ni la tradición— lo distingue.

Esta inversión es también la advertencia que cierra el método: la ablación de una sola variable es potentísima para atribuir causalidad, pero precisamente porque cambia una cosa cada vez, **no captura las interacciones**. La escala y la arquitectura interactúan: el efecto de una depende del valor de la otra. Por eso conviene leer las seis barras como lo que son —el peso de cada pieza *dentro de esta receta concreta*— y no como verdades universales del aprendizaje por refuerzo.

Quédate con esto

Una **ablación** quita un ingrediente de una receta que funciona y mide el daño, dejando todo lo demás igual; al cambiar una sola variable, el cambio en el resultado es atribuible a esa pieza (causalidad, no correlación). Sobre la base **DQN = 77%** (TEST-ID, 5 semillas, 1,5M), los seis resultados dibujan una jerarquía clarísima: la **escala de la señal** de ladrillos es crítica —bajarla a 0,25 hunde el éxito a ~1% con colapso total (−76)—; el **timeout proporcional** (−23) y el **encoder convolucional** (−20) pesan del orden de veinte puntos cada uno; el ϵ -decay y el currículo apenas mueven la aguja (−2); y **quitar el shaping Φ mejora** el resultado en +8, porque su pista continua saboteara el objetivo real. La inversión histórica de la escala (curaba con rejilla pequeña y lista plana; mata con encoder convolucional) demuestra que el ingrediente óptimo depende de la arquitectura. Conclusión, y tesis del libro: lo que desbloqueó el problema fue la **formulación** —escala, reloj, convolución—, no la potencia del algoritmo.

AUTOCOMPROBACIÓN

1. Explica con tus palabras qué es una ablación y por qué se cambia *un* ingrediente cada vez. ¿Qué te permite afirmar ese diseño que no podrías afirmar si cambiaras varios a la vez?
2. De los seis ingredientes, ¿cuál es el más crítico y qué le pasa exactamente al agente cuando se quita (cifra de éxito y de colapsos)? ¿En qué se diferencia ese tipo de daño del que causan el reloj o la convolución?
3. El shaping Φ es la única barra con Δ positivo. ¿Qué significa eso y por qué «ayudar» al agente con una señal extra acabó perjudicándolo? Calcula el éxito resultante a partir de la base y del Δ .
4. Describe la inversión histórica de la escala de la señal. ¿Por qué el mismo valor 0,25 desbloquea en una arquitectura y mata en otra, y qué te dice eso sobre copiar «trucos» de un proyecto a otro?



Grid search en vivo

Algoritmo: **DQN** (el seleccionado en el laboratorio)

PARÁMETRO A

Ritmo de aprendizaje

0.0003

0.0008

0.0015

PARÁMETRO B

Descuento γ (cuánto pesa el futuro)

0.97

0.99

0.995

Pasos por combinación

5000

Entornos por combinación

64

▶ Buscar la mejor

9 combinaciones × 8000 pasos

9/9 combinaciones

#	RITMO DE APRENDIZAJE	DESCUENTO γ (CUÁNTO PESA EL FUTURO)	PROGRESO	RECOMPENSA FINAL
5	0.0008	0.99		0.198
3	0.0003	0.995		0.182
7	0.0015	0.97		0.123
9	0.0015	0.995		0.089
2	0.0003	0.99		0.082
8	0.0015	0.99		0.072
4	0.0008	0.97		0.057
1	0.0003	0.97		-0.071
6	0.0008	0.995		-0.082

🏆 Mejor combinación

Ritmo de aprendizaje = **0.0008** · Descuento γ (cuánto pesa el futuro) = **0.99** → recompensa final **0.198**

✓ Aplicar al laboratorio

Recreará el agente con estos valores y, si estabas entrenando, seguirá.

¿Qué es esto y por qué?

La **búsqueda en rejilla** (grid search) del laboratorio: prueba combinaciones de hiperparámetros —aquí ritmo de aprendizaje × descuento γ para DQN—, las entrena con presupuesto corto y las ordena por recompensa. Es una exploración rápida y ruidosa; la ablación rigurosa (5 semillas, protocolo congelado) es la que de verdad manda.



Y AHORA, PARA EXPERTOS NIVEL EXPERTO

El protocolo de este capítulo es una **ablación de una sola variable** (*one-factor-at-a-time*): cada cambio se valida aislado, manteniendo fijos algoritmo (DQN), semillas (media de 5), presupuesto (1,5M) y conjunto de evaluación (TEST-ID, *greedy*), de modo que la diferencia Δ respecto a la base del 77% sea atribuible **causalmente** a la pieza retirada y no a covariables. Su límite conocido es que, al perturbar un solo factor, **no estima las interacciones**: cuando los efectos no son aditivos, el barrido OFAT puede ordenar mal las importancias y se necesitaría un diseño factorial para cuantificar los términos cruzados. El caso de la **escala de la señal** es justamente una interacción de primer orden con la arquitectura del encoder: en la era antigua — rejilla 4×7 , estado aplanado, MLP densa— bajar la escala a $0,25$ *desbloqueaba* el aprendizaje (regularizaba la magnitud de la entrada y estabilizaba el gradiente en una red sin sesgo espacial); con el **encoder convolucional** sobre la rejilla 8×10 , esa misma escala $0,25$ lo *mata* (1%, 100% de colapsos), porque atenúa la señal por debajo del umbral que los filtros convolucionales necesitan para extraer estructura. El óptimo del hiperparámetro de entrada es, por tanto, **condicional a la arquitectura**: prueba empírica de que la receta es un sistema acoplado y no una superposición de ingredientes independientes. El otro resultado para subrayar es que quitar el **shaping potencial Φ** *mejora* en $+8$ puntos. En teoría, el *potential-based reward shaping* (Ng, Harada & Russell, 1999) con $F(s, s') = \gamma \Phi(s') - \Phi(s)$ es **invariante de política óptima**: no debería cambiar el óptimo, solo acelerar la convergencia. Que aquí *mejore* al quitarlo delata que, en la práctica, el Φ empleado o bien no era estrictamente potencial (rompiendo la garantía de invarianza y desplazando el óptimo aprendido bajo presupuesto y función de valor aproximada), o bien su potencial estaba **mal especificado** — premiaba un proxy («acercarse») desalineado con el objetivo terminal—, de modo que con horizonte y capacidad finitos el agente convergía a explotar el proxy. La lección metodológica es doble: primero, la única evidencia válida sobre el signo del shaping es la ablación, no la intuición de que «más señal ayuda»; segundo, ablacionar es la herramienta que convierte la tesis del libro en una afirmación falsable —y, contrastada, confirmada— de que la palanca dominante fue la formulación de la tarea (escala, reloj proporcional, sesgo espacial convolucional) por encima de la elección del algoritmo.

Cómo se midió (y por qué te puedes fiar)

Protocolo congelado y conjuntos disjuntos.

Supón que te digo: «mi agente limpia el nivel el 95% de las veces». Suena rotundo. Pero antes de creermelo deberías hacerme tres preguntas incómodas. ¿En qué niveles lo has medido: en los mismos donde lo entrenaste, o en otros que nunca había visto? ¿Lo dejaste explorar al azar durante el examen, o le exigiste jugar en serio? Y, sobre todo, ¿podrías repetir el experimento mañana y obtener lo mismo, o ese 95% es fruto de una tarde afortunada con una semilla concreta? Este capítulo responde a esas tres preguntas de antemano. No habla de qué algoritmo gana —eso llega en los resultados—, sino de cómo se midió todo lo que vas a leer, y de por qué puedes fiarte de ello.

El protocolo congelado: por qué te puedes fiar

❄ PROTOCOLO CONGELADO

- frozen_hash a1ab7ce18d7bad6b
- 5 semillas [0,1,2,3,4]
- presupuestos 700k / 1,5M / 3M
- evaluación GREEDY (sin exploración)
- conjuntos disjuntos train/val/test
- 160 runs reales en ledger.csv

La regla de oro:

todo se fija ANTES de medir;
nunca se re-congela. Sin
trucos a posteriori, sin
elegir la mejor semilla.

El sello del rigor: semillas, presupuestos, evaluación GREEDY y conjuntos disjuntos se fijan ANTES de medir (frozen_hash) y nunca se re-congelan. Reproducibilidad sin trucos a posteriori.

La razón de que este capítulo exista, y de que vaya antes que las cifras, es sencilla: en aprendizaje por refuerzo es **facilísimo** engañarse a uno mismo. No por mala fe, sino porque hay mil decisiones pequeñas —en qué niveles evalúo, cuántas veces repito, qué cuenta como «éxito», cuándo paro el entrenamiento— que, tomadas *después* de ver los resultados, permiten contar casi cualquier historia. La defensa contra ese autoengaño no es ser más listo; es **atarse las manos antes de empezar**. Eso es lo que hace el protocolo que describimos aquí: fija las reglas del examen antes de conocer las notas, y luego no las toca. Si entiendes este capítulo, entenderás por qué las cifras del libro no son una anécdota afortunada sino una medida en la que se puede confiar.

TRADUCCIÓN AL JUEGO

Imagina un profesor que prepara a sus alumnos para un examen de Arkanoid. Hay una forma tramposa y una honesta de ponerles nota. La tramposa: durante el curso les hace practicar diez niveles concretos, y el día del examen les pone *esos mismos diez niveles*. Todos sacan sobresaliente, claro: se los saben de memoria. Pero eso no mide si **saben jugar**; mide si tienen buena memoria. La forma honesta: el profesor guarda bajo llave un conjunto de niveles que ningún alumno ha visto jamás, y el examen se hace ahí. El que limpia un nivel nuevo es que de verdad ha aprendido a jugar, no a recitar. Todo este capítulo es la descripción de cómo montamos ese examen honesto: niveles guardados bajo llave, reglas fijadas de antemano y un tribunal que no cambia las preguntas según le convenga.

La trampa de examinar donde entrenas

Empecemos por el error más común y más grave. Un agente de refuerzo, durante el entrenamiento, ve un nivel miles de veces. Si después lo evaluamos en *ese mismo nivel*, puede que obtenga una nota altísima simplemente porque ha **memorizado** una secuencia de movimientos que funciona ahí —mover la pala así, luego asá— sin haber entendido nada general sobre cómo se juega al Arkanoid. Es el equivalente a un alumno que se aprende las respuestas del examen del año pasado: brilla mientras las preguntas no cambian, y se hunde en cuanto cambian.

El nombre técnico de lo que queremos medir es **generalización**: la capacidad de jugar bien en situaciones **nuevas**, no vistas durante el aprendizaje. Y la única forma honesta de medirla es separar tajantemente los niveles de entrenamiento de los de evaluación. Por eso el laboratorio reparte los niveles en **tres conjuntos disjuntos** —sin solapamiento alguno—:

Entrenamiento (train)	Los niveles que el agente ve mientras aprende. Puede jugarlos cuantas veces quiera; es su material de estudio.
Validación (validación)	Niveles que el agente no ve durante el entrenamiento, reservados para ir comprobando cómo progresa sin contaminar la nota final. Son el «examen de prueba» del curso.
Test (test)	Niveles guardados bajo llave que solo se usan al final, una vez todo está decidido, para poner la nota definitiva. Es el examen de verdad, y por eso no se toca hasta el último momento.

La palabra clave es **disjuntos**: un nivel que está en train no está en validación ni en test, y viceversa. Sin solapamiento no hay forma de que la memorización infle la nota, porque en el examen literalmente no aparece nada que el agente haya podido memorizar. La diferencia entre lo bien que un agente juega en sus niveles de entrenamiento y lo bien que juega en los de test tiene incluso nombre propio —el **generalization gap**—, y cuanto más pequeña es, más de fiar es el agente.

ERROR COMÚN

Pensar que «se entrenó con muchísimos niveles» equivale a «generaliza». No equivale. Un agente puede haber visto miles de niveles y aun así haberlos memorizado todos sin extraer ninguna regla general; lo que demuestra generalización no es el *volumen* de lo visto, sino el **rendimiento en lo no visto**. Por eso la separación train/test no es un tecnicismo prescindible: es justo la medida que distingue «parecer que aprende» de «de verdad generaliza». Sin conjuntos disjuntos, cualquier cifra de éxito es sospechosa por construcción.

De dónde salen los niveles: el generador y sus familias

Para que la separación en tres conjuntos tenga sentido hace falta poder fabricar muchos niveles distintos y controlados. No los dibujamos a mano: un **generador procedural** los crea automáticamente siguiendo reglas. Cada nivel pertenece a una *familia*, según la forma en que están dispuestos los ladrillos en la rejilla. Las familias son cinco:

Dispersión	Ladrillos repartidos de forma esparcida por la rejilla, sin agruparse.
Filas	Ladrillos organizados en líneas horizontales.
Columnas	Ladrillos organizados en líneas verticales.
Bloque	Ladrillos agrupados en una masa compacta.
Simétrico	Disposiciones con simetría, espejadas a un lado y otro.

Tener familias bien definidas es lo que permite construir exámenes con distintos grados de dificultad y de novedad. Porque «no visto» admite matices: un nivel puede ser nuevo pero del *mismo tipo* que los de entrenamiento, o puede ser nuevo y de un tipo que el agente jamás ha encontrado. Esa distinción da lugar a los tres exámenes del laboratorio.



La rejilla de niveles que produce el generador procedural. Cada celda es un nivel distinto; sus disposiciones de ladrillos se agrupan en las cinco familias (dispersión, filas, columnas, bloque, simétrico). De este catálogo se reparten —sin solaparse— los niveles de entrenamiento, validación y test.

Los tres exámenes: TEST-ID, OOD-patrón y OOD-dificultad

Un solo examen no basta para saber hasta dónde llega la generalización de un agente. Por eso el test no es uno, sino **tres**, ordenados de menos a más exigentes. La diferencia entre ellos está en *cuánto se parece* lo que se examina a lo que se entrenó.

TEST-ID (in-distribution)	Niveles nuevos, pero del mismo tipo de patrón que vio en entrenamiento. El agente nunca jugó <i>estos</i> niveles concretos, pero sí jugó otros de la misma familia. Es el examen más benévolo: mide si sabe generalizar dentro de lo conocido.
OOD-patrón (out-of-distribution en patrón)	Niveles cuyos patrones no se vieron nunca : familias o disposiciones ausentes del entrenamiento. Aquí no vale la memoria de un tipo concreto; hay que haber entendido algo más general sobre el juego.
OOD-dificultad (out-of-distribution en dificultad)	Niveles con densidades de ladrillos no vistas : más cargados, o dispuestos de un modo más difícil que cualquier cosa del entrenamiento. Mide si el agente aguanta cuando la tarea se endurece más allá de lo conocido.

Leídos en orden, los tres exámenes trazan una escalera. TEST-ID pregunta «¿sabes generalizar a niveles nuevos de un tipo familiar?». OOD-patrón sube el listón: «¿y a tipos que no has visto?». OOD-dificultad lo sube aún más: «¿y cuando además es más difícil de lo que conoces?». Un agente que solo brilla en TEST-ID y se hunde en los OOD ha aprendido menos de lo que parecía; uno que aguanta los tres ha aprendido de verdad. Esta gradación es la que convierte un número suelto en un **diagnóstico**.

QUÉ MIRAR

Cuando lleguen las cifras, no mires solo el éxito en TEST-ID: mira la **caída** de TEST-ID a OOD-patrón y a OOD-dificultad. Esa pendiente es la radiografía de la generalización. Un agente con notas altas y planas a lo largo de los tres exámenes ha entendido el juego; uno que empieza alto y se desploma en cuanto cambia el patrón o sube la dificultad ha memorizado más de lo que ha comprendido. La diferencia entre exámenes dice tanto como cada nota por separado.

Evaluación GREEDY: el examen se juega en serio

Hay un detalle del examen tan importante como dónde se hace: **cómo** juega el agente mientras lo evaluamos. Durante el entrenamiento, un agente necesita **explorar** —probar de vez en cuando movimientos al azar, aunque parezcan malos, para descubrir si esconden algo mejor—. La exploración es imprescindible para aprender, pero sería absurda en un examen: equivaldría a un alumno que, sabiendo la respuesta correcta, lanza una moneda para decidir si la escribe.

Por eso la evaluación se hace en modo **GREEDY** («voraz», codicioso): sin exploración, sin azar. En cada situación el agente elige **siempre** la acción que su política considera mejor, sin reservarse un porcentaje de jugadas aleatorias. Así medimos la *política aprendida* en su mejor versión, no contaminada por el ruido de exploración que solo tenía sentido mientras aprendía. La nota refleja lo que el agente **sabe hacer cuando juega a ganar**, que es justo lo que queremos saber.

El oráculo de limpiabilidad: que el examen sea justo

Hay un último problema sutil. El generador procedural crea niveles automáticamente, y nada garantiza que *todos* sean ganables. Por la geometría del rebote, ciertas disposiciones de ladrillos pueden ser **imposibles** de limpiar por completo, hagas lo que hagas con la pala. Si uno de esos niveles imposibles cayera en el examen, penalizaría al agente por algo que no es culpa suya: ningún jugador, ni humano ni artificial, podría aprobar ahí.

Para evitar esa injusticia existe un **oráculo de limpiabilidad**: un verificador que, antes de admitir un nivel en el examen, comprueba que *existe* al menos una forma de limpiarlo del todo. Los niveles imposibles se **descartan**. Así nos aseguramos de que cada nivel del test tiene solución, y de que un fallo del agente es un fallo de verdad —no juega bien—, no el resultado de haberle puesto un examen sin respuesta. El oráculo es lo que hace que el examen sea **justo**: solo se pregunta lo que tiene respuesta.

PARA LOS CURIOSOS

El oráculo no le dice al agente *cómo* limpiar el nivel, ni le da pista alguna durante el juego: solo se usa **antes**, como filtro de admisión, para certificar que el nivel es resoluble. El agente nunca ve esa información. De hecho, separar «¿es este nivel resoluble?» (lo que comprueba el oráculo) de «¿sabe el agente resolverlo?» (lo que mide el examen) es justo lo que mantiene limpia la medida: el oráculo garantiza que la pregunta es justa, pero no ayuda a responderla.

El protocolo congelado: por qué puedes repetir el experimento

Llegamos al corazón de la credibilidad. Todo lo anterior —los conjuntos disjuntos, los tres exámenes, el modo GREEDY, el oráculo— no serviría de nada si las reglas pudieran cambiarse *después* de ver los resultados. La tentación es humana: «esta semilla salió mal, la quito»; «si mido el éxito de esta otra forma, las cifras mejoran»; «dejo entrenar un poco más, que aún sube». Cada uno de esos ajustes a

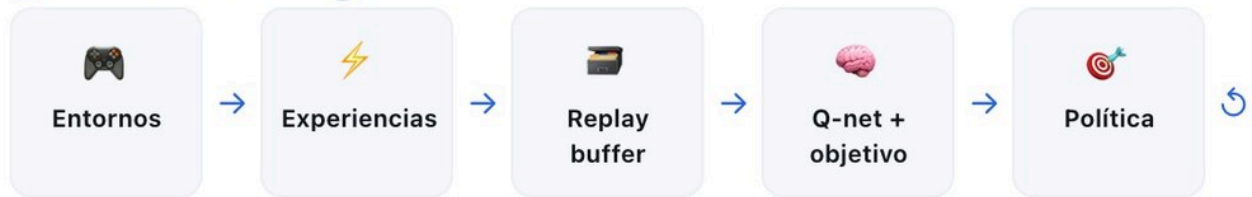
posteriori es una pequeña libertad del investigador —lo que en la jerga se llaman **researcher degrees of freedom**—, y sumadas permiten fabricar el resultado que uno quiera.

La defensa es un **protocolo congelado**: un documento que fija, *antes* de correr los experimentos, absolutamente todas las reglas del juego —semillas, presupuestos, métrica, qué cuenta como fracaso— y que después **no se toca jamás**. Su vocabulario es el siguiente:

Semillas	Cada experimento se repite con 5 semillas distintas [0, 1, 2, 3, 4]. Una semilla fija el azar inicial (pesos de la red, orden de los datos); repetir con cinco evita confundir una racha afortunada con una habilidad real, y permite reportar la dispersión entre ellas.
Presupuestos	El número de pasos de entrenamiento permitido, fijado de antemano en tres niveles: 700k, 1, 5M y 3M pasos. Nadie puede «dejar entrenar un poco más» a un algoritmo rezagado: el presupuesto es el que es.
Métrica principal	Lo que se mide es el éxito —limpiar el nivel por completo—, <i>no</i> la recompensa acumulada. Es deliberado: la recompensa puede inflarse rascando puntos sin terminar la partida; «limpiar el nivel» es un sí o un no que no admite trampa.
Colapso	Una semilla se declara en colapso si queda por debajo del 10% de éxito. Reportar el % de colapsos junto a la media evita esconder que un método, aunque a veces brille, otras veces se hunde por completo.
frozen_hash	Una huella criptográfica —a1ab7ce18d7bad6b— que identifica de forma única esta versión congelada del protocolo. Si una sola regla cambiara, la huella cambiaría; que se mantenga demuestra que las reglas son exactamente las mismas con las que se midió todo.

Sobre ese protocolo se ejecutaron **160 runs reales**, anotados uno a uno en el registro `results/ledger.csv`. El entorno técnico también está fijado: `torch 2.12.0` con `numpy 2.4.4` sobre Python 3.13, ejecutado en MPS (la GPU de un Apple M3 Ultra). No son detalles decorativos: son lo que hace que el experimento sea **reproducible**. Cualquiera con el mismo protocolo, las mismas semillas y el mismo entorno debería obtener lo mismo. Y la regla de oro, la que blinda todo: el protocolo **nunca se re-congela**. Una vez fijado, se respeta hasta el final; cambiarlo a mitad sería abrir de nuevo la puerta a las libertades del investigador que tanto cuesta cerrar.

FLUJO DE DATOS I



Off-policy: las experiencias se guardan en un replay buffer y se reutilizan en minibatches aleatorios, rompiendo la correlación temporal y mejorando la eficiencia de datos.

El flujo de datos del protocolo, de principio a fin. Los niveles del generador se reparten en conjuntos disjuntos; el agente entrena con presupuesto fijo y cinco semillas; la evaluación GREEDY se hace sobre los tres exámenes (TEST-ID, OOD-patrón, OOD-dificultad), filtrados por el oráculo de limpiabilidad; y cada resultado queda anotado en el registro. Nada de este camino se altera una vez congelado.

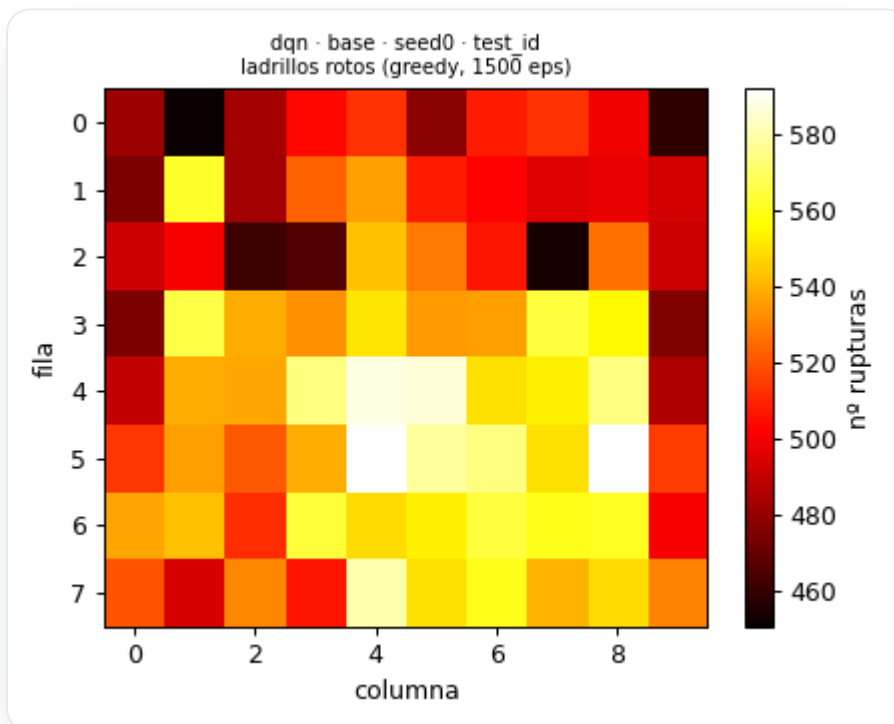
HASTA AQUÍ LLEGA

Un protocolo congelado garantiza honestidad y reproducibilidad, pero no omnisciencia. Mide lo que decidió medir: éxito en limpiar niveles de un Arkanoid de rejilla, con estas familias y estos presupuestos. No dice cómo se comportarían los mismos algoritmos en otro juego, con otra física o con presupuestos diez veces mayores. La fuerza del método no está en abarcarlo todo, sino en que, *dentro* de lo que abarca, las cifras no se pueden cocinar. Eso es exactamente lo que un protocolo congelado promete: no la verdad universal, sino una medida limpia de una pregunta concreta.

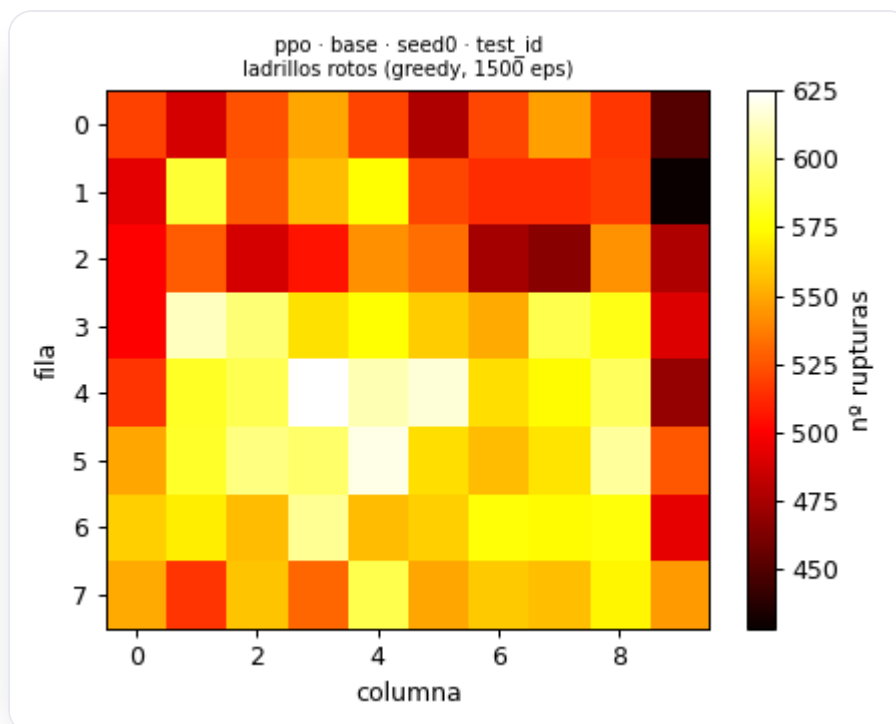
Quédate con esto

Las cifras de este libro son de fiar porque el examen se montó para que no se pudieran cocinar. Se mide en niveles **nunca vistos** (conjuntos disjuntos train/validación/test), no donde se entrenó, y en tres grados de novedad (TEST-ID, OOD-patrón, OOD-dificultad). El agente se examina **en serio** (evaluación GREEDY, sin azar), solo en niveles que **tienen solución** (oráculo de limpiabilidad), y bajo un **protocolo congelado** que fijó de antemano semillas, presupuestos y métrica —y que no se toca jamás—. La tesis es simple: medir donde no entrenas es lo que convierte «parece que aprende» en «de verdad generaliza».

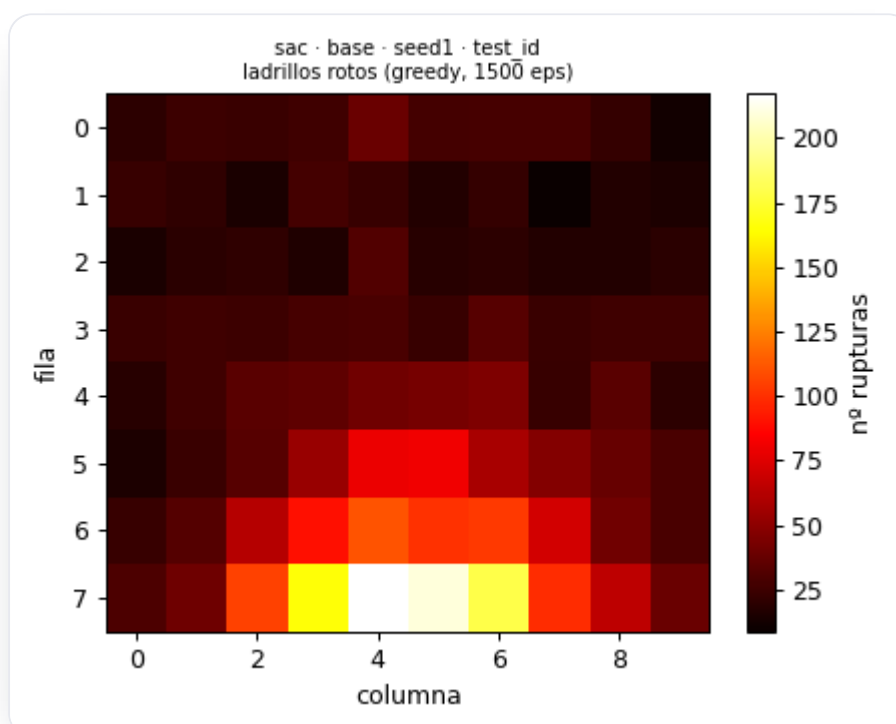
Autocomprobación. (1) ¿Por qué examinar a un agente en los mismos niveles donde entrenó da una nota engañosa? (2) Ordena de menos a más exigente los tres exámenes y di qué mide cada uno. (3) ¿Qué diferencia hay entre evaluar en modo GREEDY y dejar al agente explorar, y por qué importa en un examen? (4) ¿Para qué sirve el oráculo de limpiabilidad y por qué el agente nunca ve su información? (5) Explica con tus palabras qué evita el protocolo congelado y por qué el frozen_hash es la prueba de que las reglas no cambiaron.



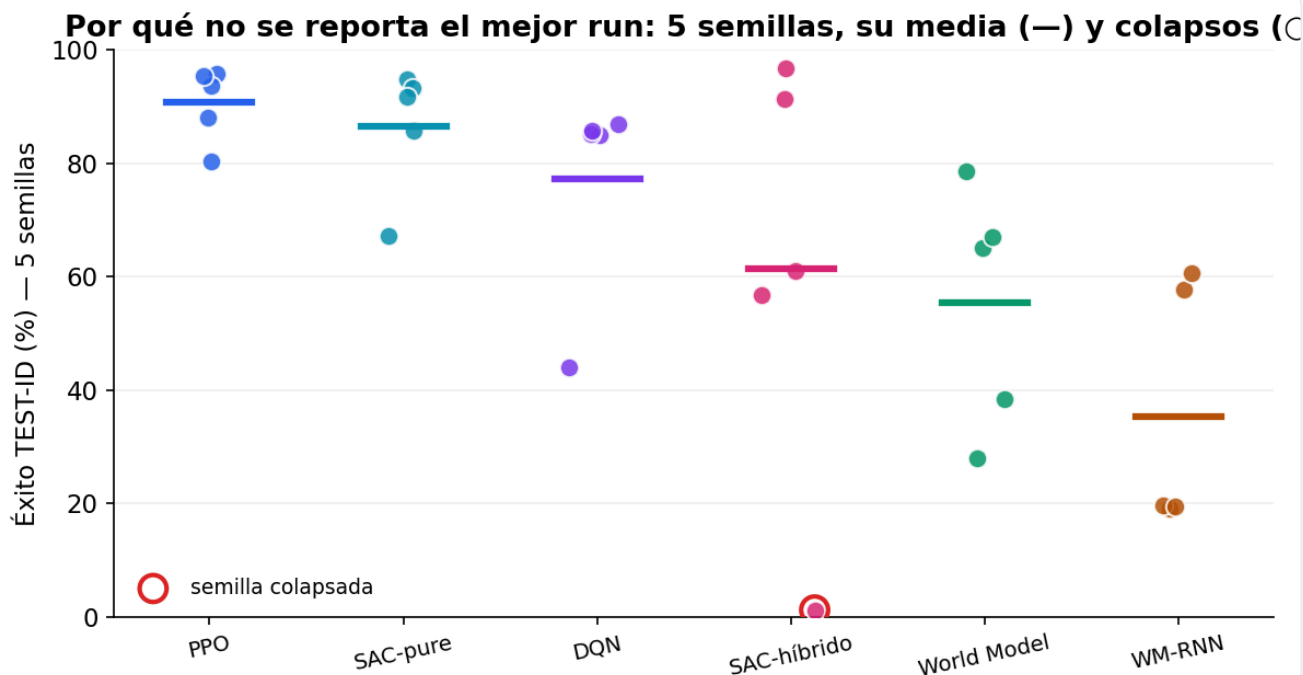
Mapa de **limpiabilidad** de DQN: cada celda cuenta cuántas veces se rompió ese ladrillo en la evaluación. El color caliente repartido por todo el tablero dice que el agente **apunta** a cualquier ladrillo, no solo a los que la bola toca de paso.



El mismo mapa para PPO: cobertura caliente y homogénea de las 80 celdas, incluidas las filas de arriba (las difíciles).
Romper ladrillos en todo el tablero —y no solo sobrevivir— es lo que distingue a un agente que generaliza.



Y así se ve un **colapso** (una semilla que se atascó): fíjate en la escala, diez veces menor, y en que las pocas roturas se amontonan en las filas de abajo, por donde la bola pasa por física. Rebota, pero no apunta: supervivencia degenerada. Por eso el veredicto se mide en éxito (limpiar), no en recompensa.



Por qué no se reporta la mejor ejecución: cada algoritmo se entrena con 5 semillas; mostramos sus cinco puntos, la media (raya) y los colapsos (círculo rojo). SAC-híbrido tiene una semilla hundida.

De dónde salen las cifras: el pipeline experimental



De dónde salen las cifras: generador → conjuntos disjuntos → entrenar (5 semillas) → evaluación GREEDY → ledger.csv → las tablas y figuras de este libro.

Y AHORA, PARA EXPERTOS **NIVEL EXPERTO**

El diseño persigue una sola cosa: una estimación insesgada de la generalización. Los **conjuntos disjuntos train/validación/test** aíslan memorización de comprensión y permiten cuantificar el **generalization gap**; el desglose en TEST-ID / OOD-patrón / OOD-dificultad descompone esa generalización en interpolación dentro de la distribución y extrapolación en patrón y en dificultad, de modo que la *pendiente* entre regímenes es tan informativa como cada nivel absoluto. La **evaluación GREEDY** (sin exploración) mide la política aprendida en sí misma, separándola de la varianza del muestreo de acciones que solo servía durante el entrenamiento. El **oráculo de limpiabilidad** descarta niveles imposibles para que el examen sea justo: el fracaso refleja una política deficiente, no un entorno sin solución, evitando un sesgo a la baja en la métrica. Y el **protocolo congelado** (`frozen_hash a1ab7ce18d7bad6b`) fija semillas `[0,1,2,3,4]` , presupuestos `700k/1,5M/3M` y la métrica —éxito, no recompensa, con umbral de colapso al 10% y reporte explícito de dispersión y % de colapsos sobre los 160 runs del `ledger.csv` — *antes* de observar resultado alguno: esto elimina los **researcher degrees of freedom** (selección de semillas, parada oportunista, redefinición de la métrica a posteriori) y, con un entorno fijado (`torch 2.12.0 + numpy 2.4.4 + py3.13` sobre MPS / M3 Ultra), garantiza reproducibilidad. La regla que lo cierra todo: el protocolo **nunca se re-congela**: re-congelar a la vista de los datos reabriría exactamente las puertas que el preregistro venía a cerrar.

Jugar: los modelos entrenados, en acción

El laboratorio web, en vivo.

¿Y si pudieras asomarte a un agente mientras aprende a jugar, ver subir su recompensa en directo y abrir su «cerebro» para mirar en qué se fija?

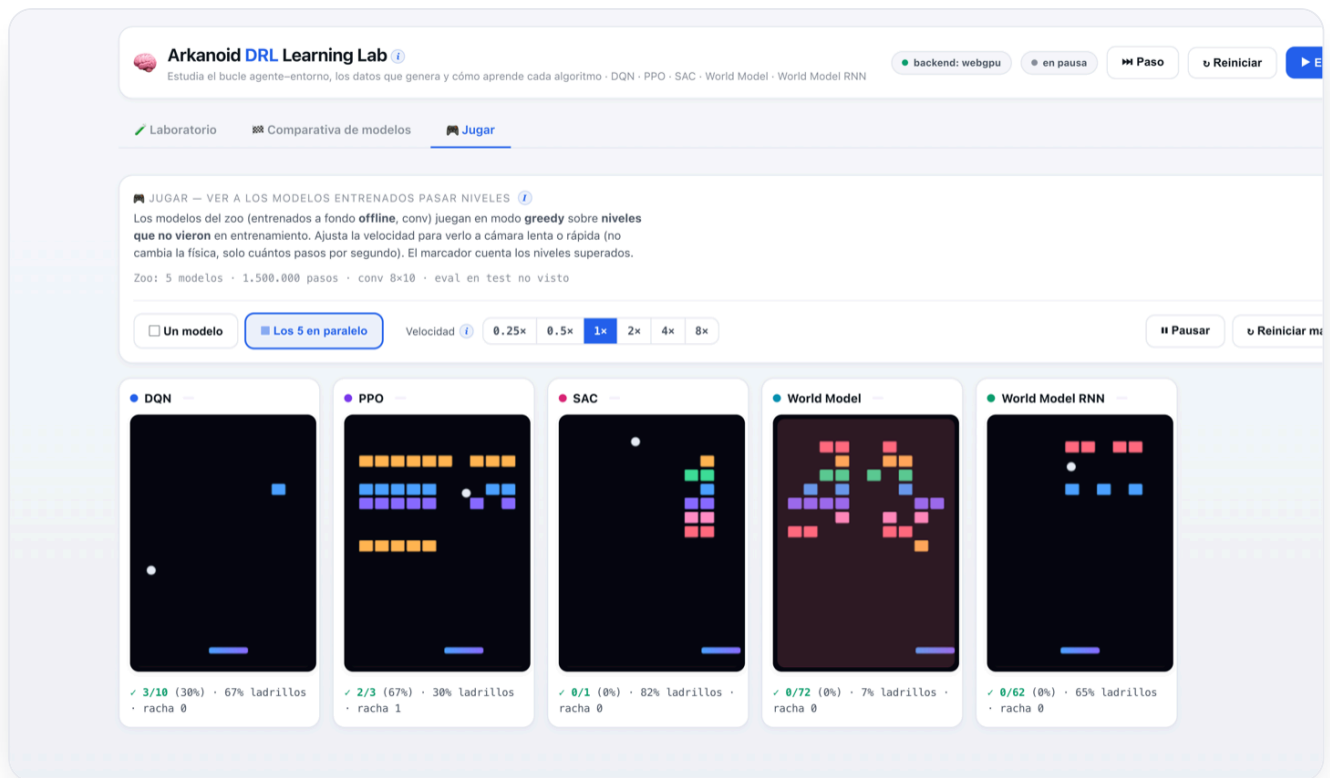
Cómo leer las curvas en vivo (cuatro formas)



Cómo mirar las curvas en vivo con criterio: una sana sube y se estabiliza; un colapso sube y se desploma; el ruido oscila; una plana es estancamiento.

Los capítulos anteriores te han dado el mapa: los cinco algoritmos, lo que cada uno aprende, sus piezas internas y los números del protocolo congelado. Este capítulo es distinto. No vamos a demostrar nada ni a defender un ranking. Vamos a **invitarte a tocar**. El laboratorio web es una página que corre *en tu navegador* y deja que entrenes, en vivo, cualquiera de los cinco algoritmos sobre el mismo Arkanoid del libro: **DQN**, **PPO**, **SAC**, **World Model** y **World Model RNN**. Verás la pala moverse, los ladrillos caer, la curva de recompensa abrirse paso y —si te animas— podrás inspeccionar la red y lanzar una búsqueda de hiperparámetros mientras todo sucede.

La idea es sencilla y vale la pena decirla antes de nada: **leer sobre aprendizaje por refuerzo y verlo ocurrir son dos cosas muy distintas**. El laboratorio existe para la segunda. Para que la intuición que has ido construyendo —ventaja, exploración, valor, política— deje de ser una palabra en una página y se convierta en algo que *reconoces a simple vista* cuando lo ves moverse.



Vista general del laboratorio jugando: a un lado el tablero del Arkanoïd con la partida observada, al otro los controles, el selector de algoritmo y las curvas. Todo ocurre en el navegador, sin instalar nada.

Qué es (y qué no es) el laboratorio

El laboratorio es una web interactiva. No es un vídeo grabado ni una simulación de adorno: dentro del navegador hay un bucle de entrenamiento real que recoge experiencia, actualiza una red neuronal y mejora —o no— una política, igual que haría un entrenamiento «de verdad», solo que a una escala que cabe en una pestaña. Puedes arrancarlo, pararlo, cambiar de algoritmo y observar cómo evoluciona el comportamiento sin tocar una línea de código.

TRADUCCIÓN AL JUEGO

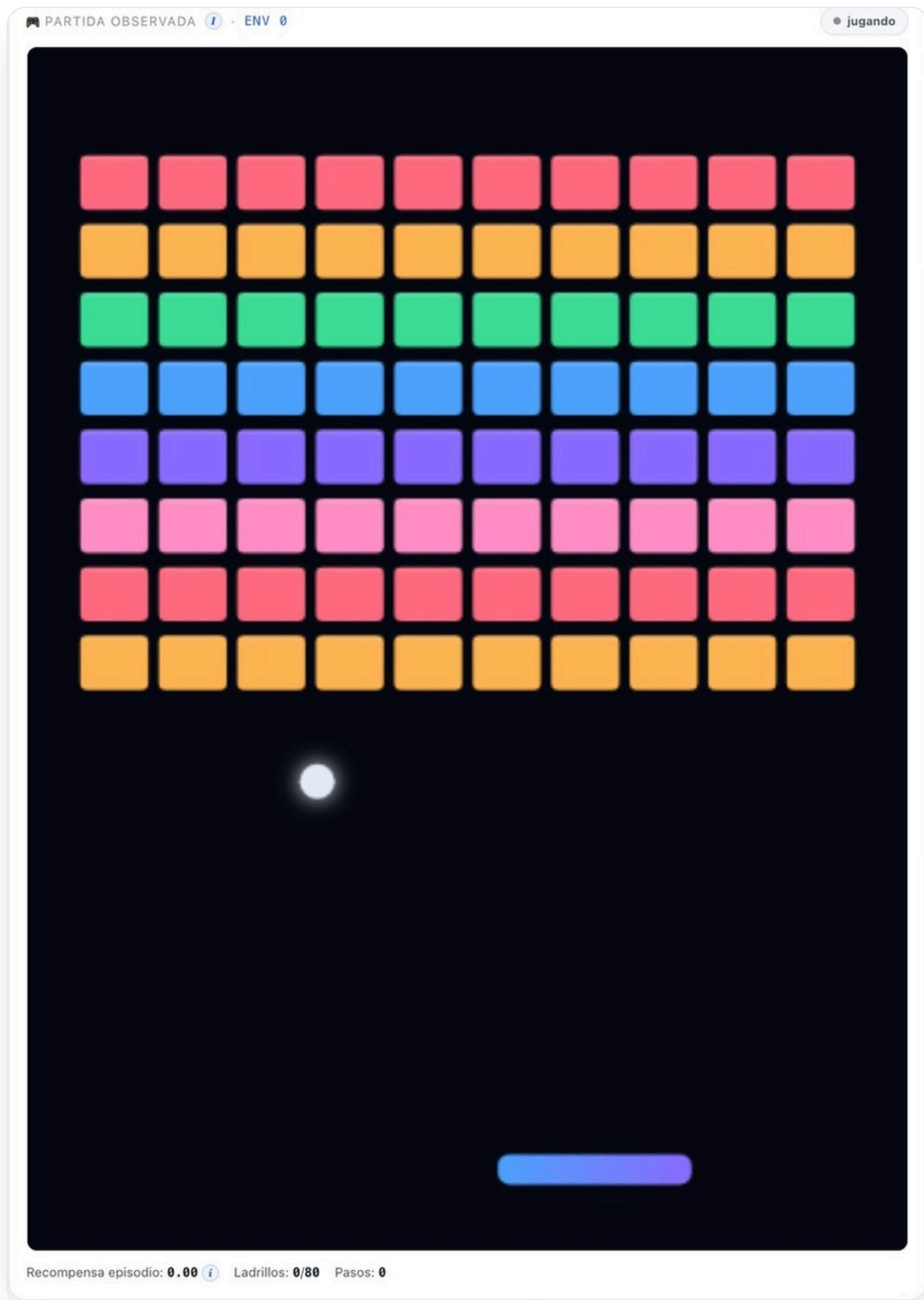
Piensa en el laboratorio como en un acuario con cristales transparentes. Dentro, un agente aprende a jugar al Arkanoïd; fuera, tú miras desde todos los ángulos: cómo se mueve la pala, qué probabilidad da a cada acción, cómo sube su recompensa partida tras partida. El agente no sabe que lo observas; tú lo ves todo. Ese es el valor del laboratorio: hacer *visible* un proceso que normalmente vive escondido en logs y ficheros.

Conviene fijar desde ya la frontera, porque es la advertencia honesta que recorre todo el libro. Lo que el laboratorio te muestra —sobre todo la recompensa que ves crecer en directo— es estupendo para la **intuición**, pero **no es el veredicto**. El logro de un agente no se mide por cuánta recompensa acumula mientras entrena, sino por su *éxito limpiando niveles que nunca ha visto*, en evaluación **greedy** (sin azar) y bajo el protocolo congelado. La recompensa en vivo es la aguja temblorosa de un velocímetro: te dice que el coche se mueve y hacia dónde, pero el resultado de la carrera se mide en otra parte. Volveremos a esto al final; por ahora, quédate con la consigna: **mira para entender, no para puntuar**.

El tablero: la partida observada

El panel central es el juego. Lo encabeza el rótulo PARTIDA OBSERVADA · ENV 0 y una insignia de estado —jugando— que te dice que hay un agente actuando ahí dentro. Es exactamente el Arkanoïd del

que habla todo el libro: una rejilla de 8×10 ladrillos arriba, una bola que rebota y una pala que el agente controla en la base. Nada de gráficos recargados; lo justo para ver el comportamiento con claridad.



El tablero de la partida observada. Arriba, la matriz 8×10 de ladrillos (el mismo estado que «ve» la rama convolucional de la red); en el centro, la bola; abajo, la pala que controla el agente. La barra inferior resume el episodio en curso: recompensa, ladrillos rotos y pasos.

Lo más informativo está en la **barra inferior**, y merece que la leas despacio:

Recompensa episodio

la recompensa acumulada en la partida que estás viendo ahora mismo. Sube cuando el agente rompe ladrillos y rebota bien. Es la señal en vivo —útil para intuir, no para juzgar.

Ladrillos

cuántos ladrillos lleva rotos sobre el total del nivel (por ejemplo, 0/80). Esta es la pista más cercana al éxito real: *limpiar el nivel* es vaciar esta cuenta.

Pasos

cuántas decisiones lleva tomadas el agente en este episodio. Relacionado con la eficiencia: un buen agente limpia el nivel en menos pasos.

QUÉ MIRAR

No te quedes solo en la recompensa: vigila la pareja **ladrillos / pasos**. Un agente que aún no ha aprendido deja la pala temblando o la planta en una esquina mientras la bola se pierde; la cuenta de ladrillos apenas avanza. Un agente que *ya sabe* coloca la pala donde la bola va a caer, encadena rebotes y la cuenta de ladrillos sube de forma sostenida. Ese cambio —de movimiento errático a interceptación limpia— es la cara visible del aprendizaje, y se aprecia mucho mejor en el tablero que en cualquier número.

El selector de algoritmo

El panel de VERSIÓN DEL EXPERIMENTO es donde eliges con qué de los cinco quieres jugar. Cada algoritmo aparece con su nombre y una etiqueta corta que resume su «carné de identidad» —las mismas tres preguntas del capítulo de panorama: ¿aprende valor o política?, ¿reutiliza experiencia (*off-policy*) o exige datos frescos (*on-policy*)?, ¿tiene modelo del mundo o no?

 VERSIÓN DEL EXPERIMENTO ⓘ

DQN ⓘ

Model-free · basado en valor · off-policy

recomendado

PPO ⓘ

Model-free · actor-crítico · on-policy

política

SAC ⓘ

Model-free · actor-crítico · off-policy

avanzado

World Model ⓘ

Model-based · off-policy

model-based

Selector de algoritmo. Cada opción muestra su familia bajo el nombre: DQN («model-free · basado en valor · off-policy»), PPO («model-free · actor-crítico · on-policy»), SAC («model-free · actor-crítico · off-policy»), World Model («model-based · off-policy»). Las insignias de la derecha (recomendado, política, avanzado, model-based) ayudan a orientarte.

Esas etiquetas no son decoración: son un atajo para que sepas qué vas a estar mirando antes de pulsar. Si eliges DQN, esperarás ver una red que estima el *valor* de cada acción; si eliges PPO o SAC, una pareja actor-crítico donde el actor reparte *probabilidad* entre las acciones; si eliges un World Model, un agente que además construye una maqueta interna del juego. Te recomendamos empezar por el que

aparece marcado como punto de partida y, una vez te hagas con la interfaz, ir saltando de uno a otro para **comparar comportamientos con tus propios ojos**.

PARA CURIOSOS

¿Por qué hay un algoritmo señalado como «recomendado» para empezar y no, simplemente, el que mejores números saca en el libro? Porque el laboratorio busca *claridad pedagógica*, no reproducir el ranking. El objetivo de tu primera sesión es entender la interfaz —dónde mirar, qué significa cada cifra— con un algoritmo que aprende de forma legible. El veredicto sobre cuál rinde mejor ya lo da el protocolo congelado en los capítulos de resultados; aquí lo que importa es que *veas aprender*.

Los controles: el ritmo del experimento

El panel de CONTROL DE EJECUCIÓN regula *cómo* corre el entrenamiento. Son pocos mandos y conviene entenderlos, porque cambian lo que ves sin cambiar lo que el agente aprende:



Controles de ejecución. «Velocidad» fija cuántos lotes se procesan por fotograma; «Entornos de entrenamiento (headless)» son las partidas invisibles que alimentan el aprendizaje en paralelo; «Entornos visuales» son las que sí se dibujan; y el interruptor activa o desactiva el moldeado de recompensa potencial.

Velocidad (lotes/frame)

cuántos lotes de aprendizaje se procesan en cada fotograma de la animación. Más velocidad = el agente aprende más rápido en tiempo de reloj, pero la imagen va más «a saltos». Es un mando de comodidad, no afecta a lo que se aprende.

Entornos de entrenamiento (headless)

el número de partidas que corren *sin dibujarse*, en paralelo, solo para recoger experiencia. Aquí está el músculo: muchas partidas a la vez aceleran muchísimo el aprendizaje. «Headless» significa, justamente, «sin pantalla».

Entornos visuales

cuántas de esas partidas se dibujan para que las veas. Son una ventana representativa; el grueso del trabajo ocurre en las invisibles.

Reward shaping potencial

un interruptor que activa o no el *moldeado de recompensa* por potencial: una ayuda extra a la señal de aprendizaje. Es una palanca metodológica delicada; tócala solo cuando quieras experimentar con ella a conciencia.

TRADUCCIÓN AL JUEGO

La distinción entre entornos **de entrenamiento** y **visuales** es la clave para no confundirte. Imagina una academia de tenis con cien pistas: en noventa y muchas se entrena a puerta cerrada (los entornos *headless*, que aportan casi toda la experiencia) y solo en unas pocas hay gradas para que el público mire (los entornos visuales). El agente aprende de *todas*; tú solo ves *algunas*. Por eso la recompensa puede mejorar de prisa aunque la partida concreta que estás mirando no sea espectacular: detrás hay decenas de partidas invisibles empujando.

Las curvas y el inspector: dónde se ve aprender

Más allá del tablero, el laboratorio ofrece dos vistas que son el verdadero corazón pedagógico. La primera son las **curvas de recompensa**: la evolución de la señal a lo largo del entrenamiento. Una curva que sube y se estabiliza cuenta una historia de aprendizaje sano; una que se desploma de golpe delata un *colapso*; una que oscila sin rumbo, un agente que no acaba de encontrar el camino. Aprender a leer la *forma* de la curva —no su valor exacto— es una de las intuiciones más valiosas que te llevarás del laboratorio.

La segunda es el **inspector de la red**: la posibilidad de abrir el «cerebro» del agente y ver en qué se apoya cada decisión. Según el algoritmo, verás el valor que asigna a cada acción (en DQN) o la distribución de probabilidad del actor junto al valor del crítico (en PPO y SAC). Es la diferencia entre ver *qué* hace el agente y entender *por qué* lo hace.

QUÉ MIRAR

En el inspector, contrasta **estados claros frente a estados ambiguos**. Cuando la bola cae limpia hacia un lado, una acción debería destacar con fuerza (valor alto o probabilidad alta); cuando la situación es dudosa, la decisión se reparte más entre opciones. Esa modulación —decidido cuando toca, prudente cuando no está claro— es la firma de una política que ha aprendido de verdad, y se ve mejor mirando el inspector que cualquier resumen numérico.

La búsqueda en rejilla de hiperparámetros

El laboratorio incluye una **búsqueda en rejilla** (en inglés, *grid search*) de hiperparámetros en vivo. La idea es probar, de forma sistemática, varias combinaciones de los ajustes que gobiernan el aprendizaje —el peso de la exploración, el tamaño del paso, etc.— y ver cuáles funcionan mejor. Es la manera honesta de descubrir que un algoritmo no es «bueno» o «malo» en abstracto: su rendimiento *depende* de cómo lo configures, y pequeñas variaciones en un hiperparámetro pueden cambiar mucho el resultado.

PARA CURIOSOS

Ver una búsqueda en rejilla en marcha es uno de los mejores antídotos contra la idea mágica de que «existe el algoritmo definitivo». Lo que descubres es lo contrario: que detrás de cada cifra bonita hay un trabajo paciente de ajuste, que dos configuraciones del *mismo* algoritmo pueden rendir de forma muy distinta, y que parte del oficio del aprendizaje por refuerzo es precisamente *encontrar* esa buena configuración. Es trabajo artesanal disfrazado de matemáticas.

Antes de pulsar: la consigna honesta

Ya tienes el mapa de los paneles. Antes de lanzarte, repasemos la advertencia metodológica, porque es lo que separa este capítulo de un folleto publicitario. El laboratorio es una herramienta de **intuición**, no un banco de pruebas de *logro*. Lo que ves —la recompensa subiendo, la pala mejorando— alimenta tu comprensión, pero **no es la métrica con la que se juzga a un agente** en este libro.

DÓNDE ESTÁ EL LÍMITE

La recompensa que crece en el laboratorio se mide *sobre los niveles que el agente está entrenando* y mientras explora (con algo de azar en sus decisiones). El veredicto del libro es otra cosa: **éxito en niveles no vistos**, en evaluación **greedy** (eligiendo siempre la mejor acción, sin azar) y como **media de varias semillas** bajo el protocolo congelado. Un agente puede acumular mucha recompensa en entrenamiento y luego generalizar mal a niveles nuevos: por eso no confundas la aguja del velocímetro con el cronómetro de la meta. Mira el laboratorio para *entender* el aprendizaje; busca el logro en los capítulos de resultados.

Dónde encontrarlo

El laboratorio y todo el código del proyecto están abiertos. Puedes explorarlo, ejecutarlo en tu propio navegador y mirar por dentro cada algoritmo:

Repositorio github.com/rcanalescoder/DRLArkanoid — el código completo del laboratorio y de los experimentos.

Web del autor www.robertocanales.com — más material y contexto del proyecto.

No necesitas instalar nada: basta con abrir la página, elegir un algoritmo y pulsar para empezar. Dedícale una sesión sin prisa. Arranca el algoritmo de partida, observa cómo la cuenta de ladrillos despega, abre el inspector en un par de estados y luego cambia de algoritmo para sentir la diferencia. Cuando una curva «de libro» deje de ser una figura y se convierta en algo que *esperas* ver, sabrás que el laboratorio ha hecho su trabajo.

Quédate con esto

El laboratorio web es una página que corre **en tu navegador** y deja entrenar en vivo los cinco algoritmos (DQN, PPO, SAC, World Model, World Model RNN) sobre el Arkanoid del libro. El **tablero** muestra la partida observada y, en su barra inferior, la pareja clave **ladrillos / pasos** —la pista más cercana al éxito real. El **selector** elige el algoritmo y recuerda su familia; los **controles** regulan velocidad y entornos (muchos invisibles, pocos visibles, todos enseñando). Las **curvas** y el **inspector** dejan ver, y entender, cómo aprende. Pero la consigna es honesta: **la recompensa en vivo sirve para la intuición, no es el veredicto**. El logro se mide aparte —éxito en niveles no vistos, greedy, media de semillas, protocolo congelado. Está todo abierto en github.com/rcanalescoder/DRLArkanoid.

AUTOCOMPROBACIÓN

1. En la barra inferior del tablero, ¿qué cifra está más cerca de medir el «éxito real» de un agente y por qué: la recompensa del episodio o la cuenta de ladrillos?
2. ¿Qué diferencia hay entre los «entornos de entrenamiento (headless)» y los «entornos visuales»? ¿De cuáles aprende el agente?
3. Estás viendo subir la recompensa en directo. ¿Por qué eso no basta para afirmar que el agente «ha aprendido bien»? ¿Dónde se mide de verdad el logro?
4. Abre el inspector y compara un estado claro con uno ambiguo. ¿Qué esperas ver en la decisión del agente en cada caso?
5. ¿Para qué sirve una búsqueda en rejilla de hiperparámetros y qué idea sobre los algoritmos ayuda a desmontar?



Y AHORA, PARA EXPERTOS

NIVEL EXPERTO

El laboratorio entrena **en el navegador** sobre un bucle de juego en JavaScript (estilo *Vita-like loop*) con **pools de entornos en paralelo**: la inmensa mayoría corren *headless* para alimentar la recogida de experiencia y solo un puñado se renderizan. La interfaz permite **inspeccionar la red** (valores de acción en DQN; distribución del actor y $V(s)$ del crítico en PPO/SAC), seguir las **curvas de recompensa** en tiempo real y lanzar una **búsqueda en rejilla** (grid search) de hiperparámetros en vivo, lo que hace tangible la sensibilidad del rendimiento a la configuración. **Advertencia metodológica**, y es la que de verdad importa: la recompensa del laboratorio es *ruido útil para intuición*, no la métrica de logro. Se mide sobre los niveles en entrenamiento, con exploración activa y a escala reducida —tres factores que la separan del criterio del estudio. El veredicto vive en la **evaluación GREEDY sobre niveles no vistos** del protocolo congelado: sin azar en la política, como media de varias semillas y midiendo *éxito* (limpiar el nivel), no recompensa acumulada. Quien confunda la curva del laboratorio con el resultado del protocolo estará leyendo el velocímetro y anotándolo como tiempo de meta.

Lo que nos llevamos · Glosario · Código

Síntesis, vocabulario y reproducibilidad.

Hemos llegado al final. Empezamos con un agente que parecía aprender y no ganaba, y terminamos con una política que limpia niveles que nunca había visto. Entre un punto y otro hay un libro entero de intuiciones, fórmulas, código y experimentos. Antes de cerrar, conviene detenerse y preguntar: ¿qué nos llevamos de verdad? ¿Cuál era la lección, más allá de los nombres de los algoritmos? Y, para quien quiera reproducirlo o seguir, ¿dónde está todo y bajo qué condiciones se puede volver a montar?

El viaje completo, de un vistazo



El viaje completo: del agente ciego (0%) al diagnóstico de los tres muros, la receta, los fundamentos, los cinco algoritmos, la medición honesta y PPO al 91% reproducible.

Cuatro lecciones para llevarte

1 · Mide en lo NO visto

El éxito en niveles de entrenamiento no dice casi nada. El 91% cuenta porque se midió fuera.

2 · Recompensa $\neq$ éxito

Una recompensa al alza puede convivir con 0% de niveles limpiados. Mide el logro, no el proxy.

3 · La forma de los datos importa

Reloj, escala y convolución (la formulación) pesaron más que cambiar de algoritmo.

4 · A veces, quitar mejora

Quitar el shaping Φ subió +8. Más piezas no es mejor; las correctas sí.

Las cuatro lecciones para llevarse: medir en lo no visto, recompensa $\neq$ éxito, la formulación pesa más que el algoritmo, y a veces quitar (el shaping Φ) mejora.

Este último capítulo no introduce nada nuevo. Es un cierre en cuatro partes. Primero, **lo que nos llevamos**: la síntesis del viaje y la tesis que lo recorre entero. Después, un **glosario** con los términos clave del libro, para tener a mano una definición breve de cada uno. Luego, **cómo está organizado el código**, para que cualquiera pueda reentrenar respetando el protocolo. Y por último, la **licencia**. Termina, como cada capítulo, con una pista para quien quiera ir más lejos.

Lo que nos llevamos

El arco del libro es una sola historia contada con datos. Arrancamos con un agente **ciego**: veía solo seis valores cinemáticos (las posiciones y velocidades de la pala y la bola), jugaba sobre una rejilla fija de 4×7

ladrillos y disponía de un tiempo por partida *constante* de 600 pasos, daba igual cuántos ladrillos quedaran. Ese agente sacaba un **0 % de éxito** en niveles dispersos —los que tienen pocos ladrillos sueltos— y solo alcanzaba un **56 %** en la rejilla completamente llena. Y ese 56 % no era buena señal, sino una *supervivencia degenerada*: el agente no aprendía a limpiar el nivel, aprendía a mantener la bola en juego el máximo tiempo posible y a esperar que algún ladrillo cayera de rebote. Parecía aprender; no ganaba.

Lo que vino después no fue cambiar de algoritmo, sino **reformular la tarea**. Cambiamos seis cosas a la vez, y todas apuntaban a lo mismo: dejar de pedirle al agente que sobreviviera y empezar a pedirle que *resolviera*. Le dimos un **reloj proporcional** (90 pasos por ladrillo, en vez de un tope fijo), de modo que un nivel con tres ladrillos no diera el mismo tiempo que uno con treinta. Le **quitamos el *shaping***, esa recompensa de ayuda Φ que premiaba acercar la pala a la bola y que, en la práctica, le enseñaba a perseguir la pelota en lugar de a ganar. Le dimos una **observación en matriz 8×10** —un mapa real de dónde está cada ladrillo— en lugar de seis números. Le pusimos un **encoder convolucional** para leer esa matriz como quien lee una imagen. **Escalamos la señal** de entrada para que la red no tuviera que pelear con magnitudes dispares. Y montamos un **currículo** con un **generador de niveles**, para que practicara con variedad y no se aprendiera de memoria un único tablero.

El resultado de esa reformulación es el dato que da sentido a todo el libro: con la nueva tarea, **PPO alcanzó un 91 % de éxito en niveles que nunca había visto** —media de cinco semillas, evaluación *greedy* (siempre la acción más probable, sin azar), cero colapsos de entrenamiento y el 100 % de las semillas por encima del 80 %—, y lo logró con un presupuesto modesto de 1,5 millones de pasos. El mismo agente que sacaba 0 % en dispersos.

TRADUCCIÓN AL JUEGO

Imagina dos profesores con el mismo alumno. El primero le pone siempre el mismo examen, le da puntos por «estar cerca de la respuesta» y le concede el mismo tiempo tenga el examen tres preguntas o treinta. El alumno aprende a parecer aplicado: rellena hueco, sobrevive a la hora, saca un aprobado raspado en el examen lleno y un cero en cualquier otro. El segundo profesor cambia el examen cada vez, solo puntúa respuestas *correctas* y ajusta el tiempo al número de preguntas. Con ese segundo profesor —y exactamente el mismo alumno— las notas se disparan en exámenes nunca vistos. No cambió el alumno: cambió cómo se le planteaba la tarea.

De ahí sale la **tesis central** del libro, y conviene enunciarla sin rodeos: **el cuello de botella nunca fue el algoritmo, ni el cómputo, sino la formulación de la tarea**. Probamos cinco familias muy distintas —valor, política, máxima entropía, modelo del mundo y modelo con memoria— y ninguna resolvió el problema mientras la tarea estuvo mal planteada. En cuanto la replanteamos, varias empezaron a funcionar. Si la pieza que faltaba hubiera sido «el algoritmo correcto», una de esas familias habría brillado sola desde el principio. No ocurrió. Lo que movió la aguja fue qué veía el agente, cómo se le repartía la recompensa y cuánto tiempo se le daba por partida.

El cuadro final, con todos los algoritmos sobre la formulación buena, ordena el estudio:

Algoritmo	Éxito en niveles no vistos	Nota
PPO	91 % (a 1,5M)	El mejor y el más fiable: 0 colapsos, 100 % de semillas >80 %
SAC (puro)	87 %	Muy estable, pero el más caro de entrenar
DQN	77 % → 85 % (a 3M)	El cimientó; mejora con más presupuesto
SAC (híbrido)	61 %	La variante híbrida no compensó
World Model	55 %	Techo por <i>model bias</i> : aprende a jugar a su maqueta, no al juego
WM-RNN	35 %	La memoria añadió coste y exigió explorar más sin pagar en éxito

Si hay que destilar el viaje en cuatro lecciones que sobreviven a este Arkanoid concreto, son estas:

Medir en lo no visto	Un éxito alto en los niveles de entrenamiento no dice casi nada. La pregunta honesta es cómo se comporta el agente en niveles que nunca vio . Todo el salto del 0 % al 91 % solo significa algo porque el 91 % se midió fuera del entrenamiento. Generalizar, no memorizar.
Recompensa ≠ éxito	Lo que el agente <i>optimiza</i> (la recompensa acumulada) y lo que nosotros <i>queremos</i> (limpiar el nivel) son cosas distintas. El agente ciego maximizaba supervivencia y nos engañaba con un 56 %. Si la recompensa premia lo que no es el objetivo, el agente aprenderá lo que no queríamos, con toda la diligencia del mundo.
La forma de los datos importa	La escala de las entradas, el reloj proporcional y la convolución sobre la matriz 8×10 no son adornos: fueron parte decisiva del salto. Darle al agente una buena representación del problema pesa tanto como el método de aprendizaje.
Quitar puede mejorar	Retirar el <i>shaping</i> Φ —una ayuda bienintencionada— mejoró el resultado, porque esa ayuda enseñaba la conducta equivocada. Menos ingeniería de recompensa, mejor objetivo. Y, sobre todo: medir en vez de suponer . Cada una de estas afirmaciones se sostiene en números de la tabla, no en intuiciones.

QUÉ NO DEMUESTRA

Nada de esto demuestra que PPO sea «el mejor algoritmo» en abstracto, ni que el *shaping* sea siempre malo, ni que los modelos del mundo no sirvan. Demuestra algo más modesto y más útil: **sobre esta formulación concreta** del Arkanoid, con este protocolo de medida, este fue el orden que salió. Cambiar la tarea podría cambiar el ranking. La lección transferible no es un ganador, es un método: fórmula con cuidado, mide en lo no visto y no confundas la recompensa con el objetivo.

Glosario

Los términos que han ido apareciendo a lo largo del libro, con una definición breve. No sustituye a los capítulos —donde cada idea se desarrolla con ejemplos—, sino que sirve de recordatorio rápido.

Agente	El que aprende y decide. En nuestro caso, la red neuronal que controla la pala.
Entorno	El mundo con el que interactúa el agente: el juego del Arkanoid, que responde a las acciones y devuelve estados y recompensas.
Estado	La situación completa del entorno en un instante (posiciones, velocidades, ladrillos presentes).
Observación	Lo que el agente <i>ve</i> del estado. Puede ser todo el estado o solo una parte; aquí, la cinemática más la matriz 8×10 .
Acción	Lo que el agente puede hacer. En el Arkanoid, tres: mover la pala a la izquierda, mantenerla o moverla a la derecha.
Recompensa	La señal numérica que el entorno da tras cada acción. Lo que el agente intenta maximizar a la larga.
Episodio	Una partida completa, desde el saque inicial hasta que se pierde la bola o se limpia el nivel.
Política	La estrategia del agente: la función $\pi(a s)$ que, dado un estado, indica qué acción tomar (o con qué probabilidad).
Valor V	$V(s)$: cuánta recompensa total se espera obtener a partir de un estado, siguiendo la política actual.
Valor Q	$Q(s, a)$: cuánta recompensa total se espera tras tomar la acción a en el estado s y seguir luego la política.
Retorno	La suma de recompensas futuras de un episodio, normalmente con descuento. Es la cantidad que de verdad se quiere maximizar.
Descuento γ	Factor entre 0 y 1 que pondera el futuro: las recompensas lejanas valen menos que las inmediatas. Controla cuánto «mira a largo plazo» el agente.
MDP	Proceso de decisión de Markov: el marco formal de estados, acciones, recompensas y transiciones donde el futuro depende solo del estado presente.
POMDP	MDP <i>parcialmente observable</i> : el agente no ve el estado completo, solo una observación. Justifica añadir memoria.
Bellman	La ecuación que relaciona el valor de un estado con el de los siguientes: el valor de hoy es la recompensa inmediata más el valor descontado de mañana. Cimiento de casi todo.
TD-error	Error de diferencia temporal: la discrepancia entre lo que el agente <i>predijo</i> que valdría un estado y lo que <i>resultó</i> ser. La señal con la que se corrige.
On-policy	Aprender solo de datos generados por la política actual (PPO). Las experiencias caducan en cuanto la política cambia.
Off-policy	Aprender también de experiencia pasada, jugada por versiones anteriores (DQN, SAC). Permite reutilizar datos.
Replay	Memoria de repetición: el almacén de transiciones pasadas que los métodos off-policy reutilizan para entrenar muchas veces.
Red objetivo	Copia «congelada» de la red que se actualiza despacio, usada en DQN para dar estabilidad a las predicciones de valor.

ϵ-greedy	Regla de exploración: con probabilidad ϵ se actúa al azar y, el resto del tiempo, se toma la mejor acción conocida.
Entropía	Medida de cuán «repartida» está la política entre acciones. Premiarla fomenta explorar y no encasillarse demasiado pronto.
Convolución	Operación que lee una matriz por zonas locales, como quien reconoce formas en una imagen. Lo que usa el encoder para leer la rejilla 8×10 .
Shaping Φ	Recompensa de ayuda añadida a la del entorno para guiar el aprendizaje. En este libro, quitarla mejoró el resultado.
Generalización / OOD	Capacidad de rendir en situaciones <i>fuera de distribución</i> (out-of-distribution): niveles no vistos en el entrenamiento. La prueba honesta de aprendizaje.
GREEDY	Modo de evaluación sin azar: se elige siempre la acción más probable. Así se mide lo que el agente <i>de verdad</i> ha aprendido.
Colapso	Desplome brusco del rendimiento durante el entrenamiento, del que a veces no se recupera. PPO tuvo cero.
Protocolo congelado	El conjunto fijo de reglas de evaluación (semillas, presupuestos, niveles de test) que no se toca, para que las comparaciones sean justas.

| Cómo está organizado el código

Todo lo que se ha contado es reproducible. El código vive en el repositorio público `github.com/rcanalescoder/DRLArkanoid`, y los experimentos se rigen por un **protocolo congelado** cuya identidad se sella en un hash: `frozen_hash a1ab7ce18d7bad6b`. Ese hash es la garantía de que las comparaciones del libro se hicieron todas bajo las mismas reglas; cambiar las reglas cambiaría el hash.

El protocolo fija las condiciones de medida. Cada configuración se entrena con **5 semillas** distintas — para no confundir un buen resultado con una inicialización afortunada— y bajo tres **presupuestos** de cómputo: 700k, 1,5M y 3M de pasos. Los resultados de todas las ejecuciones se vuelcan a un único registro, `results/ledger.csv`, que recoge los **160 runs** del estudio. Cada fila es un experimento con su semilla, su presupuesto, su algoritmo y sus métricas; las tablas de los capítulos no son más que lecturas de ese fichero.

El marco técnico es `torch 2.12.0` con `numpy 2.4.4` sobre `Python 3.13`, ejecutado en **MPS** —el backend de cómputo de Apple— sobre un **Apple M3 Ultra**. No hace falta ese hardware concreto para reproducir el estudio: cualquiera puede clonar el repositorio, instalar el entorno y **reentrenar respetando el protocolo**. Si se mantienen las semillas, los presupuestos y las reglas de evaluación selladas en el hash, los números deberían ser comparables con los del libro. Esa es justamente la idea de un protocolo congelado: que el resultado no dependa de quién, cuándo ni en qué máquina se ejecutó.

PARA CURIOSOS

¿Por qué tantos runs? Sale de multiplicar: varios algoritmos $\times$ 5 semillas $\times$ 3 presupuestos $\times$ las variantes de cada familia. Ese producto es lo que da las 160 ejecuciones del `ledger.csv`. La disciplina de registrarlo *todo* en un solo fichero, fila a fila, es lo que permite que una afirmación como «PPO hace 91 %» se pueda rastrear hasta las cinco filas concretas que la sostienen. Sin ese registro, los porcentajes del libro serían creíbles; con él, son verificables.

Licencia

Este proyecto —código y libro— se publica bajo licencia **MIT**: puede usarse, copiarse, modificarse y redistribuirse libremente, incluso con fines comerciales, conservando el aviso de copyright. Es la licencia más permisiva de uso común, elegida precisamente para que cualquiera pueda partir de aquí.

Autor: **Roberto Canales Mora**, con Claude. Web del autor: www.robertocanales.com. Repositorio: github.com/rcanalescoder/DRLArkanoid.

Quédate con esto

El viaje fue de un agente **ciego** que sacaba **0 %** en niveles dispersos —y un engañoso 56 % de pura supervivencia en la rejilla llena— a un **PPO con 91 %** de éxito en niveles **nunca vistos**. Lo que desbloqueó ese salto no fue el algoritmo ni más cómputo, sino **reformular la tarea**: reloj proporcional, fuera el *shaping* Φ , observación 8×10 , encoder convolucional, escala y currículo. Cuatro lecciones perduran: **medir en lo no visto**, recordar que **recompensa $\neq$ éxito**, cuidar la **forma de los datos** (escala, reloj, convolución) y aceptar que a veces **quitar mejora** —y, sobre todo, **medir en vez de suponer**. Todo es reproducible: repositorio público, protocolo congelado `a1ab7ce18d7bad6b`, 5 semillas, 160 runs en `ledger.csv`, bajo licencia MIT.

Mapa de conceptos: el glosario por familias

Entorno y experiencia

- estado s
- acción a
- recompensa r
- episodio
- transición

Recompensa y retorno

- retorno G
- descuento γ
- *shaping* Φ

Valor y política

- $V(s)$
- $Q(s,a)$
- política π
- Bellman
- TD-error

Redes

- neurona
- convolución
- dos ramas
- gradiente

Algoritmos

- DQN
- PPO
- SAC
- World Model
- WM-RNN

Medición

- éxito
- TEST/OOD
- colapso
- GREEDY
- semillas

El glosario como sistema, no como lista: los términos agrupados por familias (entorno, recompensa, valor/política, redes, algoritmos, medición).

Del código al resultado: el ciclo reproducible



El ciclo reproducible cerrado: del repositorio al entorno, el entrenamiento, la evaluación, el ledger, las figuras y, finalmente, este libro.



Y AHORA, PARA EXPERTOS NIVEL EXPERTO

Si este libro despertó las ganas de seguir, estas son las lecturas que conviene tener a mano. El texto de referencia, base de casi todo lo que aquí se contó en versión intuitiva, es **Sutton & Barto, «Reinforcement Learning: An Introduction»** (2.^a ed., 2018): MDP, Bellman, TD, on/off-policy y gradiente de política, con rigor y claridad. Sobre la familia de valor, el artículo fundacional de Deep RL es **DQN** (Mnih et al., *Nature* 2015), que combinó **Q-learning**, replay y red objetivo; su corrección del sesgo optimista llegó con **Double DQN** (van Hasselt et al., 2016). Sobre la familia de política, los dos pilares de este estudio: **PPO** (Schulman et al., 2017) con su objetivo sustituto recortado, apoyado en la estimación de ventaja **GAE** (Schulman et al., 2016). En máxima entropía, **SAC** (Haarnoja et al., 2018) y su adaptación a acciones discretas, **SAC discreto** (Christodoulou, 2019), la que aquí se empleó. La raíz de los modelos del mundo está en **Dyna** (Sutton, 1991), que ya mezclaba experiencia real e imaginaria. Y, transversal a todo, un recordatorio metodológico imprescindible: **la evaluación estadística en Deep RL** (Agarwal et al., 2021, «*Deep RL at the Edge of the Statistical Precipice*»), que argumenta por qué reportar medias sobre pocas semillas engaña y propone alternativas robustas —exactamente el espíritu de las 5 semillas y el protocolo congelado de este trabajo. Como **direcciones abiertas**, lo que no se exploró y promete: *prioritized replay* (muestrear las transiciones más informativas), retornos *n-step* (interpolan entre TD y Monte Carlo), *distributional RL* (aprender la distribución del retorno, no solo su media) y mejores *world models* que recorten el *model bias* que aquí impuso el techo del 55 %. El mapa está dibujado; queda mucho terreno por andar.